宇宙安全声明

- 这部分没有找到系统的学习资料所以课程大部分的观点和结论都是参考零零散散的博客和结合实验现象的猜测，因此难免有错误，介意者勿看~
- 所有参考均会以链接的形式提供，并在视频中强调出处，坚决维护原创者的劳动成果~

付费通知

- 所有课程都是免费的，即使不购买PPT和代码也是可以正常学习的～
- 收费的原因主要是 为了更好的呈现课程 制作ppt和整理代码花费了我大量的时间，望各位理解～
- 如果想购买课程PPT和代码 或者单纯想支持一下UP（感激不尽）一共是50块钱（包含reduce sgemm ncu以及后面会不断更新的cuda内容）加微信购买～
- 不断更新的代码和ppt会给付费者拉一个交流群在群里同步，但是这个群不是我的答疑群，大家可以一起交流～



扫一扫上面的二维码图案，加我为朋友。

【个人微信号】：
hkl793831293



全网首发！

ggml源码逐行调试带读

希望成为你大模型部署的第一课！

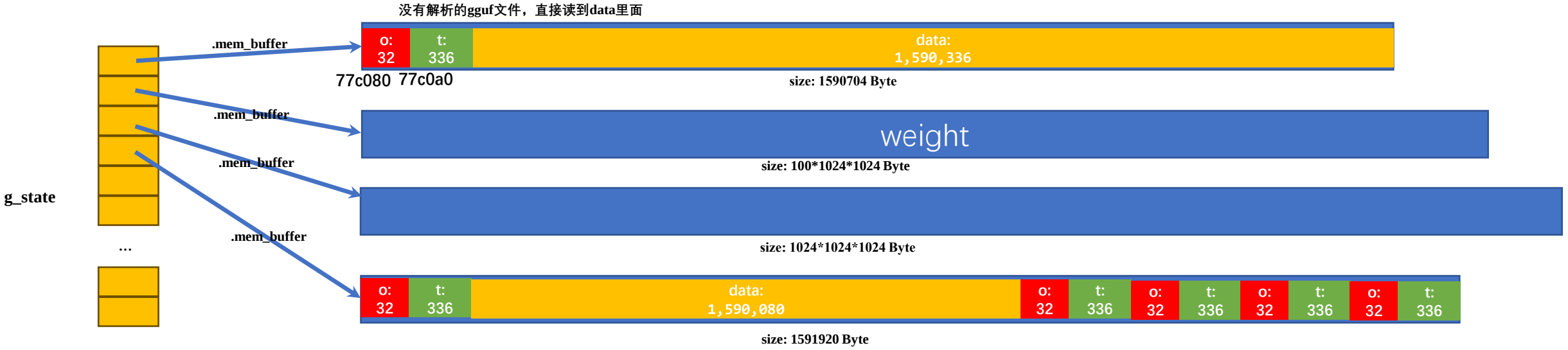
ggml_context: 一个装载各类对象 (如张量、计算图、其他数据) 的“容器”

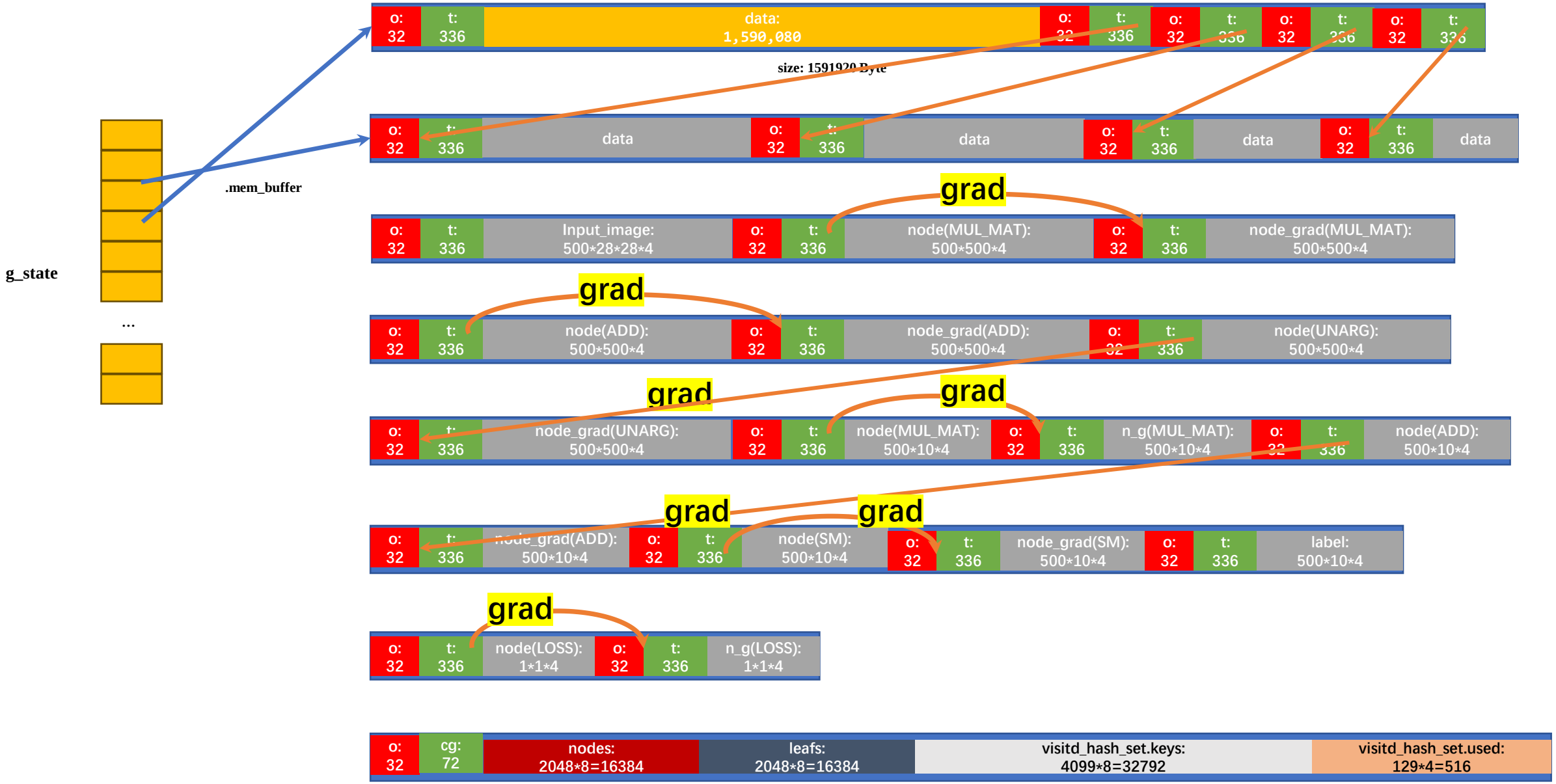
```
Georgi Gerganov, 3 months ago | 2 authors (kunnis and one other)
struct ggml_context {
    size_t mem_size;
    void* mem_buffer;
    bool   mem_buffer_owned;
    bool   no_alloc;
    bool   no_alloc_save; // this is used to save the no_alloc state when using scratch buffers

    int    n_objects;

    struct ggml_object * objects_begin;
    struct ggml_object * objects_end;

    struct ggml_scratch scratch;      kunnis, 5 months ago • ggml : use dynamic thread scheduling f
    struct ggml_scratch scratch_save;
};
```




g_state
...
.mem_buffer
o: 32
t: 336
data: 1,590,080
size: 1591920 Byte
data
Input_image: 500*28*28*4
node(MUL_MAT): 500*500*4
node_grad(MUL_MAT): 500*500*4
node(ADD): 500*500*4
node_grad(ADD): 500*500*4
node(UNARG): 500*500*4
node_grad(UNARG): 500*500*4
node(MUL_MAT): 500*10*4
n_g(MUL_MAT): 500*10*4
node(ADD): 500*10*4
node_grad(ADD): 500*10*4
node(SM): 500*10*4
node_grad(SM): 500*10*4
label: 500*10*4
node(LOSS): 1*1*4
n_g(LOSS): 1*1*4
grad
cg: 72
nodes: 2048*8=16384
leafs: 2048*8=16384
visitd_hash_set.keys: 4099*8=32792
visitd_hash_set.used: 129*4=516

ggml_cgraph: 计算图的表示，可以理解为将要传给后端的“计算执行顺序”。

```
// computation graph
slaren, 2 months ago | 2 authors (Georgi Gerganov and one other)
struct ggml_cgraph {
    int size;
    int n_nodes;
    int n_leafs;

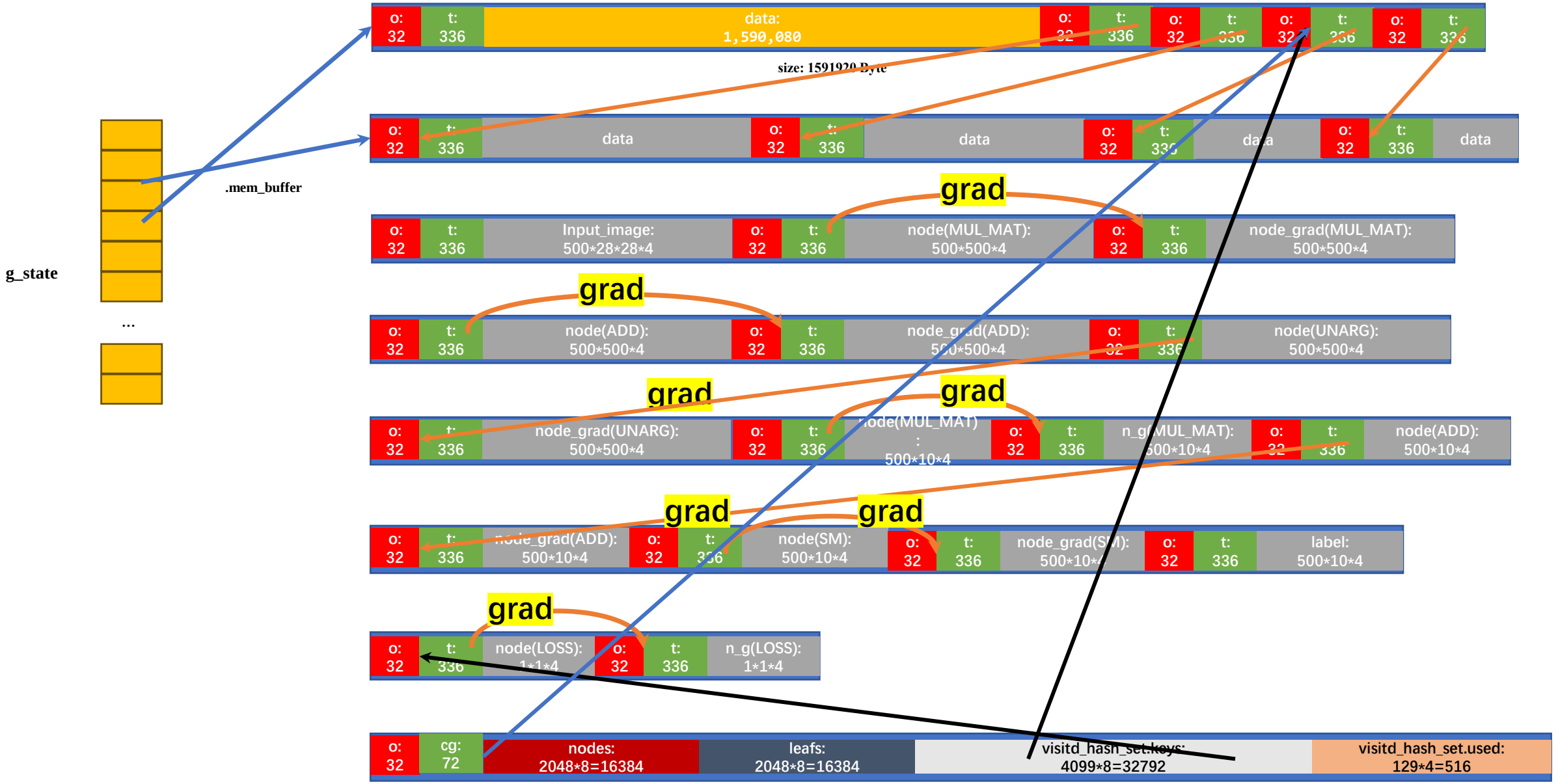
    struct ggml_tensor ** nodes;
    struct ggml_tensor ** grads;
    struct ggml_tensor ** leafs;

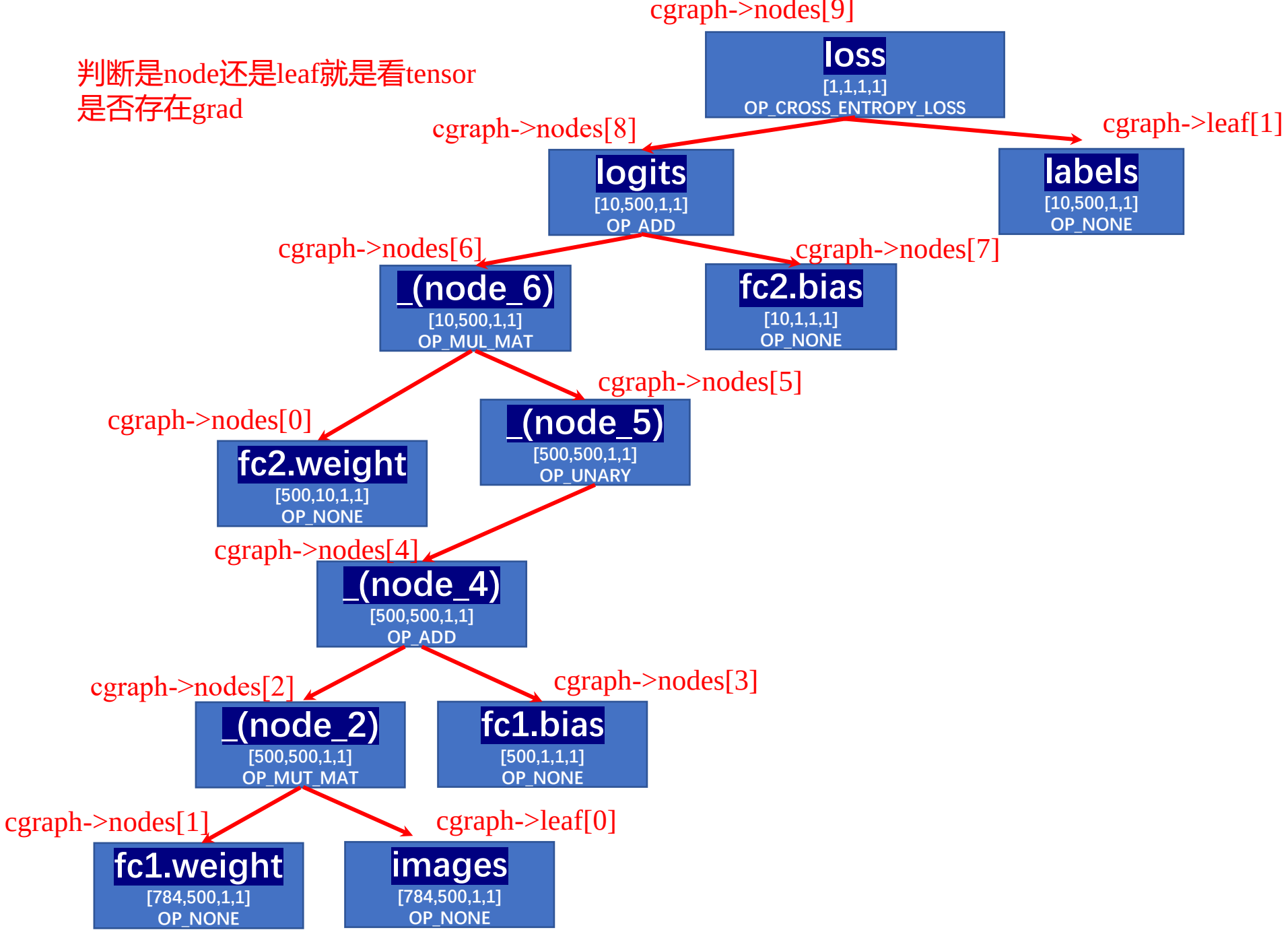
    struct ggml_hash_set visited_hash_set;

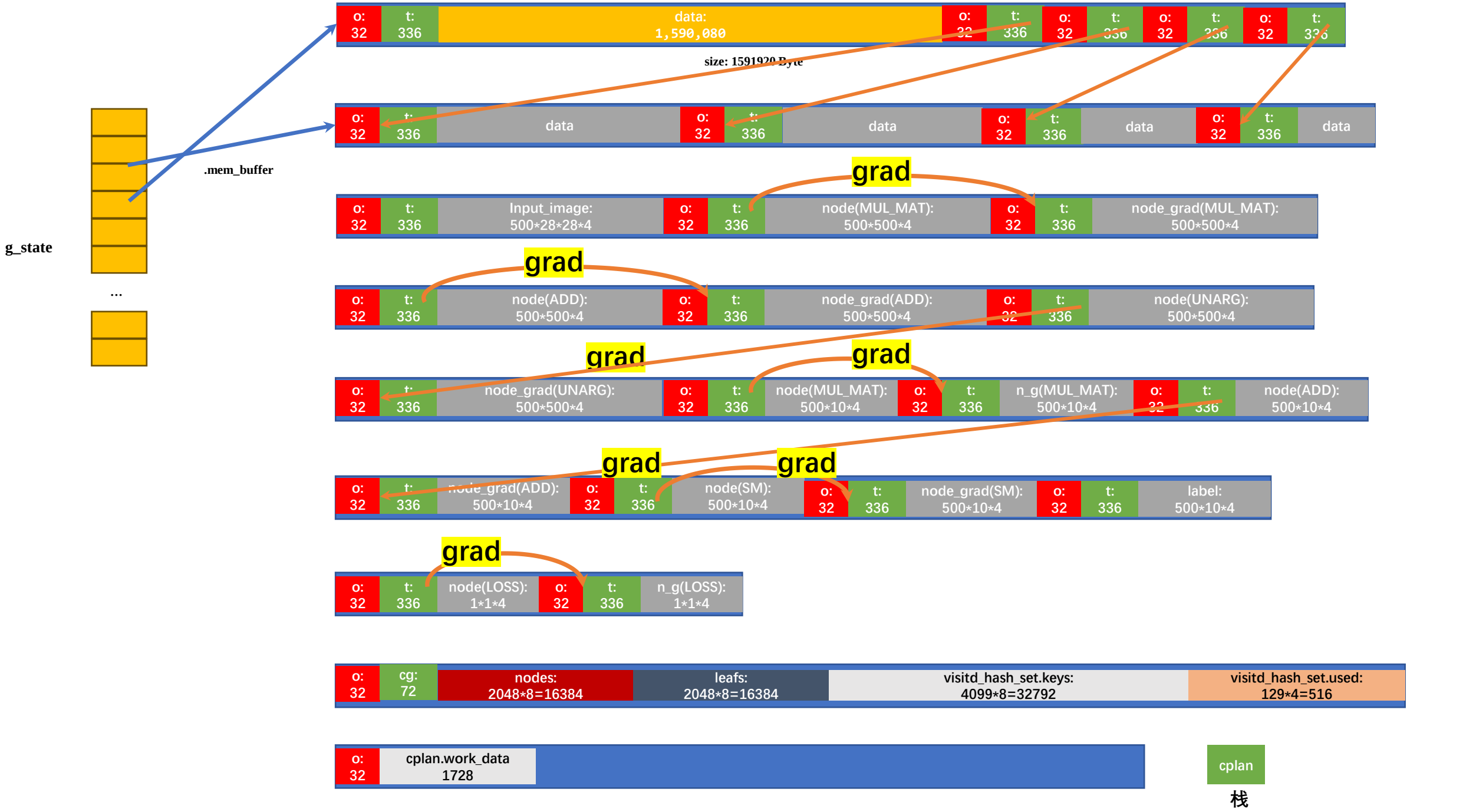
    enum ggml_cgraph_eval_order order;
};
```

```
Johannes Gäßler, last month | 2 authors (slaren and one other)
struct ggml_hash_set {
    size_t size;
    ggml_bitset_t * used;       // whether or not the keys are in use i.e. set
    struct ggml_tensor ** keys; // actual tensors in the set, keys[i] is only defined if ggml_bitset_get(used, i)
};
    slaren, 11 months ago • ggml-backend v2 : add ggml_backend_sched (#586)
```

o: 32	cg: 72	nodes: 2048*8=16384	leafs: 2048*8=16384	visitd_hash_set.keys: 4099*8=32792	visitd_hash_set.used: 129*4=516


g_state
...
.mem_buffer
o: 32
t: 336
data: 1,590,080
size: 1591920 Byte
data
Input_image: 500*28*28*4
node(MUL_MAT): 500*500*4
node_grad(MUL_MAT): 500*500*4
node(ADD): 500*500*4
node_grad(ADD): 500*500*4
node(UNARG): 500*500*4
node_grad(UNARG): 500*500*4
node(MUL_MAT): 500*10*4
node(ADD): 500*10*4
node_grad(ADD): 500*10*4
node(SM): 500*10*4
label: 500*10*4
node(LOSS): 1*1*4
n_g(LOSS): 1*1*4
grad
cg: 72
nodes: 2048*8=16384
leafs: 2048*8=16384
visitd_hash_set.keys: 4099*8=32792
visitd_hash_set.used: 129*4=516


判断是node还是leaf就是看tensor
是否存在grad
cgraph->nodes[9]
loss
[1,1,1,1]
OP_CROSS_ENTROPY_LOSS
cgraph->nodes[8]
cgraph->leaf[1]
logits
[10,500,1,1]
OP_ADD
labels
[10,500,1,1]
OP_NONE
cgraph->nodes[6]
cgraph->nodes[7]
_(node_6)
[10,500,1,1]
OP_MUL_MAT
fc2.bias
[10,1,1,1]
OP_NONE
cgraph->nodes[5]
cgraph->nodes[0]
fc2.weight
[500,10,1,1]
OP_NONE
_(node_5)
[500,500,1,1]
OP_UNARY
cgraph->nodes[4]
_(node_4)
[500,500,1,1]
OP_ADD
cgraph->nodes[2]
cgraph->nodes[3]
_(node_2)
[500,500,1,1]
OP_MUT_MAT
fc1.bias
[500,1,1,1]
OP_NONE
cgraph->nodes[1]
cgraph->leaf[0]
fc1.weight
[784,500,1,1]
OP_NONE
images
[784,500,1,1]
OP_NONE


g_state
...
.mem_buffer
o: 32
t: 336
data: 1,590,080
size: 1591920 Byte
data
Input_image: 500*28*28*4
node(MUL_MAT): 500*500*4
node_grad(MUL_MAT): 500*500*4
node(ADD): 500*500*4
node_grad(ADD): 500*500*4
node(UNARG): 500*500*4
node_grad(UNARG): 500*500*4
node(MUL_MAT): 500*10*4
n_g(MUL_MAT): 500*10*4
node(ADD): 500*10*4
node_grad(ADD): 500*10*4
node(SM): 500*10*4
node_grad(SM): 500*10*4
label: 500*10*4
node(LOSS): 1*1*4
n_g(LOSS): 1*1*4
grad
cg: 72
nodes: 2048*8=16384
leafs: 2048*8=16384
visitd_hash_set.keys: 4099*8=32792
visitd_hash_set.used: 129*4=516
cplan.work_data 1728
cplan
栈

```c
struct ggml_cplan {       Georgi Gerganov, 15 months ago • ggml : sync llama.cpp (changes to ggml_graph_co…
    size_t    work_size; // size of work buffer, calculated by `ggml_graph_plan()`
    uint8_t * work_data; // work buffer, to be allocated by caller before calling to `ggml_graph_compute()`

    int n_threads;
    struct ggml_threadpool * threadpool;

    // abort ggml_graph_compute when true
    ggml_abort_callback abort_callback;
    void *              abort_callback_data;
};
```



```
// Threadpool def
Faisal Zaghloul, last month | 3 authors (Faisal Zaghloul and others)
struct ggml_threadpool {
    ggml_mutex_t mutex;       // mutex for cond.var
    ggml_cond_t  cond;        // cond.var for waiting for new work

    struct ggml_cgraph * cgraph;
    struct ggml_cplan  * cplan;

    // synchronization primitives
    atomic_int n_graph;       // incremented when there is work to be done (i.e each graph)
    atomic_int n_barrier;
    atomic_int n_barrier_passed;
    atomic_int current_chunk; // currently processing chunk during Mat_Mul, shared between all the threads.

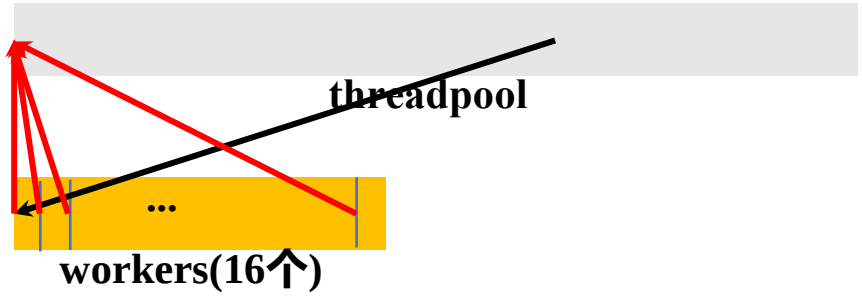
    // these are atomic as an annotation for thread-sanitizer
    atomic_bool stop;         // Used for stopping the threadpool altogether
    atomic_bool pause;        // Used for pausing the threadpool or individual threads

    struct ggml_compute_state * workers;   // per thread state
    int          n_threads_max; // number of threads in the pool
    int          n_threads_cur; // number of threads used in the current graph

    int32_t      prio;        // Scheduling priority
    uint32_t     poll;        // Polling level (0 - no polling)

    enum ggml_status ec;
};
```

```
// Per-thread state
Faisal Zaghloul, last month | 3 authors (Faisal Zaghloul and others)
struct ggml_compute_state {
#ifndef GGML_USE_OPENMP
    ggml_thread_t thrd;
    bool cpumask[GGML_MAX_N_THREADS];
    int  last_graph;
    bool pending;
#endif
    struct ggml_threadpool * threadpool;
    int ith;
};
```



create_backend

```
static ggml_backend_registry & get_reg() {
    static ggml_backend_registry reg;
    return reg;
}
```

```
Diego Devesa, 7 days ago | 2 authors (Diego Devesa and one other)
struct ggml_backend_registry {
    std::vector<ggml_backend_reg_t> backends;
    std::vector<ggml_backend_dev_t> devices;
```

```
Diego Devesa, 2 weeks ago | 1 author (Diego Devesa)
struct ggml_backend_device {
    struct ggml_backend_device_i iface;
    ggml_backend_reg_t reg;
    void * context;
};
```

```
struct ggml_backend_cuda_device_context {
    int device;
    std::string name;
    std::string description;
};
```

```
Diego Devesa, 2 weeks ago | 1 author (Diego Devesa)
struct ggml_backend_reg {
    // int api_version; // TODO: for dynamic loading
    struct ggml_backend_reg_i iface;
    void * context;
};
```

```
Diego Devesa, 2 weeks ago | 1 author (Diego Devesa)
struct ggml_backend_cuda_reg_context {
    std::vector<ggml_backend_dev_t> devices;
```

static ggml_backend_reg reg【cuda】;

static ggml_backend_reg ggml_backend_cpu_reg;

```
Diego Devesa, 2 weeks ago | 1 author (Diego Devesa)
struct ggml_backend_reg {
    // int api_version; // TODO: for dynamic loading
    struct ggml_backend_reg_i iface;
    void * context;
};
```

Null

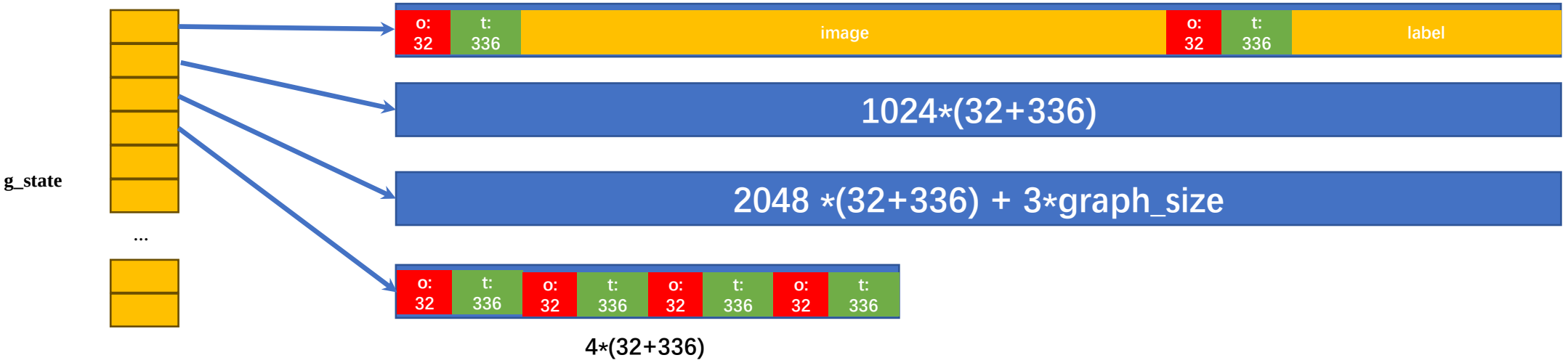
model.backend

```
Diego Devesa, 2 weeks ago | 3 authors (slaren and others)
struct ggml_backend {
    ggml_guid_t guid;
    struct ggml_backend_i iface;
    ggml_backend_dev_t device;
    void * context;
};
```

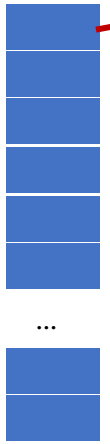
```
struct ggml_backend_cuda_context {
    int device;
    std::string name;
    cudaEvent_t copy_event = nullptr;

    cudaStream_t streams[GGML_CUDA_MAX_DEVICES][GGML_CUDA_MAX_STREAMS] = { { nullptr } };
    cublasHandle_t cublas_handles[GGML_CUDA_MAX_DEVICES] = {nullptr};

    std::unique_ptr<ggml_cuda_graph> cuda_graph;
```


g_state
...
o:
32
t:
336
image
o:
32
t:
336
label
1024*(32+336)
2048 *(32+336) + 3*graph_size
o:
32
t:
336
o:
32
t:
336
o:
32
t:
336
o:
32
t:
336
4*(32+336)

ggml_backend_cuda_buffer_types



...

```
Diego Devesa, 2 weeks ago | 2 authors (slaren and one other)
struct ggml_backend_buffer_type {
    struct ggml_backend_buffer_type_i  iface;
    ggml_backend_dev_t device;
    void * context;
};
```

```
struct ggml_backend_cuda_buffer_type_context {
    int device;
    std::string name;
};
```

model.buf_weight

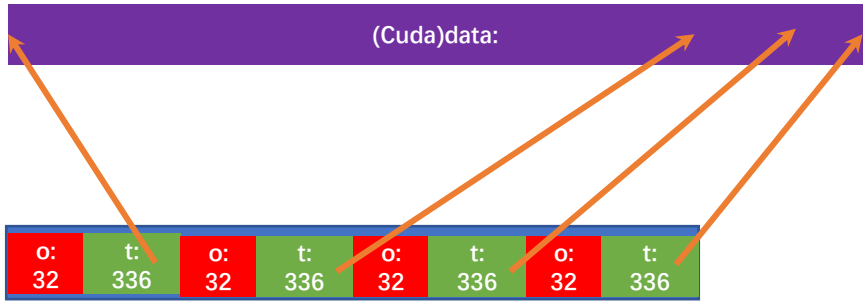
```
Diego Devesa, 2 weeks ago | 2 authors (slaren and one other)
struct ggml_backend_buffer {
    struct ggml_backend_buffer_i  iface;
    ggml_backend_buffer_type_t    buft;
    void * context;
    size_t size;
    enum ggml_backend_buffer_usage usage;
};
```

```
slaren, 7 months ago | 2 authors (slaren and one other)
struct ggml_backend_cuda_buffer_context {
    int device;
    void * dev_ptr = nullptr;
    std::string name;

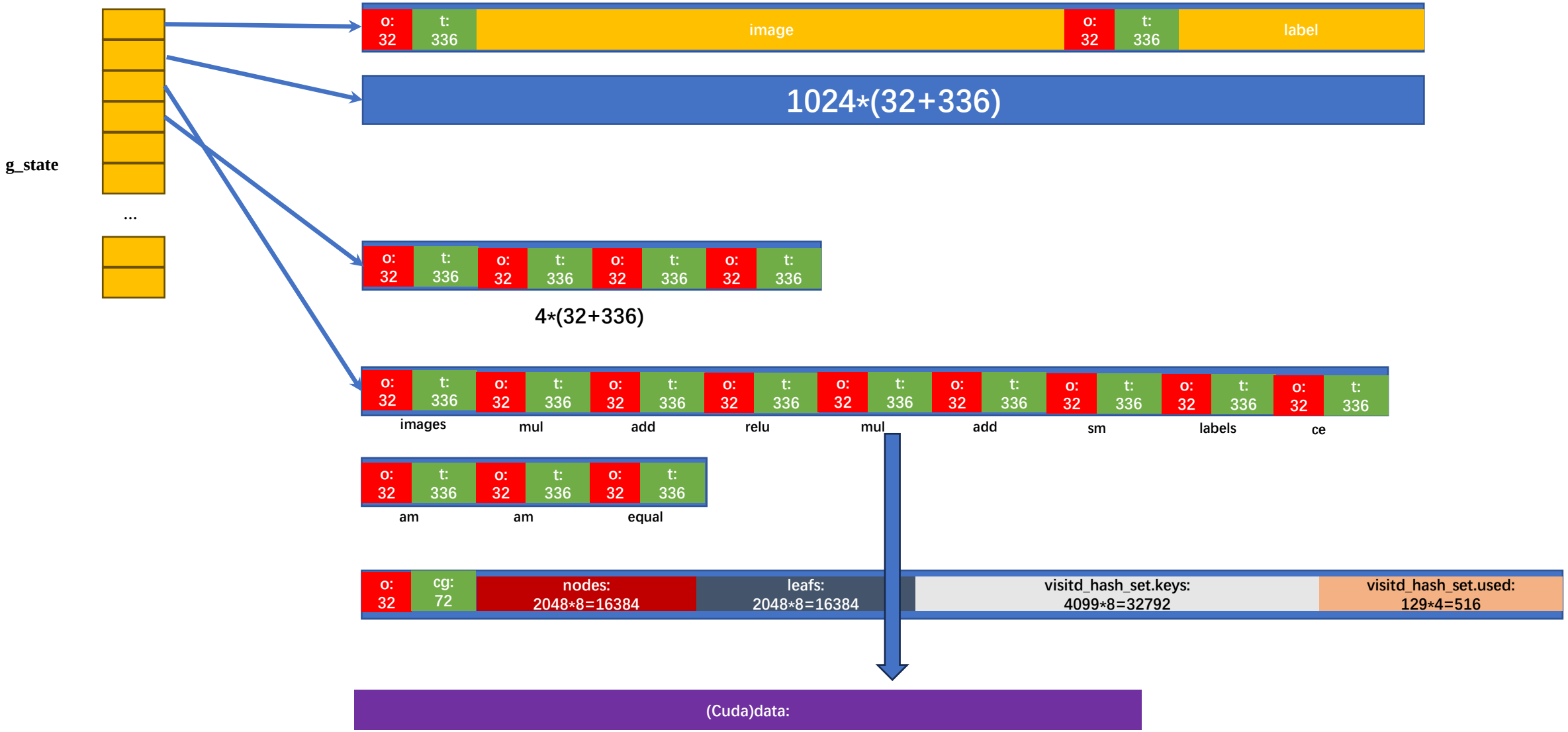
    ggml_backend_cuda_buffer_context(int device, void * dev_ptr) :
        device(device), dev_ptr(dev_ptr),
        name(GGML_CUDA_NAME + std::to_string(device)) {
    }

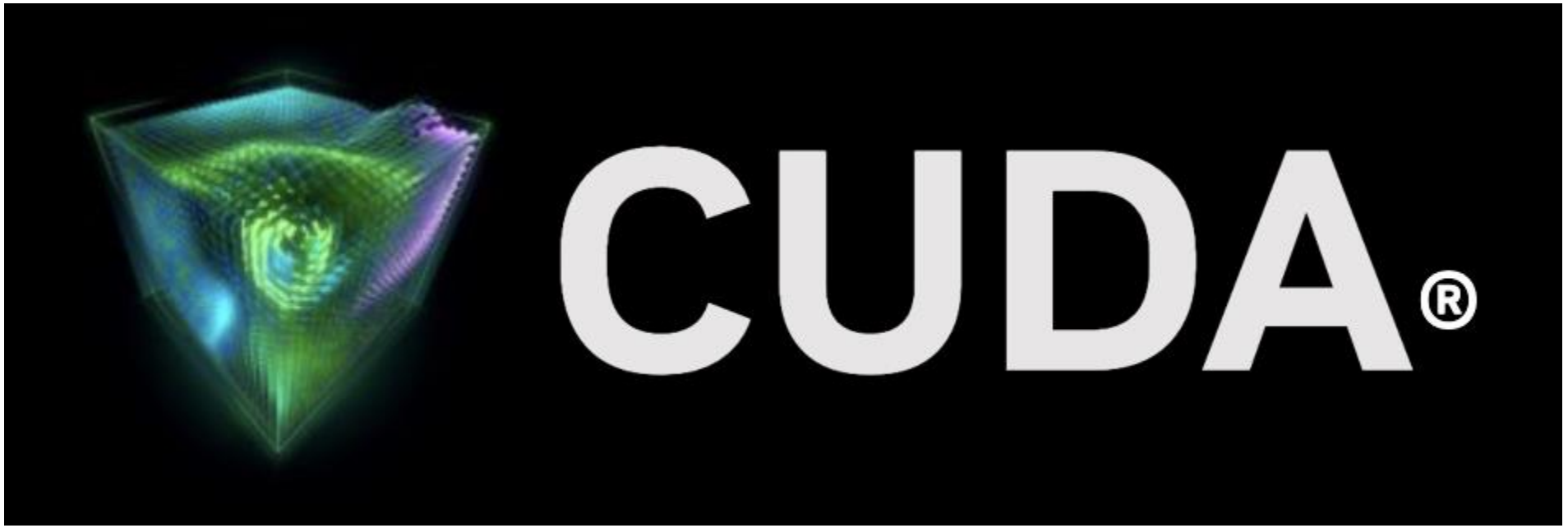
    ~ggml_backend_cuda_buffer_context() {
        CUDA_CHECK(cudaFree(dev_ptr));
    }
};
```

显存地址



4*(32+336)


g_state
...
o: 32
t: 336
image
o: 32
t: 336
label
1024*(32+336)
o: 32
t: 336
o: 32
t: 336
o: 32
t: 336
o: 32
t: 336
4*(32+336)
o: 32
t: 336
o: 32
t: 336
o: 32
t: 336
o: 32
t: 336
o: 32
t: 336
o: 32
t: 336
o: 32
t: 336
o: 32
t: 336
o: 32
t: 336
o: 32
t: 336
images
mul
add
relu
mul
add
sm
labels
ce
o: 32
t: 336
o: 32
t: 336
o: 32
t: 336
am
am
equal
o: 32
cg: 72
nodes: 2048*8=16384
leafs: 2048*8=16384
visitd_hash_set.keys: 4099*8=32792
visitd_hash_set.used: 129*4=516
(Cuda)data:



宇宙最强CUDA课程（不是）

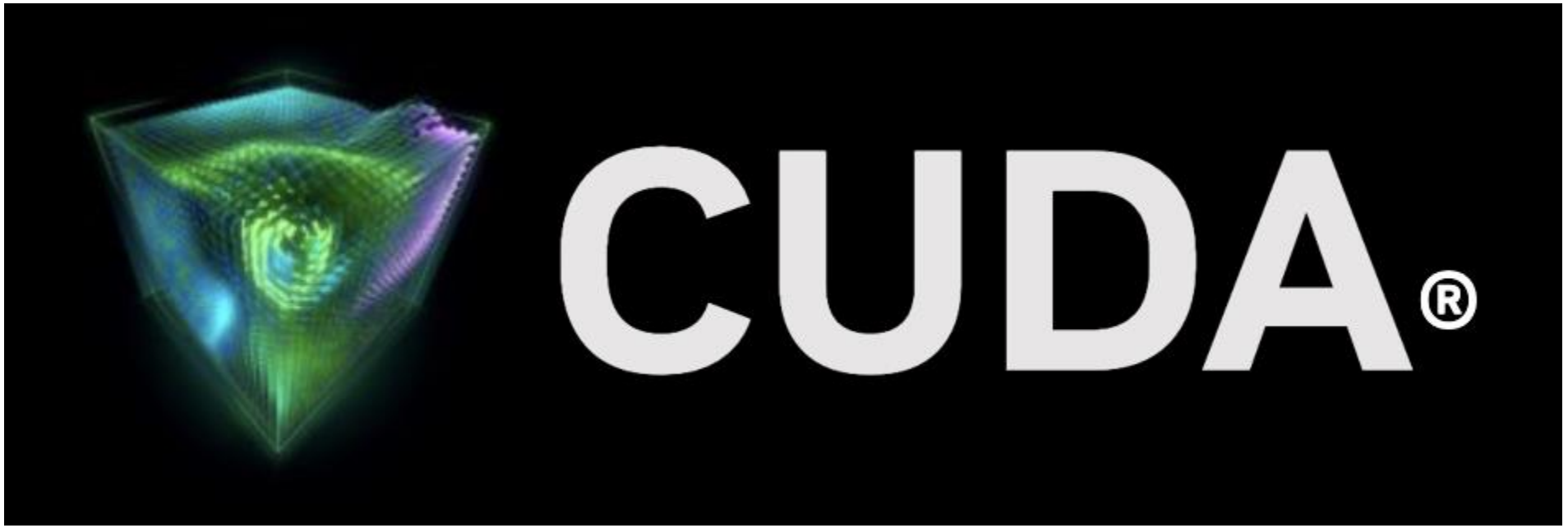
比飞鸟贵重的多_HKL

当前已有教程问题：

1.基本上千篇一律的在讲cuda语法
2.都说cuda编程与硬件关系重大，没人告诉我关系是什么
3.Cuda代码听着还行但写不出来
4.写出来有问题怎么调试
5.代码如何调优（工具/手段）
6.Cuda是一个技术栈不仅仅是代码

宇宙安全声明

- 同样作为初学者，没有能力自行整理知识点，所以基本上所有内容都是参考各种视频和文章，再加一些自己学习过程的观点和总结。
- 所有参考均会以链接的形式提供，并在视频中强调出处，坚决维护原创者的劳动成果~

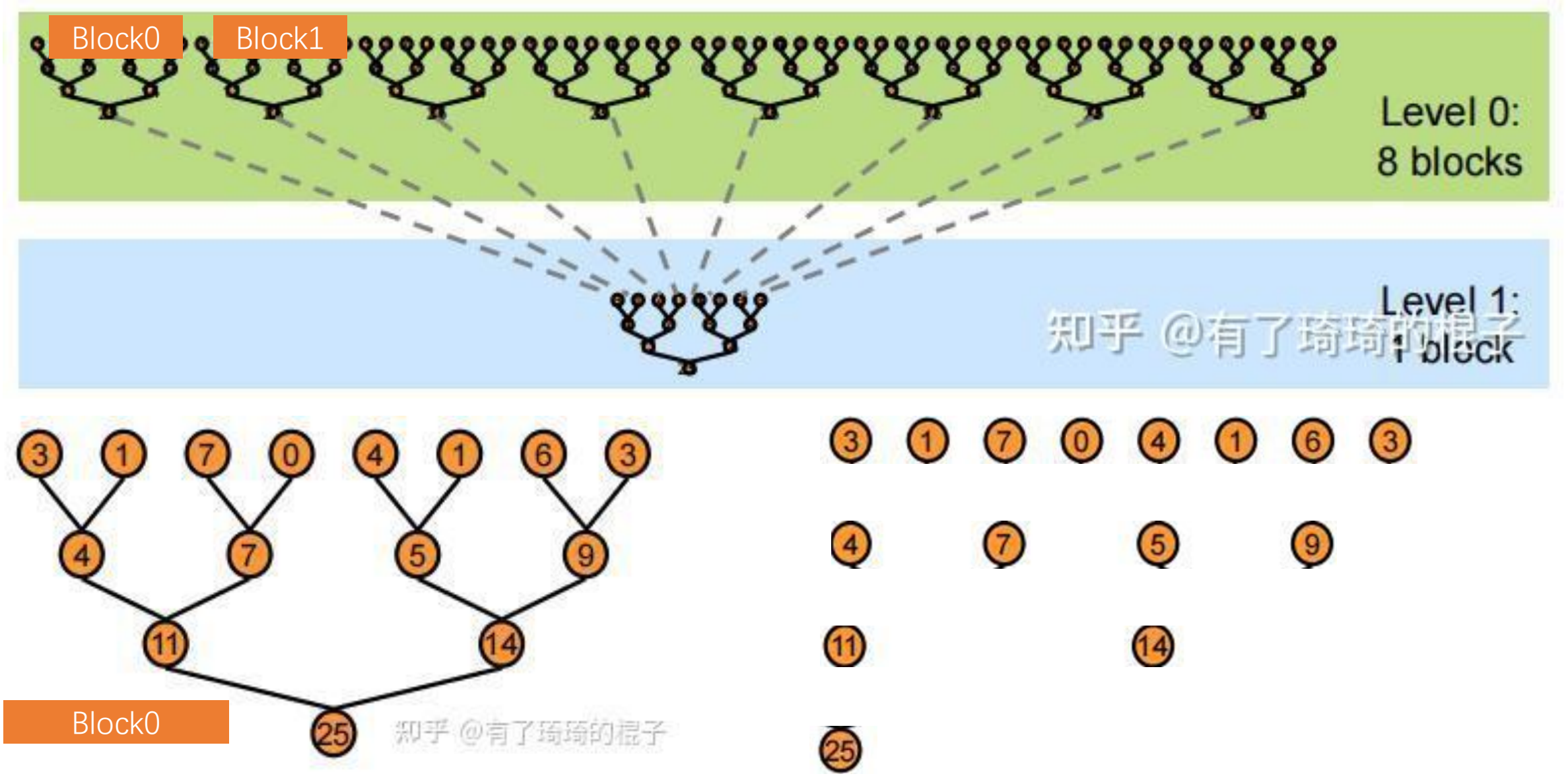


宇宙最强CUDA课程（不是）

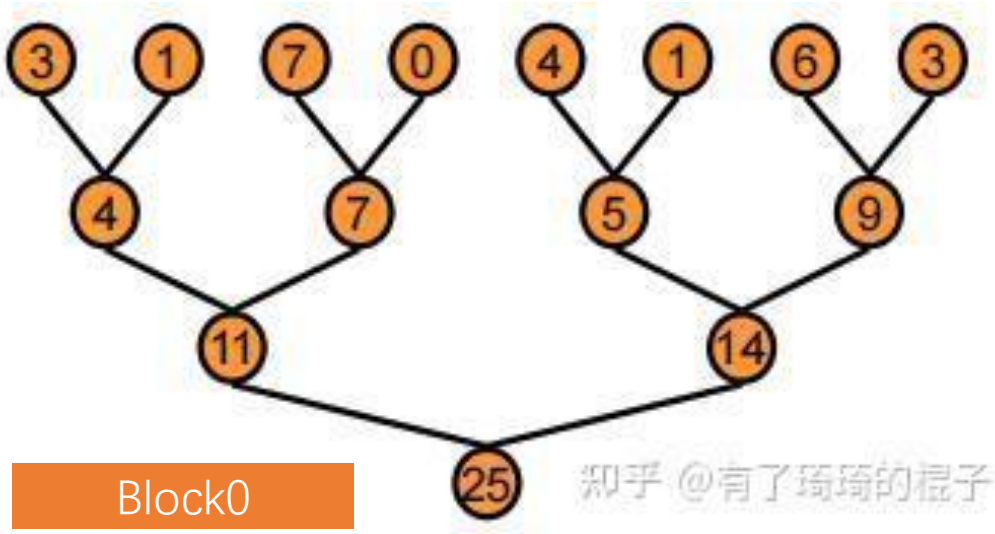
Reduce规约求和

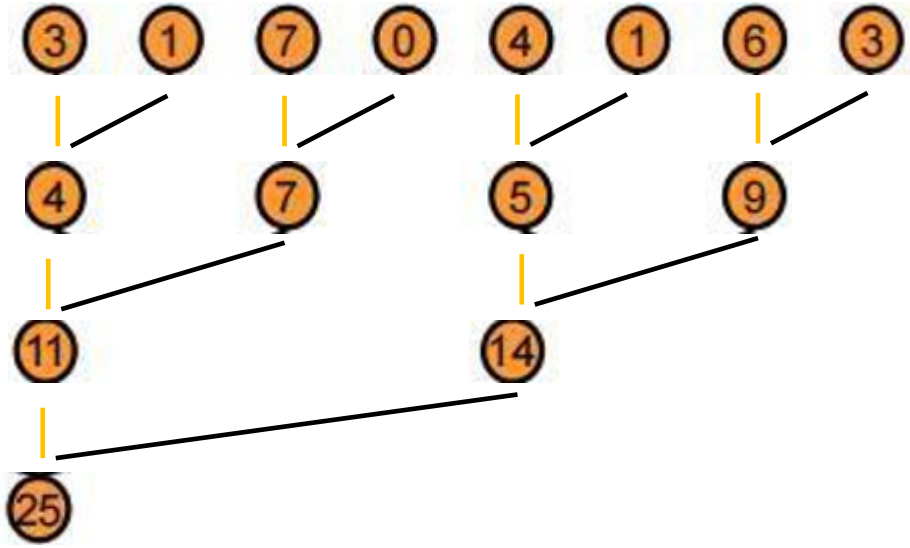
深入浅出GPU优化系列：reduce优化 - 知乎

How_to_optimize_in_GPU/reduce at master · Liu-xiandong/How_to_optimize_in_GPU · GitHub



Reduce_v0_global_memory

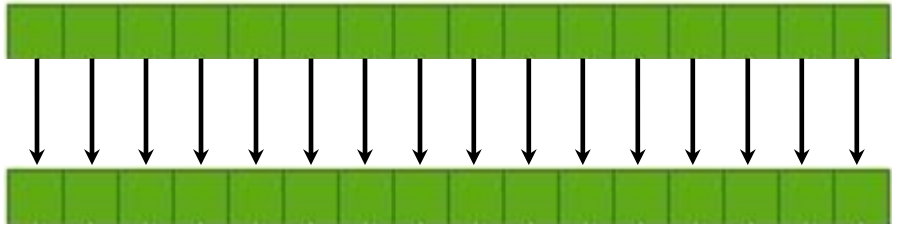






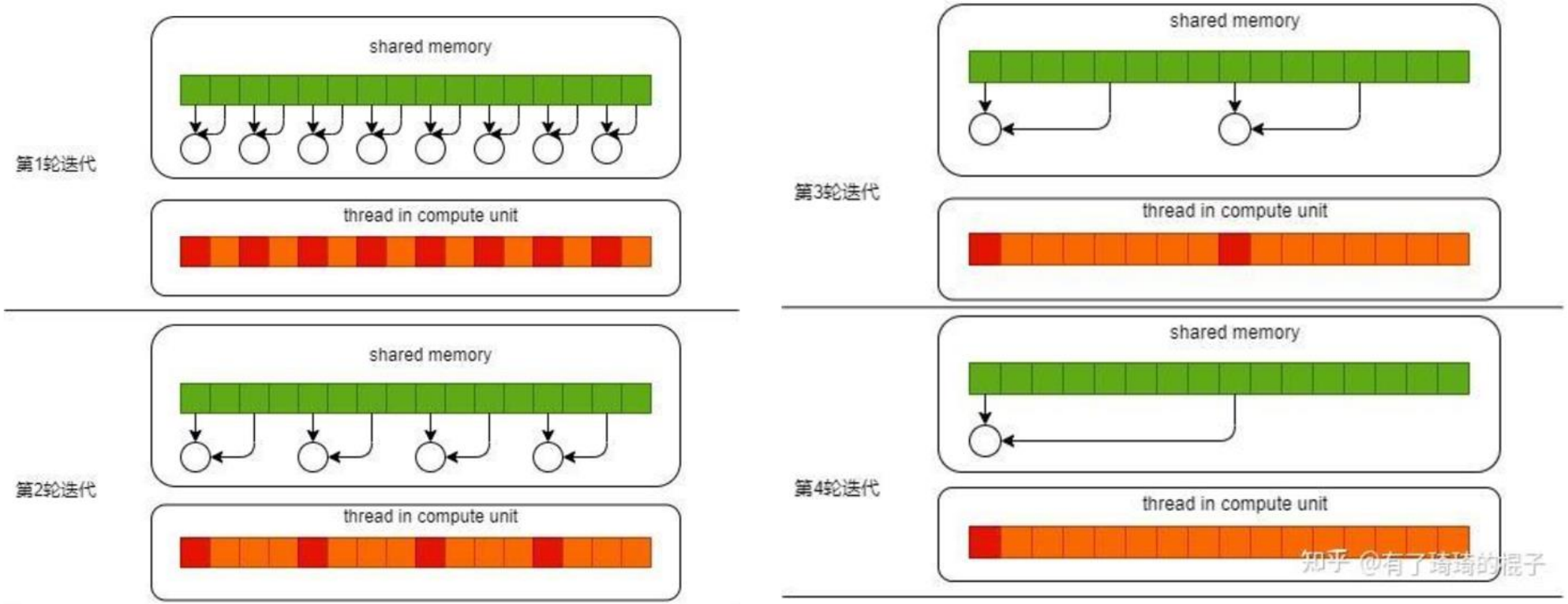
- 设计算法的时候是按照Block设计的，但是写程序的时候是按照thread写的
- 索引很容易乱套的，所以最好为每一个Block设计一个更进一步的索引
- 绘制一个清晰的Block算法图是很有必要的

Reduce_v1_shared_memory



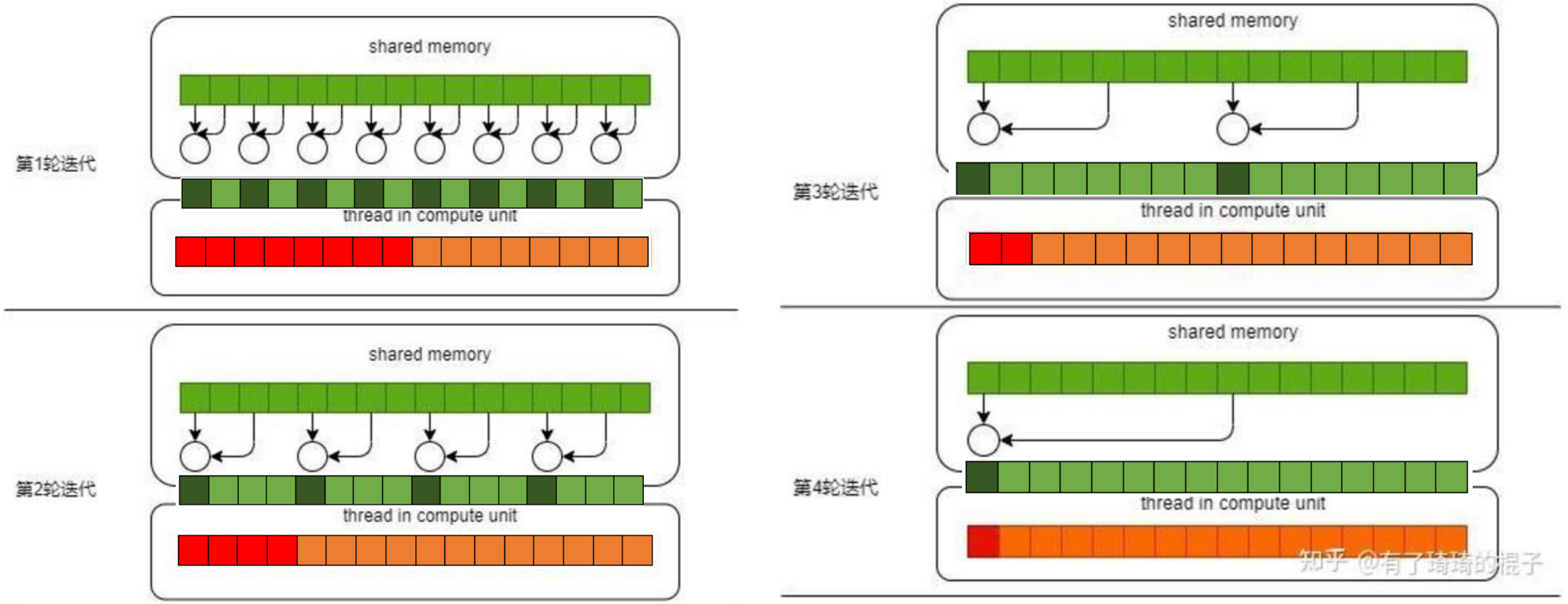
global memory

shared memory

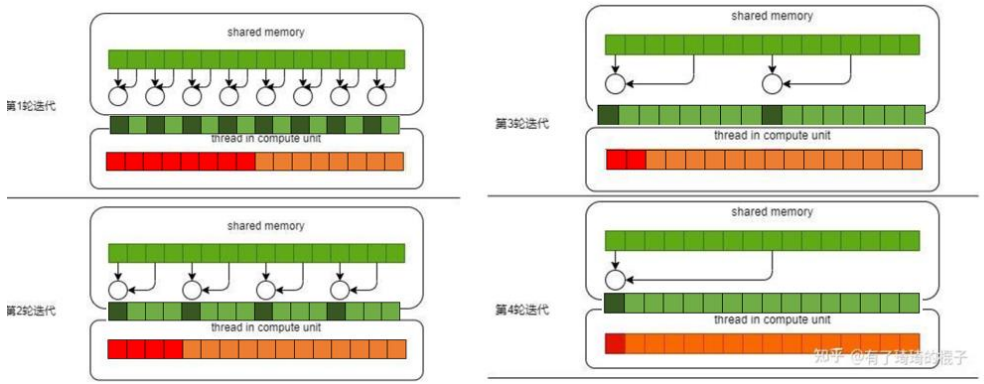


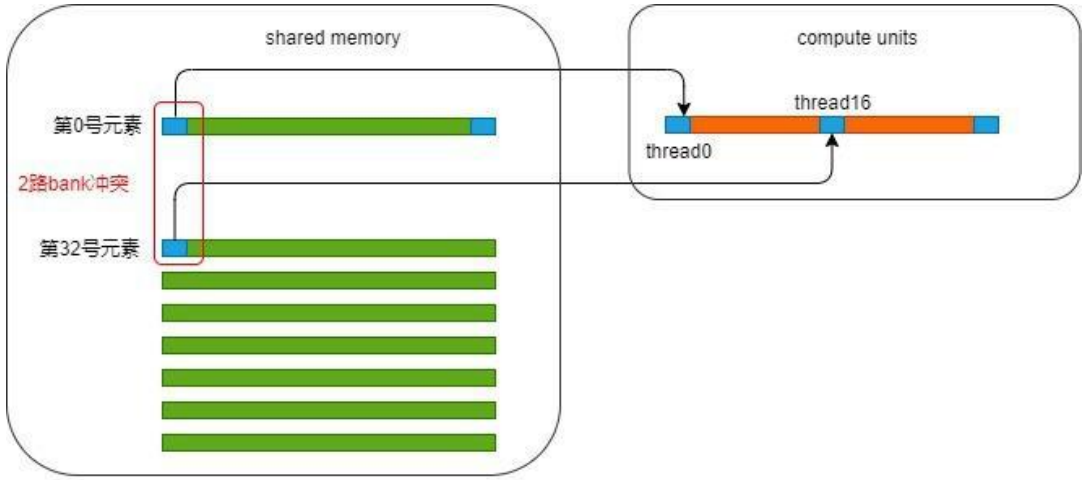
知乎 @有了琦琦的棍子

Reduce_v2_no_divergence_branch



Reduce_v3_no_bank_conflict





只看一个block中的0号warp

- 第一次迭代：

thread0访问：0号，1号

thread16访问：32号，33号

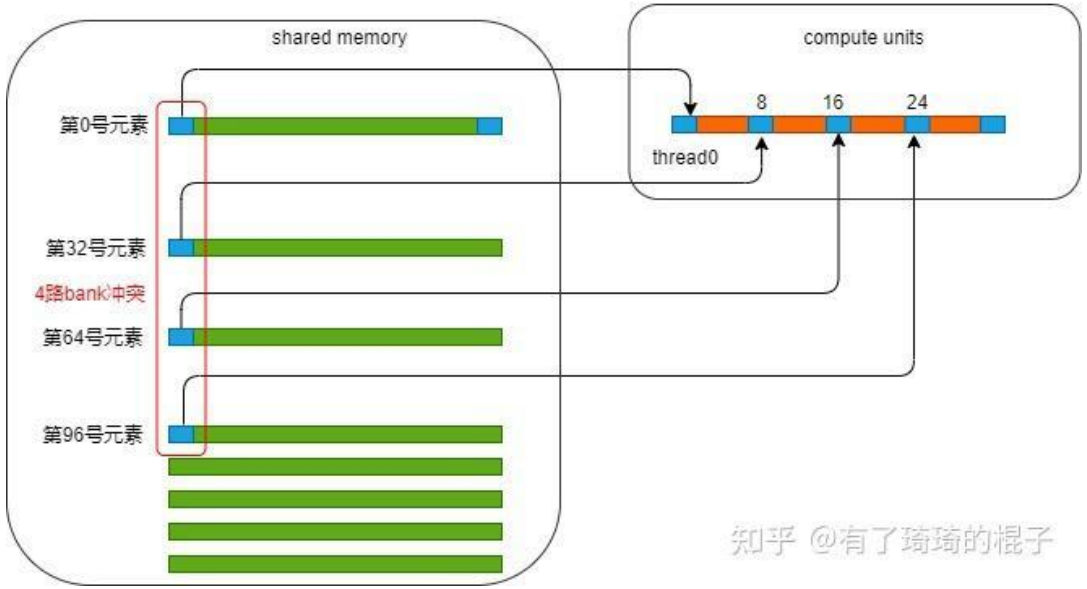
- 第二次迭代：

thread0访问：0号，2号

thread8访问：32号，34号

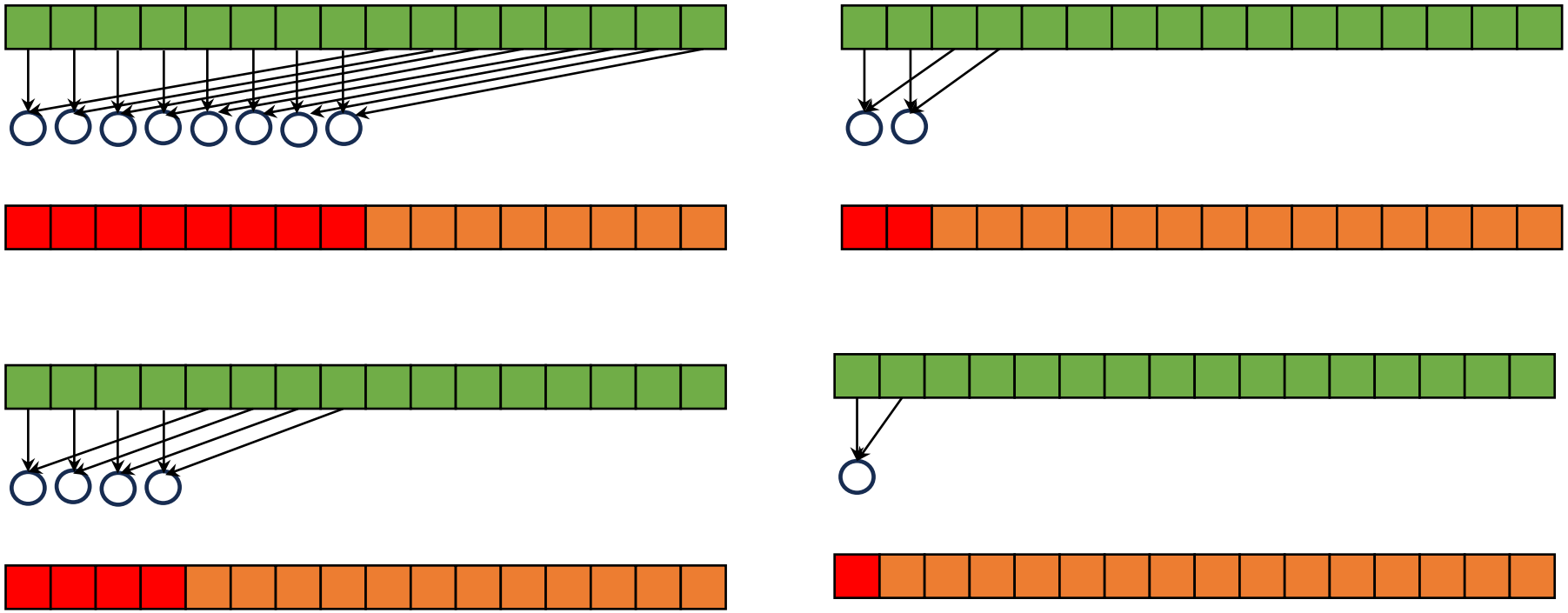
thread16访问：64号，66号

thread24访问：96号，98号



知乎 @有了琦琦的棍子

Reduce_v3_no_bank_conflict



只看一个block中的0号warp

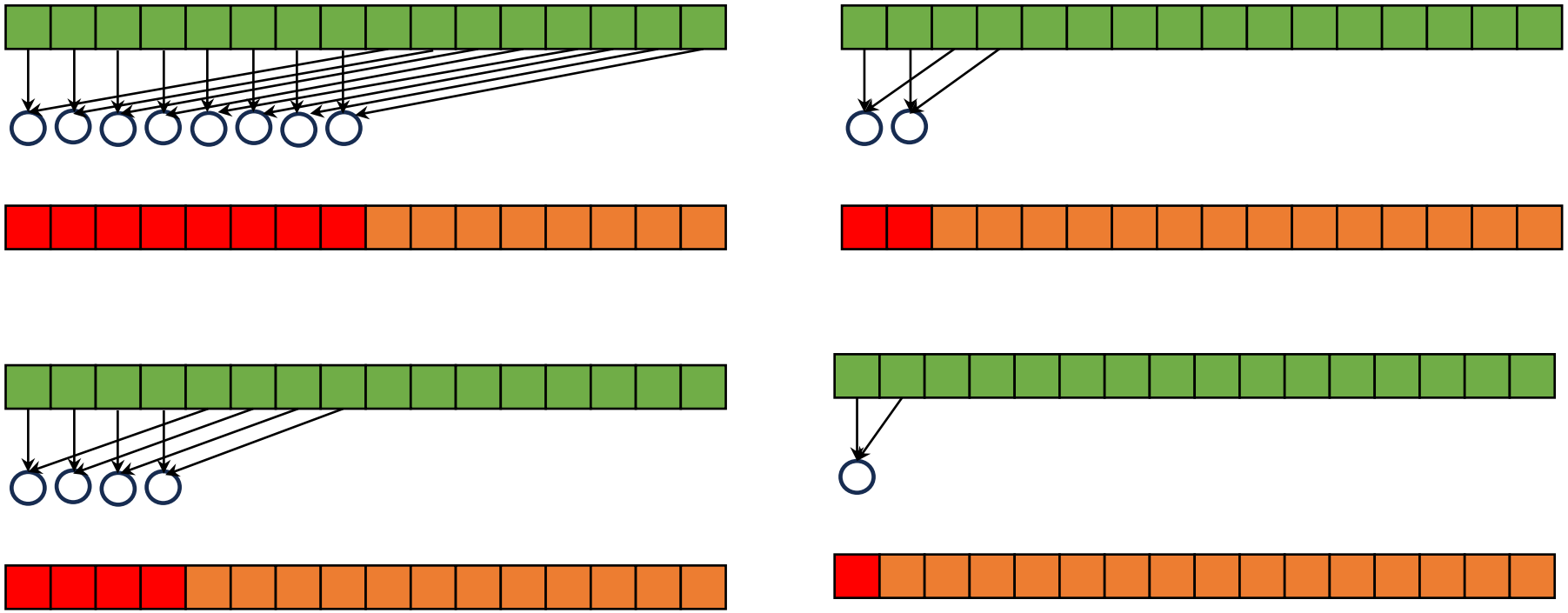
- 第一次迭代：

thread0访问：0号，128号
thread8访问：8号，136号
thread16访问：16号，144号
thread24访问：24号，152号

- 第二次迭代：

thread0访问：0号，64号
thread8访问：8号，72号
thread16访问：16号，80号
thread24访问：24号，88号

Reduce_v3_no_bank_conflict



只看一个block中的0号warp

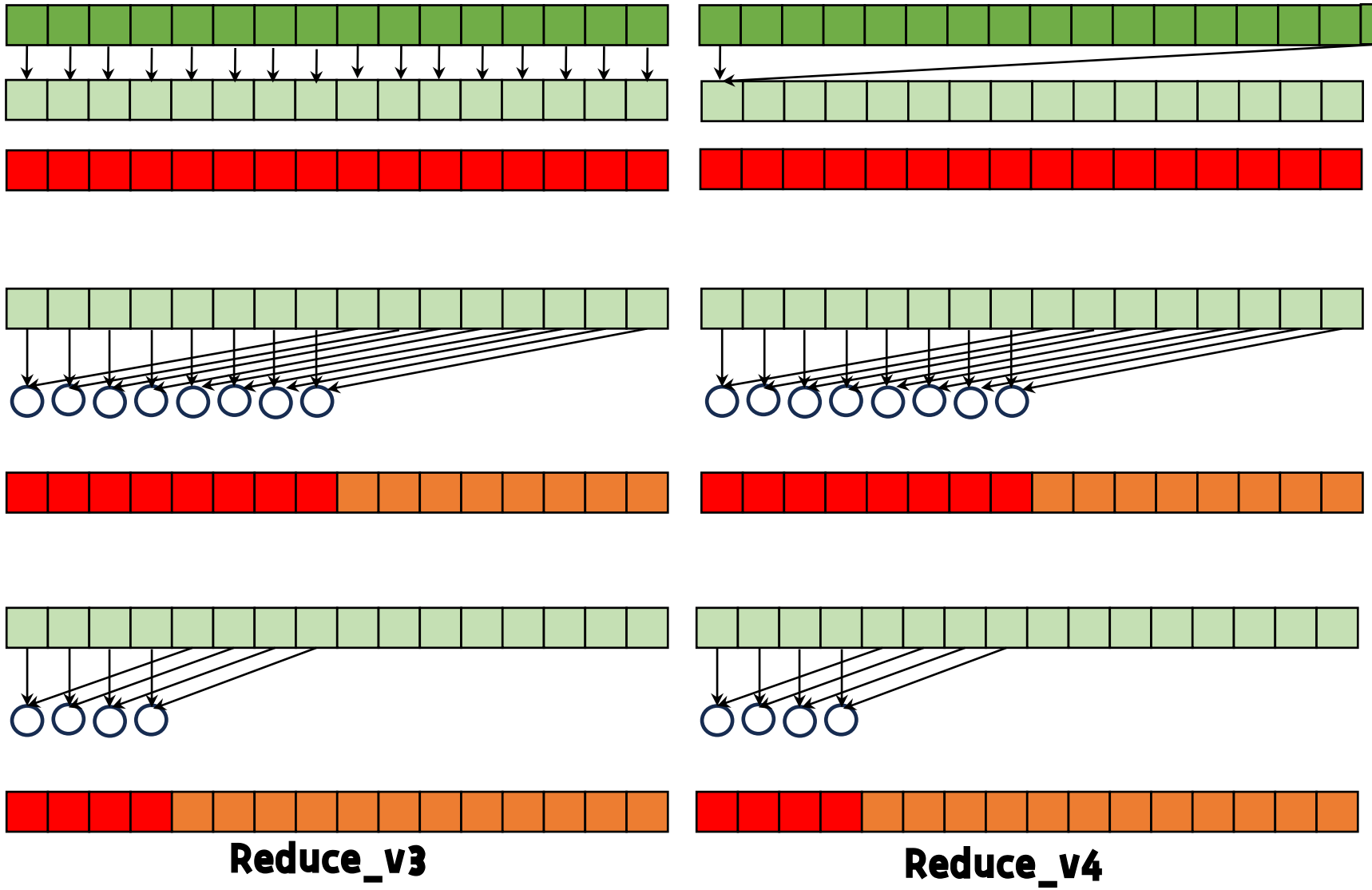
- 第一次迭代：

thread0访问：0号，128号
thread8访问：8号，136号
thread16访问：16号，144号
thread24访问：24号，152号

- 第二次迭代：

thread0访问：0号，64号
thread8访问：8号，72号
thread16访问：16号，80号
thread24访问：24号，88号

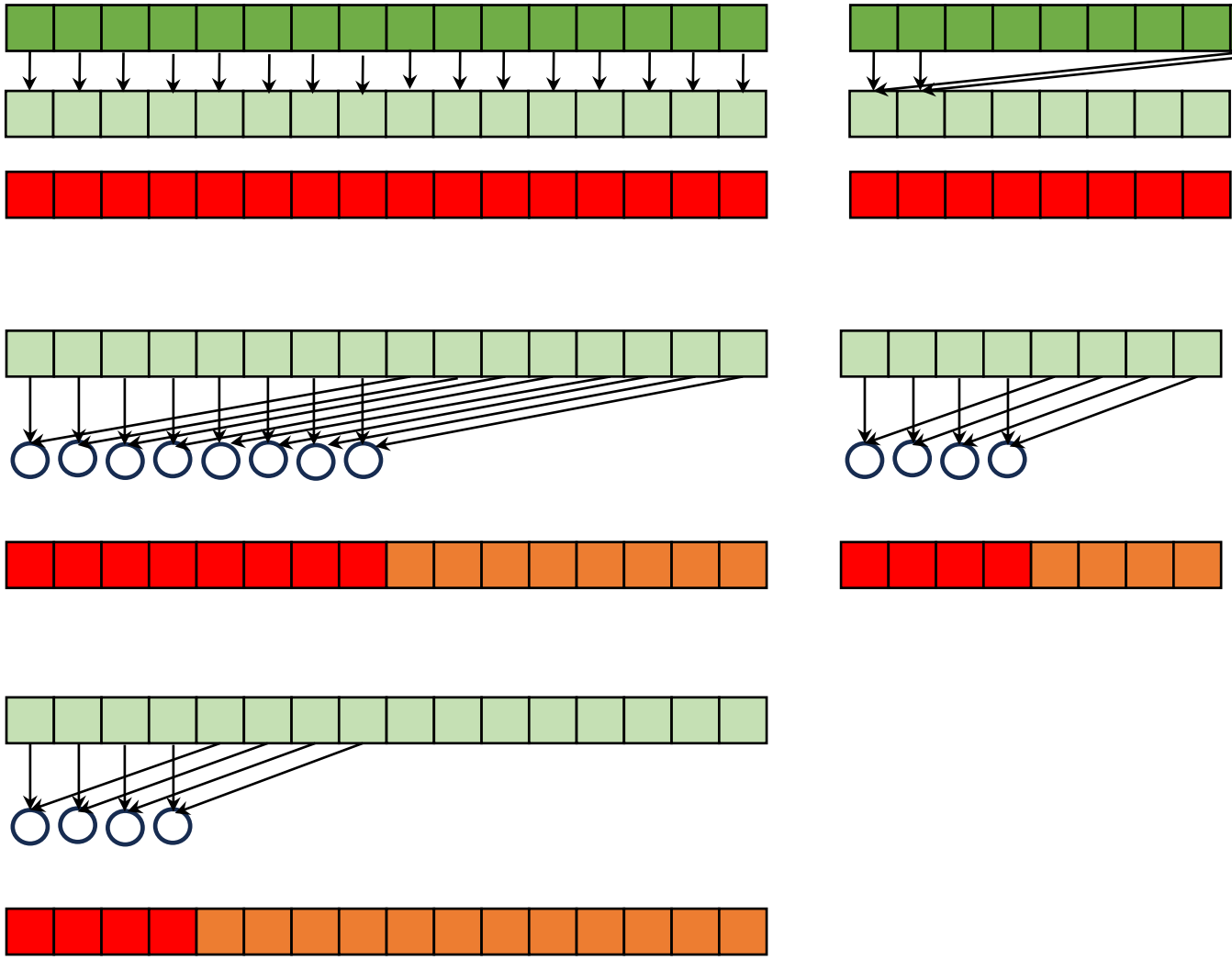
Reduce_v4_add_during_load



Plan A:

- 减少block的数量
- 保持block中thread的数量
- 让每个thread处理更多的数据

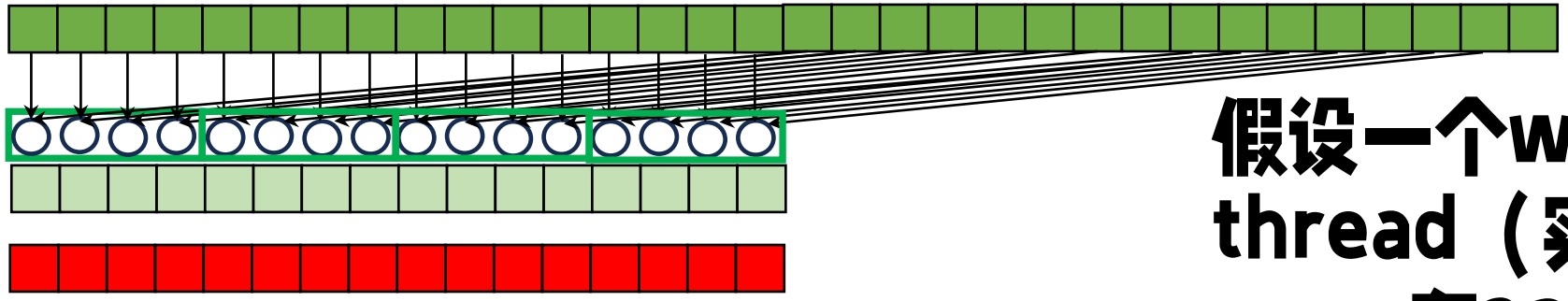
Reduce_v4_add_during_load



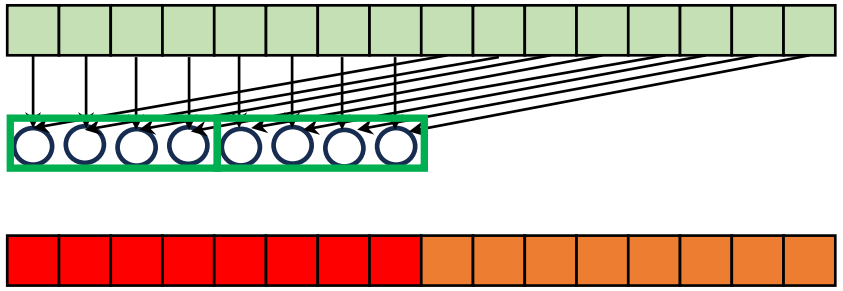
Plan B:

- 保持block的数量
- 减少block中thread的数量
- 让每个thread处理更多的数据

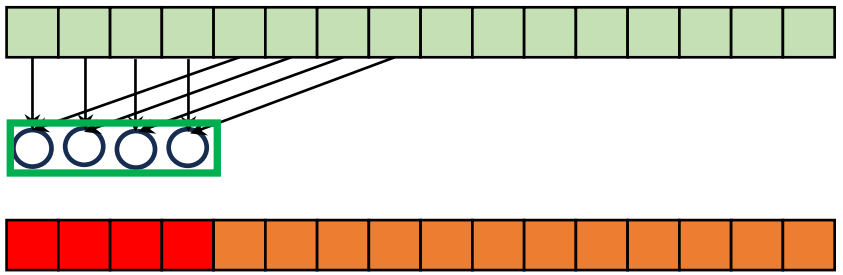
Reduce_v5_unroll_last_warp



__syncthreads();

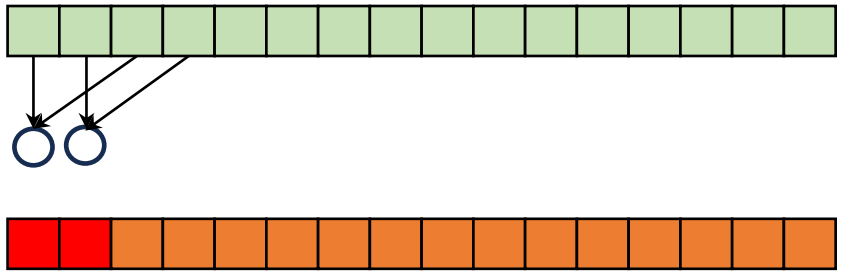


__syncthreads();

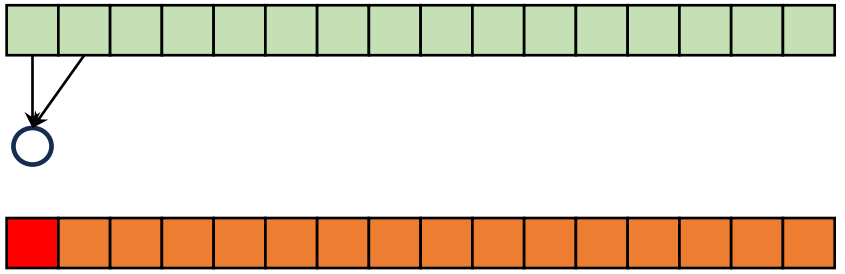


~~__syncthreads();~~

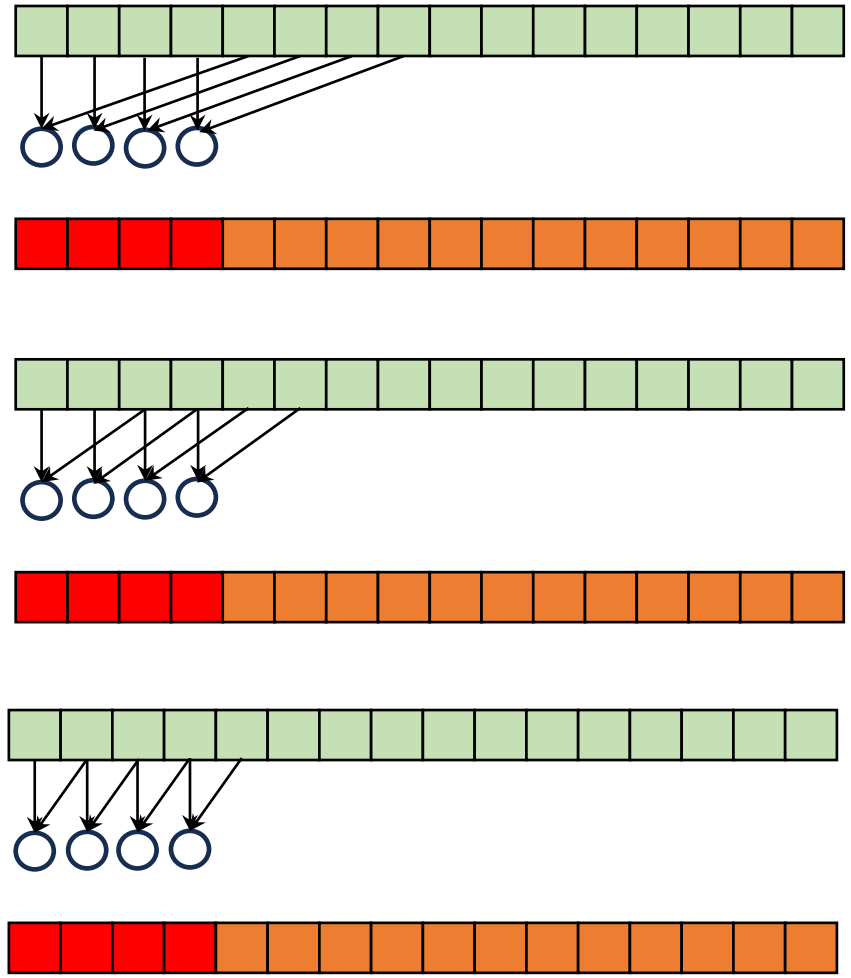
假设一个warp有4个thread（实际一个warp有32个thread）



~~__syncthreads();~~



Reduce_v5_unroll_last_warp

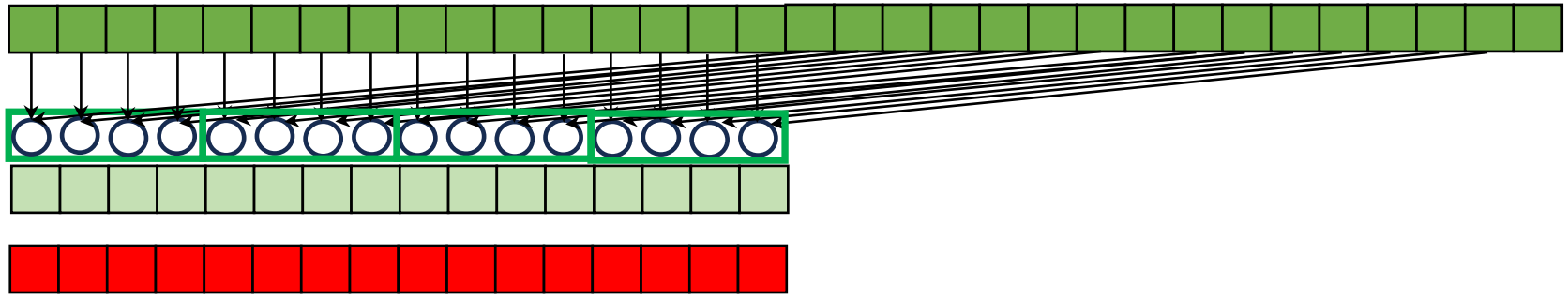


```
cache[tid]+=cache[tid+4];
cache[tid]+=cache[tid+2];
cache[tid]+=cache[tid+1];
```

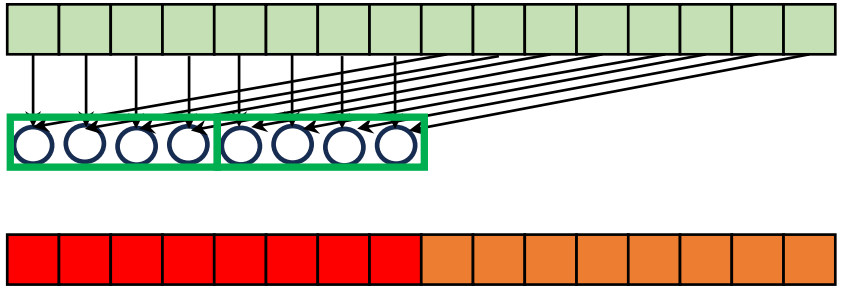
猜测：

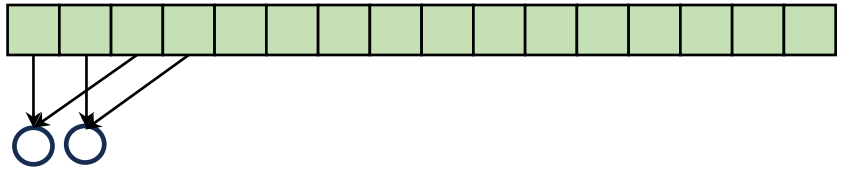
将tid放到reg_1中
将tid+4放到reg_2中
将tid+2放到reg_3中
将tid+1放到reg_4中
从cache出读取reg_1位置的数据放到reg_5中
从cache出读取reg_2位置的数据放到reg_6中
从cache出读取reg_3位置的数据放到reg_7中
从cache出读取reg_4位置的数据放到reg_8中
reg_5 = reg_5 + reg_6
reg_5 = reg_5 + reg_7
reg_5 = reg_5 + reg_8

Reduce_v6_completely_unroll



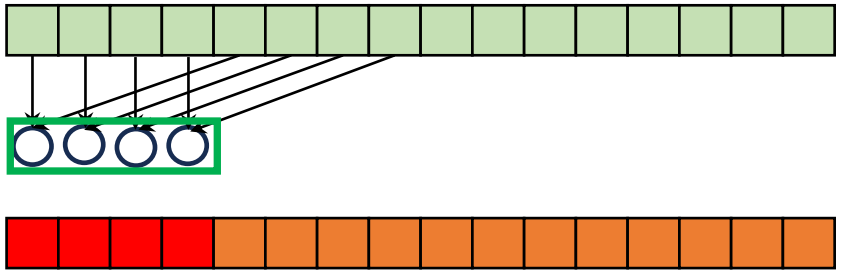
```
__syncthreads();
```

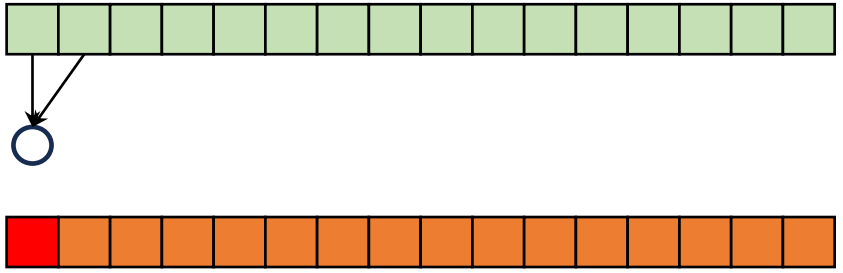




```
__syncthreads();
```

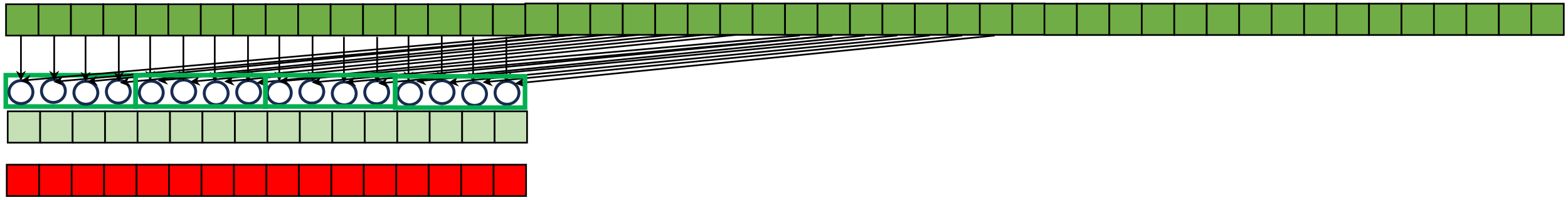
~~`__syncthreads();`~~



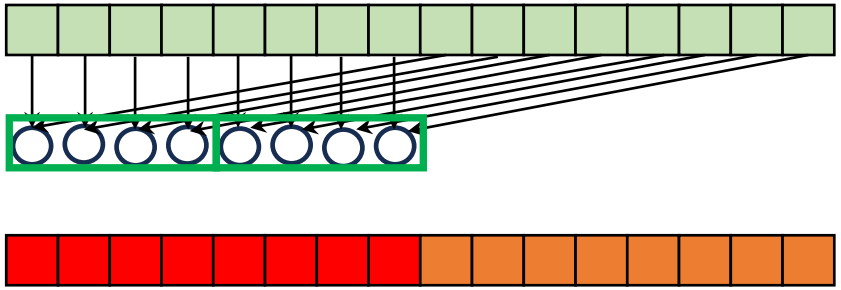


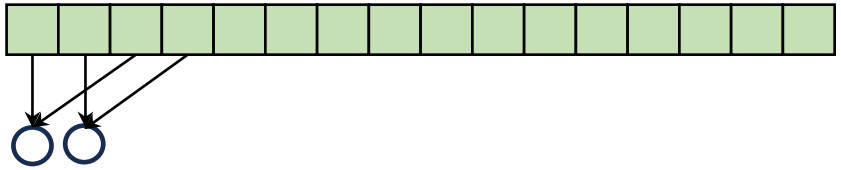
~~`__syncthreads();`~~

Reduce_v7_multi_add



```
__syncthreads();
```

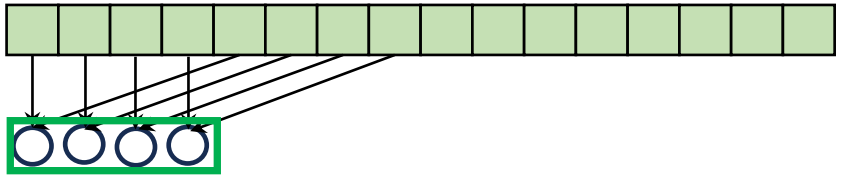


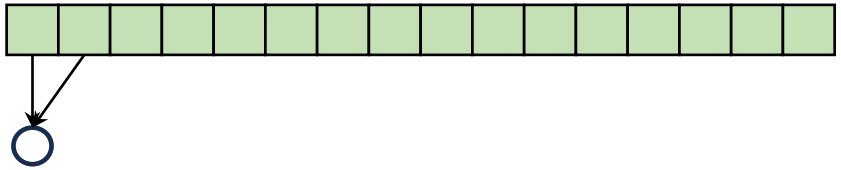



```
__syncthreads();
```



~~__syncthreads();~~

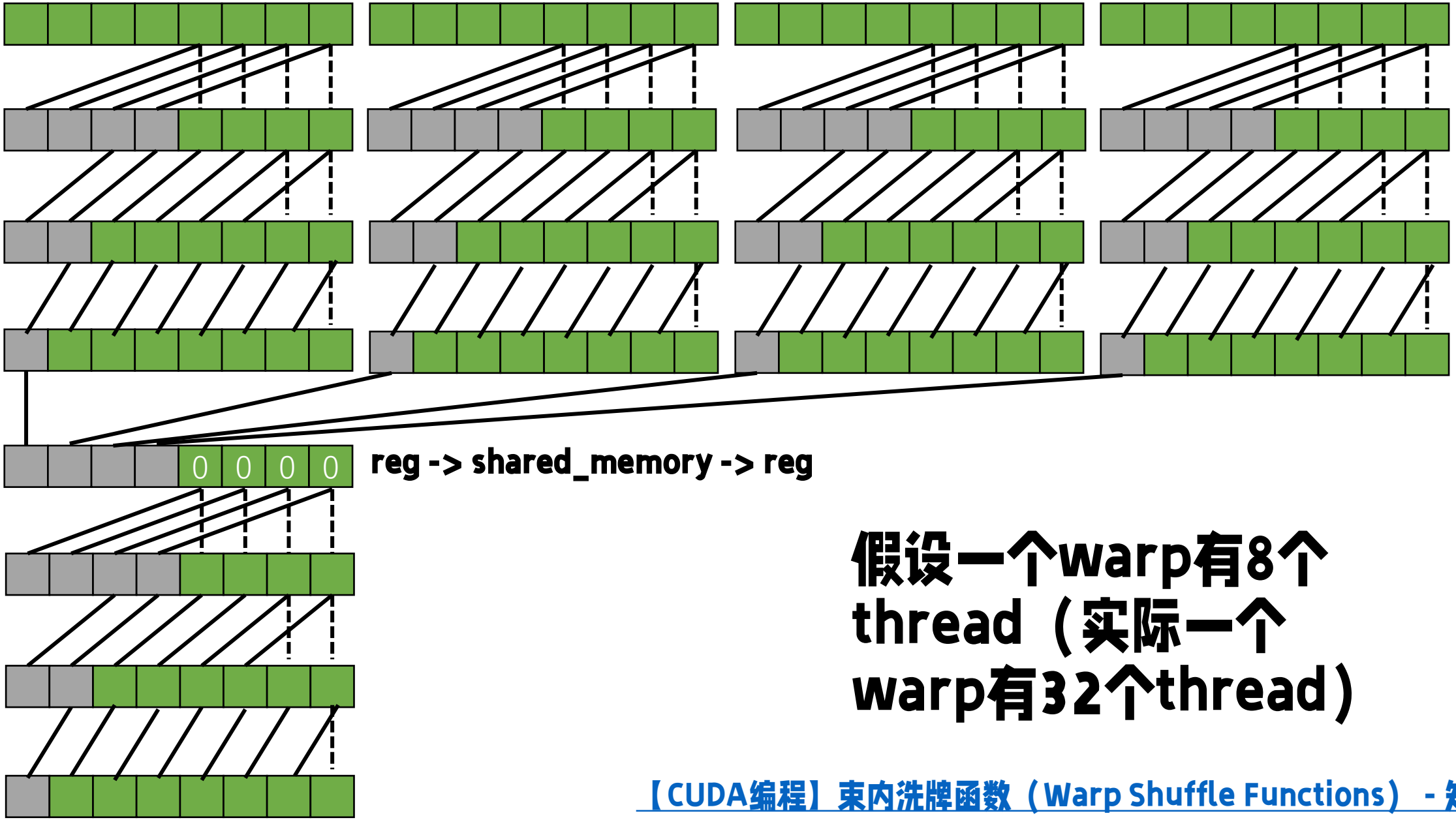






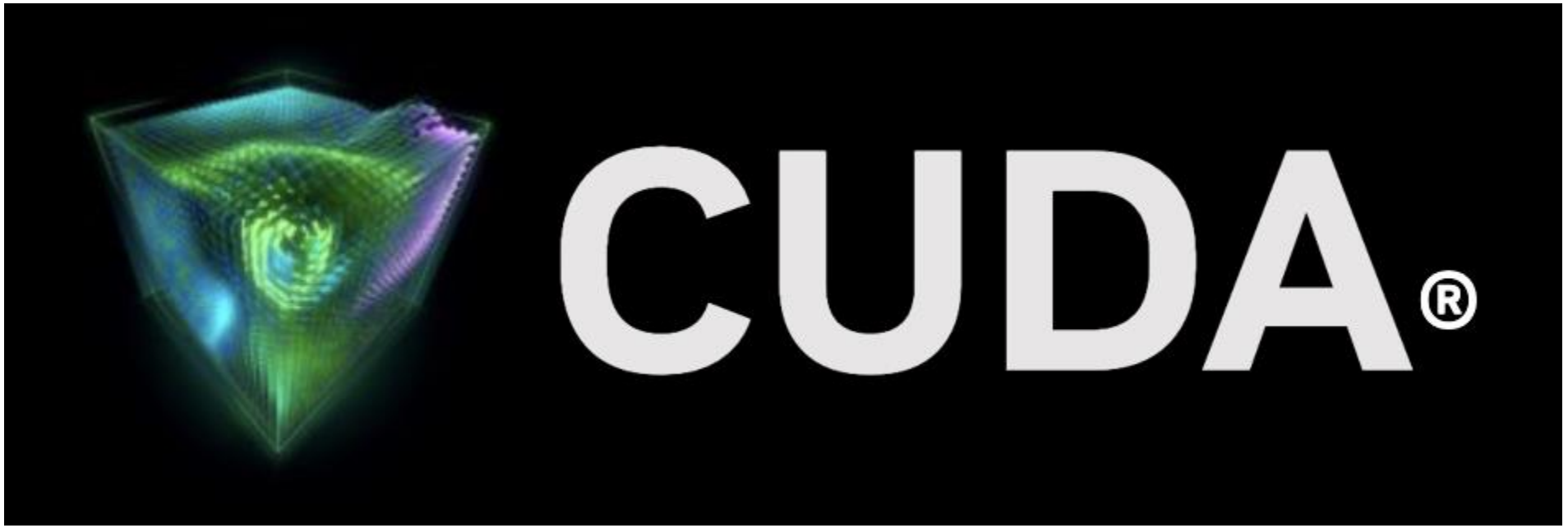
~~__syncthreads();~~

Reduce_v8_shuffle



假设一个warp有8个thread（实际一个warp有32个thread）

【CUDA编程】束内洗牌函数（Warp Shuffle Functions） - 知乎



宇宙最強CUDA課程（不是）

Sgemm単精度通用矩阵乘法

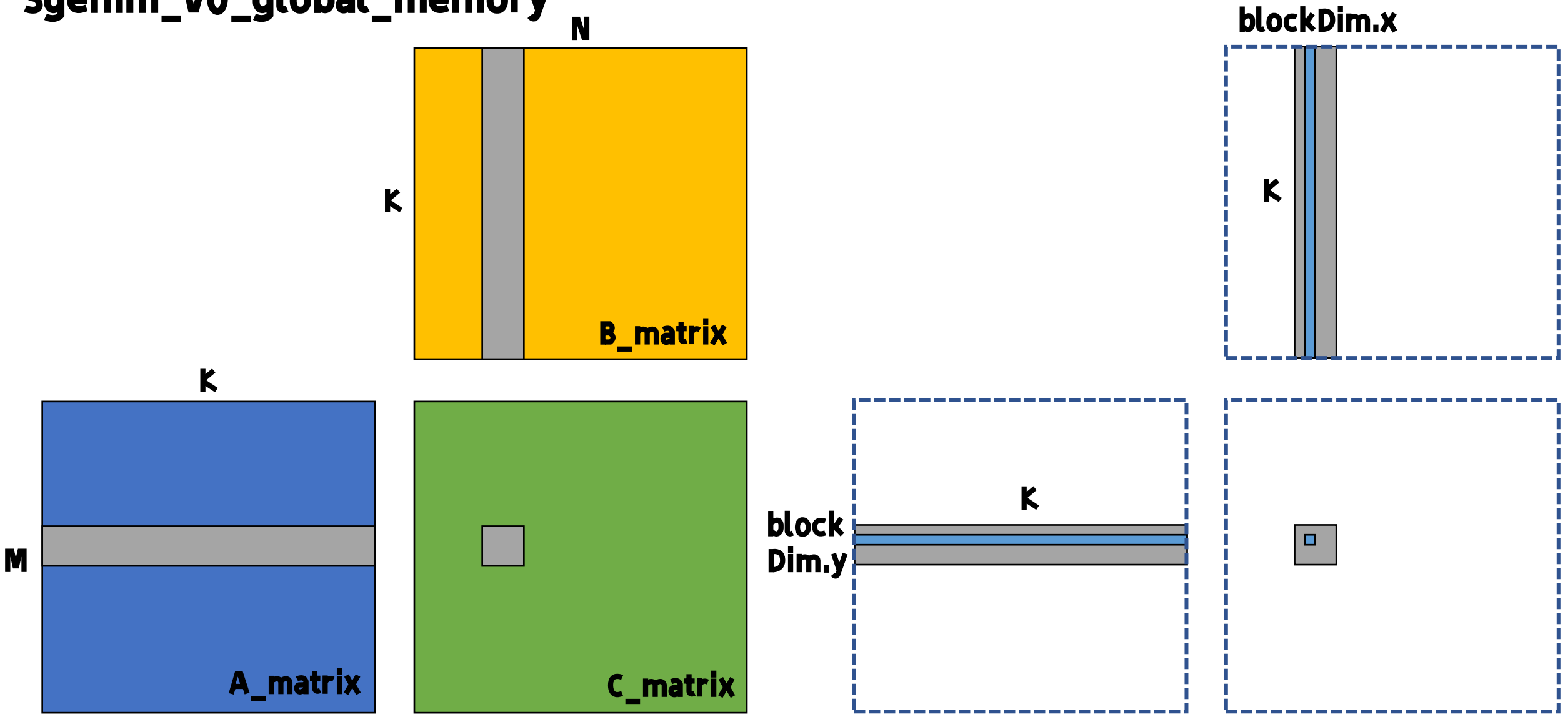
[施工中] CUDA GEMM 理论性能分析与 kernel 优化 - 知乎

cuda 入门的正确姿势：how-to-optimize-gemm - 知乎

CUDA 矩阵乘法终极优化指南 - 知乎

深入浅出GPU优化系列：GEMM优化（一） - 知乎

Sgemm_v0_global_memory

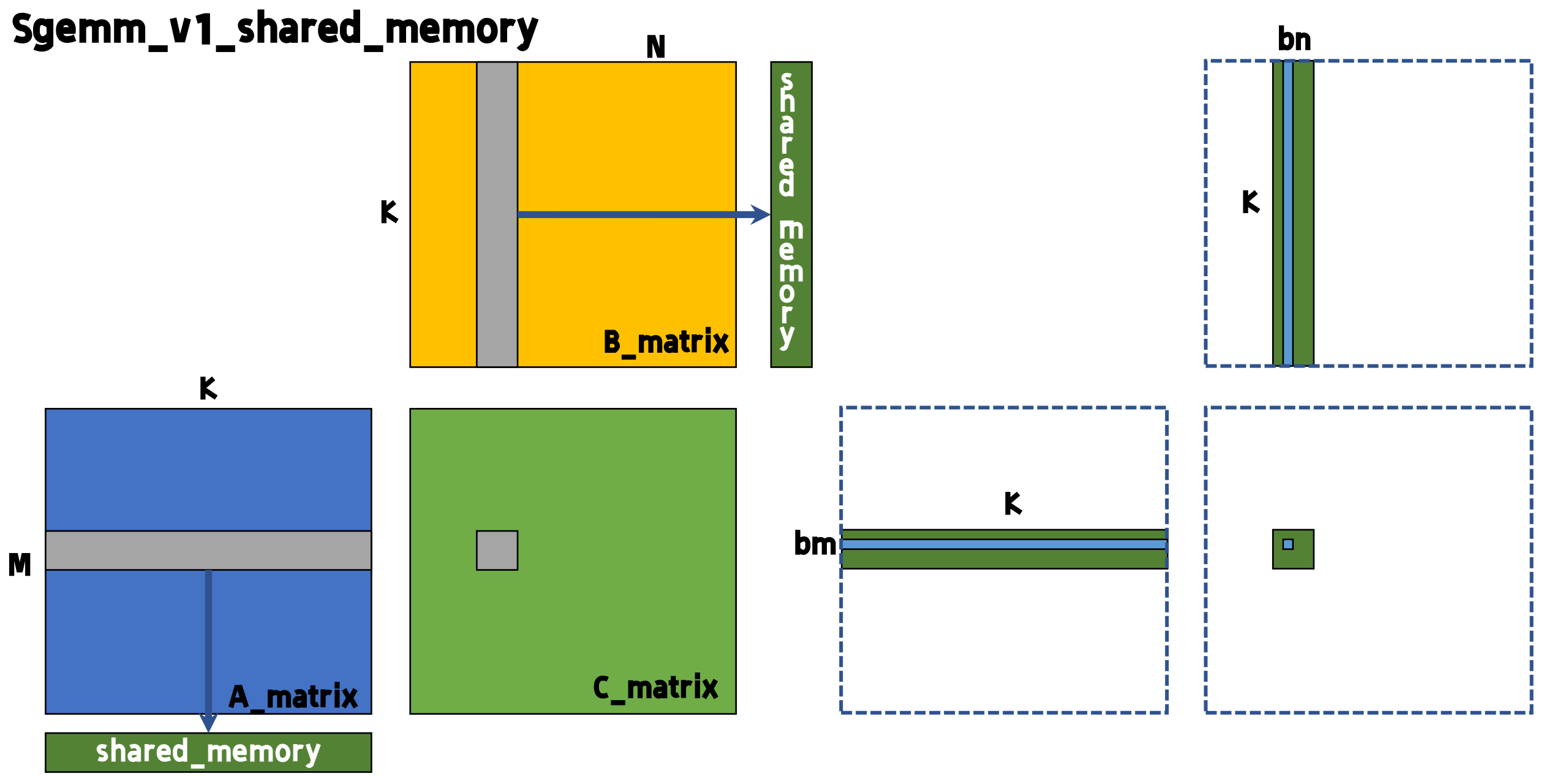


global_memory

- read:2MNK
- write:MN

深入浅出GPU优化系列：GEMM优化（一） - 知乎

Sgemm_v1_shared_memory



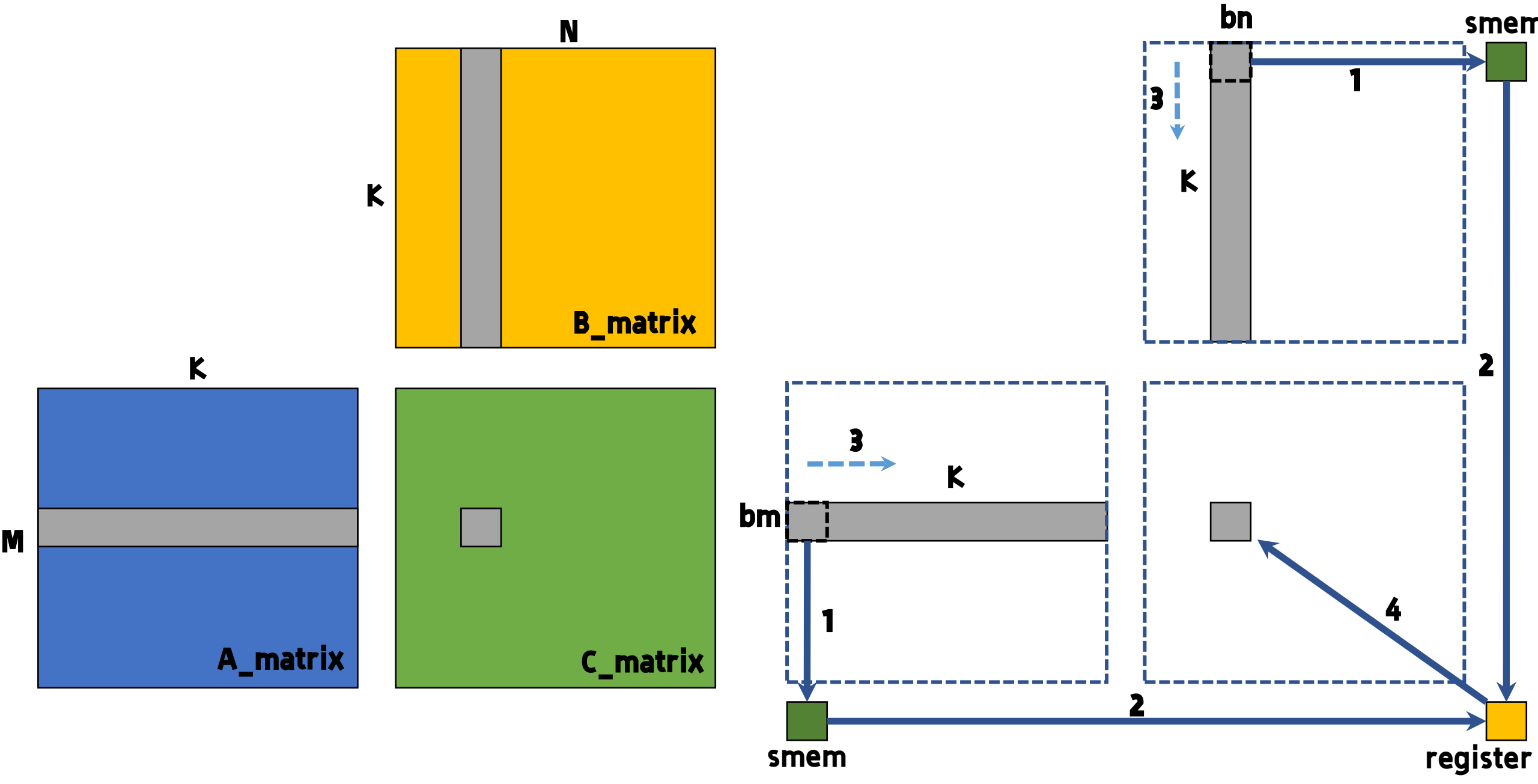
global_memory

read:bm*K*(N/bn)*(M/bm)+bn*K*(N/bn)*(M/bm)=KNM/bn+KNM/bm=KNM(1/bn+1/bm)

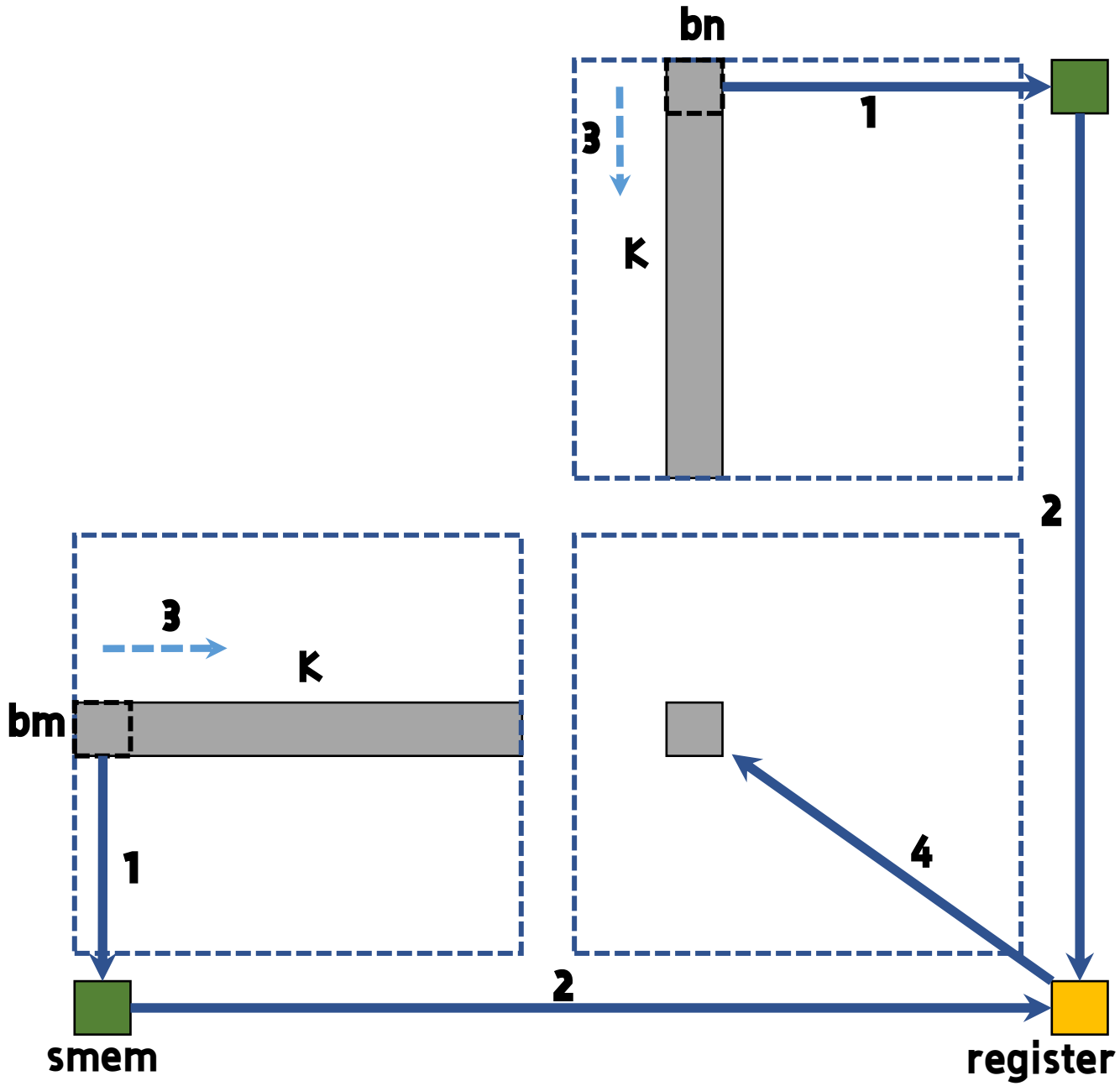
write:MN

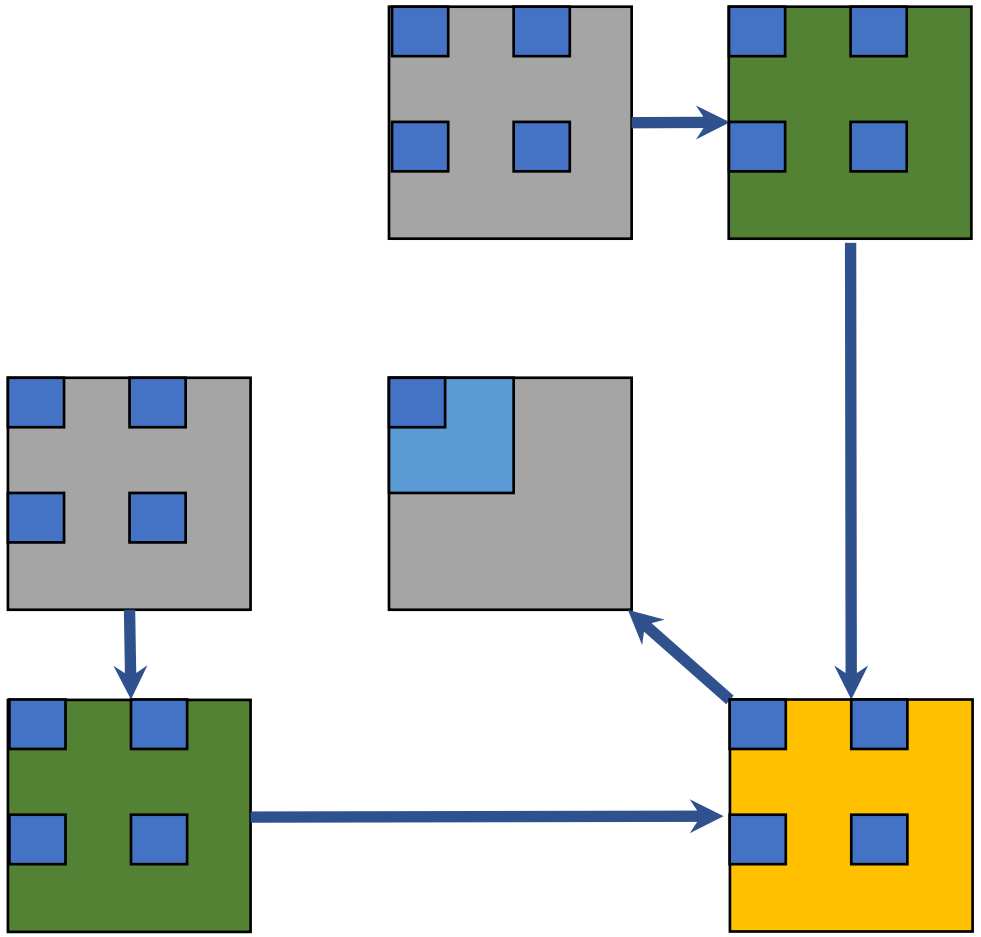
CUDA实现矩阵乘法的8种优化策略编程介绍_哔哩哔哩_bilibili

Sgemm_v2_shared_memory_sliding_window

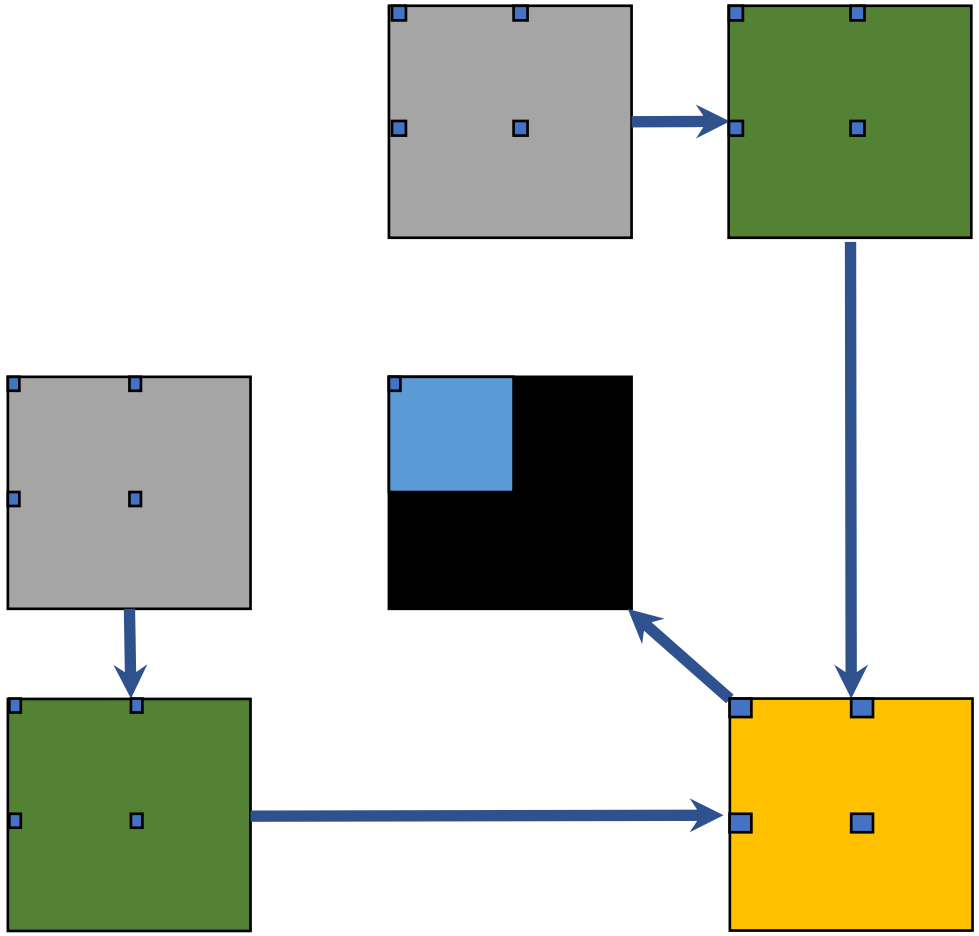


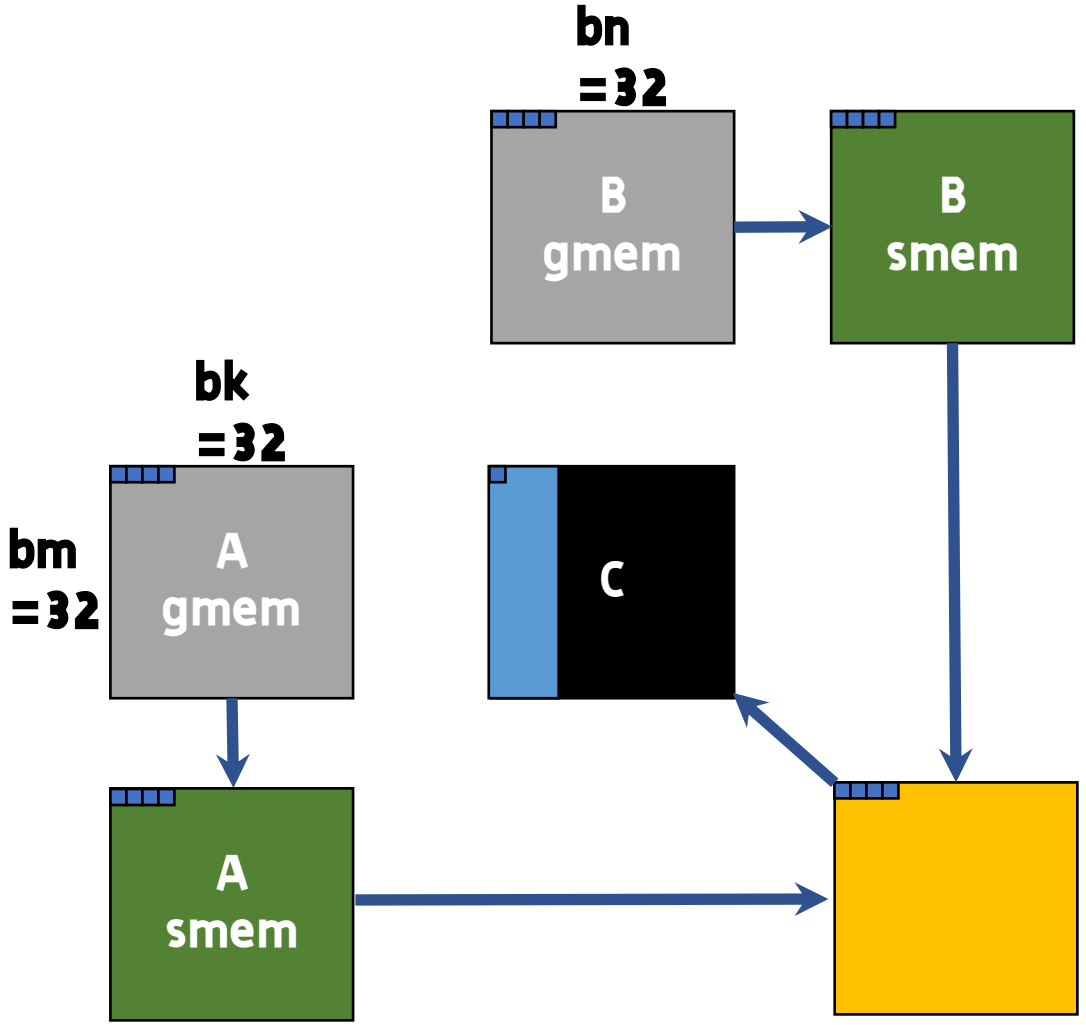
Sgemm_v3_increase_work_of_per_thread





Sgemm_v4_using_float4

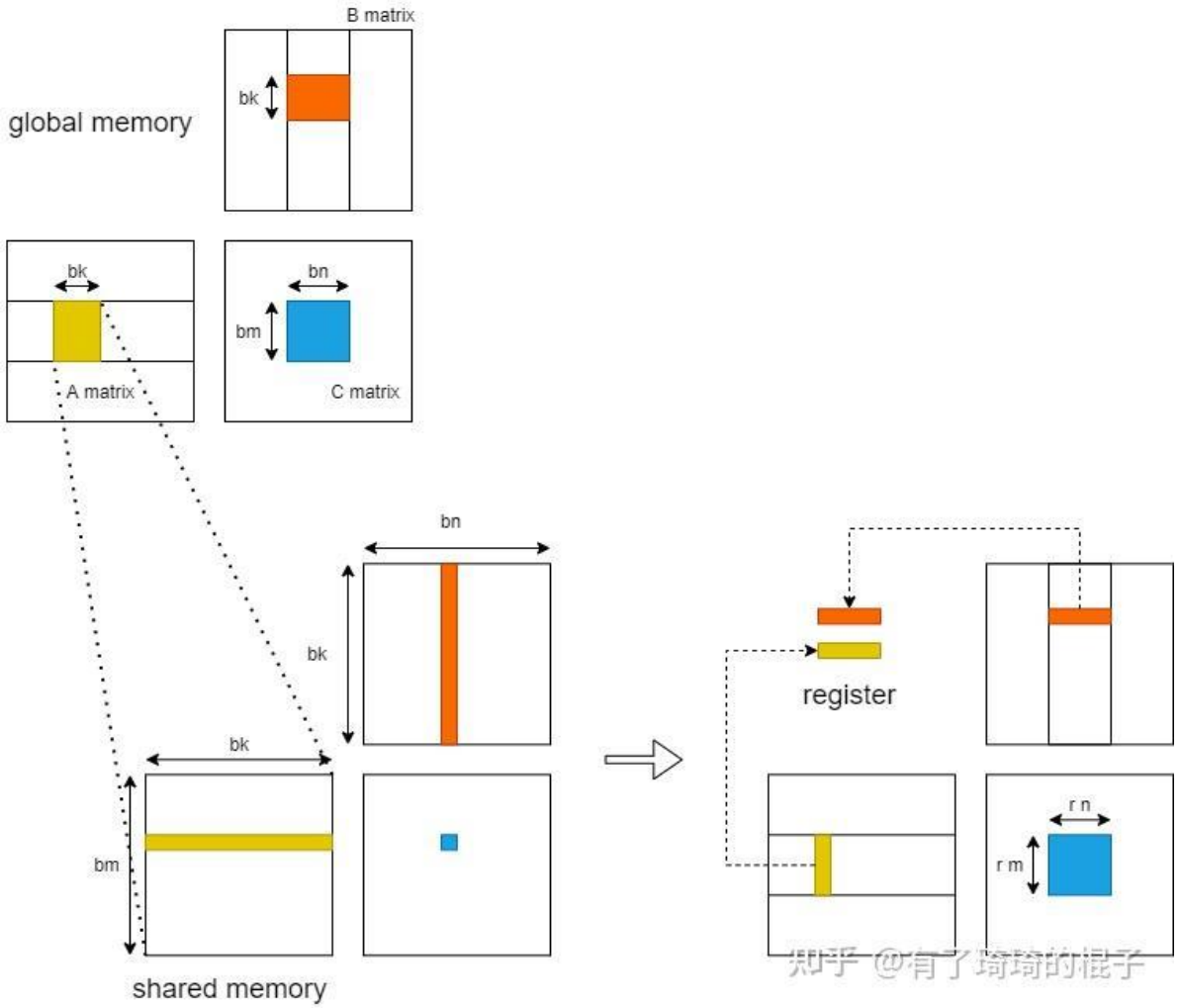





```
#define FETCH_FLOAT4(pointer)
(reinterpret_cast<float4 *>(&(pointer))[0])
```

block(16,16) → block(8,32
)

Sgemm_v5_register_outer_product



假设有矩阵A和B如下：

$$A = \begin{pmatrix} a_{11} & a_{12} \end{pmatrix}, B = \begin{pmatrix} b_{11} \\ b_{21} \end{pmatrix}$$

那么：

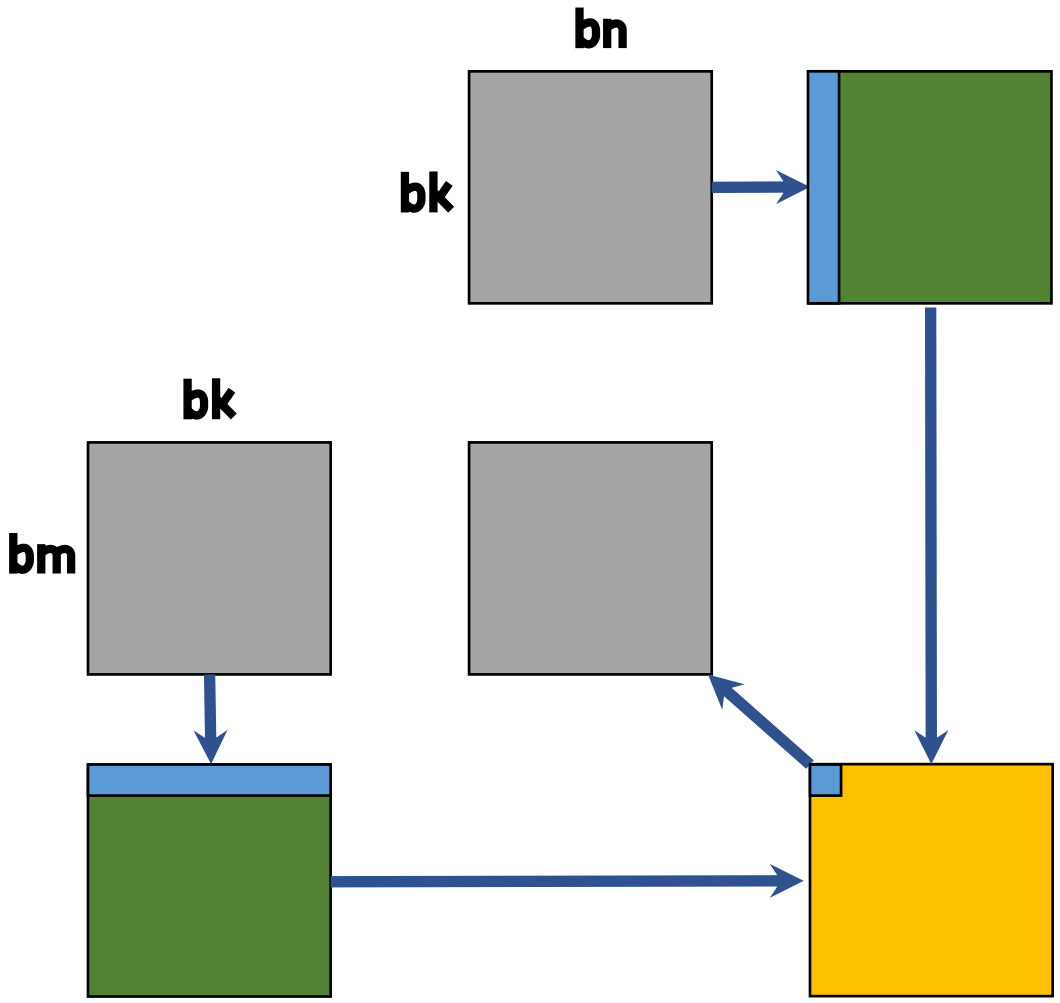
内积 $A:B$ 将是 $a_{11}b_{11} + a_{12}b_{21}$。

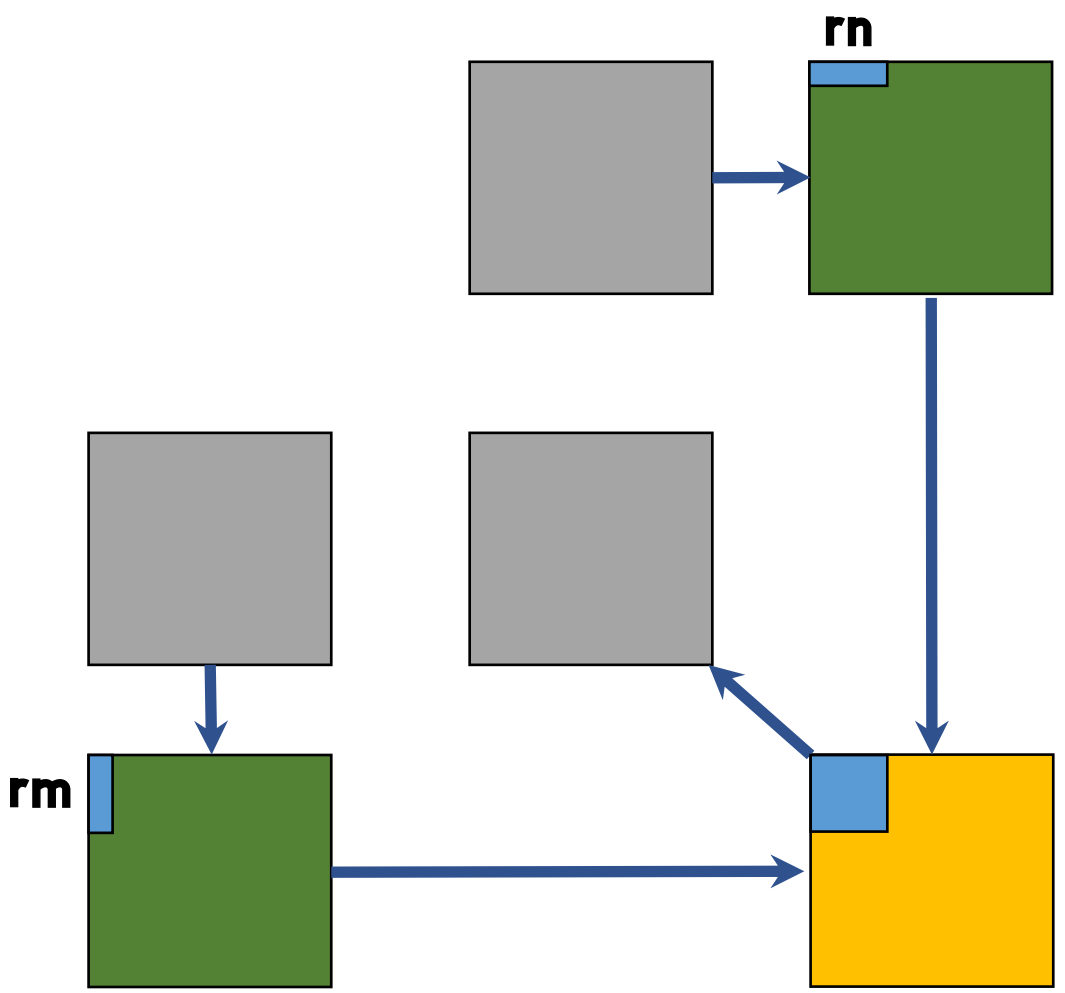
外积 $A \otimes B$ 将是一个2×2的矩阵：

$$A \otimes B = \begin{pmatrix} a_{11}b_{11} & a_{11}b_{21} \\ a_{12}b_{11} & a_{12}b_{21} \end{pmatrix}$$

[深入浅出GPU优化系列：GEMM优化（一） - 知乎]

Sgemm_v5_register_outer_product

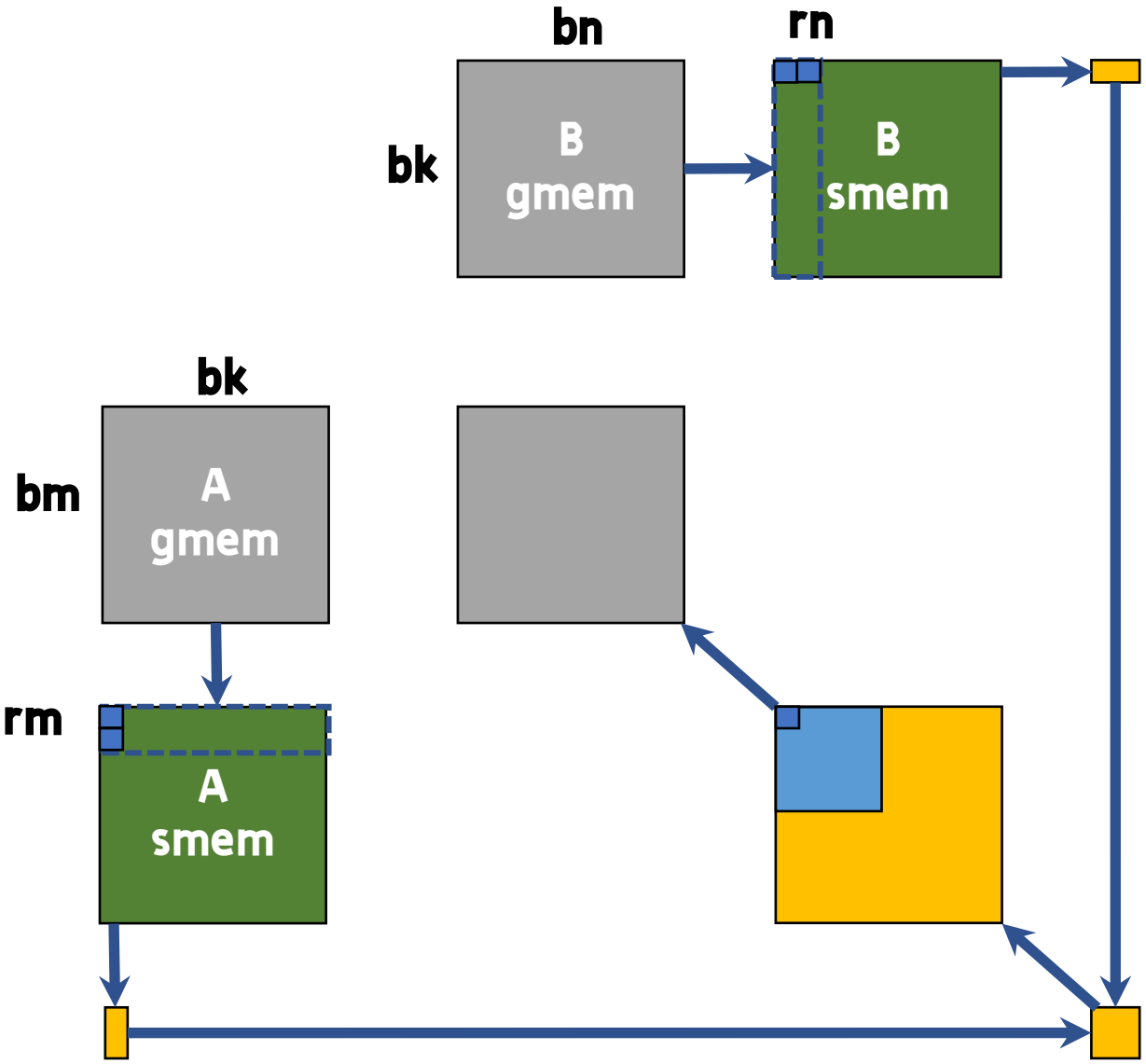





```
for m from 0 to bm:
   for n from 0 to bn:
      for k from 0 to bk:
         C_tile[m][n] += A_tile[m][k] * B_tile[k][n];
```

```
for k from 0 to bk:
   for m from 0 to bm:
      for n from 0 to bn:
         C_tile[m][n] += A_tile[m][k] * B_tile[k][n];
```

Sgemm_v5_register_outer_product



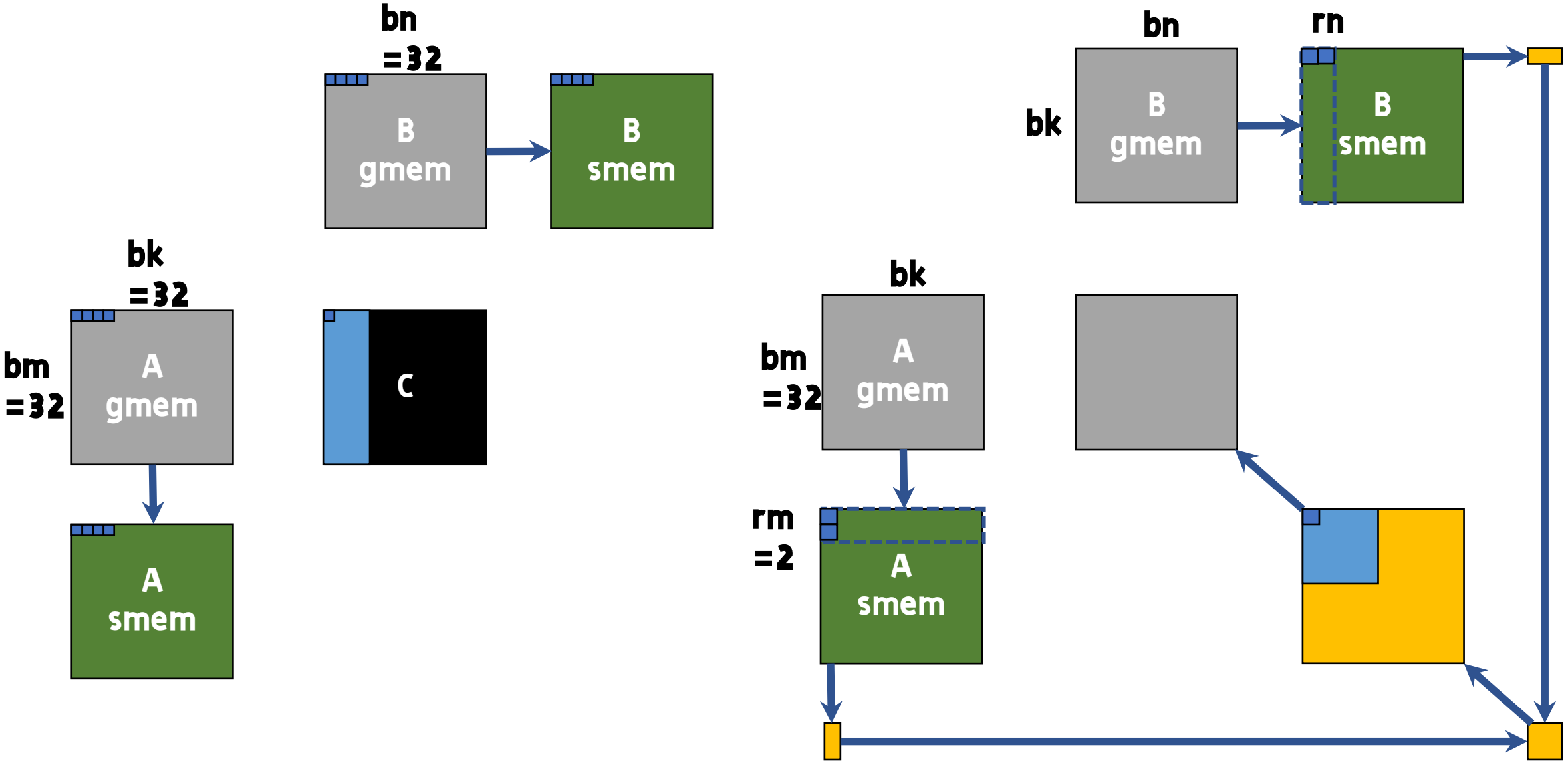

```
for k from 0 to bk:
   for m from 0 to bm:
      for n from 0 to bn:
         C_tile[m][n] += A_tile[m][k] * B_tile[k][n];
```

A_frag, B_frag, C_frag: registers
A_tile, B_tile: shared memory

```
for k from 0 to bk:
   A_frag[rm] <= A_tile[0 to rm][k]
   B_frag[rn] <= B_tile[k][0 to rn]
   for m from 0 to bm:
      for n from 0 to bn:
         C_frag[m][n] += A_frag[k] * B_frag[k];
```

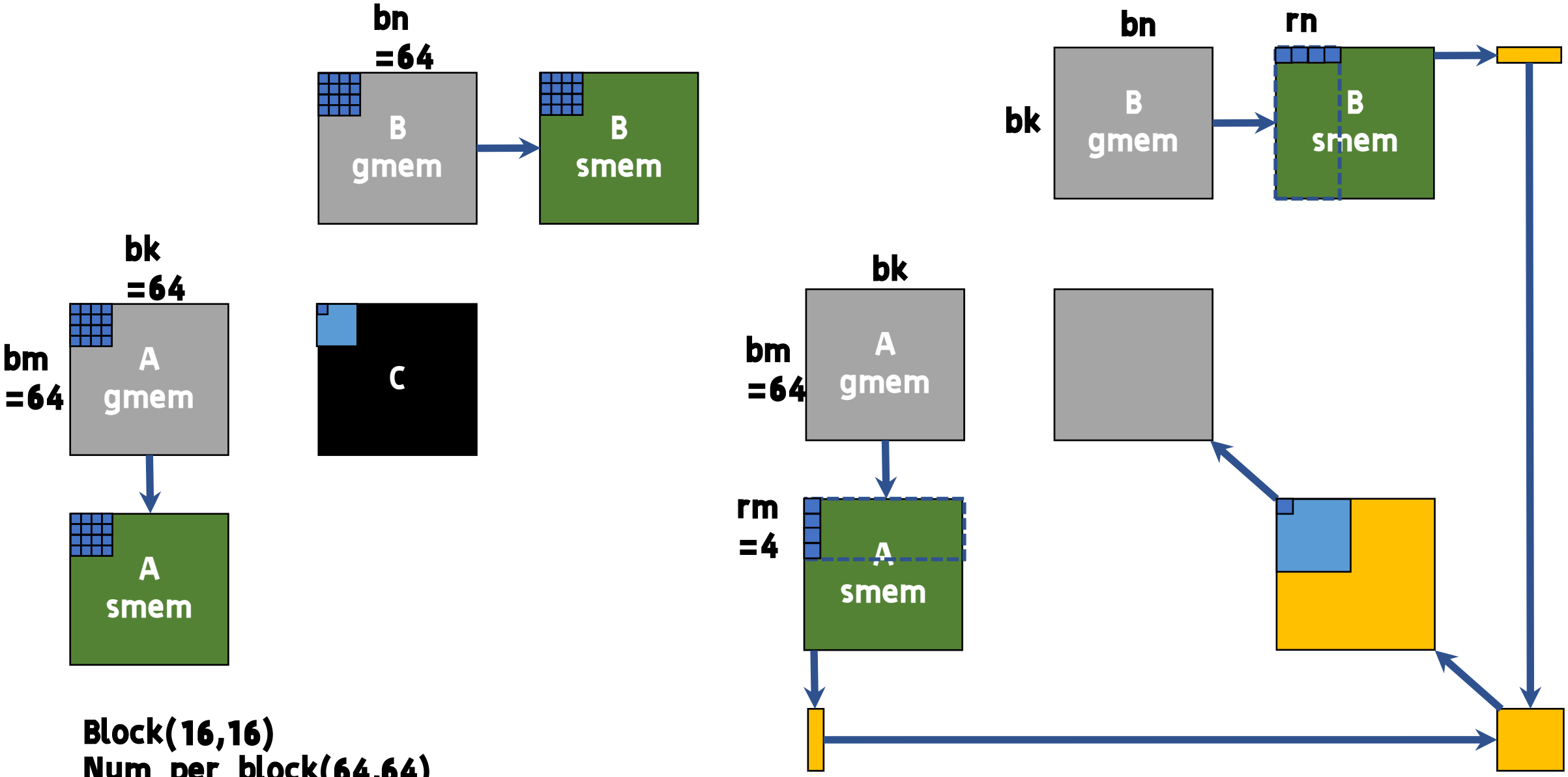
[施工中] CUDA GEMM 理论性能分析与 kernel 优化 - 知乎

Sgemm_v5_register_outer_product



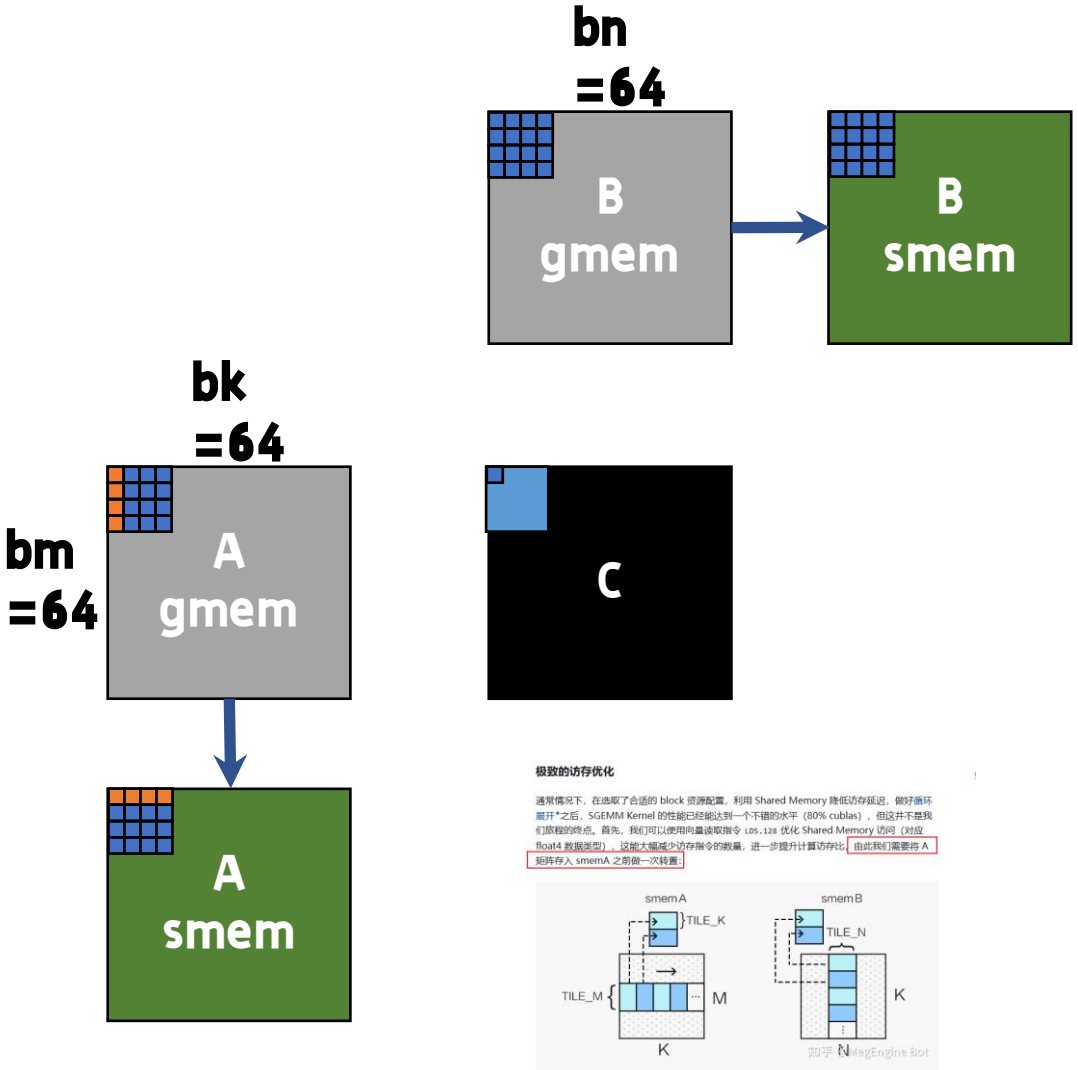
[施工中] CUDA GEMM 理论性能分析与 kernel 优化 - 知乎

Sgemm_v6_register_outer_product_float4

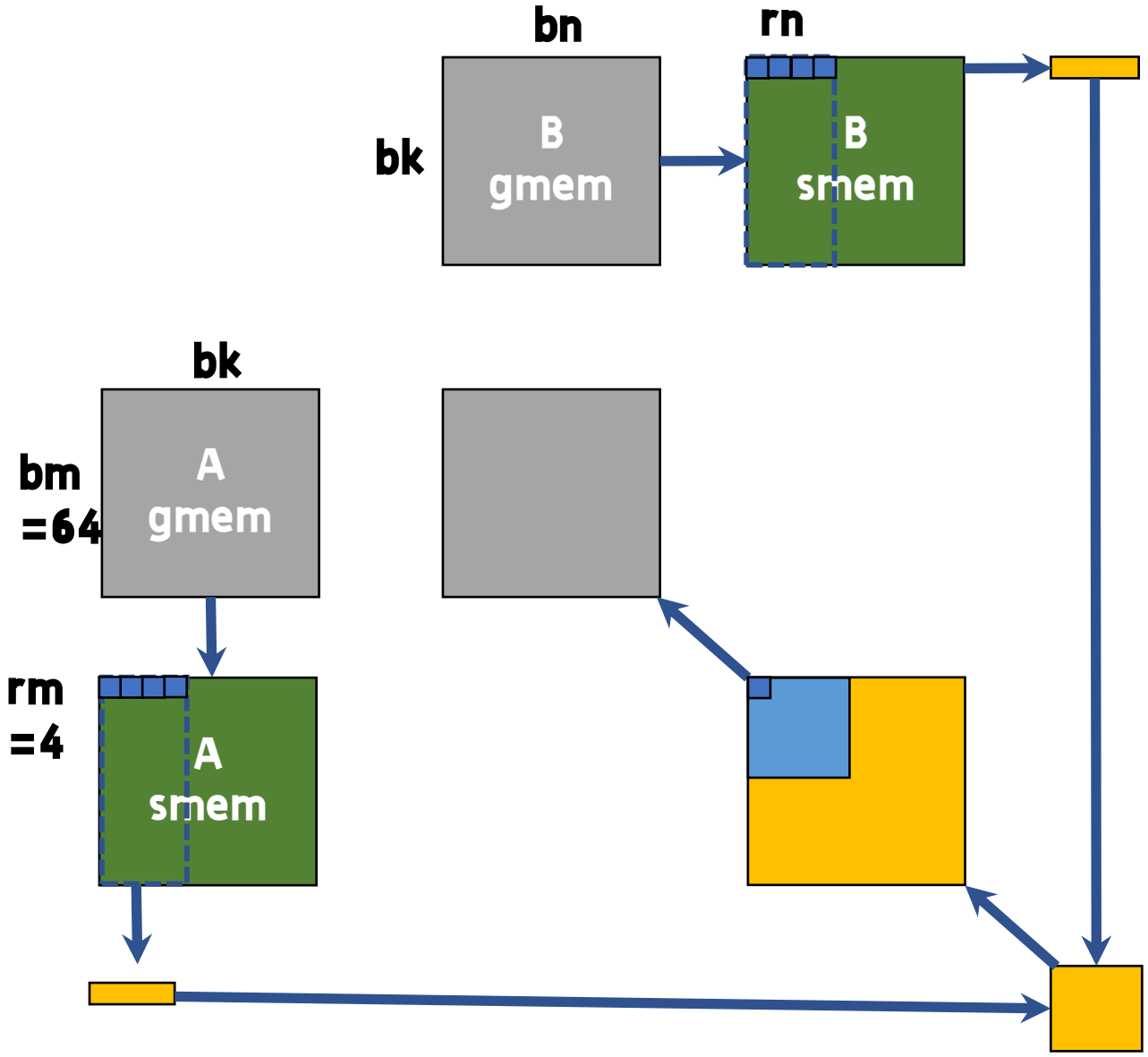


[施工中] CUDA GEMM 理论性能分析与 kernel 优化 - 知乎

Sgemm_v7_A_smem_transpose

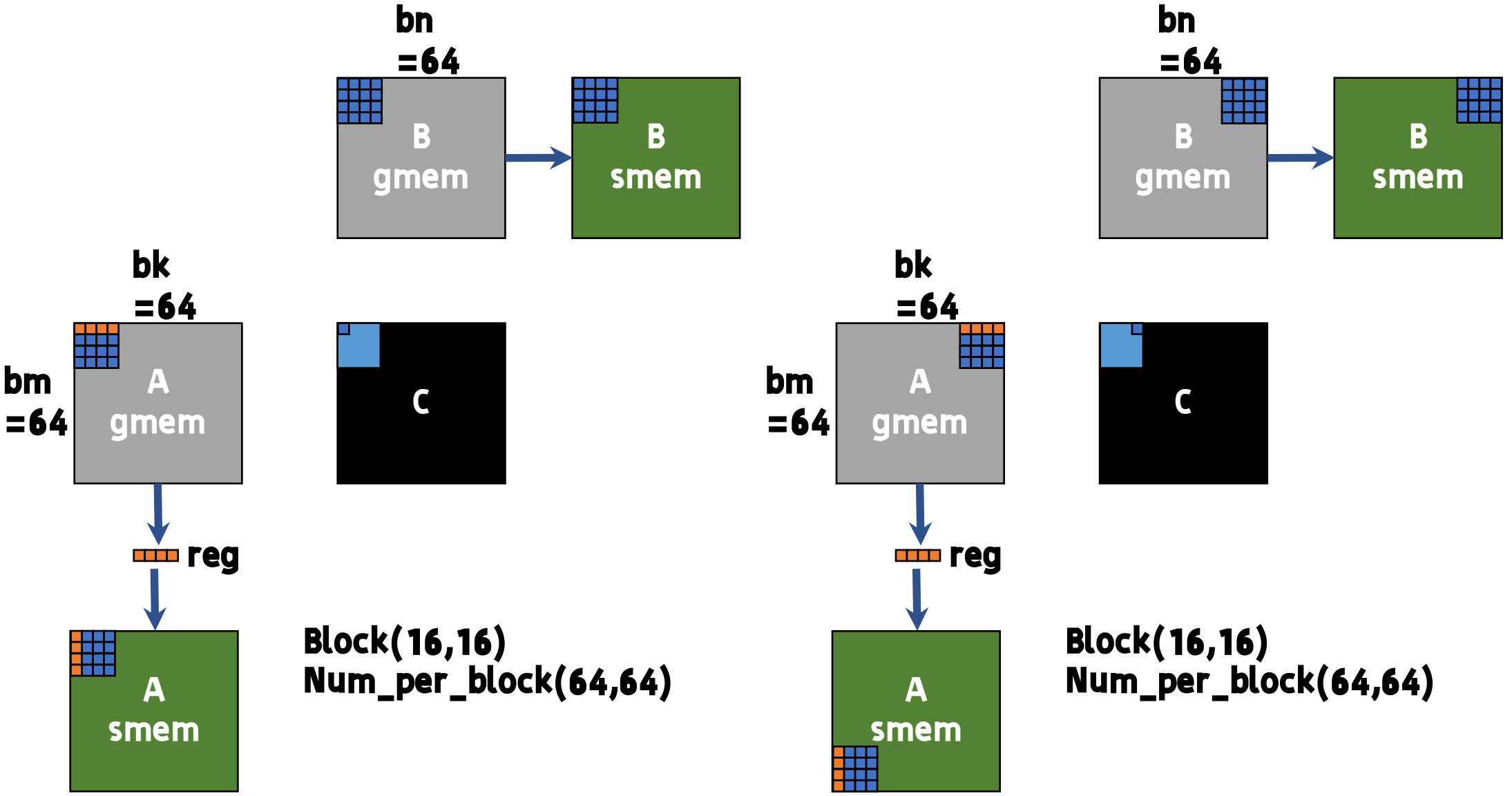


Block(16,16)
Num_per_block(64,64)



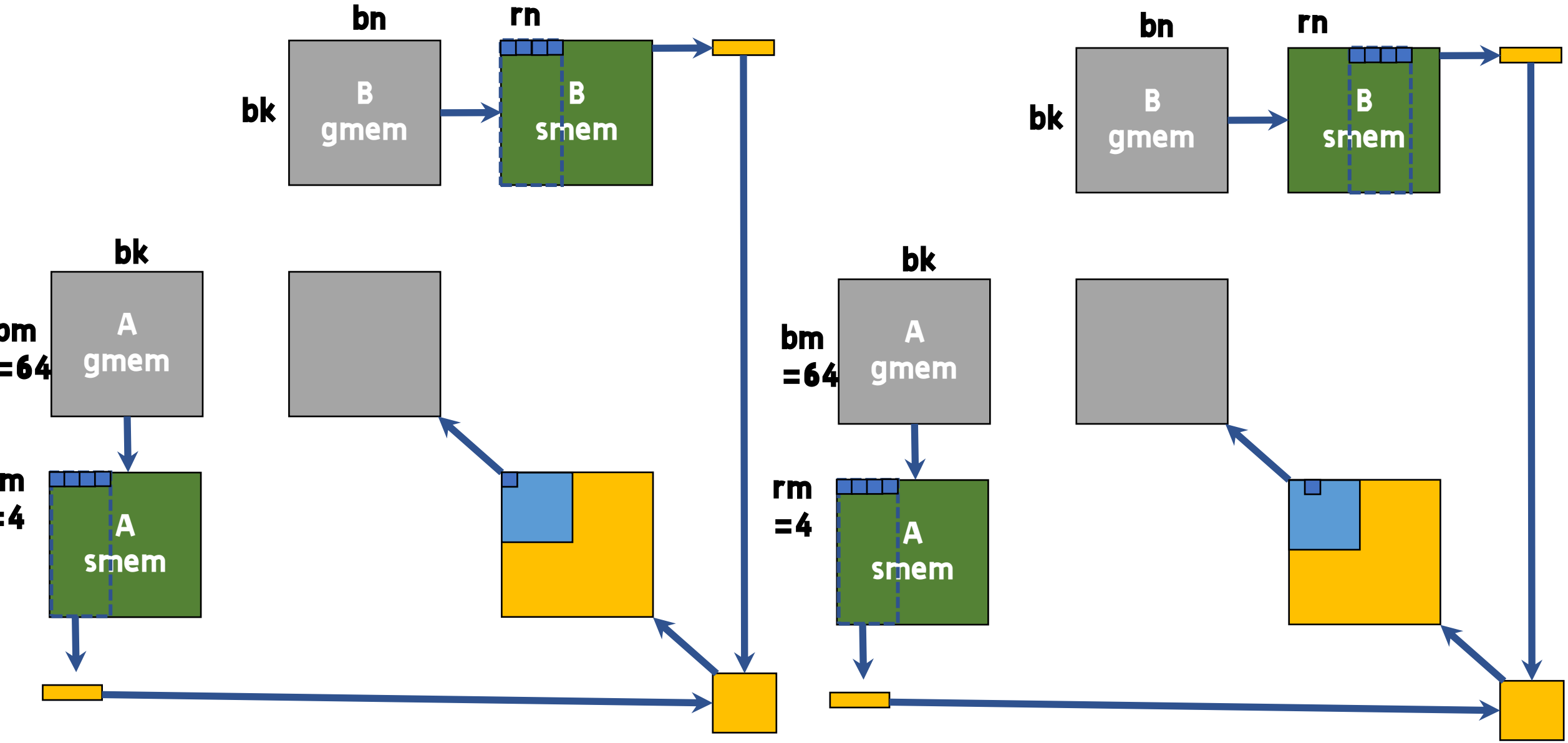
CUDA 矩阵乘法终极优化指南 - 知乎

Sgemm_v7_A_smem_transpose



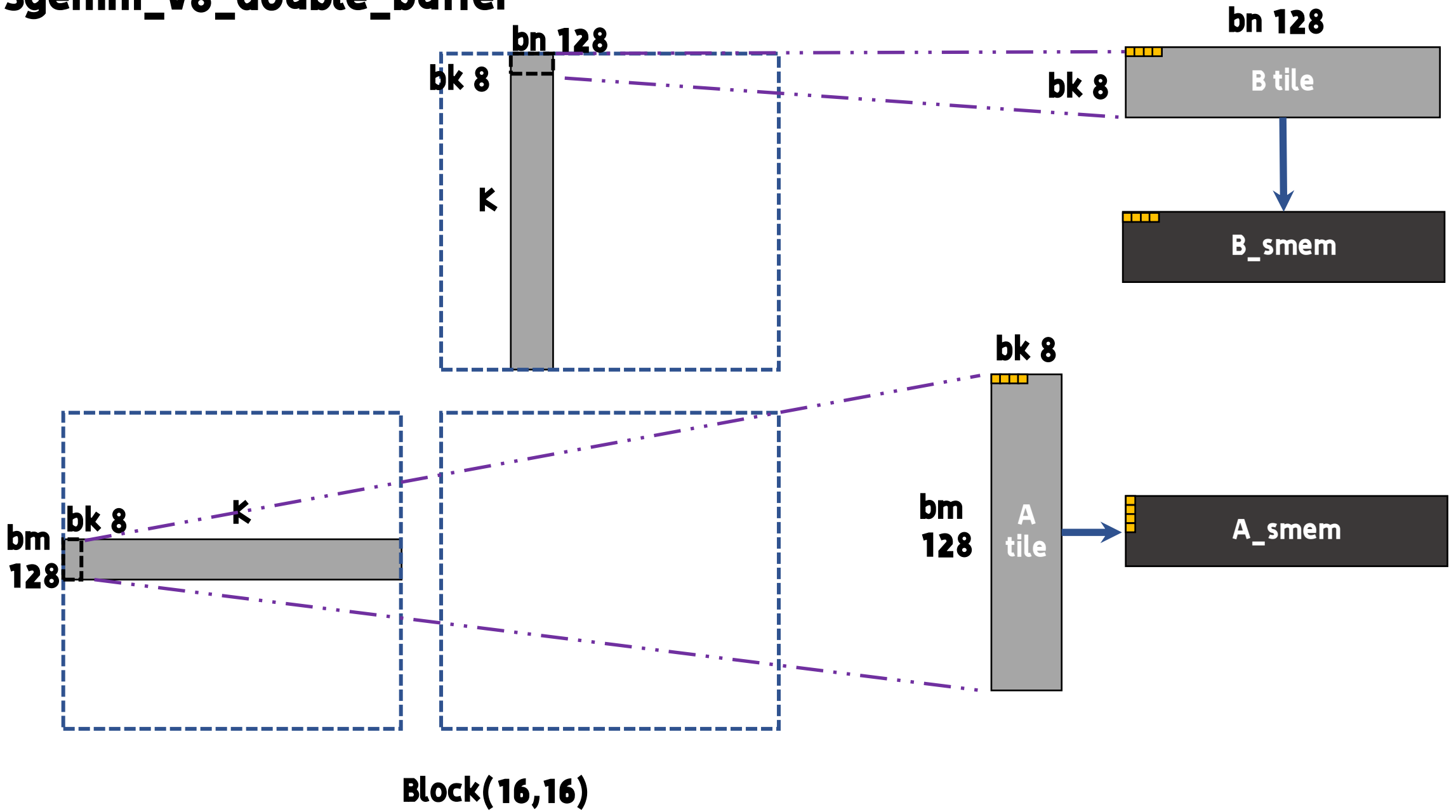
CUDA 矩阵乘法终极优化指南 - 知乎

Sgemm_v7_A_smem_transpose



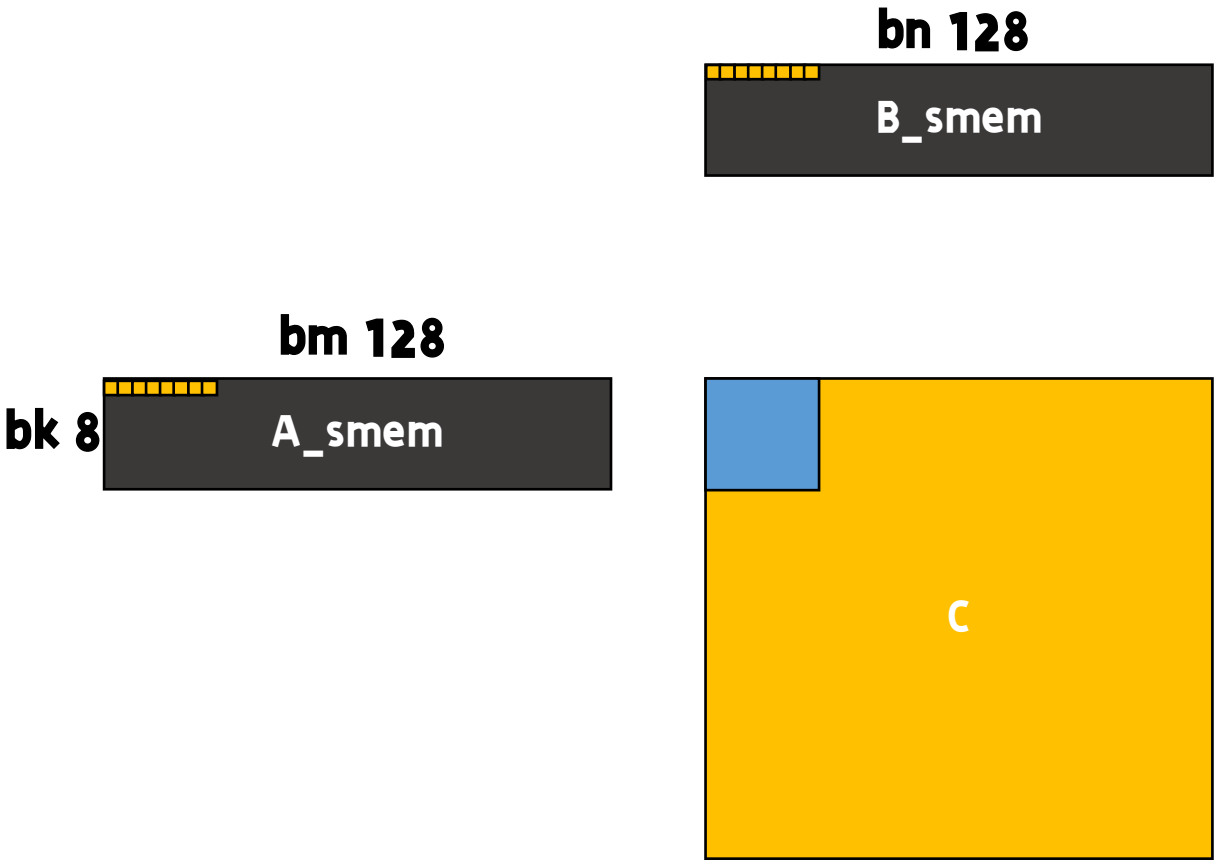
CUDA 矩阵乘法终极优化指南 - 知乎

Sgemm_v8_double_buffer



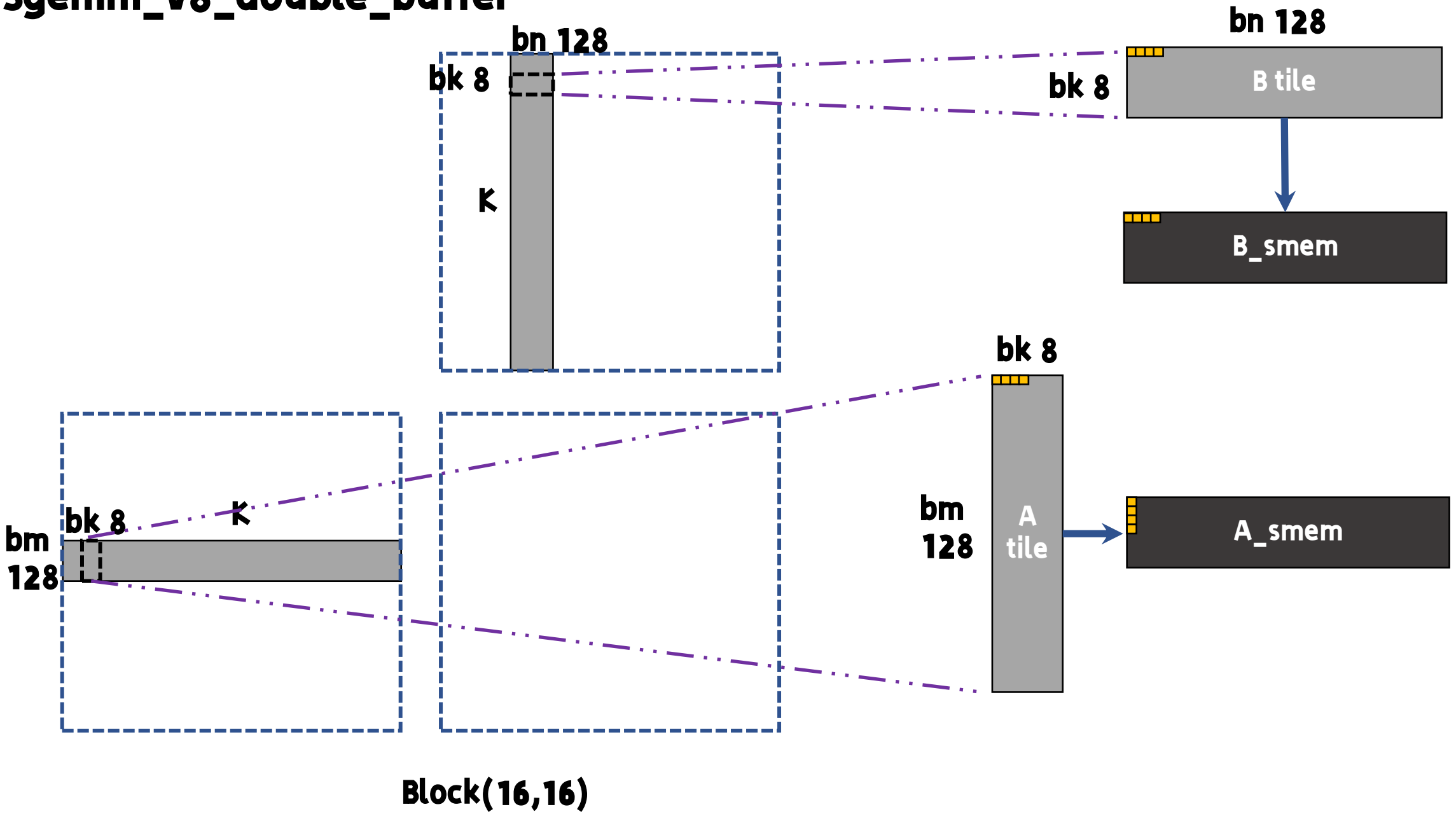
深入浅出GPU优化系列：GEMM优化（二） - 知乎

Sgemm_v8_double_buffer



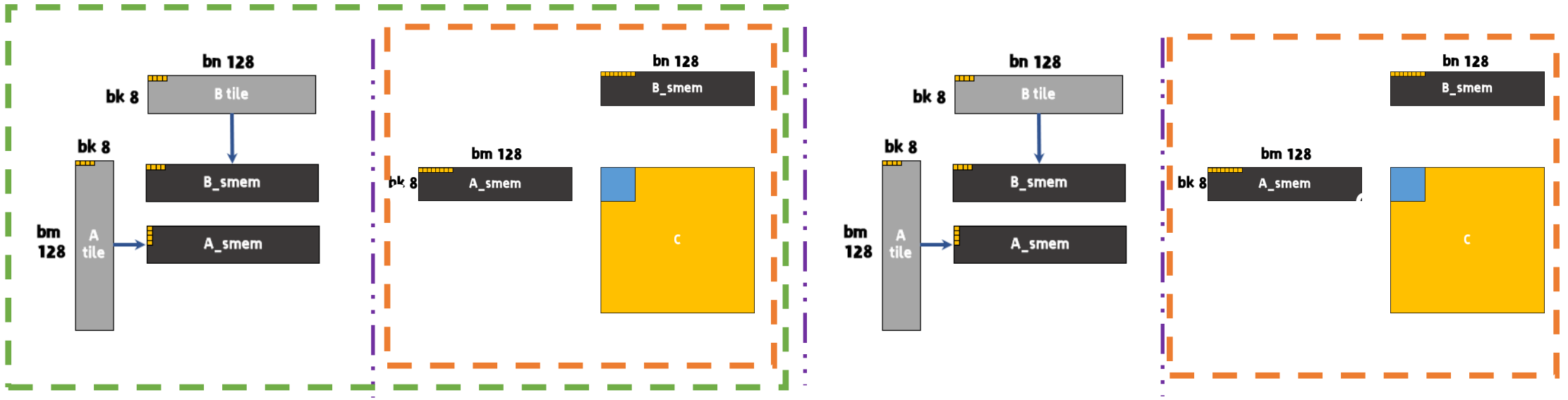
深入浅出GPU优化系列：GEMM优化（二） - 知乎

Sgemm_v8_double_buffer



深入浅出GPU优化系列：GEMM优化（二） - 知乎

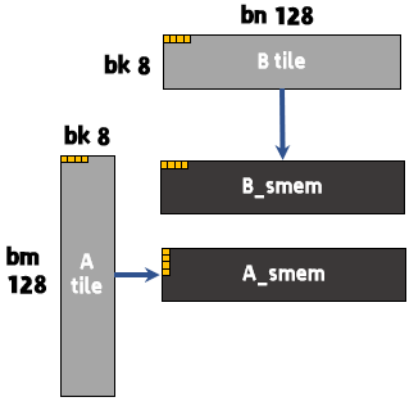
Sgemm_v8_double_buffer



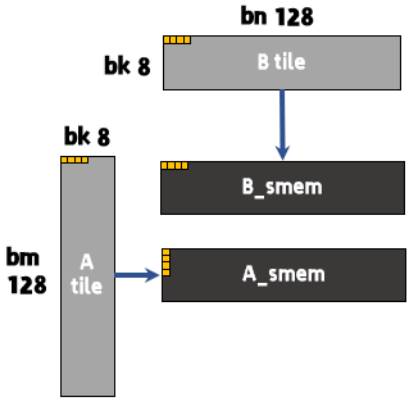
小循环

大循环

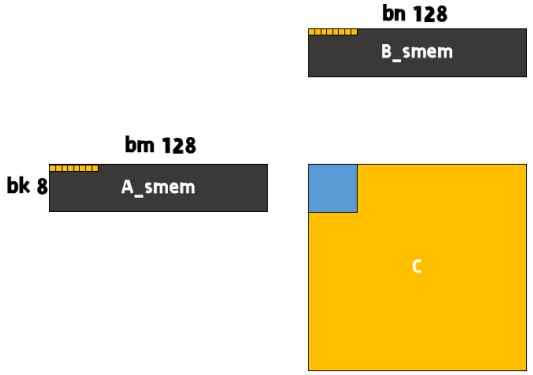
深入浅出GPU优化系列：GEMM优化（二） - 知乎



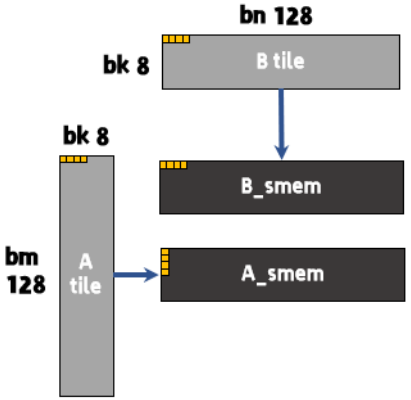
shared_0



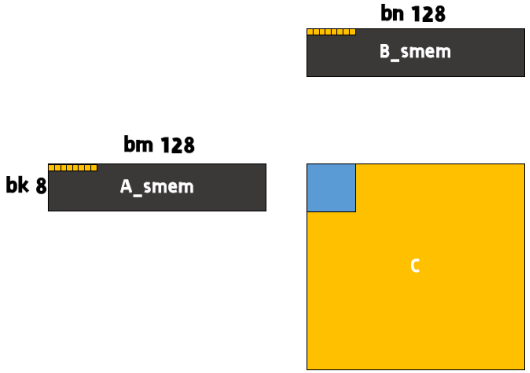
shared_1



shared_0 → reg

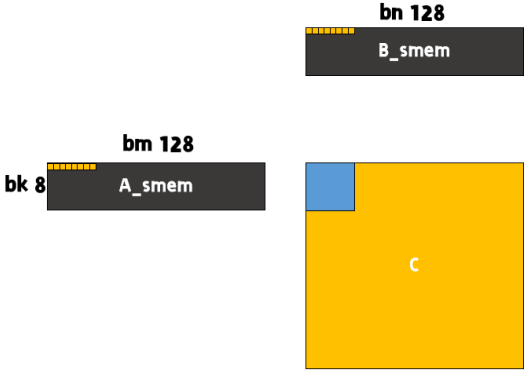


shared_0



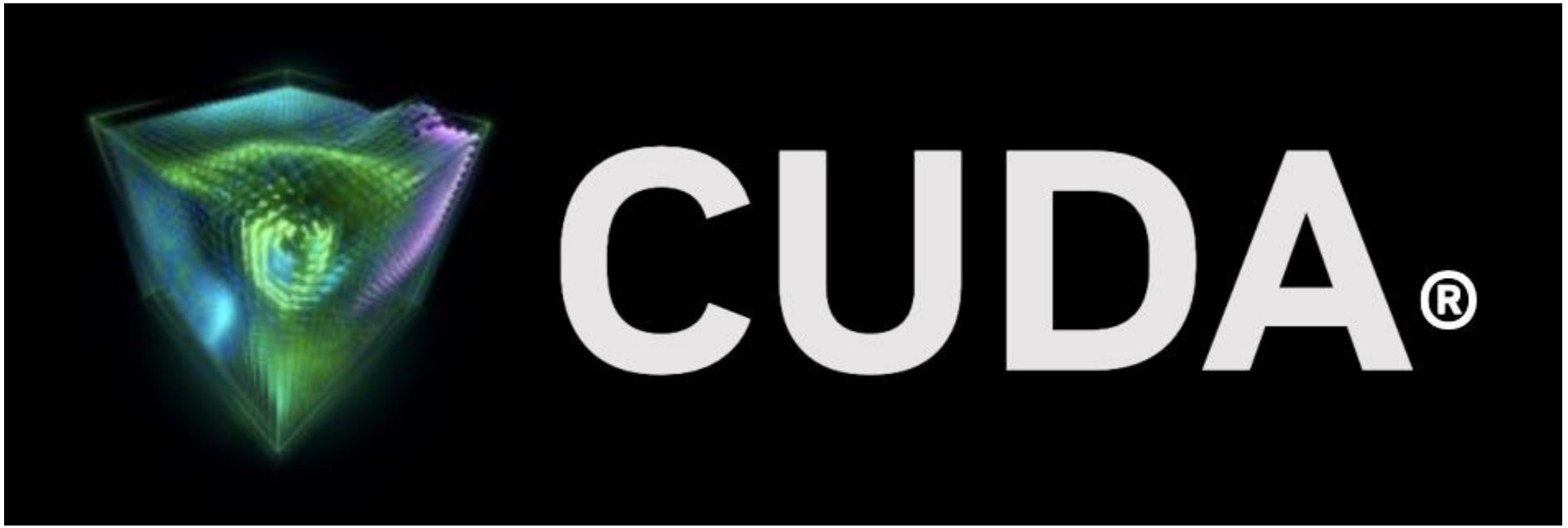
shared_1 → reg

.....



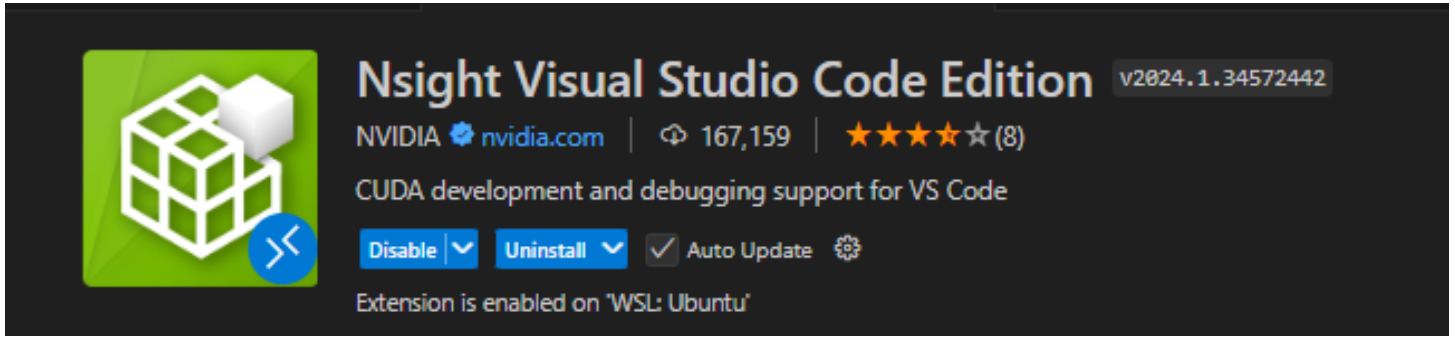
shared_x → reg

Sgemm_v8_double_buffer



宇宙最强CUDA课程（不是）
硬件与工具探索合集

使用VSCode调试CUDA（win不支持）




```
if(CMAKE_BUILD_TYPE STREQUAL "Debug")
   target_compile_options(my_reduce_v0_global_memory PRIVATE $<$<COMPILE_LANGUAGE:CUDA>:-G>)
endif()
set_target_properties(my_reduce_v0_global_memory PROPERTIES CUDA_SEPARABLE_COMPILATION ON)
```

- 如果发现某些博客里面写的东西有问题，一定要试着往更本质的地方去思考
  - Cmake最终还是会使用nvcc，所以要去看nvcc是否有什么设置错误的标志位

CUDA 番外篇 | Visual Studio Code的CUDA环境 - 知乎

使用Vscode调试cuda代码 - 李理的博客

GPU基础数据查询

NVIDIA GeForce RTX 4090 Specs | TechPowerUp GPU Database

https://www.techpowerup.com/gpu-specs/

NVIDIA GeForce RTX 4090

Report an Error

AD102	16384	512	176	24 GB	GDDR6X	384 bit
GRAPHICS PROCESSOR	CORES	TMUS	ROPS	MEMORY SIZE	MEMORY TYPE	BUS WIDTH





Recommended Gaming Resolutions: 1920x1080 2560x1440 3840x2160

The GeForce RTX 4090 is an enthusiast-class graphics card by NVIDIA, launched on September 20th, 2022. Built on the 5 nm process, and based on the AD102 graphics processor, in its AD102-300-A1 variant, the card supports DirectX 12 Ultimate. This ensures that all modern games will run on GeForce RTX 4090. Additionally, the DirectX 12 Ultimate capability guarantees support for hardware-raytracing, variable-rate shading and more, in upcoming video games. The AD102 graphics processor is a large chip with a die area of 609 mm² and 76,300 million transistors. Unlike the fully unlocked TITAN Ada, which uses the same GPU but has all 18432 shaders enabled, NVIDIA has disabled some shading units on the GeForce RTX 4090 to reach the product's target shader count. It features 16384 shading units, 512 texture mapping units, and 176 ROPs. Also included are 512 tensor cores which help improve the speed of machine learning applications. The card also has 128 raytracing acceleration cores. NVIDIA has paired 24 GB GDDR6X memory with the GeForce RTX 4090, which are connected using a 384-bit memory interface. The GPU is operating at a frequency of 2235 MHz, which can be boosted up to 2520 MHz, memory is running at 1313 MHz (21 Gbps effective).

Being a triple-slot card, the NVIDIA GeForce RTX 4090 draws power from 1x 16-pin power connector, with power draw rated at 450 W maximum. Display outputs include: 1x HDMI 2.1, 3x DisplayPort 1.4a. GeForce RTX 4090 is connected to the rest of the system using a PCI-Express 4.0 x16 interface. The card's dimensions are 304 mm x 137 mm x 61 mm, and it features a triple-slot cooling solution. Its price at launch was 1599 US Dollars.

Graphics Processor

GPU Name:	AD102
GPU Variant:	AD102-300-A1
Architecture:	Ada Lovelace
Foundry:	TSMC
Process Size:	5 nm
Transistors:	76,300 million
Density:	125.3M / mm²
Die Size:	609 mm²

Clock Speeds

Base Clock:	2235 MHz
Boost Clock:	2520 MHz
Memory Clock:	1313 MHz 21 Gbps effective

Board Design

Slot Width:	Triple-slot
Length:	304 mm 12 inches
Width:	137 mm 5.4 inches
Height:	61 mm 2.4 inches
TDP:	450 W
Suggested PSU:	850 W
Outputs:	1x HDMI 2.1 3x DisplayPort 1.4a
Power Connectors:	1x 16-pin
Board Number:	PG139 SKU 330

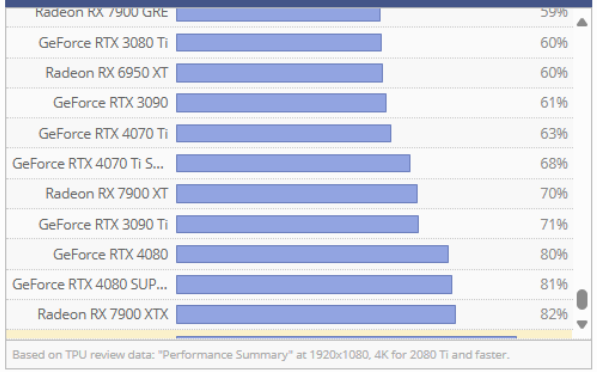
Graphics Card

Release Date:	Sep 20th, 2022
Availability:	Oct 12th, 2022
Generation:	GeForce 40
Predecessor:	GeForce 30
Successor:	GeForce 50
Production:	Active
Launch Price:	1,599 USD
Current Price:	Amazon / Newegg
Bus Interface:	PCIe 4.0 x16
Reviews:	74 in our database

Theoretical Performance

Pixel Rate:	443.5 GPixel/s
Texture Rate:	1,290 GTexel/s
FP16 (half):	82.58 TFLOPS (1:1)
FP32 (float):	82.58 TFLOPS
FP64 (double):	1,290 GFLOPS (1:64)

Relative Performance



Memory

Memory Size:	24 GB
Memory Type:	GDDR6X
Memory Bus:	384 bit
Bandwidth:	1.01 TB/s

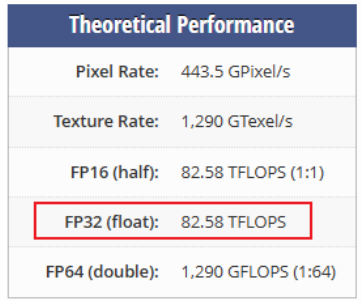
Graphics Features

DirectX:	12 Ultimate (12_2)
OpenGL:	4.6
OpenCL:	3.0
Vulkan:	1.3
CUDA:	8.9
Shader Model:	6.8

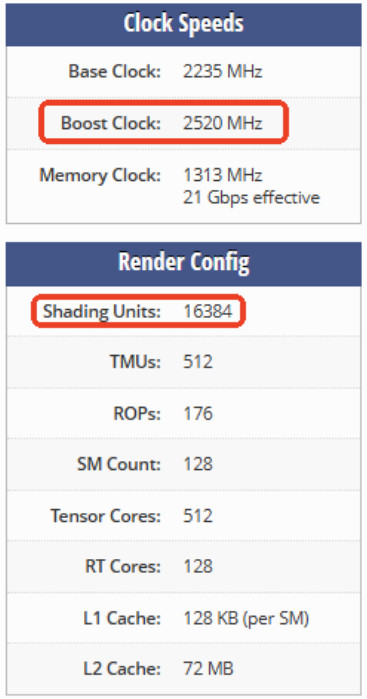
Render Config

Shading Units:	16384
TMUs:	512
ROPs:	176
SM Count:	128
Tensor Cores:	512
RT Cores:	128
L1 Cache:	128 KB (per SM)
L2 Cache:	72 MB

显卡算力计算



Theoretical Performance	
Pixel Rate:	443.5 GPixel/s
Texture Rate:	1,290 GTexel/s
FP16 (half):	82.58 TFLOPS (1:1)
FP32 (float):	82.58 TFLOPS
FP64 (double):	1,290 GFLOPS (1:64)



Clock Speeds	
Base Clock:	2235 MHz
Boost Clock:	2520 MHz
Memory Clock:	1313 MHz 21 Gbps effective

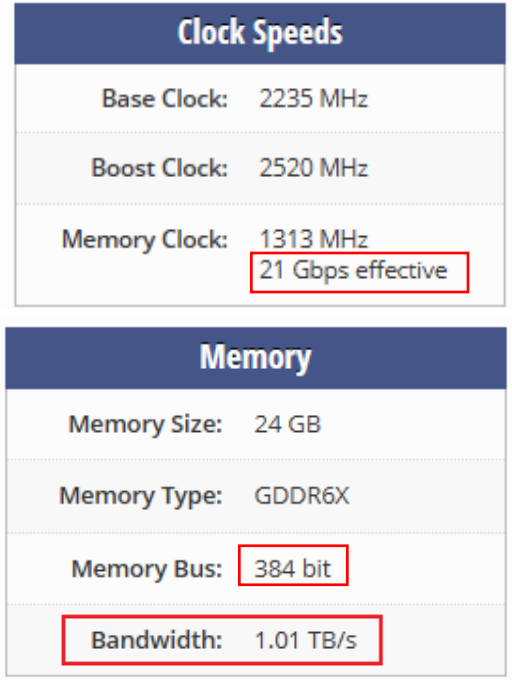
Render Config	
Shading Units:	16384
TMUs:	512
ROPs:	176
SM Count:	128
Tensor Cores:	512
RT Cores:	128
L1 Cache:	128 KB (per SM)
L2 Cache:	72 MB

FLOPS （Floating-point operations per second的缩写），即每秒浮点运算次数。

FLOPS =
【GPU核数】*【单核主频】*【GPU单个周期浮点计算能力】
=16384* 2520MHz * 2(乘加)
=82.58TFLOPS

【算力基础】GPU算力计算和其他相关基础(TFLOPS/TOPS/FP32/INT8...)_fp32 tflops-CSDN博客

显卡带宽计算



Clock Speeds	
Base Clock:	2235 MHz
Boost Clock:	2520 MHz
Memory Clock:	1313 MHz 21 Gbps effective

Memory	
Memory Size:	24 GB
Memory Type:	GDDR6X
Memory Bus:	384 bit
Bandwidth:	1.01 TB/s

带宽memory bandwidth:也即计算平台的带宽上限,指的是一个计算平台每秒所能完成的内存交换量。单位是Byte/s

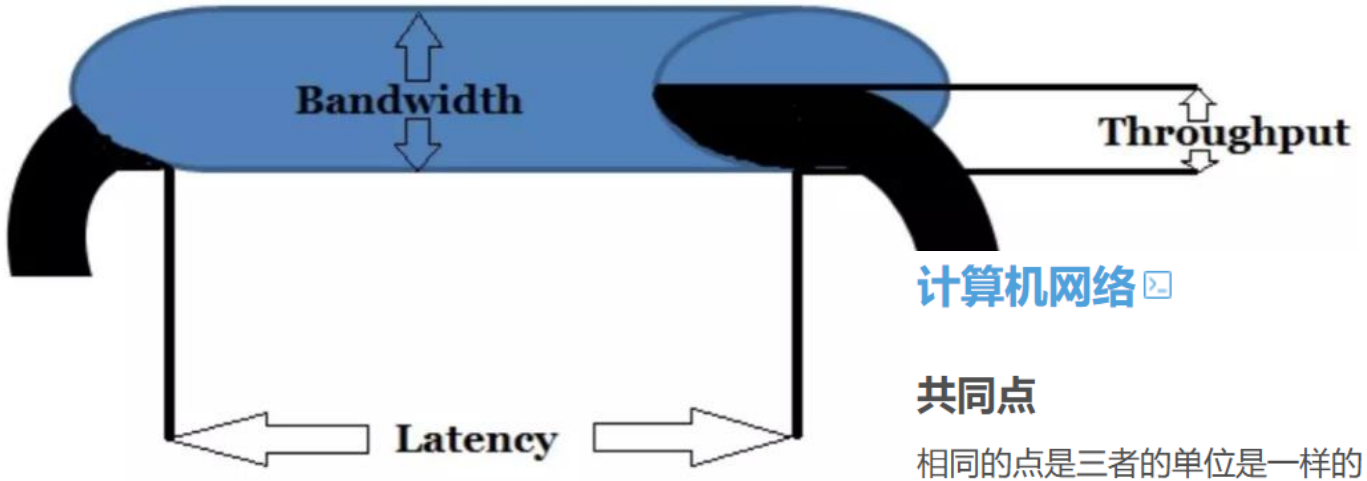
bandwidth =
【内存频率】*【Prefetch】*【内存位宽】/【8】
=1313MHz * 16 * 384bit / 8
=1.01TB/s

[SOLVED] - Effective memory clock speed confusions | Tom's Hardware Forum

GDDR6 vs DDR4 vs HBM2?为什么CPU还不用GDDR？异构内存的未来在哪里？ - 知乎

【九零科普】LPDDR5 DDR5 GDDR6都有啥区别？——常见内存区别科普_哔哩哔哩_bilibili

显卡带宽计算



计算机网络

共同点

相同的点是三者的单位是一样的，b/s,kb/s,Mb/s。

区别

带宽是理想状况下数据传送速率——假设一个人理想状态下能每秒吃10个鸡蛋

吞吐量是某一个时间点通过某个网络的传输速度——在鸡蛋只有5个的情况，实际上每秒只吃5个鸡蛋

速率在我的理解上更整体——相当于汽车从A到B的平均速度

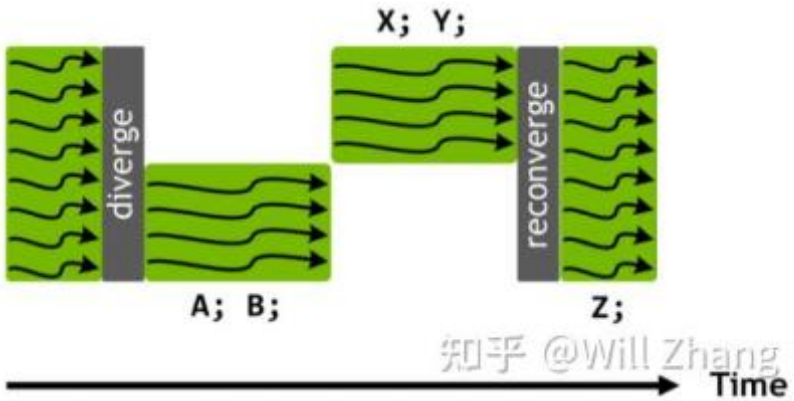
带宽，传输速率，吞吐量三者之间的关系与区别_带宽和吞吐量的区别和关系-CSDN博客

延时（Latency）、吞吐（Throughput）、带宽（Bandwidth）和响应时间（Response Time） - luckylan - 博客园

Warp divergence

从Volta开始，线程调度发生了变化，在Pascal以及之前的GPU上，每个Warp里的32个线程共享一个Program Counter (简称PC)，并且使用一个Active Mask表示任意时刻哪些线程是可运行的，一个经典的运行如下

```
if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
```



直到第一个分支完整结束，才会执行另一个分支。这意味着同一个warp内不同分支失去了并发性，不同分支的线程互相无法发送信号或者交换数据，但同时，不同warp之间的线程又保留了并发性，这当中的线程并发存在着不一致，事实上如果程序员不注意这点，很可能导致死锁。

- **Old way: threads share program counter, but have an "active mask".**
- **Careful not to do inter-thread communication / sync inside if (without mask)!**
- **Automatic reconvergence.**
- **N.B.: As there are load/store instructions the typical pattern (cond ? x[i]: 0.f) will not cause divergence**

—— CUDA MODE 4

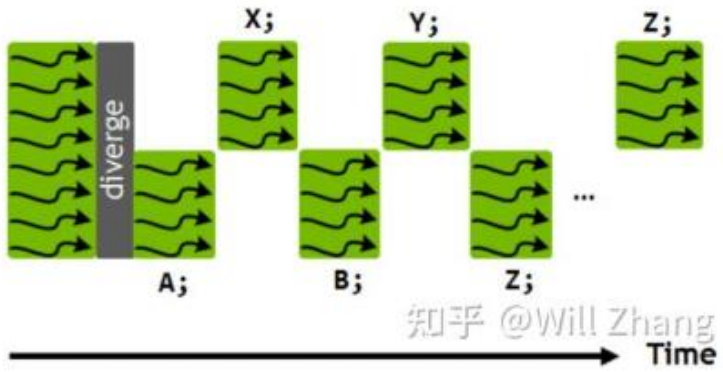
[Lecture 4 Compute and Memory Basics]

[英伟达GPU架构演进近十年，从费米到安培 - 知乎]

Warp divergence

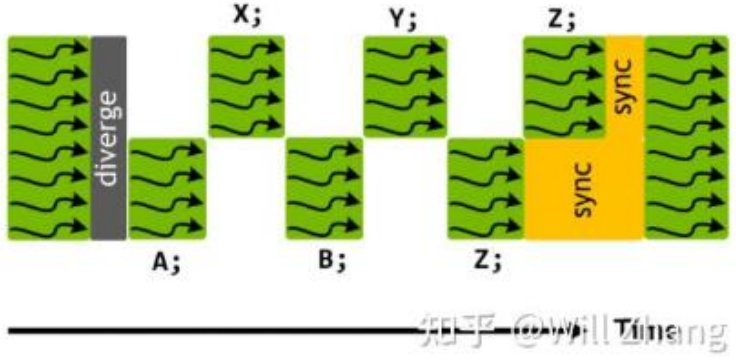
由于运行时仍然要符合SIMT，所以存在一个调度优化器负责将可运行的线程分组，使用SIMT模式执行。经典运行如下：

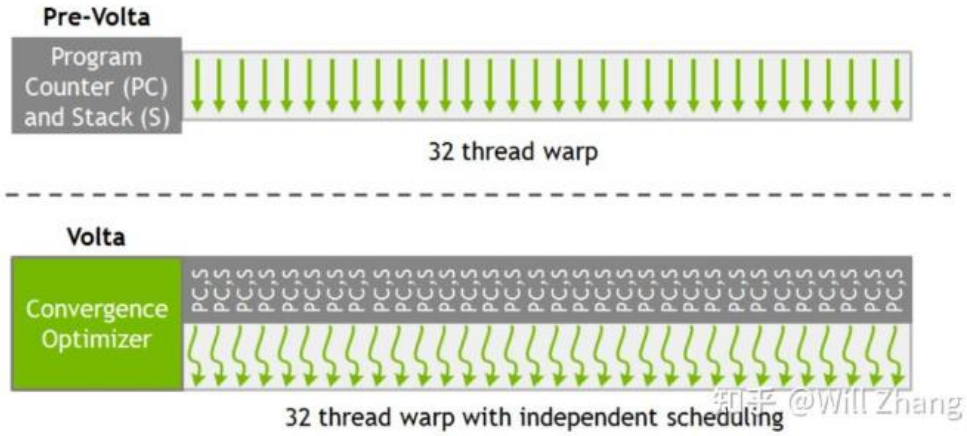
```
if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
```



上图可以注意到，Z的执行并没有被合并，这是因为Z可能会产生一些被其他分支需要的数据，所以调度优化器只有在确定安全的情况下才会合并Z，所以上图Z未合并只是一种情况，一般来说，调度优化器足够聪明可以发现安全的合并。程序员也可以通过一个API来强制合并，如下

```
if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
__syncwarp()
```





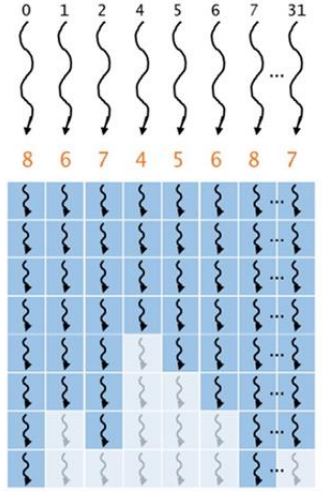
We're not done with a warp until everything is done

(note: this is a pre-volta image)

```
N = a[threadIdx.x];
for(i = 0; i < N; ++i) {

    A

}
```



Loops with variable upper bound also cause divergence.

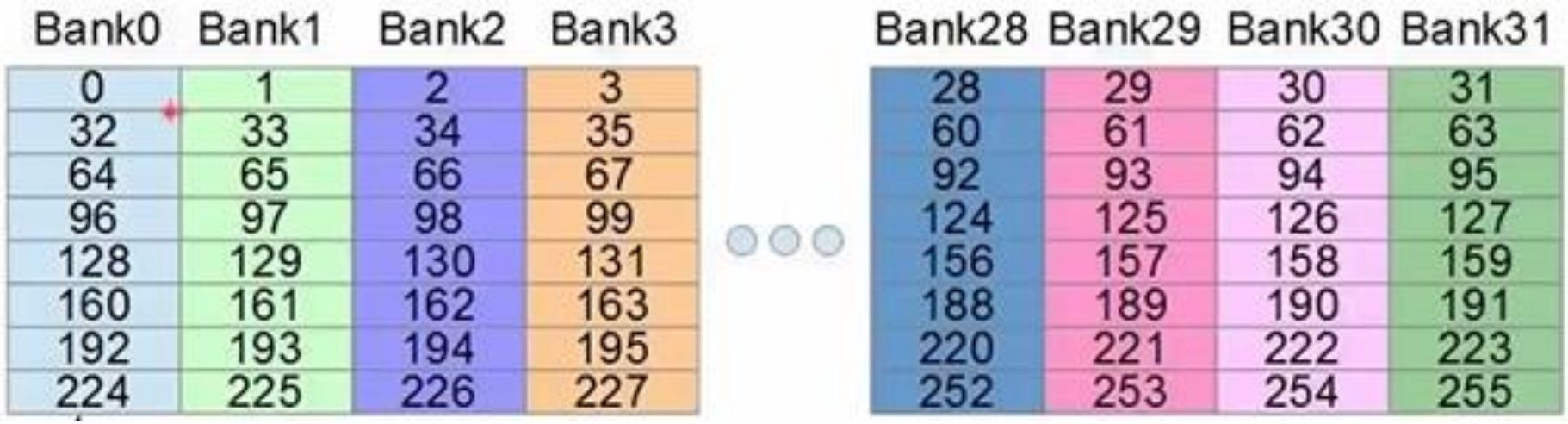
[Lecture 4 Compute and Memory Basics]

[英伟达GPU架构演进近十年，从费米到安培 - 知乎]

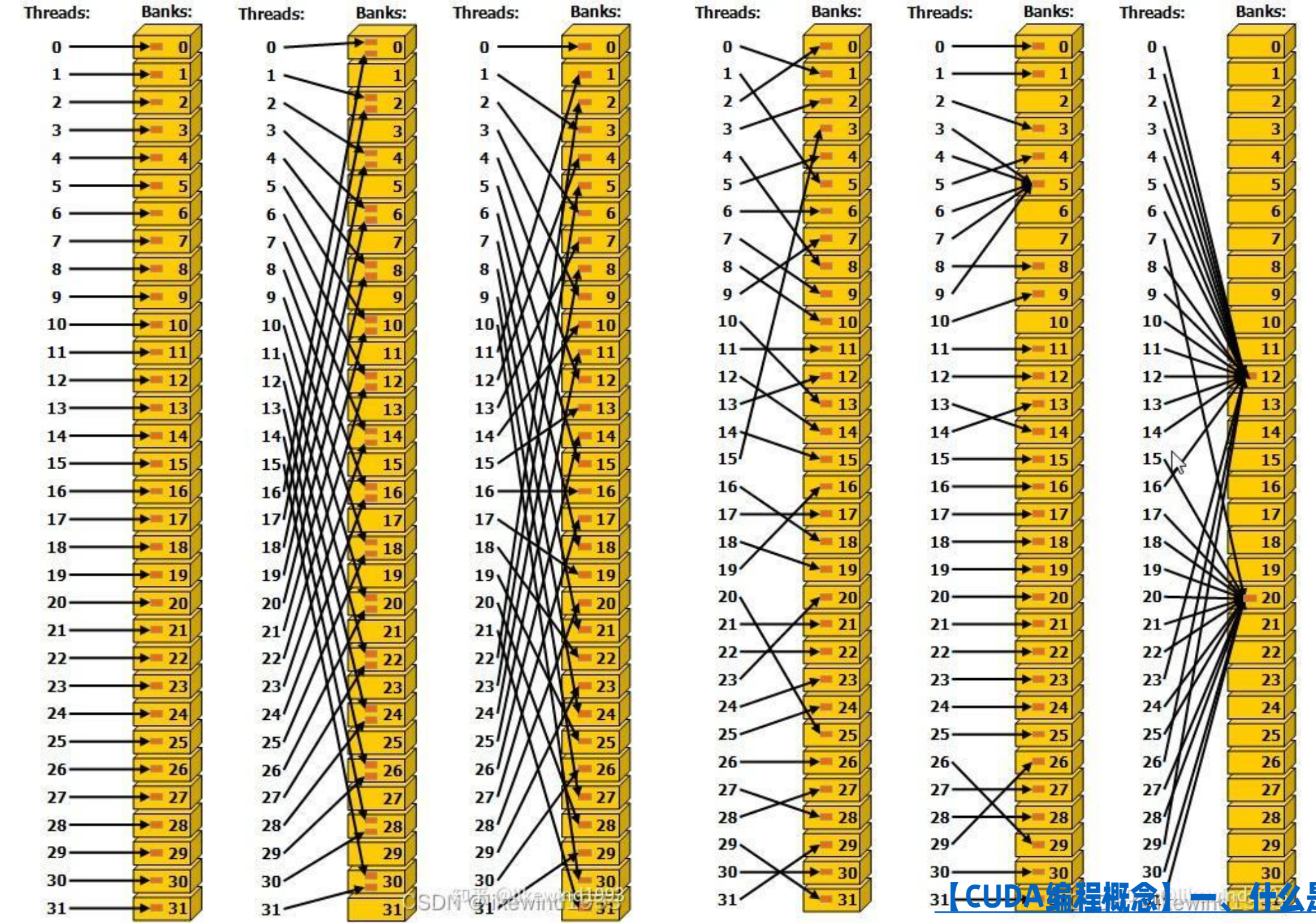
Bank Conflict

为了提高内存读写带宽，共享内存被分割成了32个等大小的内存块，即Bank。因为一个Warp有32个线程，相当于一个线程对应一个内存Bank

```
__shared__ float data[256];
```



【CUDA编程概念】一、什么是bank conflict？ - 知乎



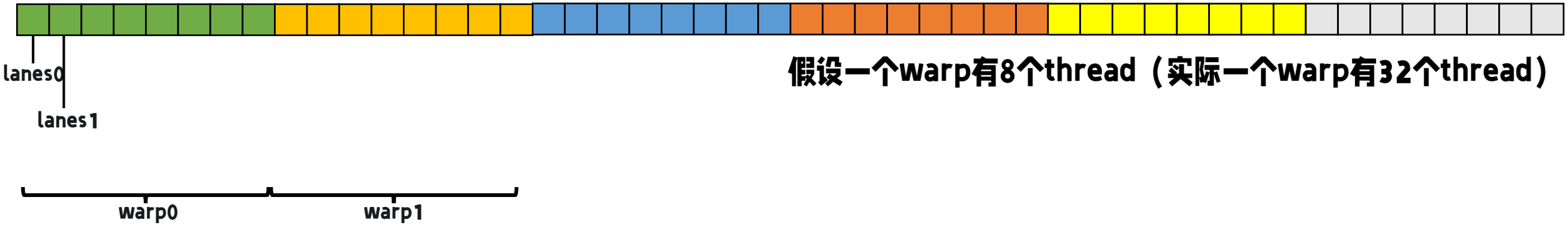
- 只有第二种情况才会触发bank conflict
- 第二种情况是2way-bank
- 最后两种都会变成broadcast

CSDN @Rewind1993

【CUDA编程概念】一、什么是bank conflict？ - 知乎

shuffle

主要用于进行一些 warp 内(寄存器)数据交换操作



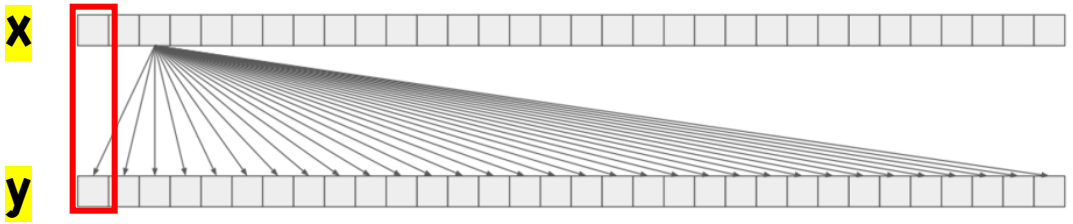

```
const int laneId = tid % WARP_SIZE;
const int warpId = tid / WARP_SIZE;
```

【CUDA编程】束内洗牌函数（Warp Shuffle Functions） - 知乎

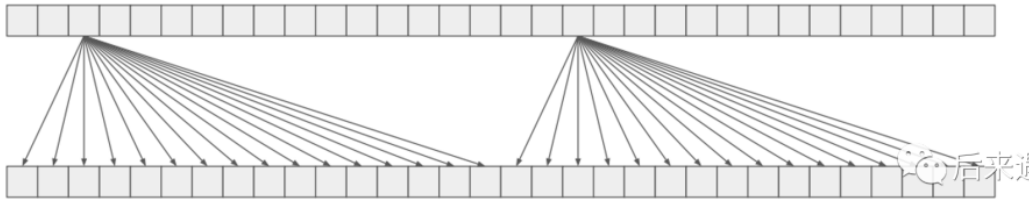
Warp Broadcast

T __shfl_sync(unsigned mask, T var, int srcLane, int width=warpSize);

int y = __shfl_sync(0xffffffff, x, 2);



int y = __shfl_sync(0xffffffff, x, 2, 16);

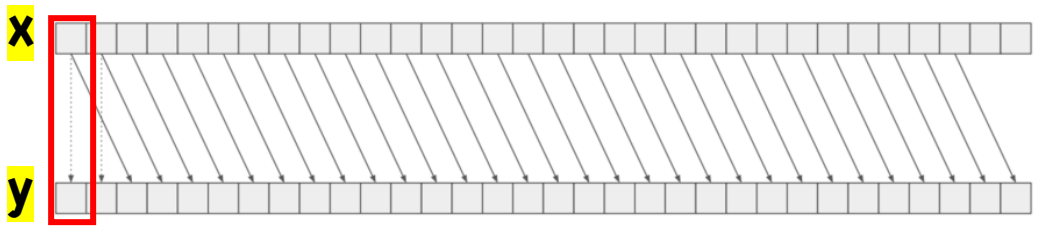


后来遇见AI

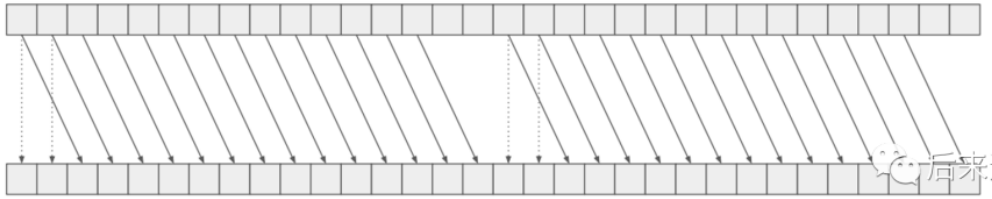
Warp Broadcast

T __shfl_up_sync(unsigned mask, T var, int srcLane, int width=warpSize);

int y = __shfl_up_sync(0xffffffff, x, 2);



int y = __shfl_up_sync(0xffffffff, x, 2, 16);

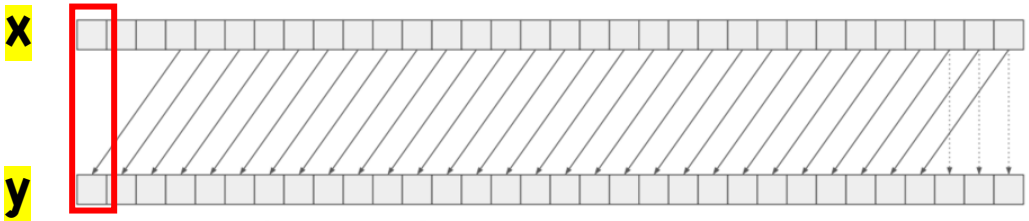


后来遇见AI

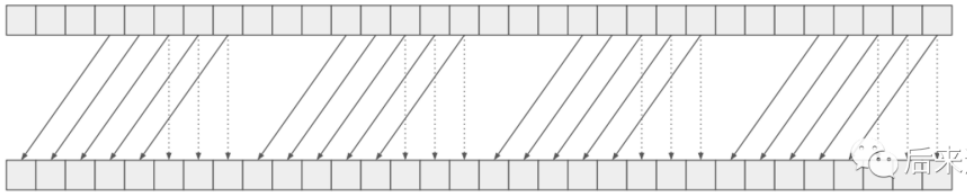
Warp Broadcast

T __shfl_down_sync(unsigned mask, T var, unsigned int delta, int width=warpSize)

int y = __shfl_down_sync(0xffffffff, x, 3);



int y = __shfl_down_sync(0xffffffff, x, 3, 8);

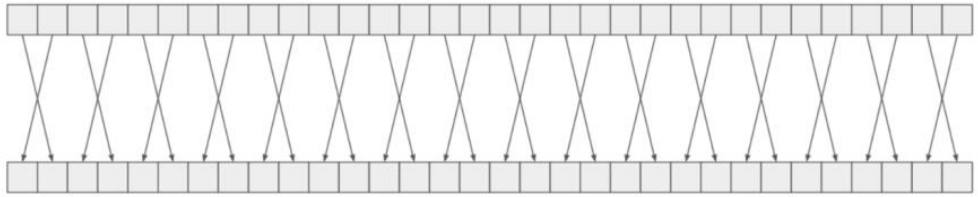


后来遇见AI

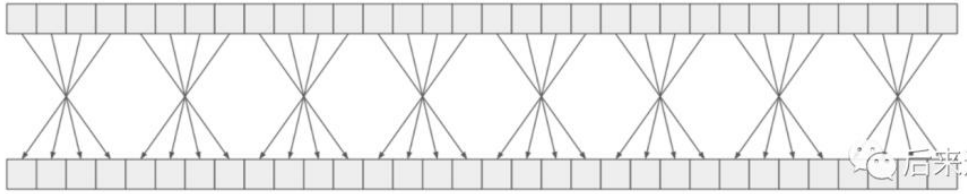
Warp Broadcast

T __shfl_xor_sync(unsigned mask, T var, int laneMask, int width=warpSize);

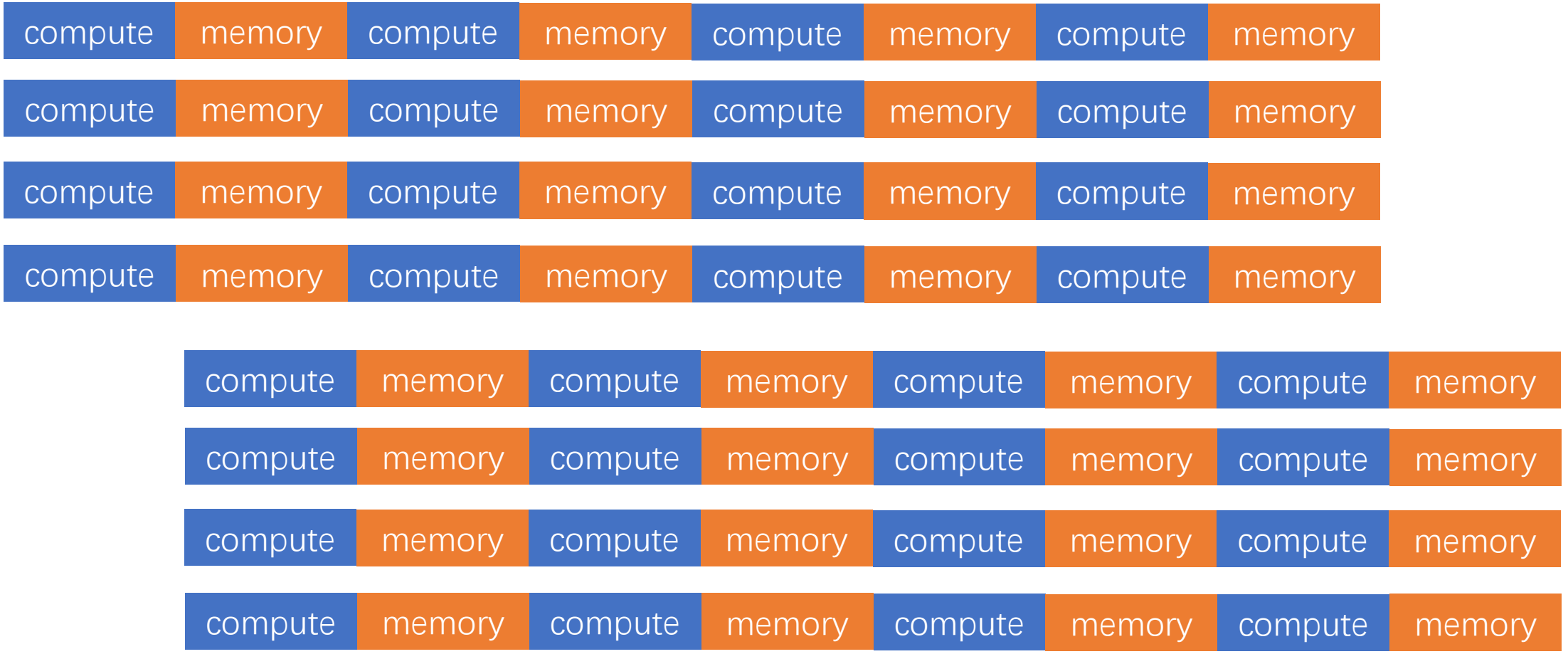
int y = __shfl_xor_sync(0xffffffff,x, 1);



int y = __shfl_ xor_sync(0xffffffff,x, 3);



后来遇见AI


compute memory compute memory compute memory compute memory
compute memory compute memory compute memory compute memory
compute memory compute memory compute memory compute memory
compute memory compute memory compute memory compute memory
compute memory compute memory compute memory compute memory
compute memory compute memory compute memory compute memory
compute memory compute memory compute memory compute memory
compute memory compute memory compute memory compute memory



宇宙最强CUDA课程

ncu与kernel调优指南

宇宙安全声明

- 这部分没有找到系统的学习资料所以课程大部分的观点和结论都是参考零零散散的博客和结合实验现象的猜测，因此难免有错误，介意者勿看~
- 所有参考均会以链接的形式提供，并在视频中强调出处，坚决维护原创者的劳动成果~

付费通知

- 所有课程都是免费的，即使不购买PPT和代码也是可以正常学习的～
- 收费的原因主要是 为了更好的呈现课程 制作ppt和整理代码花费了我大量的时间，望各位理解～
- 如果想购买课程PPT和代码 或者单纯想支持一下UP（感激不尽）一共是50块钱（包含reduce sgemm ncu以及后面会不断更新的cuda内容）加微信购买～
- 不断更新的代码和ppt会给付费者拉一个交流群在群里同步，但是这个群不是我的答疑群，大家可以一起交流～



扫一扫上面的二维码图案，加我为朋友。

【个人微信号】：
hkl793831293



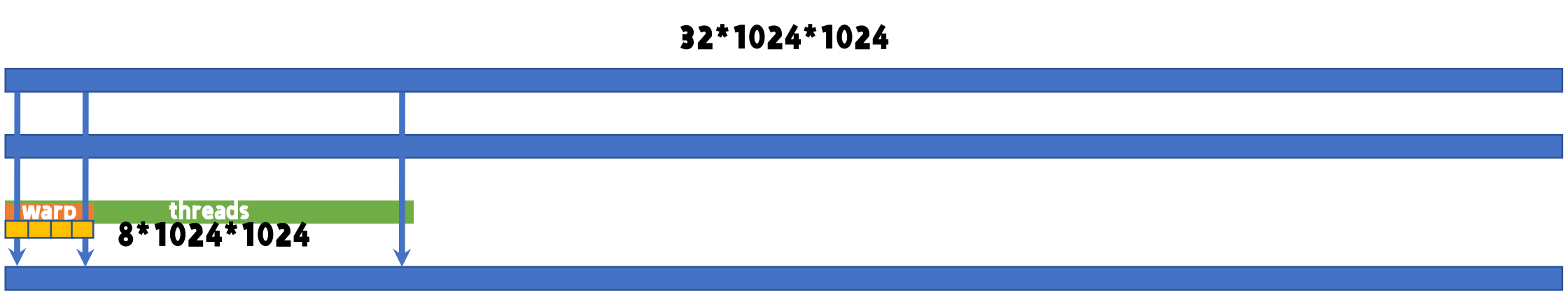
ncu与kernel调优指南

合并访存

全局内存的访问模式，有合并访存和非合并访存两种方式。若一个线程束对全局内存的一次访问导致最少的数据传输，则称该访问为合并访存，否则为非合并访存。

定量地，可以定义合并度，它等于线程束请求的字节数除以由该次请求导致的所有数据传输的字节数。合并度可以理解为资源利用率。如果数据传输中处理的数据都是线程束所需要的，那么合并度就是100%，访存的资源利用率也就是100%，对应的访问模式为合并访存。利用率越高，核函数的访存性能就更好；反之，显存带宽利用率较低

CUDA笔记 内存合并访问 - 知乎





L1/TEX Cache

	Instructions	Requests	Wavefronts	% Peak	Sectors	Sectors/Req	Hit Rate	Bytes	Sector Misses to L2	% Peak to L2
Global Load	524288	524288			2097152	4	0	67108864	2097152	14.27
Global Load To Shared Store (access)	0	0	524288	3.57	0	0		0		
Global Load To Shared Store (bypass)	0	0			0	0	-	0	0	0
Surface Load	0	0	0	0	0	0	0	0	0	0
Texture Load	0	0	0	0	0	0	0	0	0	0
Global Store	262144	262144	262144	1.78	1048576	4	0	33554432	1048576	7.14

```
void __global__ add1(float *x, float *y, float *z)
{
  int n = threadIdx.x + blockIdx.x * blockDim.x;
  z[n] = x[n] + y[n];
}
```

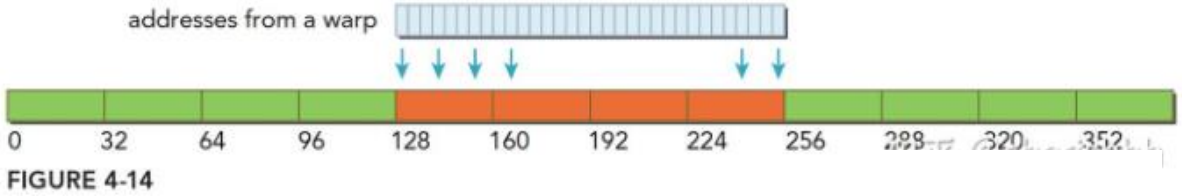
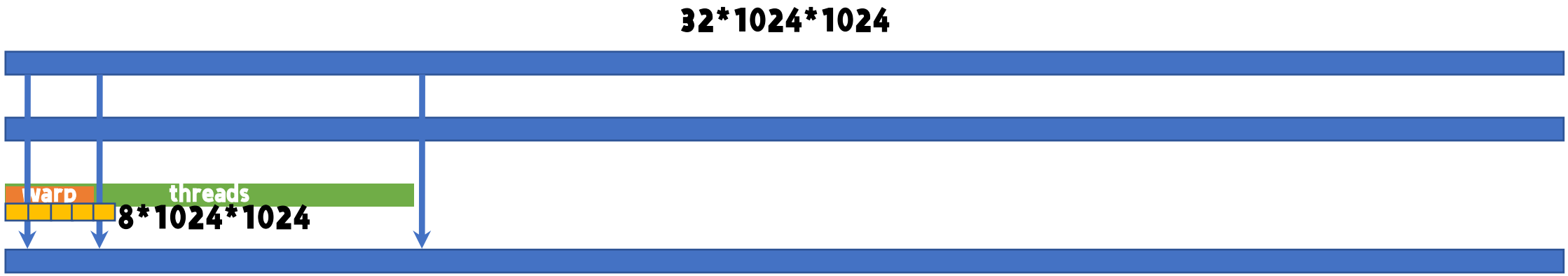


FIGURE 4-14

[CUDA编程学习笔记-03(内存访问) - 知乎]

[CUDA学习(二)矩阵转置及优化(合并访问、共享内存、bank conflict) - 知乎]





L1/TEX Cache

	Instructions	Requests	Wavefronts	% Peak	Sectors	Sectors/Req	Hit Rate	Bytes	Sector Misses to L2	% Peak to L2
Global Load	524288	524288			2621440	5	13.36	83886080	2271436	11.25
Global Load To Shared Store (access)	0	0	559240	2.77	0	0		0		
Global Load To Shared Store (bypass)	0	0			0	0	-	0	0	0
Surface Load	0	0	0	0	0	0	0	0	0	0
Texture Load	0	0	0	0	0	0	0	0	0	0
Global Store	262144	262144	279620	1.38	1310720	5	30.26	41943040	1226067	6.07
Local Store	0	0	0	0	0	0	0	0		

```
void __global__ add2(float *x, float *y, float *z)
{
  int n = threadIdx.x + blockIdx.x * blockDim.x + 1;
  z[n] = x[n] + y[n];
}
```

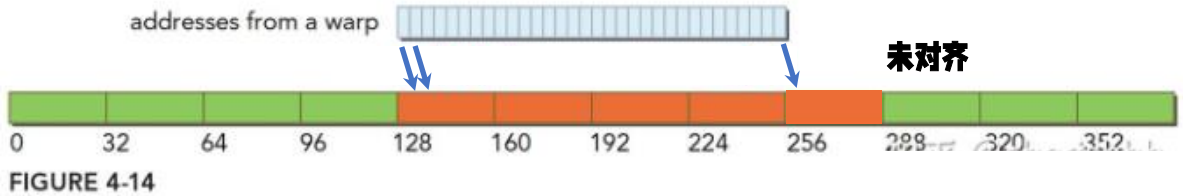


FIGURE 4-14

[CUDA编程学习笔记-03(内存访问) - 知乎]

[CUDA学习(二)矩阵转置及优化(合并访问、共享内存、bank conflict) - 知乎]

L1TEX Global Load Access Pattern
Est. Speedup: 0.21%

The memory access pattern for global loads in L1TEX might not be optimal. On average, this kernel accesses 4.0 bytes per thread per memory request; but the address pattern, possibly caused by the stride between threads, results in 5.0 sectors per request, or 5.0*32 = 160.0 bytes of cache data transfers per request. The optimal thread address pattern for 4.0 byte accesses would result in 4.0*32 = 128.0 bytes of cache data transfers per request, to maximize L1TEX cache performance. Check the Source Counters section for uncoalesced global loads.

全局加载在L1TEX中的内存访问模式可能不是最优的。平均而言，此内核每个线程每个内存请求访问4.0字节；但是由线程之间的步长引起的地址模式导致每个请求5.0个扇区，或者每个请求5.0*32 = 160.0字节的缓存数据传输。为了最大化L1TEX缓存的性能，对于4.0字节的访问，最优的线程地址模式将导致4.0*32 = 128.0字节的缓存数据传输每个请求。检查源计数器部分以获取未合并的全局加载信息。

L1TEX Global Store Access Pattern
Est. Speedup: 0.21%

The memory access pattern for global stores in L1TEX might not be optimal. On average, this kernel accesses 4.0 bytes per thread per memory request; but the address pattern, possibly caused by the stride between threads, results in 5.0 sectors per request, or 5.0*32 = 160.0 bytes of cache data transfers per request. The optimal thread address pattern for 4.0 byte accesses would result in 4.0*32 = 128.0 bytes of cache data transfers per request, to maximize L1TEX cache performance. Check the Source Counters section for uncoalesced global stores.

L2 Load Access Pattern
Est. Speedup: 4.62%

The memory access pattern for loads from L1TEX to L2 is not optimal. The granularity of an L1TEX request to L2 is a 128 byte cache line. That is 4 consecutive 32-byte sectors per L2 request. However, this kernel only accesses an average of 3.0 sectors out of the possible 4 sectors per cache line. Check the Source Counters section for uncoalesced loads and try to minimize how many cache lines need to be accessed per memory request.

从L1TEX到L2的加载的内存访问模式不是最优的。L1TEX对L2的请求的粒度是一个128字节的缓存行。也就是说，每个L2请求有4个连续的32字节扇区。然而，此内核平均只访问了每个缓存行中可能的4个扇区中的3.0个扇区。检查“源计数器”部分以获取未合并的加载信息，并尽量减少每个内存请求需要访问的缓存行数量。

L2 Store Access Pattern
Est. Speedup: 6.71%

The memory access pattern for stores from L1TEX to L2 is not optimal. The granularity of an L1TEX request to L2 is a 128 byte cache line. That is 4 consecutive 32-byte sectors per L2 request. However, this kernel only accesses an average of 2.8 sectors out of the possible 4 sectors per cache line. Check the Source Counters section for uncoalesced stores and try to minimize how many cache lines need to be accessed per memory request.

DRAM Excessive Read Sectors
Est. Speedup: 80.41%

The memory access pattern for loads from device memory causes 2,097,154 sectors to be read from DRAM, which is 1.0x of the 2,096,186 sectors which cause a miss in the L2 cache. The DRAM fetch granularity for read misses in L2 is 64 bytes, i.e. the lower or upper half of an L2 cache line. Try changing your access pattern to make use of both sectors returned by a DRAM read request for optimal usage of the DRAM throughput. For strided memory reads, avoid strides of 64 bytes or larger to avoid moving unused sectors from DRAM to L2.

从设备内存加载数据的内存访问模式导致从DRAM中读取了2,097,154个扇区，这是导致L2缓存未命中的2,096,186个扇区的1.0倍。L2缓存中读未命中的DRAM提取粒度是64字节，即L2缓存行的上半部分或下半部分。尝试更改您的访问模式，以充分利用DRAM读取请求返回的两个扇区，以实现DRAM吞吐量的最佳使用。对于跨步内存读取，避免使用64字节或更大的步长，以避免将未使用的扇区从DRAM移动到L2。

L1/TEX Cache

	Instructions	Requests	Wavefronts	% Peak	Sectors	Sectors/Req	Hit Rate	Bytes
Local Load	0	0	0	0	0	0	0	0
Global Load	524288	524288			2621440	5	16.53	83886080
Global Load To Shared Store (access)	0	0	559240	2.40	0	0		0
Global Load To Shared Store (bypass)	0	0			0	0	-	0
Surface Load	0	0	0	0	0	0	0	0
Texture Load	0	0	0	0	0	0	0	0
Global Store	262144	262144	279620	1.20	1310720	5	37.45	41943040

32*1024*1024





虽然你希望访问的是Global Memory，但是还是要过L1 L2

一个sector是32Byte（最小的读取单位）

L1/TEX Cache

	Instructions	Requests	Wavefronts	% Peak	Sectors	Sectors/Req	Hit Rate	Bytes	Sector Misses to L2	% Peak to L2
Global Load	524288	524288			2097152	4	0	67108864	2097152	14.58
Global Load To Shared Store (access)	0	0	524288	3.65	0	0		0		
Global Load To Shared Store (bypass)	0	0			0	0	-	0	0	0
Surface Load	0	0	0	0	0	0	0	0	0	0
Texture Load	0	0	0	0	0	0	0	0	0	0
Global Store	262144	262144	262144	1.82	1048576	4	0	33554432	1048576	7.29

```
void __global__ add3(float *x, float *y, float *z)
{
  int tid_permuted = threadIdx.x ^ 0x1;
  int n = tid_permuted + blockIdx.x * blockDim.x;
  z[n] = x[n] + y[n];
}
```

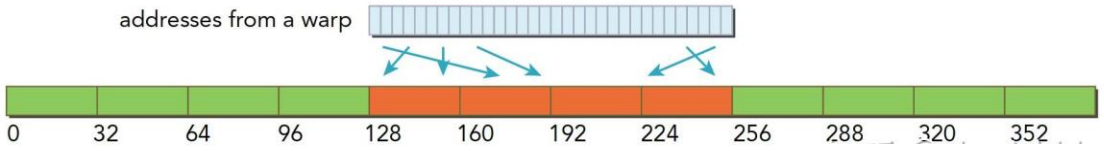


FIGURE 4-15

CUDA编程学习笔记-03(内存访问) - 知乎

CUDA学习(二)矩阵转置及优化(合并访问、共享内存、bank conflict) - 知乎

32*1024*1024





L1/TEX Cache

	Instructions	Requests	Wavefronts	% Peak	Sectors	Sectors/Req	Hit Rate	Bytes	Sector Misses to L2	% Peak to L2
Global Load	524288	524288			524288	1	60.23	16777216	207228	2.38
Global Load To Shared Store (access)	0	0	524288	6.02	0	0		0		
Global Load To Shared Store (bypass)	0	0			0	0	-	0	0	0
Surface Load	0	0	0	0	0	0	0	0	0	0
Texture Load	0	0	0	0	0	0	0	0	0	0
Global Store	262144	262144	262144	3.01	262144	1	66.09	8388608	107048	1.23
Local Store	0	0	0	0	0	0	0	0		

```
void __global__ add4(float *x, float *y, float *z)
{
  int n = threadIdx.x + blockIdx.x * blockDim.x;
  int warp_idx = n / 32;
  z[warp_idx] = x[warp_idx] + y[warp_idx];
}
```

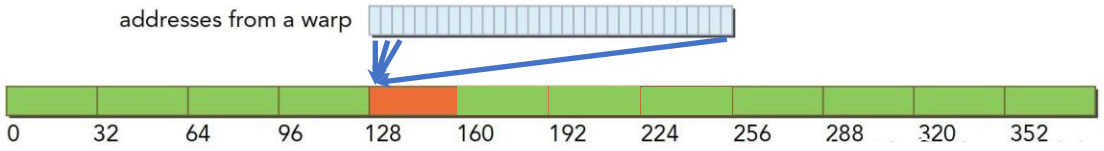


FIGURE 4-15

CUDA编程学习笔记-03(内存访问) - 知乎

CUDA学习(二)矩阵转置及优化(合并访问、共享内存、bank conflict) - 知乎

32*1024*1024



8*1024*1024



Load 8*1024*1024个float
Load 8*1024*1024个float
Store 8*1024*1024个float

虽然你希望访问的是Global Memory，但是还是要过L1 L2

一个sector是32Byte（最小的读取单位）

L1/TEX Cache

	Instructions	Requests	Wavefronts	% Peak	Sectors	Sectors/Req	Hit Rate	Bytes	Sector Misses to L2	% Peak to L2
Global Load	524288	524288			8388608	16	0	268435456	8388608	11.94
Global Load To Shared Store (access)	0	0	544591	0.77	0	0		0		
Global Load To Shared Store (bypass)	0	0			0	0	-	0	0	0
Surface Load	0	0	0	0	0	0	0	0	0	0
Texture Load	0	0	0	0	0	0	0	0	0	0
Global Store	262144	262144	271260	0.39	4194304	16	0	134217728	4194304	5.97
Local Store	0	0	0	0	0	0	0	0		
Surface Store	0	0	0	0	0	0	0	0	0	0

```
void __global__ add5(float *x, float *y, float *z)
{
  int n = (threadIdx.x + blockIdx.x * blockDim.x) * 4;
  z[n] = x[n] + y[n];
}
```

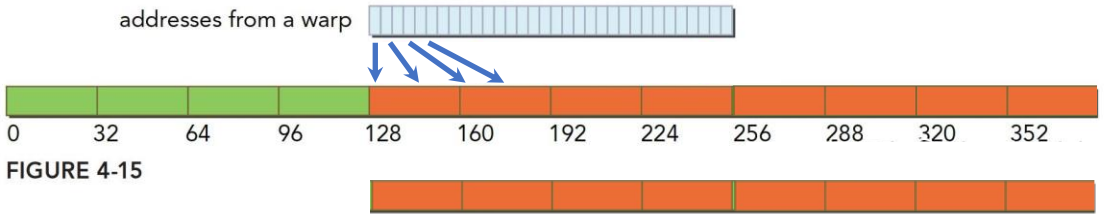
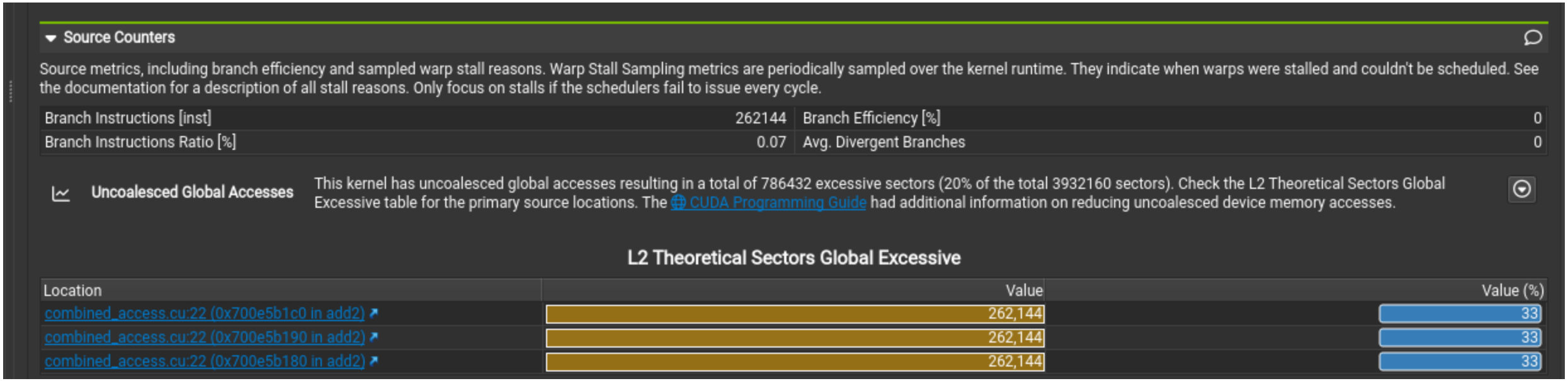


FIGURE 4-15

CUDA编程学习笔记-03(内存访问) - 知乎

CUDA学习(二)矩阵转置及优化(合并访问、共享内存、bank conflict) - 知乎

这个内核存在未合并的全局访问，导致总共出现了9437184个过度扇区（占总扇区数12582912的75%）。请检查L2理论扇区全局过度表格以了解主要源位置。CUDA编程指南中有关于减少未合并设备内存访问的额外信息。



Source Counters

Source metrics, including branch efficiency and sampled warp stall reasons. Warp Stall Sampling metrics are periodically sampled over the kernel runtime. They indicate when warps were stalled and couldn't be scheduled. See the documentation for a description of all stall reasons. Only focus on stalls if the schedulers fail to issue every cycle.

Branch Instructions [inst]	262144	Branch Efficiency [%]	0
Branch Instructions Ratio [%]	0.07	Avg. Divergent Branches	0

Uncoalesced Global Accesses

This kernel has uncoalesced global accesses resulting in a total of 786432 excessive sectors (20% of the total 3932160 sectors). Check the L2 Theoretical Sectors Global Excessive table for the primary source locations. The CUDA Programming Guide had additional information on reducing uncoalesced device memory accesses.

L2 Theoretical Sectors Global Excessive

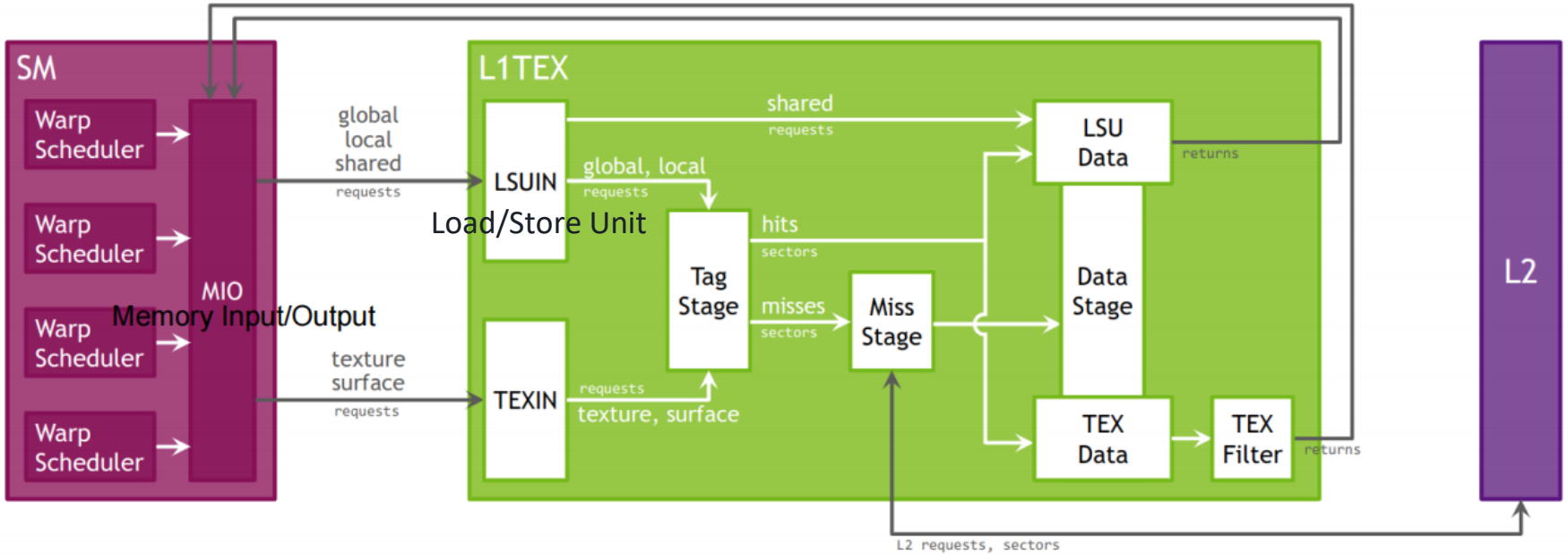
Location	Value	Value (%)
combined_access.cu:22 (0x700e5b1c0 in add2)	262,144	33
combined_access.cu:22 (0x700e5b190 in add2)	262,144	33
combined_access.cu:22 (0x700e5b180 in add2)	262,144	33



ncu与kernel调优指南

缓存&访存流程

数据从全局内存到SM的传输过程中，会去cache中查询是否有缓存。如果cache命中，则下一次的访问时间会大幅减少。在启用了L1缓存的情况下，对全局内存的访问将经过L1 Cache；如果未命中，则会接着从L2缓存中查找；如果再次未命中，则会从全局内存DRAM中读取。

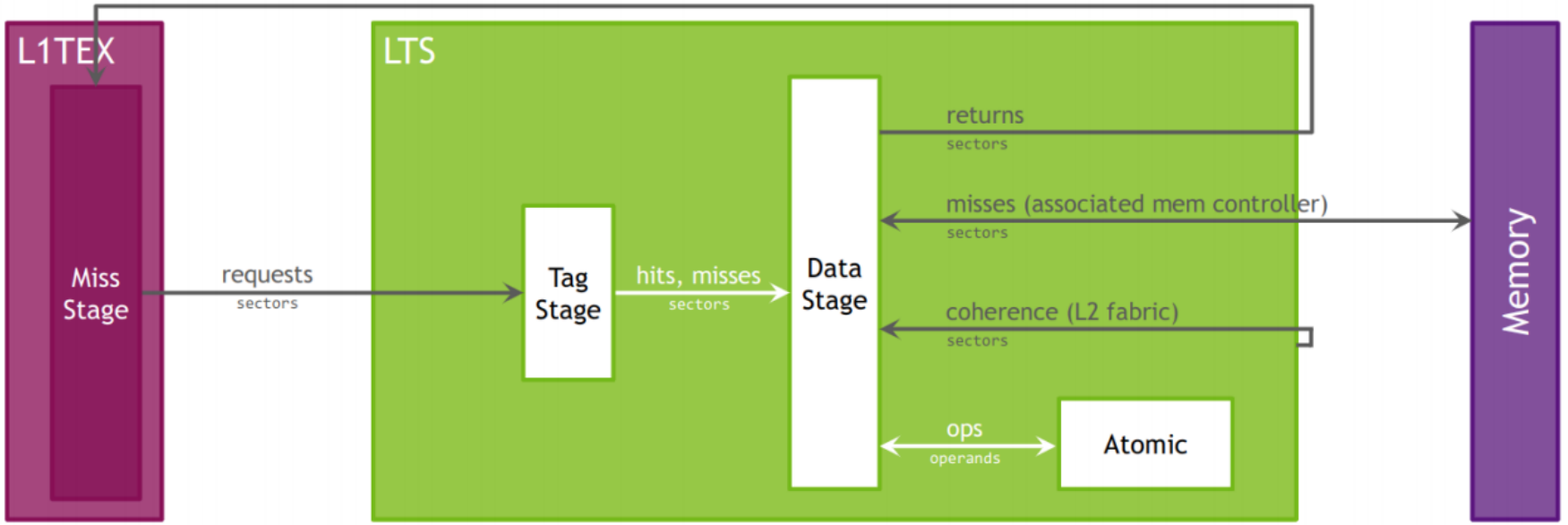


Model of Load/Store and Texture pipelines for the L1TEX cache.

[2. Kernel Profiling Guide — NsightCompute 12.6 documentation]

[CUDA笔记 内存合并访问 - 知乎]

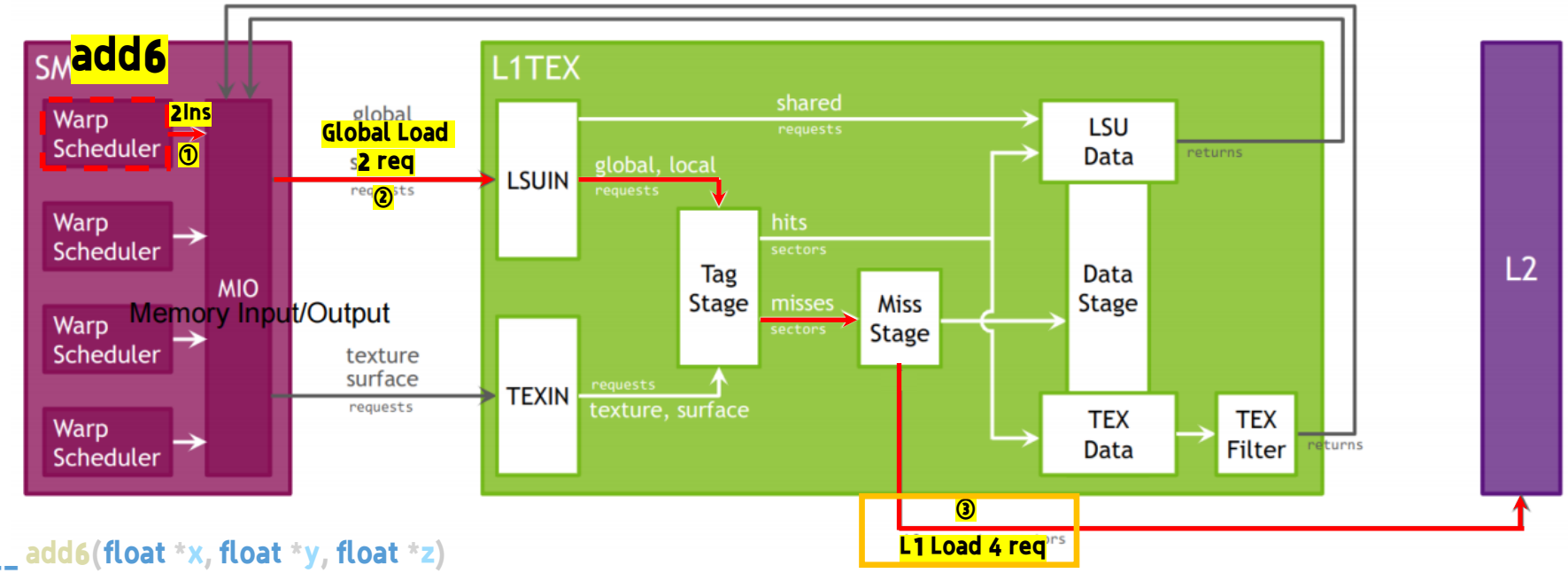
数据从全局内存到SM的传输过程中，会去cache中查询是否有缓存。如果cache命中，则下一次的访问时间会大幅减少。在启用了L1缓存的情况下，对全局内存的访问将经过L1 Cache；如果未命中，则会接着从L2缓存中查找；如果再次未命中，则会从全局内存DRAM中读取。



Model of the L2 cache.

[2. Kernel Profiling Guide — NsightCompute 12.6 documentation]

[CUDA笔记 内存合并访问 - 知乎]




```
void __global__ add6(float *x, float *y, float *z)
{
  if (blockIdx.x == 0 && threadIdx.x < 32)
  {
    int n = threadIdx.x + blockIdx.x * blockDim.x + 1;
    z[n] = x[n] + y[n];
  }
}
```

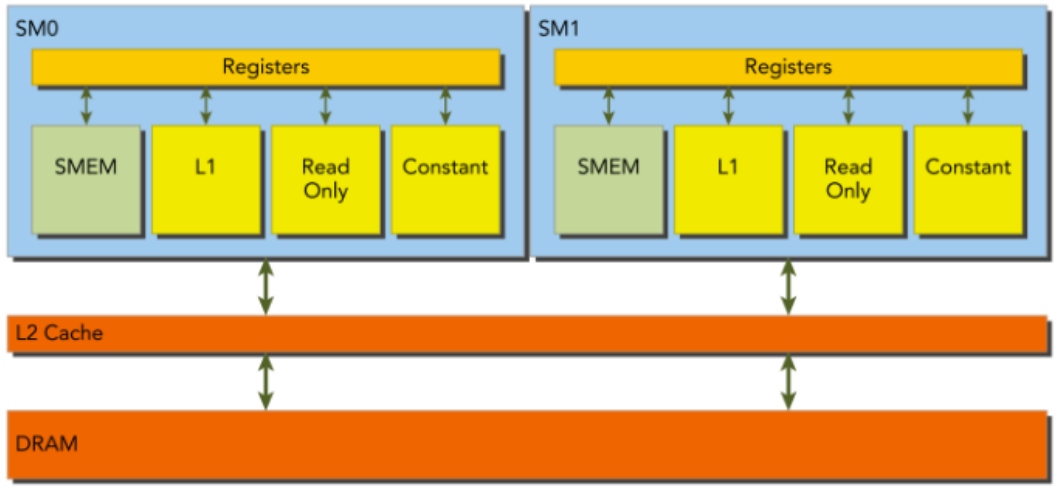
对于每种访问类型，向L2缓存发起的总请求数量。这与L1缓存的扇区缺失到L2的相关。每个请求最多访问来自单个128字节缓存行的四个扇区。

L2 Cache

	Requests	Sectors	Sectors/Req	% Peak	Hit Rate	Bytes	Throughput	Sector Misses to Device
L1/TEX Load	4	10	2.50	0.00	0	320	4338394.79	10
L1/TEX Store	2	5	2.50	0.00	100	160	2169197.40	0

2. Kernel Profiling Guide — NsightCompute 12.6 documentation

2.1 对齐合并访问



DRAM

全局内存是从逻辑角度的叫法，实际的全局内存就是设备的DRAM，从上图可以发现，所有对全局内存的访问都会经过L2 Cache，有部分会经过L1 Cache。因为CUDA可以通过指令来选择是否启用L1 Cache，由此每次**访问全局内存**有两种**粒度**：**同时使用L1 Cache 和 L2 Cache则粒度为128字节和只使用L2 Cache则粒度为32字节**

在对CUDA进行编译时，可以通过如下标志通知nvcc是否使用一级缓存

```
# 禁用一级缓存
-Xptxas -dlcm=cg
# 启用一级缓存
-Xptxas -dlcm=ca
```

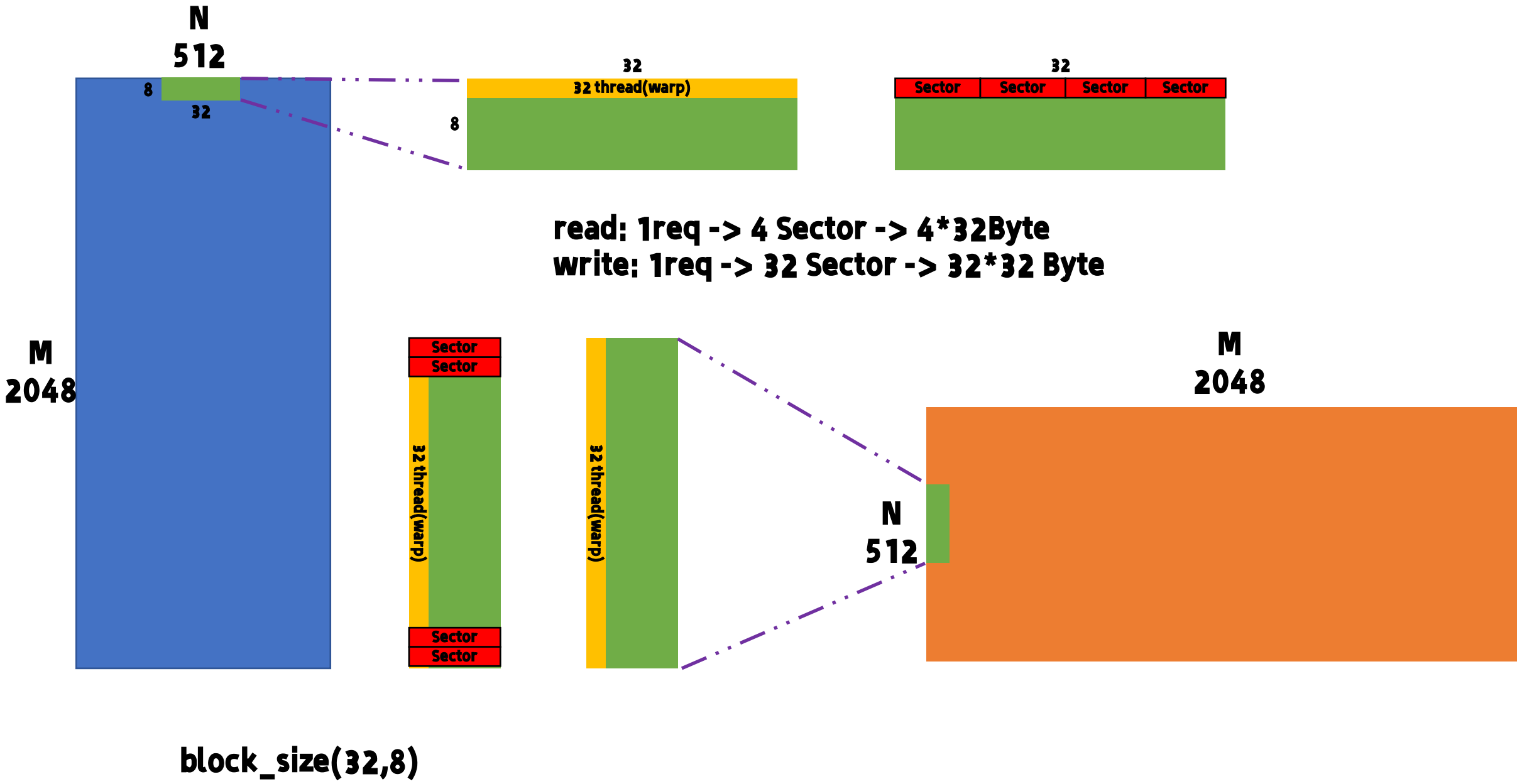
当前似乎已经无效了

[CUDA编程学习笔记-03(内存访问) - 知乎]



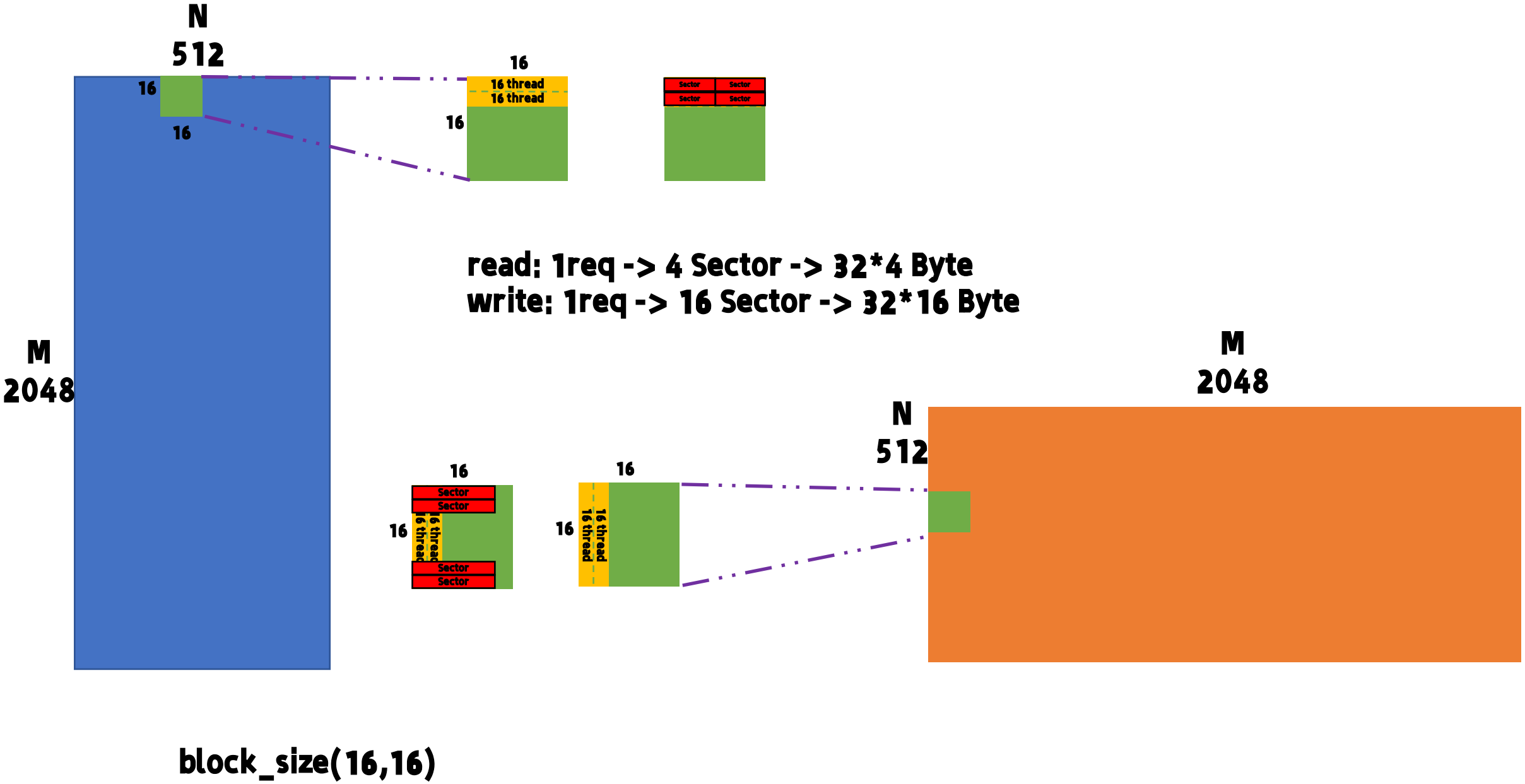
ncu与kernel调优指南 合并访存与Transpose

Transpose_v1_naive



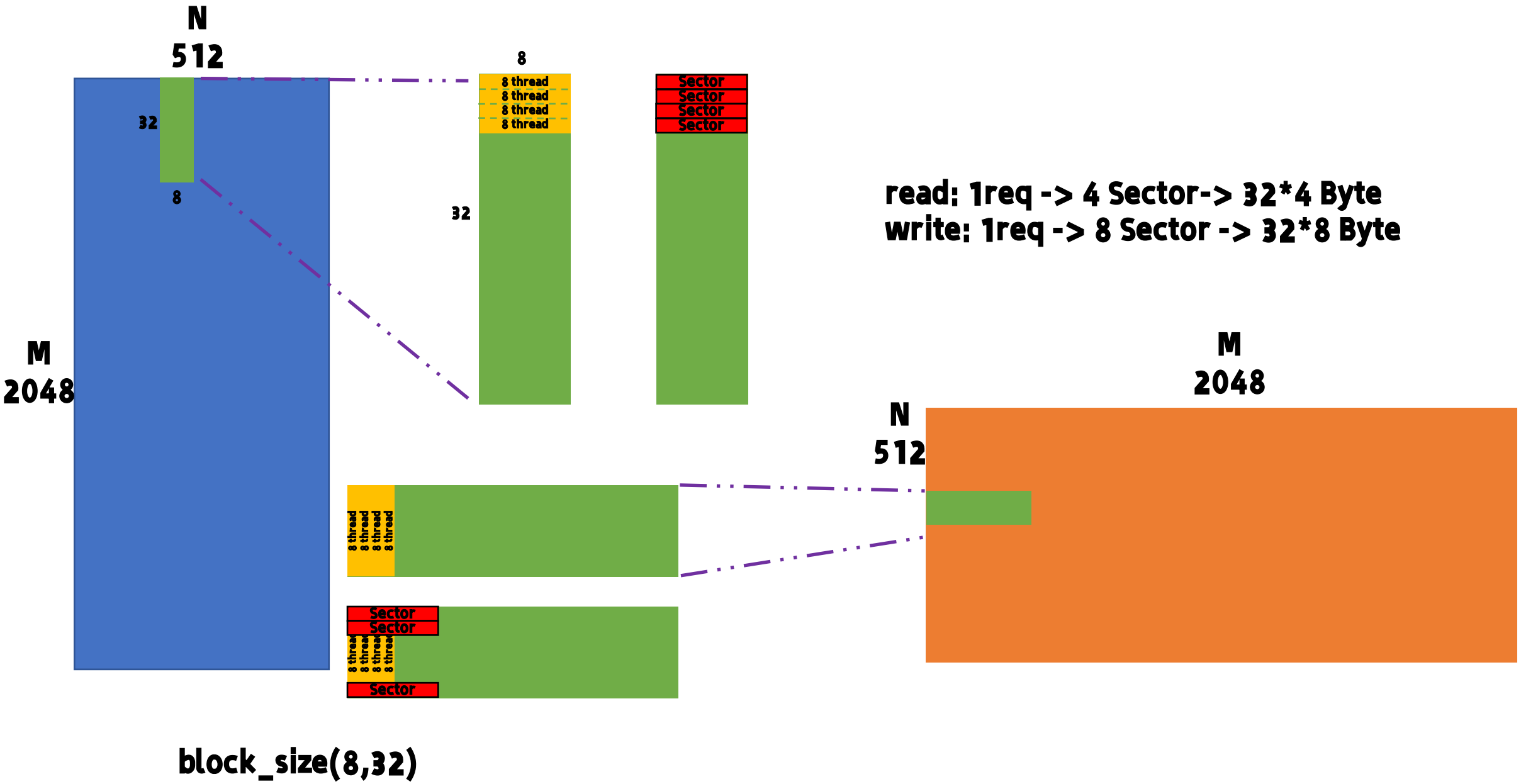
cuda性能优化笔记--Transpose性能优化 - 知乎

Transpose_v1_naive



cuda性能优化笔记--Transpose性能优化 - 知乎

Transpose_v1_naive

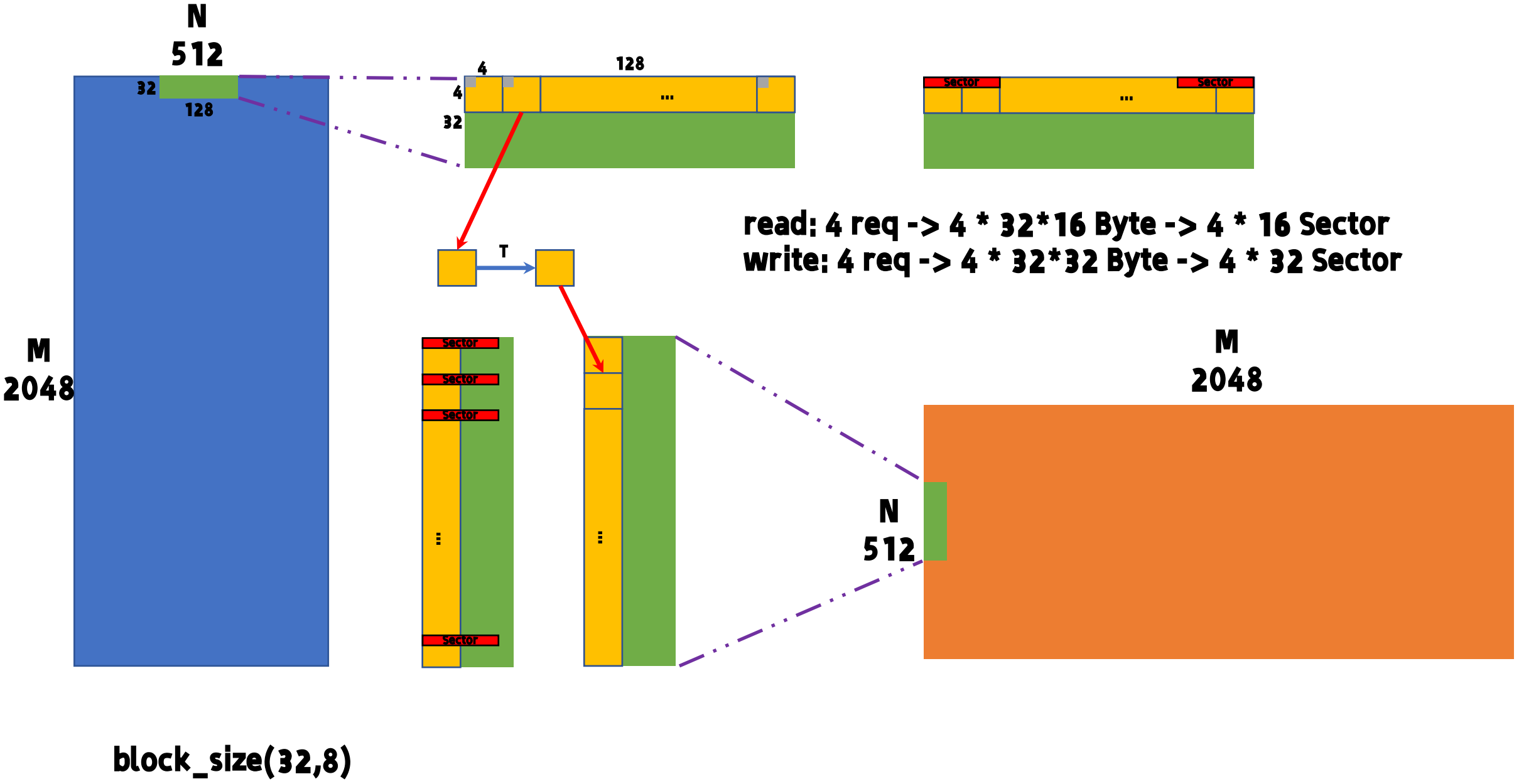


read: 1req -> 4 Sector-> 32*4 Byte
write: 1req -> 8 Sector -> 32*8 Byte

block_size(8,32)

cuda性能优化笔记--Transpose性能优化 - 知乎

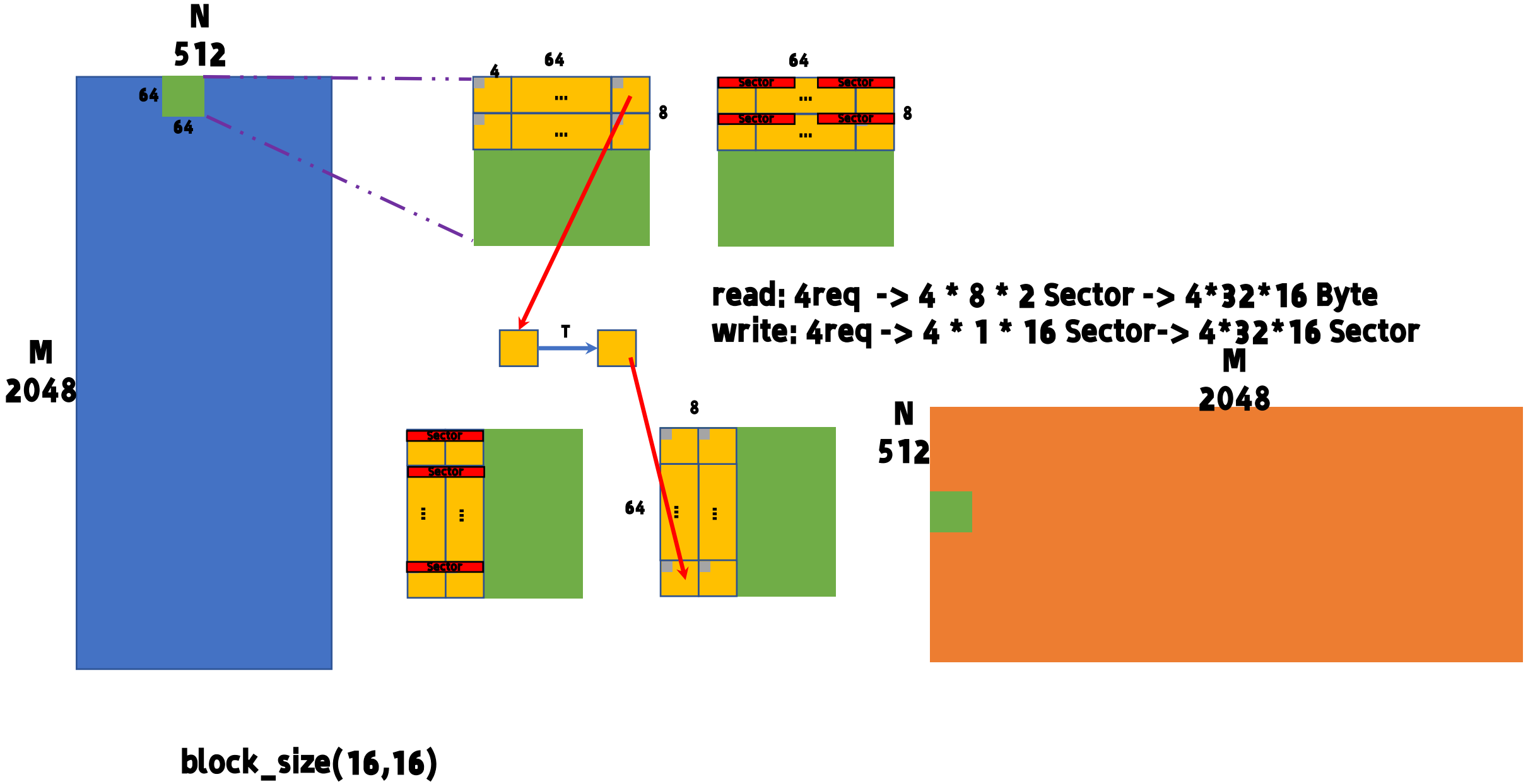
Transpose_v2_float4



block_size(32,8)

cuda性能优化笔记--Transpose性能优化 - 知乎

Transpose_v2_float4



read: 4req -> 4 * 8 * 2 Sector -> 4*32*16 Byte
write: 4req -> 4 * 1 * 16 Sector-> 4*32*16 Sector

block_size(16,16)

[cuda性能优化笔记--Transpose性能优化 - 知乎]



ncu与kernel调优指南

Occupancy Calculator

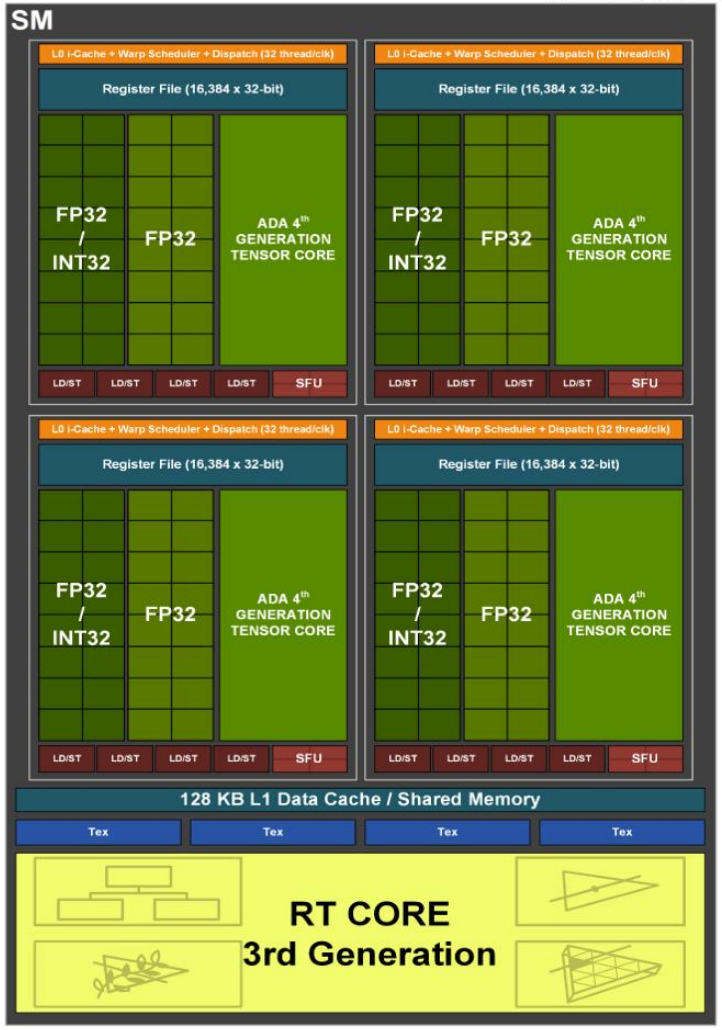


Figure 5. Ada Streaming Multiprocessor (SM)

occupancy初步理解

- occupancy中文翻译是占用率，一个SM中理论假如最大支持M个active warp（这里不理解什么是active warp可以参考这里），而实际上因为各种原因只能支持N个，这里N/M就是occupancy。

Active Warp（活跃线程束）：

- 在任何给定的时间点，GPU的SM（流处理器簇，Streaming Multiprocessors）可以同时处理多个warp。但是，由于资源限制（如执行单元、寄存器、共享内存等），并不是所有的warp都能同时活跃。
- Active warp指的是那些正在执行指令的warp。它们可能处于不同的执行阶段，比如正在执行计算任务、正在等待内存访问结果或正在执行控制流指令。

CUDA Occupancy Calculator占用率计算与显卡算力计算_cuda计算能力7.5-CSDN博客
GPU基础：Occupancy、wave and tail effect - 知乎
CUDA中Occupancy相关知识_cuda occupancy-CSDN博客

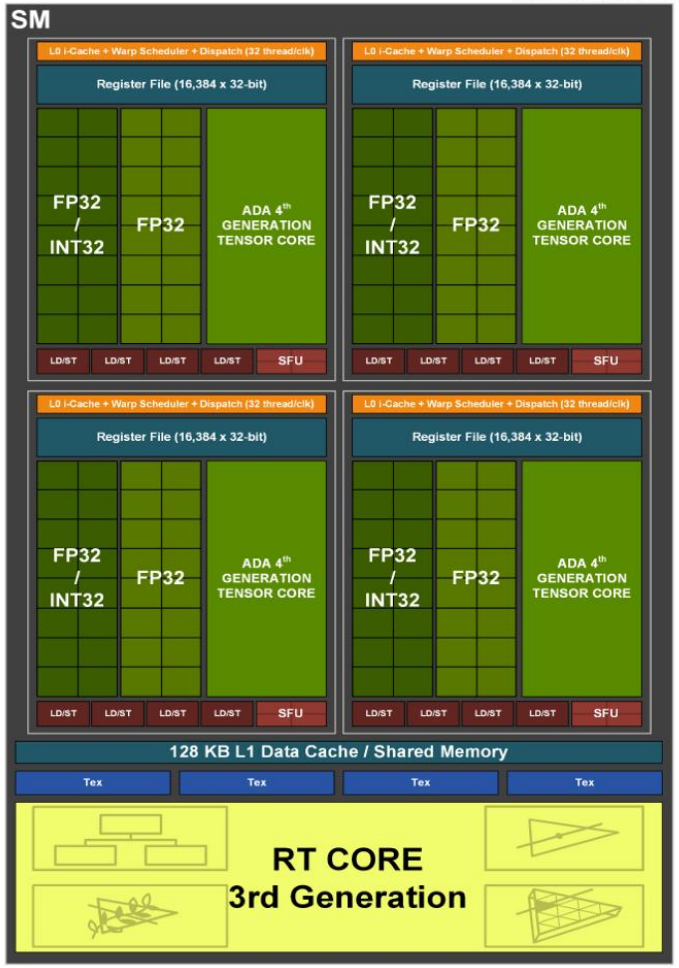


Figure 5. Ada Streaming Multiprocessor (SM)

以左侧图为例

可看出一个SM中有128个Core，所以至少是可以放下128个Thread（4个warp）为了隐藏延时，其实可以放进去更多warp（大于4个）根据显卡的架构不同，这个更多的warp仍然有一个理论上限。

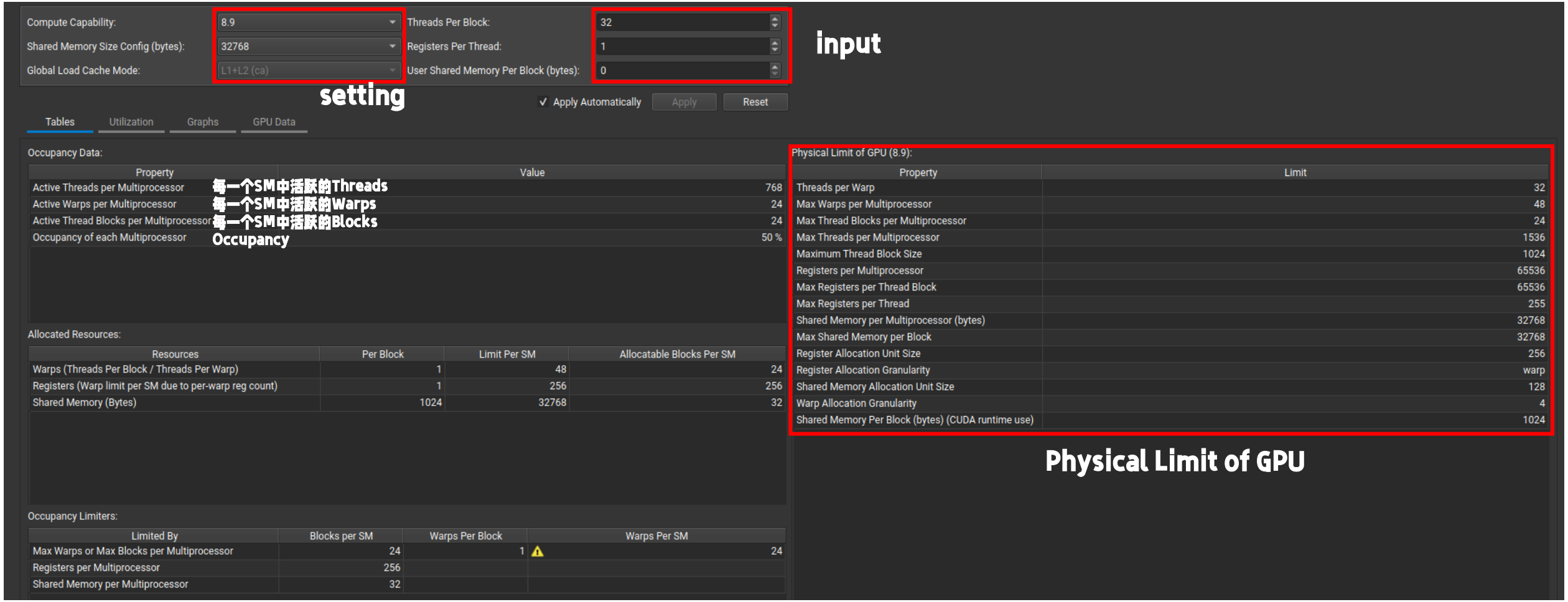
1.用户设置的block的大小会影响实际SM上可以容纳的warp数量

2.CUDA隐藏延迟的手段就是当一个warp处于等待状态的时候会立刻切到下一个warp，而这个立刻就要求线程的切换要特别的迅速（0开销），这与CPU多线程有很大的区别，CPU在进行thread切换的时候，是有开销的（要保留每个thread的上下文），但是GPU不需要因为它有大量的寄存器，每个thread的寄存器数据都是自己用所以就没有切换上下文的需要，自然也可以做到0开销，而与此同时每个SM中的寄存器数量是有上限的所以寄存器也会成为能开启多少个warp的一个衡量指标

3.共享内存和寄存器类似，每个SM只有固定大小，所以也影响warp（block块）的数量。

123再结合木桶效应，取最小值就可以得到每个SM能容纳的warp数量。

CUDA Occupancy Calculator



block_size对SM可启动warp数量的影响

Physical Limit of GPU (8.9):

Property	Limit
Threads per Warp	32
Max Warps per Multiprocessor	48
Max Thread Blocks per Multiprocessor	24
Max Threads per Multiprocessor	1536
Maximum Thread Block Size	1024
Registers per Multiprocessor	65536
Max Registers per Thread Block	65536
Max Registers per Thread	255
Shared Memory per Multiprocessor (bytes)	16384
Max Shared Memory per Block	16384
Register Allocation Unit Size	256
Register Allocation Granularity	warp
Shared Memory Allocation Unit Size	128
Warp Allocation Granularity	4
Shared Memory Per Block (bytes) (CUDA runtime use)	1024

1. 每一个Block必须分配到一个SM中
2. 每一个SM最多可放48个warp(1536个thread)
3. 每一个SM最多可放24个Block

假设需要计算的数据足够多，每个thread都只使用很少的smem和register，设置的Block_Size为(32,5)

- 根据2可得最多能放48个warp相当于9.6个Block
- 根据1可得0.6个Block是不能被分配到一个SM上的，所以只能放9个Block，相当于45个warp，所以occupany为0.9375=45/48

Allocated Resources:

Resources	Per Block	Limit Per SM	Allocatable Blocks Per SM
Warps (Threads Per Block / Threads Per Warp)	5	48	9
Registers (Warp limit per SM due to per-warp reg count)	5	256	51
Shared Memory (Bytes)	1024	65536	64

block_size对SM可启动warp数量的影响

Physical Limit of GPU (8.9):

Property	Limit
Threads per Warp	32
Max Warps per Multiprocessor	48
Max Thread Blocks per Multiprocessor	24
Max Threads per Multiprocessor	1536
Maximum Thread Block Size	1024
Registers per Multiprocessor	65536
Max Registers per Thread Block	65536
Max Registers per Thread	255
Shared Memory per Multiprocessor (bytes)	16384
Max Shared Memory per Block	16384
Register Allocation Unit Size	256
Register Allocation Granularity	warp
Shared Memory Allocation Unit Size	128
Warp Allocation Granularity	4
Shared Memory Per Block (bytes) (CUDA runtime use)	1024

1. 每一个Block必须分配到一个SM中
2. 每一个SM最多可放48个warp(1536个thread)
3. 每一个SM最多可放24个Block

假设需要计算的数据足够多，每个thread都只使用很少的smem和register，设置的Block_Size为(32)

- 根据2可得最多能放48个warp相当于48个Block
- 根据3可得SM最多可放24个Block，所以只能放24个warp
- occupany为0.5=24/48

Allocated Resources:

Resources	Per Block	Limit Per SM	Allocatable Blocks Per SM
Warps (Threads Per Block / Threads Per Warp)	1	48	24
Registers (Warp limit per SM due to per-warp reg count)	1	256	256
Shared Memory (Bytes)	1024	65536	64

Register对SM可启动warp数量的影响

Physical Limit of GPU (8.9):

Property	Limit
Threads per Warp	32
Max Warps per Multiprocessor	48
Max Thread Blocks per Multiprocessor	24
Max Threads per Multiprocessor	1536
Maximum Thread Block Size	1024
Registers per Multiprocessor	65536
Max Registers per Thread Block	65536
Max Registers per Thread	255
Shared Memory per Multiprocessor (bytes)	16384
Max Shared Memory per Block	16384
Register Allocation Unit Size	256
Register Allocation Granularity	warp
Shared Memory Allocation Unit Size	128
Warp Allocation Granularity	4
Shared Memory Per Block (bytes) (CUDA runtime use)	1024

1. 每一个SM有65536个寄存器
2. 寄存器的分配单元大小是256个
3. 寄存器的分割粒度是warp
4. warp的分配粒度是4（如果 warp Allocation Granularity 为 4，那么即使一个内核配置只需要 3 个 warp 的寄存器资源，硬件也会按照 4 个 warp 来分配寄存器资源）

假设需要计算的数据足够多，每个线程需要51个register，设置的Block_Size为(128)

- 根据3，每个thread 51 reg，所以每个warp需要51*32=1632个reg
- 根据2，1632个reg，需要按照256粒度分配，1632/256=6.375→7*256=1792，所以每个warp实际需要1792个reg
- 根据1，65536个reg，每个warp需要1792，所以65536/1792=36.57→36个warp
- occupancy=36/48=0.75

Allocated Resources:

Resources	Per Block	Limit Per SM	Allocatable Blocks Per SM
Warps (Threads Per Block / Threads Per Warp)	4	48	12
Registers (Warp limit per SM due to per-warp reg count)	4	36	9
Shared Memory (Bytes)	1024	65536	64

Register对SM可启动warp数量的影响

Physical Limit of GPU (8.9):

Property	Limit
Threads per Warp	32
Max Warps per Multiprocessor	48
Max Thread Blocks per Multiprocessor	24
Max Threads per Multiprocessor	1536
Maximum Thread Block Size	1024
Registers per Multiprocessor	65536
Max Registers per Thread Block	65536
Max Registers per Thread	255
Shared Memory per Multiprocessor (bytes)	16384
Max Shared Memory per Block	16384
Register Allocation Unit Size	256
Register Allocation Granularity	warp
Shared Memory Allocation Unit Size	128
Warp Allocation Granularity	4
Shared Memory Per Block (bytes) (CUDA runtime use)	1024

1. 每一个SM有65536个寄存器
2. 寄存器的分配单元大小是256个
3. 寄存器的分割粒度是warp
4. warp的分配粒度是4（如果 warp Allocation Granularity 为 4，那么即使一个内核配置只需要 3 个 warp 的寄存器资源，硬件也会按照 4 个 warp 来分配寄存器资源）

假设需要计算的数据足够多，每个线程需要90个register，设置的Block_Size为(128)

- 根据3，每个thread 90 reg，所以每个warp需要90*32=2880个reg
- 根据2，2880个reg，需要按照256粒度分配，2880/256=11.25→12*256=3072，所以每个warp实际需要3072个reg
- 根据1，65536个reg，每个warp需要3072，所以65536/3072=21.3333→21个warp
- 根据4，需要按照4个warp来分配，但是分配24个reg资源不够，所以只能分配20个warp
- occupancy=20/48=0.41666

Allocated Resources:

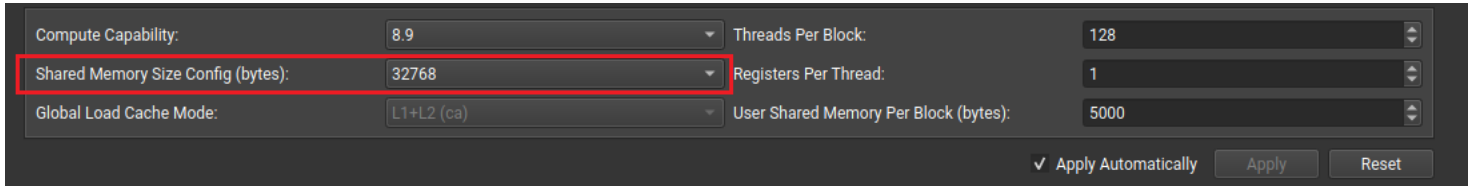
Resources	Per Block	Limit Per SM	Allocatable Blocks Per SM
Warps (Threads Per Block / Threads Per Warp)	4	48	12
Registers (Warp limit per SM due to per-warp reg count)	4	20	5
Shared Memory (Bytes)	1024	65536	64

smem对SM可启动warp数量的影响

Physical Limit of GPU (8.9):

Property	Limit
Threads per Warp	32
Max Warps per Multiprocessor	48
Max Thread Blocks per Multiprocessor	24
Max Threads per Multiprocessor	1536
Maximum Thread Block Size	1024
Registers per Multiprocessor	65536
Max Registers per Thread Block	65536
Max Registers per Thread	255
Shared Memory per Multiprocessor (bytes)	16384
Max Shared Memory per Block	16384
Register Allocation Unit Size	256
Register Allocation Granularity	warp
Shared Memory Allocation Unit Size	128
Warp Allocation Granularity	4
Shared Memory Per Block (bytes) (CUDA runtime use)	1024

1. 每一个SM有16384Byte smem
2. smem的分配单元大小是128Byte
3. 每个SM都必须分配1024Byte smem
4. smem配置的大小为32768Byte



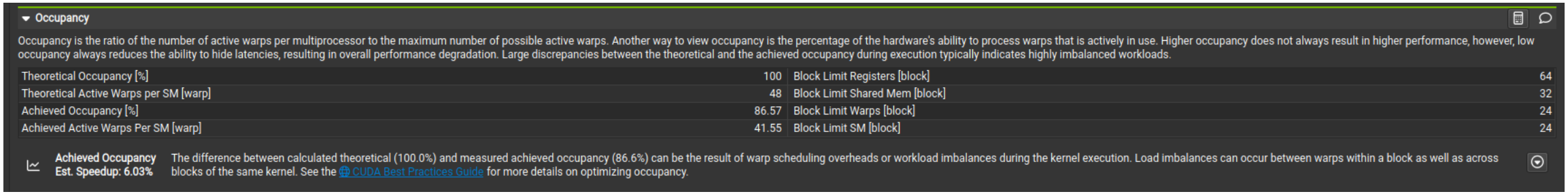
假设需要计算的数据足够多，每个线程需要5000Byte smem，设置的Block_Size为(128)

- 根据3，每个Block需要1024+5000=6024Byte smem
- 根据2，6024Byte，需要按照128Byte粒度分配，6024/128=47.06→48*128=6144，所以每个Block实际需要6144Byte
- 根据4，一共32768Byte，32768/6144=5.33，所以只能分配5个Block
- occupancy=(5*4)/48=0.41666

Allocated Resources:

Resources	Per Block	Limit Per SM	Allocatable Blocks Per SM
Warps (Threads Per Block / Threads Per Warp)	4	48	12
Registers (Warp limit per SM due to per-warp reg count)	4	256	64
Shared Memory (Bytes)	6144	32768	5

占用率是指每个多处理器上活跃线程束的数量与可能的最大活跃线程束数量之比。另一种看待占用率的方式是，将其视为硬件处理线程束的能力中正在被积极使用的那部分所占的百分比。然而，较高的占用率并不总是能带来更高的性能，不过，较低的占用率总会降低隐藏延迟的能力，进而导致整体性能下降。在执行过程中，理论占用率和实际达到的占用率之间存在较大差异，通常意味着工作负载高度不均衡。





ncu与kernel调优指南

warp scheduler

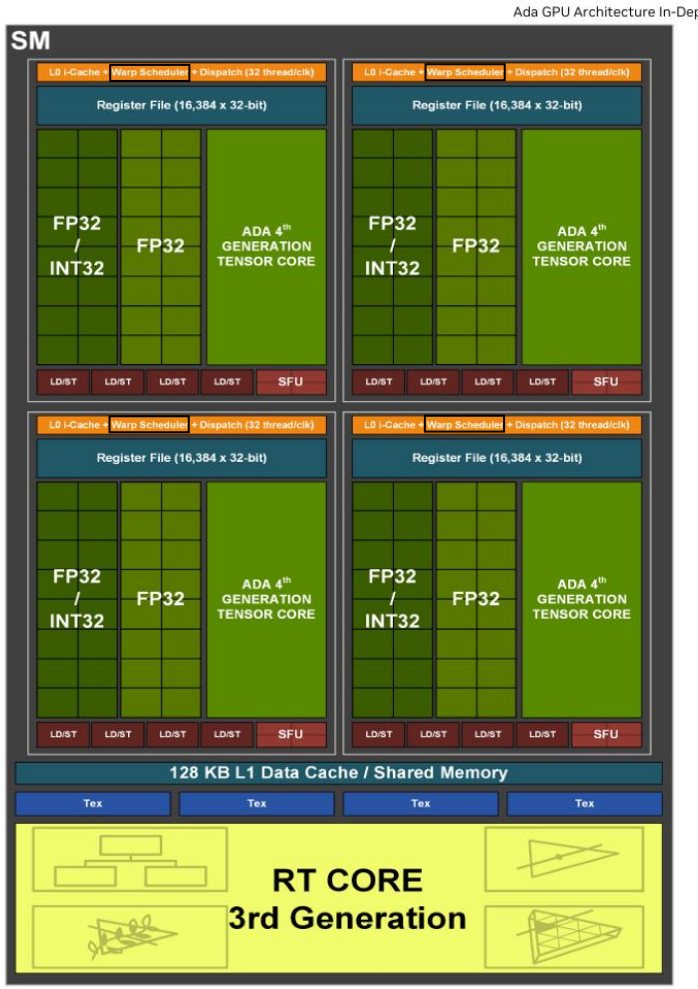
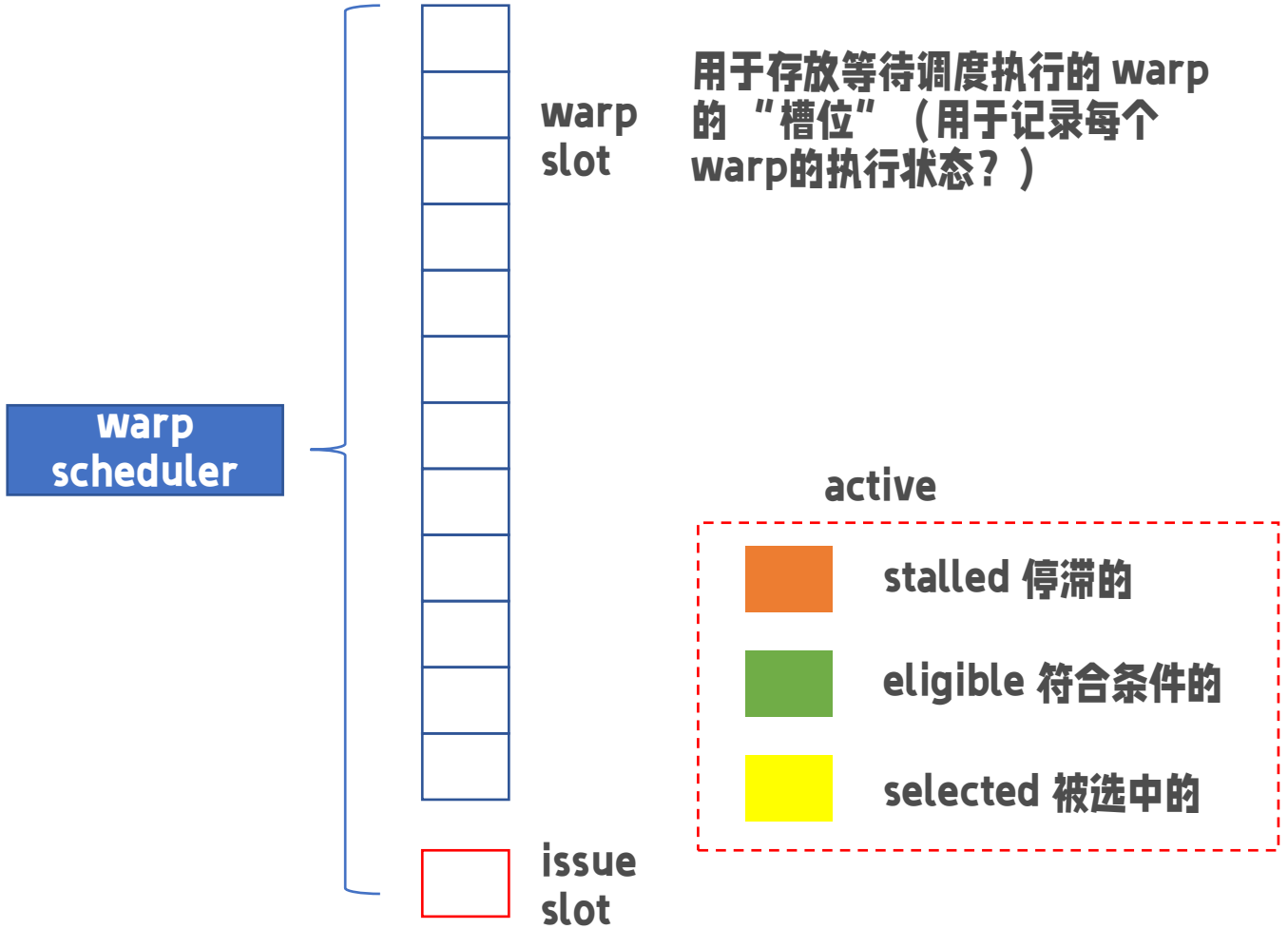


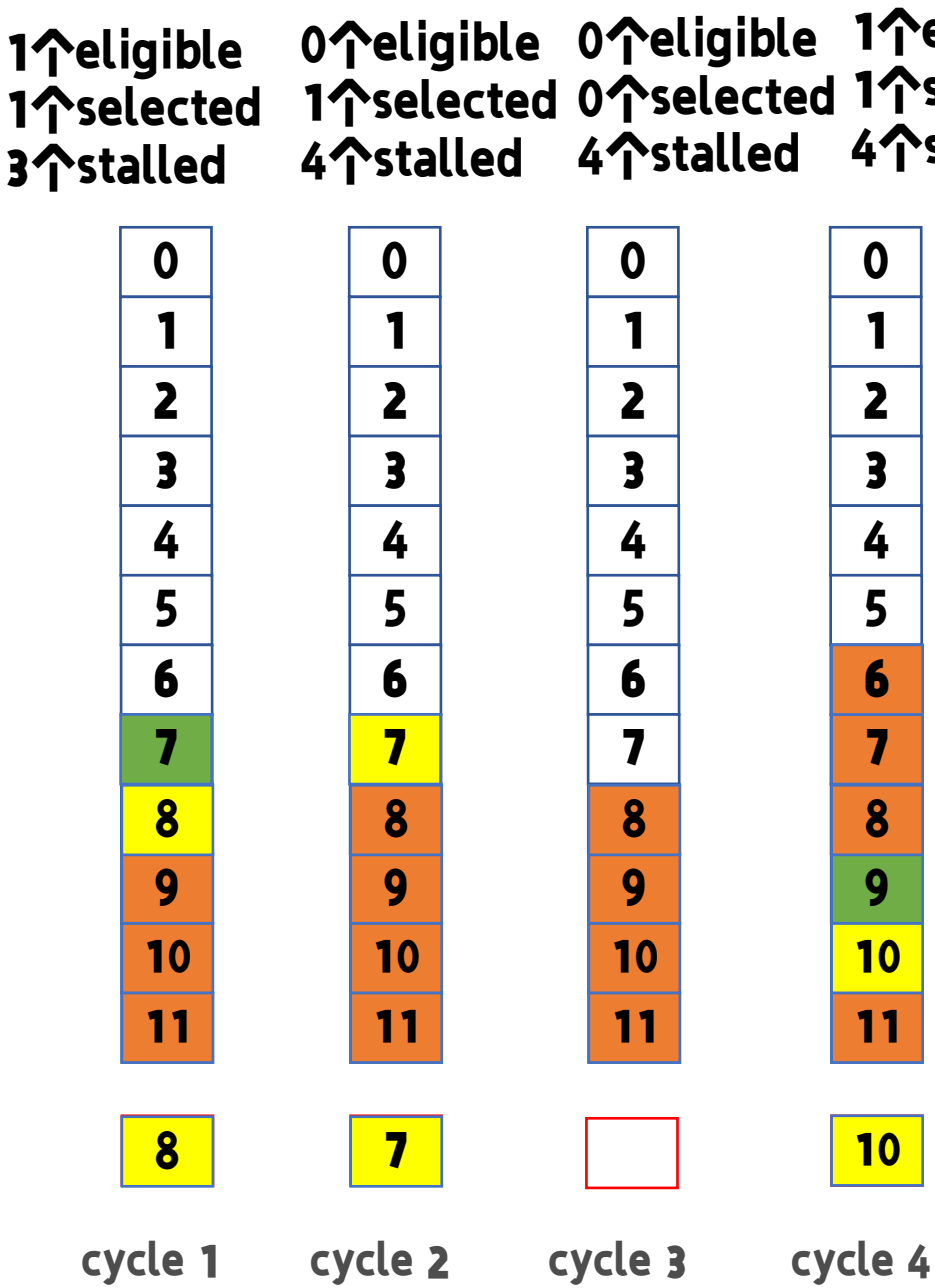
Figure 5. Ada Streaming Multiprocessor (SM)

Ada AD102 GPU SM

对于4090显卡来说，每个SM最多可以处理48个warp，而从架构图上4090上面有4个warp scheduler，所以每个warp scheduler可以负责issue 12个warp（有12个warp slots）



[NVIDIA GPU 架构与 CUDA 算力 - CuiYuhao's Blog]
[cuda的warp scheduler知识-CSDN博客]
[Nsight Compute(NCU) Scheduler Statistics 数据解读_nsys compute ncu-CSDN博客]



stalled 停滞的

eligible 符合条件的

selected 被选中的

造成stalled主要有以下5种情况：

- 在取指令
- 依赖内存指令的访存结果
- 依赖之前指令的执行结果
- pipeline正在忙
- 同步barrier

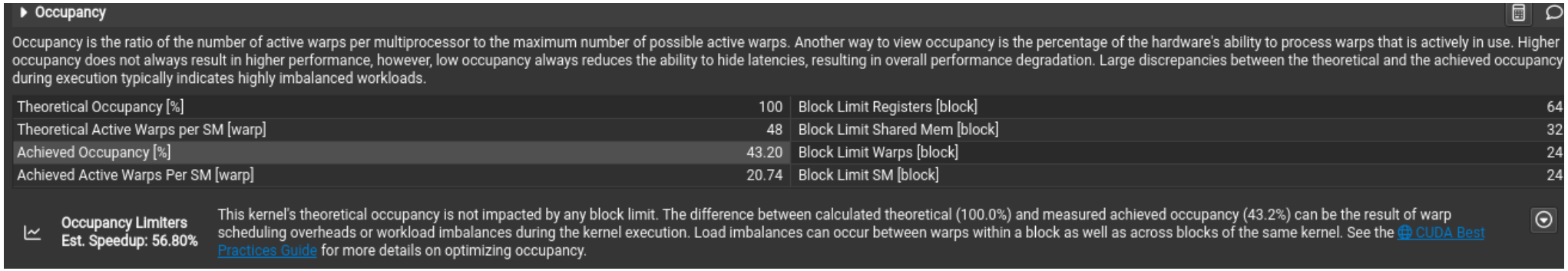
cycles_active: 4
warps_active: 20=5+5+4+6

warps_active/cycles_active: 5=20/4
achieved_occupancy: 41.67%=5/12

warps_stalled: 15
warps_eligible: 5
warps_issued: 3
warps_issued/cycle_active: 0.75=3/4
issue_slot_utilization: 0.75

【CUDA进阶】深入理解 Nsight System 和 Nsight Compute_哔哩哔哩_bilibili

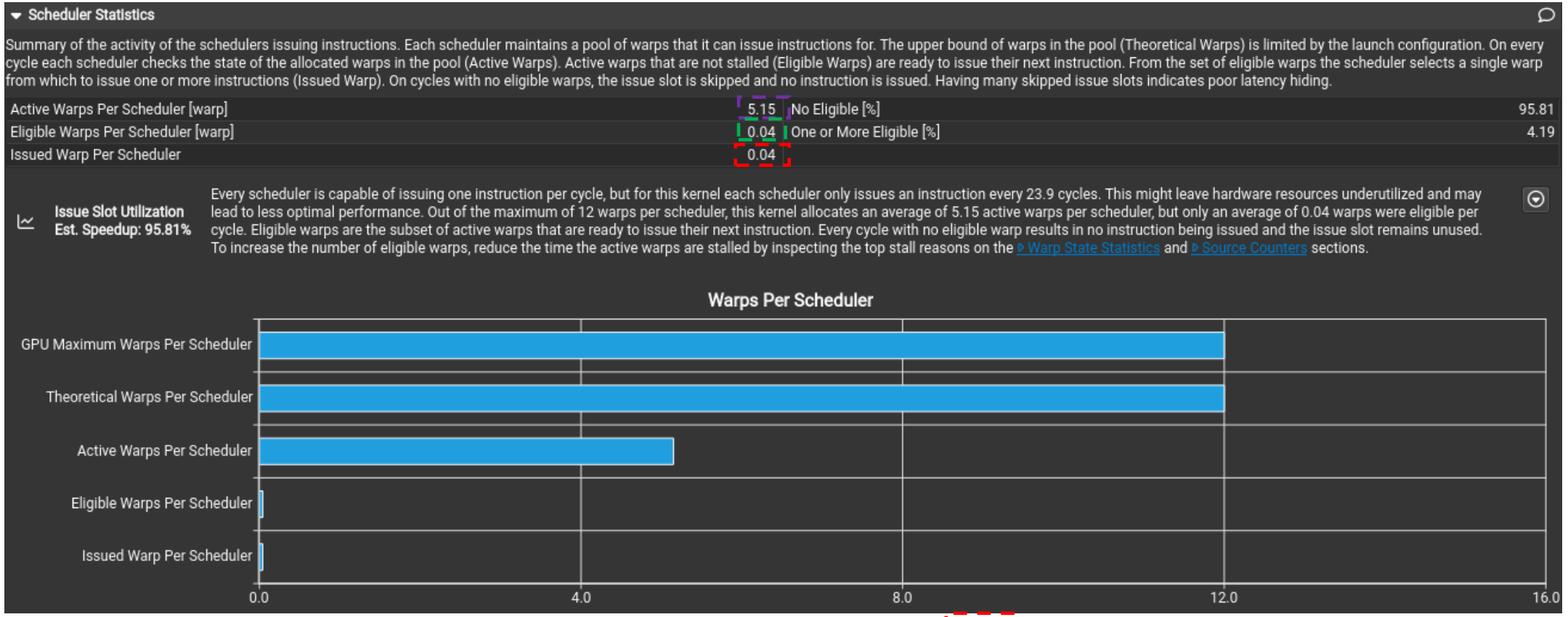
占用率是指每个多处理器上活跃线程束的数量与可能的最大活跃线程束数量之比。另一种看待占用率的方式是，将其视为硬件处理线程束的能力中正在被积极使用的那部分所占的百分比。然而，较高的占用率并不总是能带来更高的性能，不过，较低的占用率总会降低隐藏延迟的能力，进而导致整体性能下降。在执行过程中，理论占用率和实际达到的占用率之间存在较大差异，通常意味着工作负载高度不均衡。



这个内核的理论占用率不会受到任何块限制的影响。计算出的理论占用率（100.0%）与测量到的实际占用率（43.2%）之间的差异可能是由于线程束调度开销或在内核执行期间的负载不平衡造成的。负载不平衡可能发生在同一块内的线程束之间，也可能发生在同一内核的不同块之间。有关优化占用的更多详细信息，请参阅 CUDA 最佳实践指南。

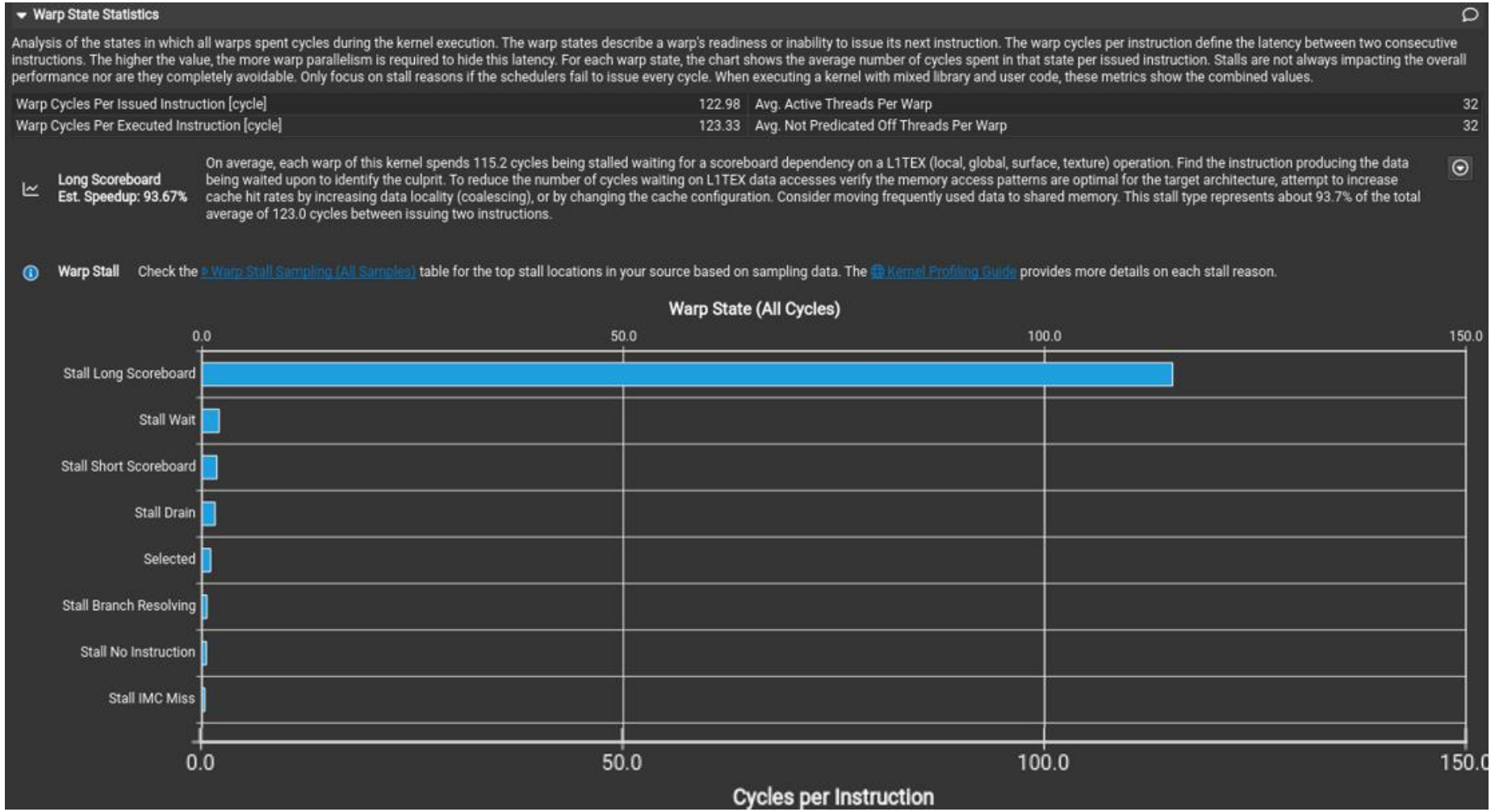
```
void __global__ add1(float *x, float *y, float *z)
{
  int n = threadIdx.x + blockIdx.x * blockDim.x;
  z[n] = x[n] + y[n];
}
```

调度器发出指令的活动摘要。每个调度器都维护一个它可以为其发出指令的线程束池。线程束池中的线程束上限（理论线程束数）受启动配置的限制。在每个周期，每个调度器都会检查池中分配的线程束的状态（活动线程束）。未停滞的活动线程束（符合条件的线程束）已准备好发出它们的下一条指令。从符合条件的线程束集合中，调度器选择一个线程束以发出一条或多条指令（已发出线程束）。在没有符合条件的线程束的周期中，指令发出槽会被跳过，且不会发出任何指令。有许多跳过的指令发出槽表明延迟隐藏效果差。



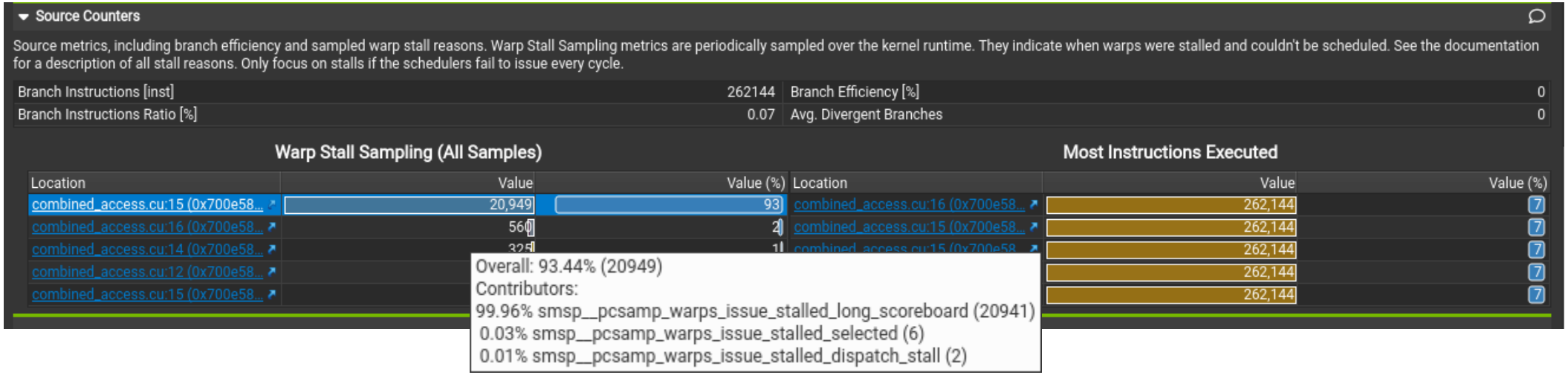
每个调度器都有能力每个周期发出一条指令，但对于这个内核，每个调度器每23.9个周期才发出一条指令。这可能会导致硬件资源未充分利用，并可能带来不太理想的表现。在每个调度器最多12个线程束的情况下，这个内核每个调度器平均分配了5.15个活动线程束，但每个周期平均只有0.04个线程束是符合条件的。符合条件的线程束是活动线程束的一个子集，它们已准备好发出下一条指令。每个周期如果没有符合条件的线程束，就不会发出指令，指令发出槽位将保持未使用。为了增加符合条件的线程束数量，可以通过检查“线程束状态统计”和“源计数器”部分中的主要停滞原因来减少活动线程束停滞的时间。

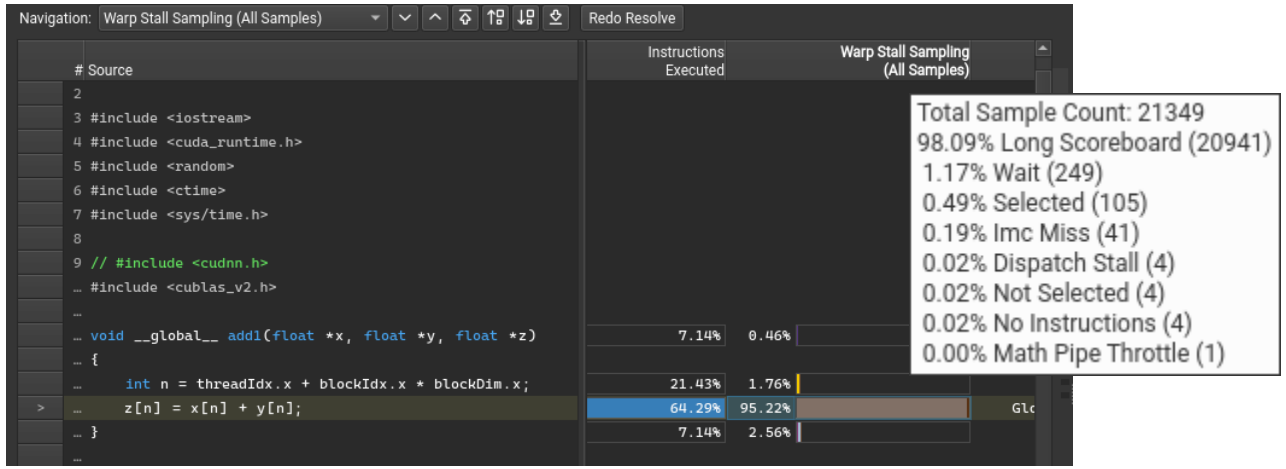
分析内核执行期间所有线程束花费周期的状态。线程束状态描述了一个线程束发出下一条指令的准备情况或无法发出指令。线程束每条指令的周期数定义了两条连续指令之间的延迟。该值越高，隐藏这种延迟所需的线程束并行度就越高。对于每个线程束状态，图表显示了在该状态下每条已发出指令平均花费的周期数。停滞并不总是影响整体性能，也不是完全可以避免的。只有当调度器无法每个周期都发出指令时，才需要关注停滞原因。当执行包含混合库代码和用户代码的内核时，这些指标显示的是综合值。



平均而言，这个内核的每个线程束花费115.2个周期因等待L1TEX（本地、全局、表面、纹理）操作的计分板依赖而停滞。找到产生被等待数据的指令以确定罪魁祸首。为了减少等待L1TEX数据访问的周期数，请验证内存访问模式是否针对目标架构进行了优化，尝试通过增加数据局部性（合并）或更改缓存配置来提高缓存命中率。考虑将频繁使用的数据移动到共享内存中。这种停滞类型占到了发布两条指令之间总平均123.0个周期的大约93.7%。

源指标，包括分支效率和采样线程束停滞原因。线程束停滞采样指标是在内核运行期间定期采样的。它们指示线程束何时停滞且无法被调度。请参阅文档以获取所有停滞原因的描述。只有当调度器无法每个周期都发出指令时，才需要关注停滞问题。





UNDERSTAND STALL REASON

Typical Stall Reason

- Common Stall Reasons:
- Long Scoreboard → L1Tex (Global, Local, Suface, Tex) result dependency.
- Short Scoreboard → Shared memory result dependency or frequent MUFU or Dynamic branching.
- LG Throttle → Waiting for the L1 instruction queue for local and global (LG) memory operations to be not full. Stall occurs only when executing local or global memory instructions extremely frequently.
- MIO Throttle → Was stalled waiting for the MIO (memory input/output) instruction queue to be not full. Stall occurs when executing LDS, MUFU or Dynamic Branching extremely frequently.
- Math Pipe Throttle → Was stalled waiting for the execution pipe to be available.

详细解释：

1. **长记分牌**
   - **含义**: 当线程因为等待某个操作的结果而导致较长时间的阻塞时，这种情况被称为"长记分牌"。例如，当一个线程依赖于L1纹理单元(Global, Local, Surface, Tex)的操作结果时，它可能会因为其他线程尚未完成相应的操作而被迫等待。
   - **解决方法**: 可以通过优化数据访问模式和算法来减少此类依赖，从而降低阻塞时间。
2. **短记分牌**
   - **含义**: 这是指线程由于共享内存的结果依赖或频繁的多功能单元(MFU)调用以及动态分支导致的短暂阻塞。
   - **解决方法**: 通过重新设计算法以减少对共享资源的竞争和提高分支预测准确性可以缓解这一问题。
3. **LG节流**
   - **含义**: 当线程频繁地进行本地和全局内存操作时，可能会遇到L1指令队列满的情况，从而导致阻塞。
   - **解决方法**: 合理管理内存操作的数量和顺序，避免过度使用某些类型的内存访问。
4. **MIO节流**
   - **含义**: 如果线程经常进行加载存储(memory input/output)操作，如LDS(load-store)、MUFU(multi-functional unit)调用或动态分支等，可能会导致MIO指令队列溢出，进而引起阻塞。
   - **解决方法**: 调整程序的执行流程和数据结构，尽量减少不必要的I/O操作。
5. **数学管道节流**
   - **含义**: 当线程试图使用数学函数库或其他复杂的计算任务时，如果执行管道繁忙或不空闲，那么线程将会被挂起直到管道可用。
   - **解决方法**: 平滑地分布计算密集型任务的执行，确保不会出现单个核心过载的情况。

【CUDA进阶】深入理解 Nsight System 和 Nsight Compute_哔哩哔哩_bilibili

UNDERSTAND STALL REASON

Long Scoreboard

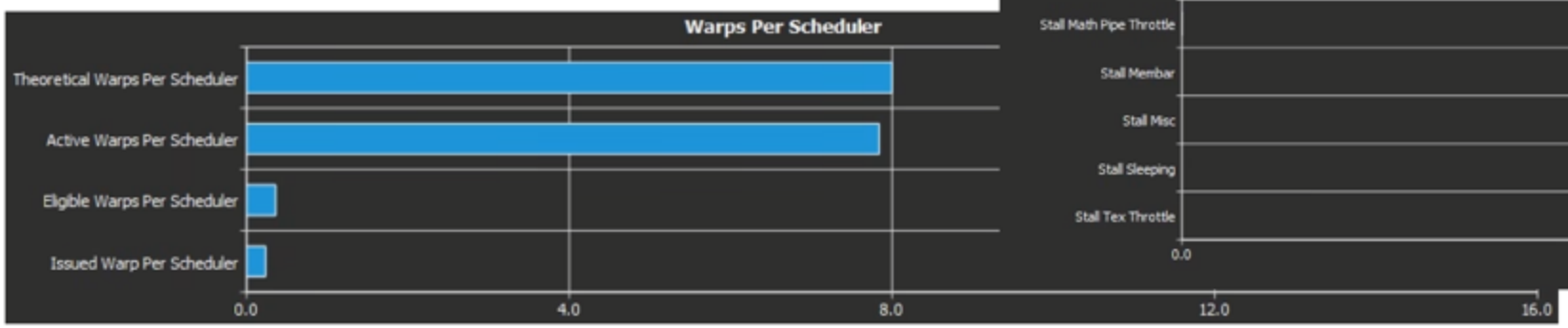
Long score board -> global memory latency

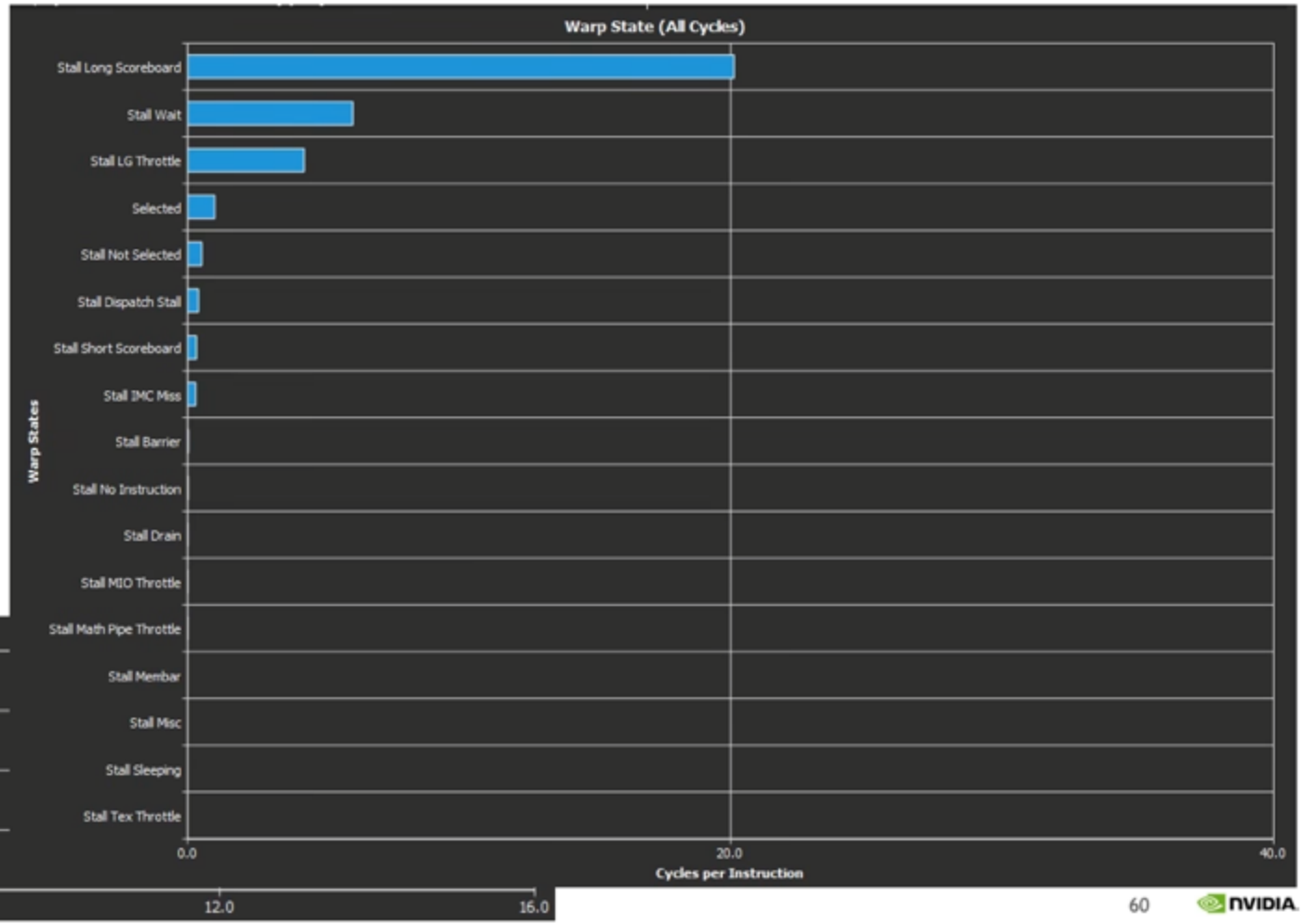
```cpp
__global__ void stall_reason_lsb(int * dramptr){

    int tid = threadIdx.x;
    int laneid = tid % 32;
    dramptr[laneid] = laneid;
    __syncthreads();

    int idx = laneid;
#pragma unroll
    for(int i = 0; i < 1000; i++)
        idx = dramptr[idx];
    dramptr[idx] = idx;


}
```





【CUDA进阶】深入理解 Nsight System 和 Nsight Compute_哔哩哔哩_bilibili

UNDERSTAND STALL REASON

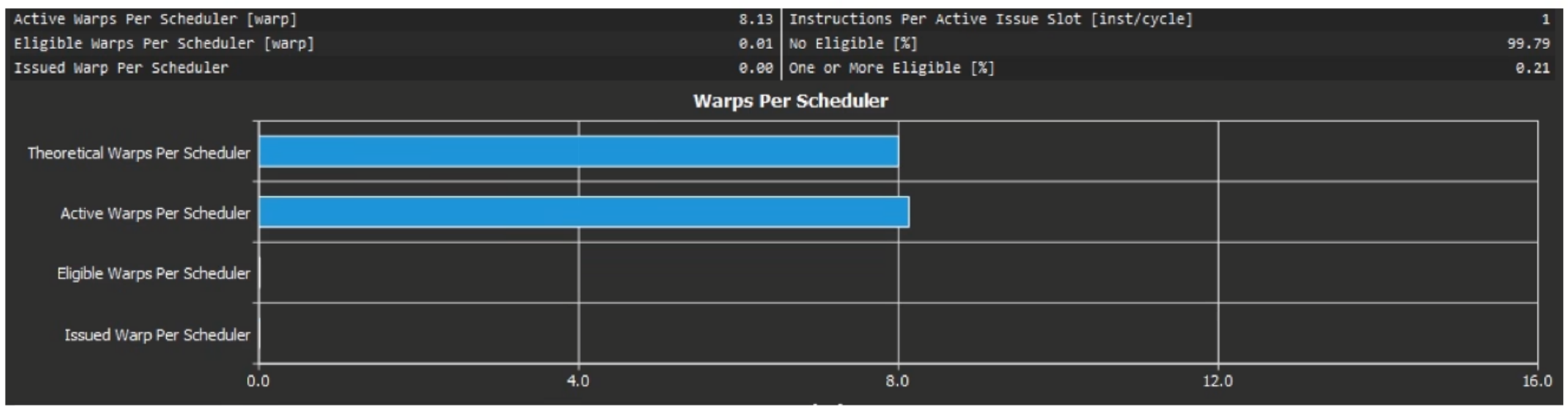
LG Throttle

LG Throttle -> Too many & frequent global access

```cpp
__global__ void stall_reason_lg_worst(int8_t * dramptr, int8_t * dramptr2){

    int tid = threadIdx.x;
    int offset = tid * 1000;
#pragma unroll
    for(int i = 0; i < 2000; i++)
        dramptr2[offset + i] = dramptr[offset + i];

}
```



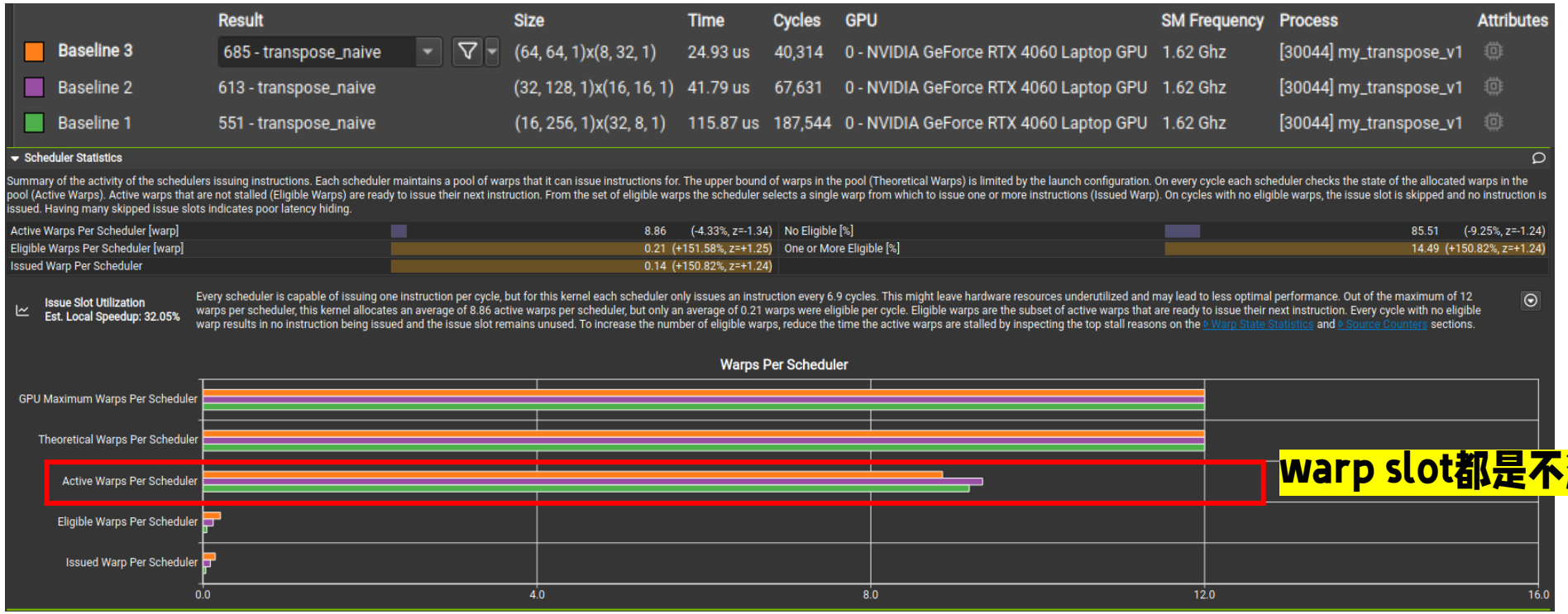
【CUDA进阶】深入理解 Nsight System 和 Nsight Compute_哔哩哔哩_bilibili



ncu与kernel调优指南

Warp调度与Transpose

Naive版本不同的block排布分析



	Result	Size	Time	Cycles	GPU	SM Frequency	Process	Attributes
Baseline 3	685 - transpose_naive	(64, 64, 1)x(8, 32, 1)	24.93 us	40,314	0 - NVIDIA GeForce RTX 4060 Laptop GPU	1.62 Ghz	[30044] my_transpose_v1	
Baseline 2	613 - transpose_naive	(32, 128, 1)x(16, 16, 1)	41.79 us	67,631	0 - NVIDIA GeForce RTX 4060 Laptop GPU	1.62 Ghz	[30044] my_transpose_v1	
Baseline 1	551 - transpose_naive	(16, 256, 1)x(32, 8, 1)	115.87 us	187,544	0 - NVIDIA GeForce RTX 4060 Laptop GPU	1.62 Ghz	[30044] my_transpose_v1	

Scheduler Statistics

Summary of the activity of the schedulers issuing instructions. Each scheduler maintains a pool of warps that it can issue instructions for. The upper bound of warps in the pool (Theoretical Warps) is limited by the launch configuration. On every cycle each scheduler checks the state of the allocated warps in the pool (Active Warps). Active warps that are not stalled (Eligible Warps) are ready to issue their next instruction. From the set of eligible warps the scheduler selects a single warp from which to issue one or more instructions (Issued Warp). On cycles with no eligible warps, the issue slot is skipped and no instruction is issued. Having many skipped issue slots indicates poor latency hiding.

Active Warps Per Scheduler [warp]	8.86 (-4.33%, z=-1.34)	No Eligible [%]	85.51 (-9.25%, z=-1.24)
Eligible Warps Per Scheduler [warp]	0.21 (+151.58%, z=+1.25)	One or More Eligible [%]	14.49 (+150.82%, z=+1.24)
Issued Warp Per Scheduler	0.14 (+150.82%, z=+1.24)		

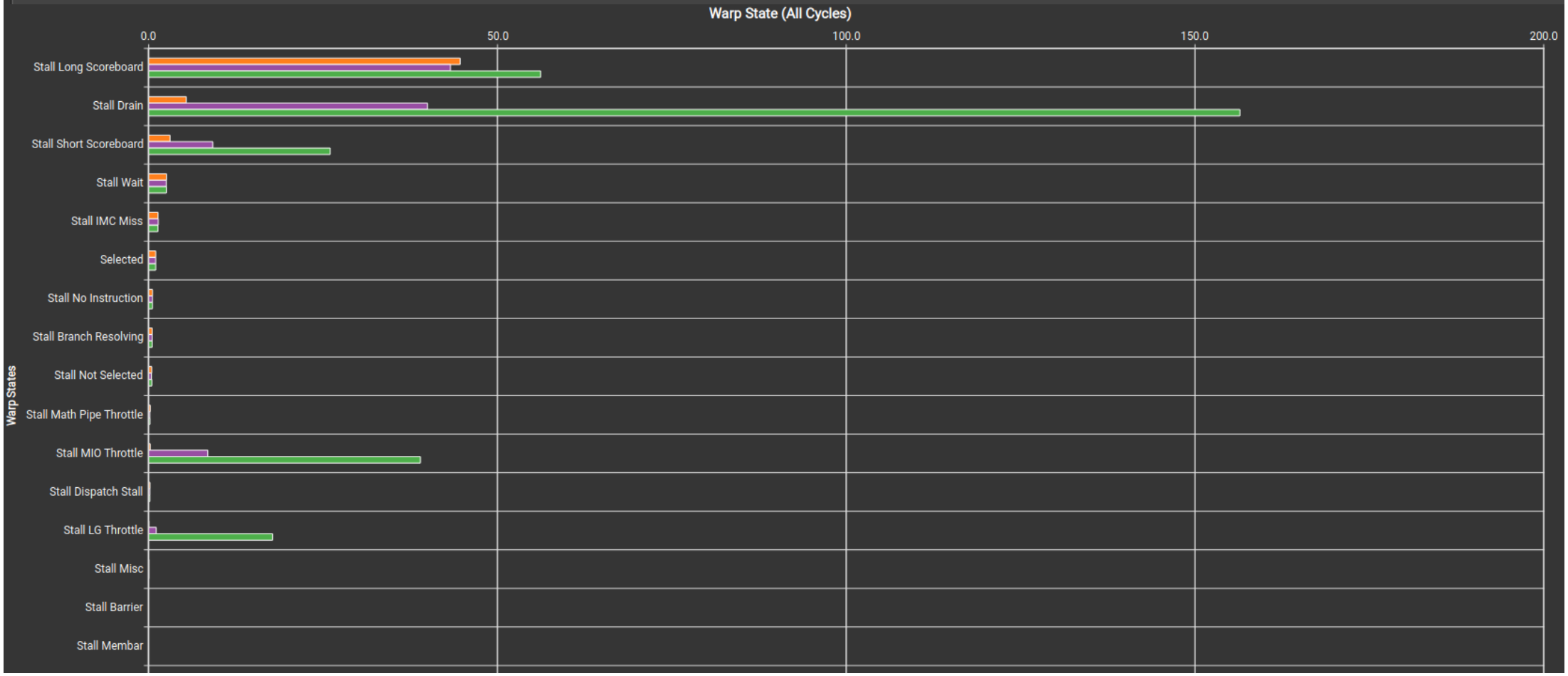
Issue Slot Utilization
Est. Local Speedup: 32.05%

Every scheduler is capable of issuing one instruction per cycle, but for this kernel each scheduler only issues an instruction every 6.9 cycles. This might leave hardware resources underutilized and may lead to less optimal performance. Out of the maximum of 12 warps per scheduler, this kernel allocates an average of 8.86 active warps per scheduler, but only an average of 0.21 warps were eligible per cycle. Eligible warps are the subset of active warps that are ready to issue their next instruction. Every cycle with no eligible warp results in no instruction being issued and the issue slot remains unused. To increase the number of eligible warps, reduce the time the active warps are stalled by inspecting the top stall reasons on the ▹ Warp State Statistics and ▹ Source Counters sections.

warp slot都是不满的

Naive版本不同的block排布分析

	Result	Size	Time	Cycles	GPU	SM Frequency	Process	Attributes
Baseline 3	685 - transpose_naive	(64, 64, 1)x(8, 32, 1)	24.93 us	40,314	0 - NVIDIA GeForce RTX 4060 Laptop GPU	1.62 Ghz	[30044] my_transpose_v1	
Baseline 2	613 - transpose_naive	(32, 128, 1)x(16, 16, 1)	41.79 us	67,631	0 - NVIDIA GeForce RTX 4060 Laptop GPU	1.62 Ghz	[30044] my_transpose_v1	
Baseline 1	551 - transpose_naive	(16, 256, 1)x(32, 8, 1)	115.87 us	187,544	0 - NVIDIA GeForce RTX 4060 Laptop GPU	1.62 Ghz	[30044] my_transpose_v1	





smsp__average_warps_issue_stalled_long_scoreboard_per_issue_active.ratio
average # of warps resident per issue cycle, waiting for a scoreboard dependency on L1TEX (local, global, surface, tex) operation
(This ratio metric represents the value expressed as a ratio across all sub-unit instances)
Warp was stalled waiting for a scoreboard dependency on a L1TEX (local, global, surface, tex) operation.
To reduce the number of cycles waiting on L1TEX data accesses verify the memory access patterns are optimal for the target architecture.
Also, attempt to increase cache hit rates by increasing data locality, or by changing the cache configuration, and consider moving frequently used data to shared memory.

smsp: Each SM is partitioned into four processing blocks, called SM sub partitions.
The SM sub partitions are the primary processing elements on the SM.
A sub partition manages a fixed size pool of warps.

warps: A a group of 32 threads within a CTA.
A warp is allocated to a sub partition and resides on the sub partition from launch to completion.

L1TEX: The Level 1 (L1)/Texture Cache is located within the GPC.
It can be used as directed-mapped shared memory and/or store global, local and texture data in its cache portion.
l1tex__t refers to its Tag stage.
l1tex__m refers to its Miss stage.
l1tex__d refers to its Data stage.

global: Global memory is a 49-bit virtual address space that is mapped to physical memory on the device, pinned system memory, or peer memory.
Global memory is visible to all threads in the GPU.
Global memory is accessed through the SM L1 and GPU L2.

surface: Surface memory

shared: Shared memory is located on chip, so it has much higher bandwidth and much lower latency than either local or global memory.
Shared memory can be shared across a compute CTA.

smsp_average_warps_issue_stalled_long_scoreboard_per_issue_active.ratio
average # of warps resident per issue cycle, waiting for a scoreboard dependency on L1TEX (local, global, surface, tex) operation (This ratio metric represents the value expressed as a ratio across all sub - unit instances) Warp was stalled waiting for a scoreboard dependency on a L1TEX (local, global, surface, tex) operation.To reduce the number of cycles waiting on L1TEX data accesses verify the memory access patterns are optimal for the target architecture. Also, attempt to increase cache hit rates by increasing data locality, or by changing the cache configuration, and consider moving frequently used data to shared memory.

每个发射周期中驻留的平均线程束数量，这些线程束在等待 L1TEX（本地、全局、表面、纹理）操作的记分牌依赖（此比率度量表示为所有子单元实例的比率值）。线程束在等待 L1TEX（本地、全局、表面、纹理）操作的记分牌依赖时停滞。为减少等待 L1TEX 数据访问的周期数，需验证内存访问模式对于目标架构是否为最优。此外，尝试通过提高数据局部性或改变缓存配置来提高缓存命中率，并考虑将频繁使用的数据移至共享内存。



smsp_average_warps_issue_stalled_drain_per_issue_active.ratio
average # of warps resident per issue cycle, waiting after EXIT for all memory instructions to complete so that warp resources can be freed (This ratio metric represents the value expressed as a ratio across all sub - unit instances)
Warp was stalled after EXIT waiting for all memory instructions to complete so that warp resources can be freed. A high number of stalls due to draining warps typically occurs when a lot of data is written to memory towards the end of a kernel. Make sure the memory access patterns of these store operations are optimal for the target architecture and consider parallelized data reduction, if applicable.

每个发射周期中驻留的平均线程束数量，在线程束退出（EXIT）后等待所有内存指令完成，以便释放线程束资源（该比率指标表示为所有子单元实例的比率值）。线程束在线束退出后停滞，等待所有内存指令完成，以便释放线程束资源。当在内核结束时大量数据写入内存时，通常会因线程束排空而出现大量停滞。确保这些存储操作的内存访问模式对于目标架构是最优的，并在适用的情况下考虑并行化数据缩减。

smsp__average_warps_issue_stalled_drain_per_issue_active.ratio
average # of warps resident per issue cycle, waiting after EXIT for all memory instructions to complete so that warp resources can be freed
(This ratio metric represents the value expressed as a ratio across all sub-unit instances)
Warp was stalled after EXIT waiting for all memory instructions to complete so that warp resources can be freed.
A high number of stalls due to draining warps typically occurs when a lot of data is written to memory towards the end of a kernel.
Make sure the memory access patterns of these store operations are optimal for the target architecture and consider parallelized data reduction, if applicable.

smsp: Each SM is partitioned into four processing blocks, called SM sub partitions.
The SM sub partitions are the primary processing elements on the SM.
A sub partition manages a fixed size pool of warps.

warps: A a group of 32 threads within a CTA.
A warp is allocated to a sub partition and resides on the sub partition from launch to completion.

instructions: An assembly (SASS) instruction.
Each executed instruction may generate zero or more requests.

stalls: A warp is stalled when the warp is waiting on an instruction fetch, a memory dependency (result of memory instruction),
an execution dependency (result of previous instruction), or a synchronization barrier.



smsp_average_warps_issue_stalled_short_scoreboard_per_issue_active.ratio
average # of warps resident per issue cycle, waiting for a scoreboard dependency on MIO operation (other than (local, global, surface, tex) (This ratio metric represents the value expressed as a ratio across all sub - unit instances)Warp was stalled waiting for a scoreboard dependency on a MIO (memory input/output) operation (not to L1TEX). The primary reason for a high number of stalls due to short scoreboards is typically memory operations to shared memory. Other reasons include frequent execution of special math instructions (e.g. MFU0) or dynamic branching (e.g. BRX, JMX).Verify if there are shared memory operations and reduce bank conflicts, if applicable.

每个发射周期中驻留的平均线程束数量，这些线程束在等待除（本地、全局、表面、纹理）操作之外的 MIO（内存输入 / 输出）操作的记分牌依赖（此比率度量表示为所有子单元实例的比率值）。线程束在等待 MIO（内存输入 / 输出）操作（而非 L1TEX）的记分牌依赖时停滞。由于短记分牌导致大量停滞的主要原因通常是对共享内存的内存操作。其他原因包括频繁执行特殊数学指令（例如 MFU0）或动态分支（例如 BRX、JMX）。如果适用，检查是否存在共享内存操作并减少存储体冲突。

smsp__average_warps_issue_stalled_short_scoreboard_per_issue_active.ratio
average # of warps resident per issue cycle, waiting for a scoreboard dependency on MIO operation other than (local, global, surface, tex)
(This ratio metric represents the value expressed as a ratio across all sub-unit instances)
Warp was stalled waiting for a scoreboard dependency on a MIO (memory input/output) operation (not to L1TEX).
The primary reason for a high number of stalls due to short scoreboards is typically memory operations to shared memory.
Other reasons include frequent execution of special math instructions (e.g. MUFU) or dynamic branching (e.g. BRX, JMX).
Verify if there are shared memory operations and reduce bank conflicts, if applicable.

smsp: Each SM is partitioned into four processing blocks, called SM sub partitions.
The SM sub partitions are the primary processing elements on the SM.
A sub partition manages a fixed size pool of warps.

warps: A group of 32 threads within a CTA.
A warp is allocated to a sub partition and resides on the sub partition from launch to completion.

MIO: Memory input/output

global: Global memory is a 49-bit virtual address space that is mapped to physical memory on the device, pinned system memory, or peer memory.
Global memory is visible to all threads in the GPU.
Global memory is accessed through the SM L1 and GPU L2.

surface: Surface memory

stalls: A warp is stalled when the warp is waiting on an instruction fetch, a memory dependency (result of memory instruction),
an execution dependency (result of previous instruction), or a synchronization barrier.

shared: Shared memory is located on chip, so it has much higher bandwidth and much lower latency than either local or global memory.
Shared memory can be shared across a compute CTA.

instructions: An assembly (SASS) instruction.
Each executed instruction may generate zero or more requests.

bank: Shared memory has 32 banks that are organized such that successive 32-bit words map to successive banks that can be accessed simultaneously.
Any 32-bit memory read or write request made of 32 addresses that fall in 32 distinct memory banks can therefore be serviced simultaneously.

conflicts: If two addresses of a memory request fall in the same memory bank, there is a bank conflict and the access has to be serialized.



smsp__average_warps_issue_stalled_lg_throttle_per_issue_active.ratio
average # of warps resident per issue cycle, waiting for a free entry in the LSU instruction queue
(This ratio metric represents the value expressed as a ratio across all sub-unit instances)
Warp was stalled waiting for the L1 instruction queue for local and global (LG) memory operations to be not full.
Typically, this stall occurs only when executing local or global memory instructions extremely frequently.
If applicable, consider combining multiple lower-width memory operations into fewer wider memory operations and try interleaving memory operations and math instructions.

smsp: Each SM is partitioned into four processing blocks, called SM sub partitions.
The SM sub partitions are the primary processing elements on the SM.
A sub partition manages a fixed size pool of warps.

warps: A a group of 32 threads within a CTA.
A warp is allocated to a sub partition and resides on the sub partition from launch to completion.

lg: Local/Global memory

LSU: Load Store Unit.
The LSU pipeline issues load, store, atomic, and reduction instructions to the L1TEX unit for global, local, and shared memory.
It also issues special register reads (S2R), shuffles, and CTA-level arrive/wait barrier instructions to the L1TEX unit.

instruction: An assembly (SASS) instruction.
Each executed instruction may generate zero or more requests.

L1: The Level 1 (L1)/Texture Cache is located within the GPC.
It can be used as directed-mapped shared memory and/or store global, local and texture data in its cache portion.
l1tex__t refers to its Tag stage.
l1tex__m refers to its Miss stage.
l1tex__d refers to its Data stage.

local: Local memory is private storage for an executing thread and is not visible outside of that thread.
It is intended for thread-local data like thread stacks and register spills.
Local memory has the same latency as global memory.

global: Global memory is a 49-bit virtual address space that is mapped to physical memory on the device, pinned system memory, or peer memory.
Global memory is visible to all threads in the GPU.
Global memory is accessed through the SM L1 and GPU L2.

每个问题周期中驻留的平均线程束数量，这些线程束在等待 LSU（加载 - 存储单元）指令队列有空余项（此比率指标表示为所有子单元实例的比率值）。线程束在等待本地和全局（LG）内存操作的 L1 指令队列未满时停滞。通常，这种停滞仅在极其频繁地执行本地或全局内存指令时才会发生。如果可行的话，考虑将多个窄位宽的内存操作合并为较少的宽位宽内存操作，并尝试将内存操作和数学指令交错执行。

smsp_average_warps_issue_stalled_lg_throttle_per_issue_active.ratio
average # of warps resident per issue cycle, waiting for a free entry in the LSU instruction queue (This ratio metric represents the value expressed as a ratio across all sub - unit instances) Warp was stalled waiting for the L1 instruction queue for local and global (LG) memory operations to be not full. Typically, this stall occurs only when executing local or global memory instructions extremely frequently. If applicable, consider combining multiple lower - width memory operations into fewer wider memory operations and try interleaving memory operations and math instructions.



smsp__average_warps_issue_stalled_mio_throttle_per_issue_active.ratio
average # of warps resident per issue cycle, waiting for a free entry in the MIO instruction queue
(This ratio metric represents the value expressed as a ratio across all sub-unit instances)
Warp was stalled waiting for the MIO (memory input/output) instruction queue to be not full.
This stall reason is high in cases of extreme utilization of the MIO pipelines, which include special math instructions, dynamic branches, as well as shared memory instructions.

smsp: Each SM is partitioned into four processing blocks, called SM sub partitions.
The SM sub partitions are the primary processing elements on the SM.
A sub partition manages a fixed size pool of warps.

warps: A a group of 32 threads within a CTA.
A warp is allocated to a sub partition and resides on the sub partition from launch to completion.

mio: Memory input/output

instruction: An assembly (SASS) instruction.
Each executed instruction may generate zero or more requests.

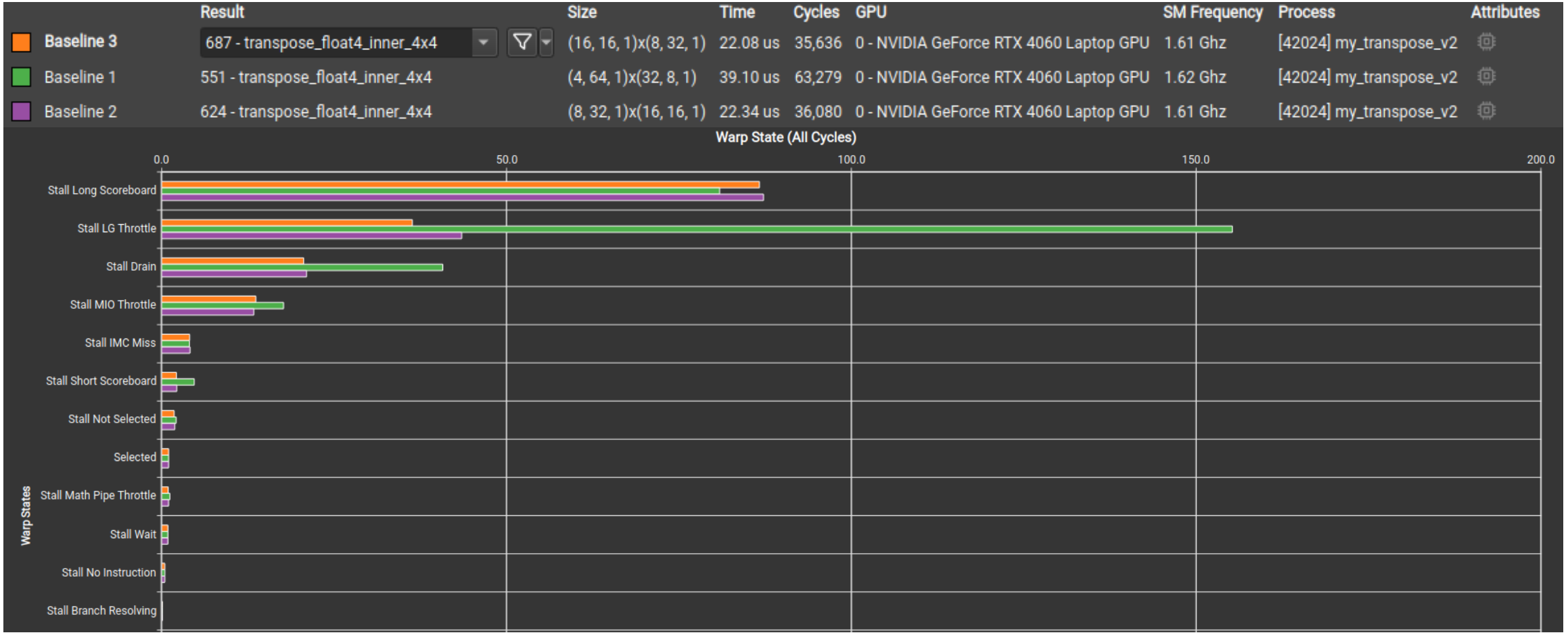
stall: A warp is stalled when the warp is waiting on an instruction fetch, a memory dependency (result of memory instruction),
an execution dependency (result of previous instruction), or a synchronization barrier.

shared: Shared memory is located on chip, so it has much higher bandwidth and much lower latency than either local or global memory.
Shared memory can be shared across a compute CTA.

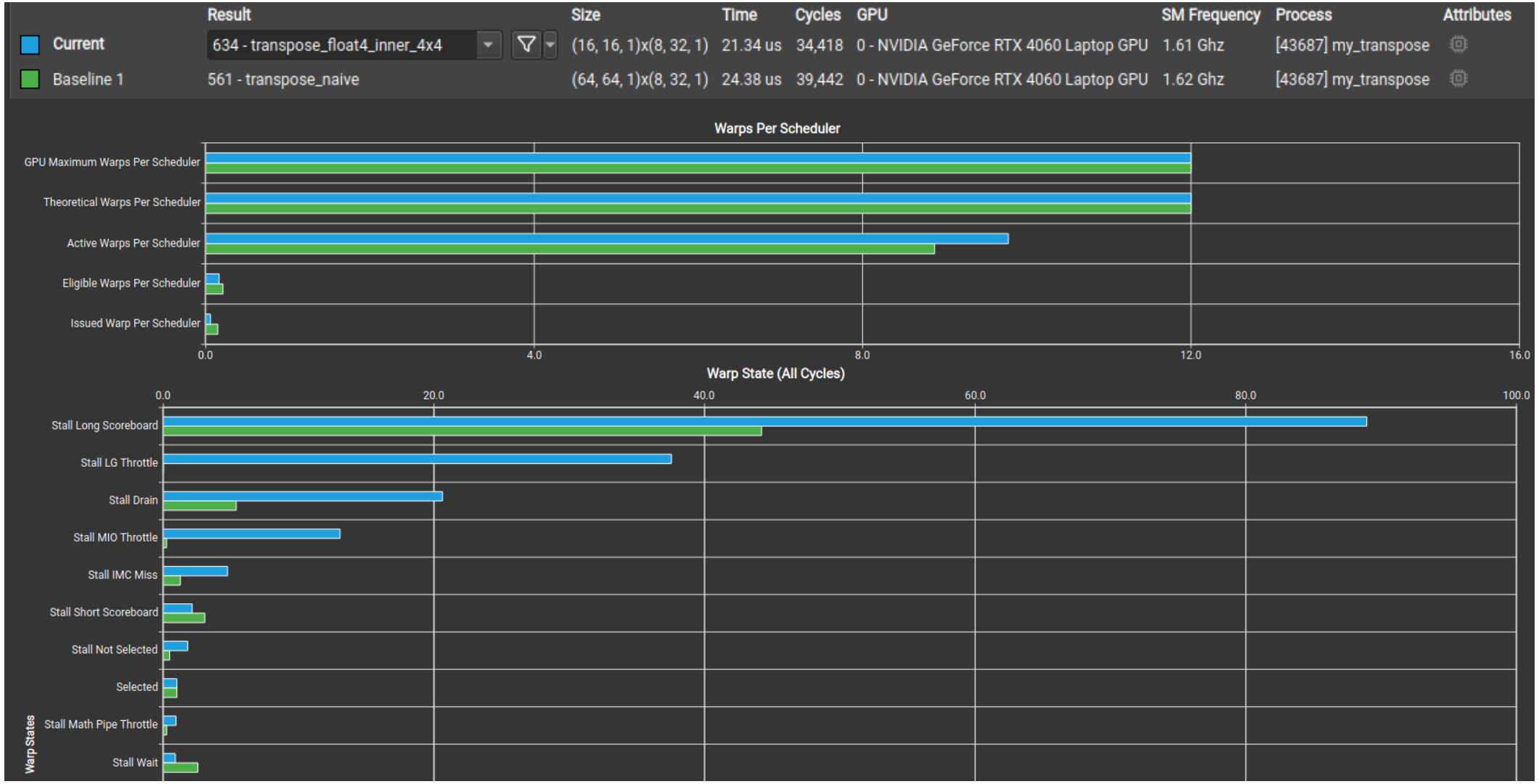
smsp_average_warps_issue_stalled_mio_throttle_per_issue_active.ratio
average # of warps resident per issue cycle, waiting for a free entry in the MIO instruction queue (This ratio metric represents the value expressed as a ratio across all sub - unit instances) Warp was stalled waiting for the MIO (memory input/output) instruction queue to be not full. This stall reason is high in cases of extreme utilization of the MIO pipelines, which include special math instructions, dynamic branches, as well as shared memory instructions.

每个发射周期中驻留的平均线程束数量，这些线程束在等待 MIO（内存输入 / 输出）指令队列有空余项（此比率度量表示为所有子单元实例的比率值）。线程束在等待 MIO（内存输入 / 输出）指令队列未满时停滞。在 MIO 流水线被极度占用的情况下，这种停滞的原因很常见，其中包括特殊数学指令、动态分支以及共享内存指令。

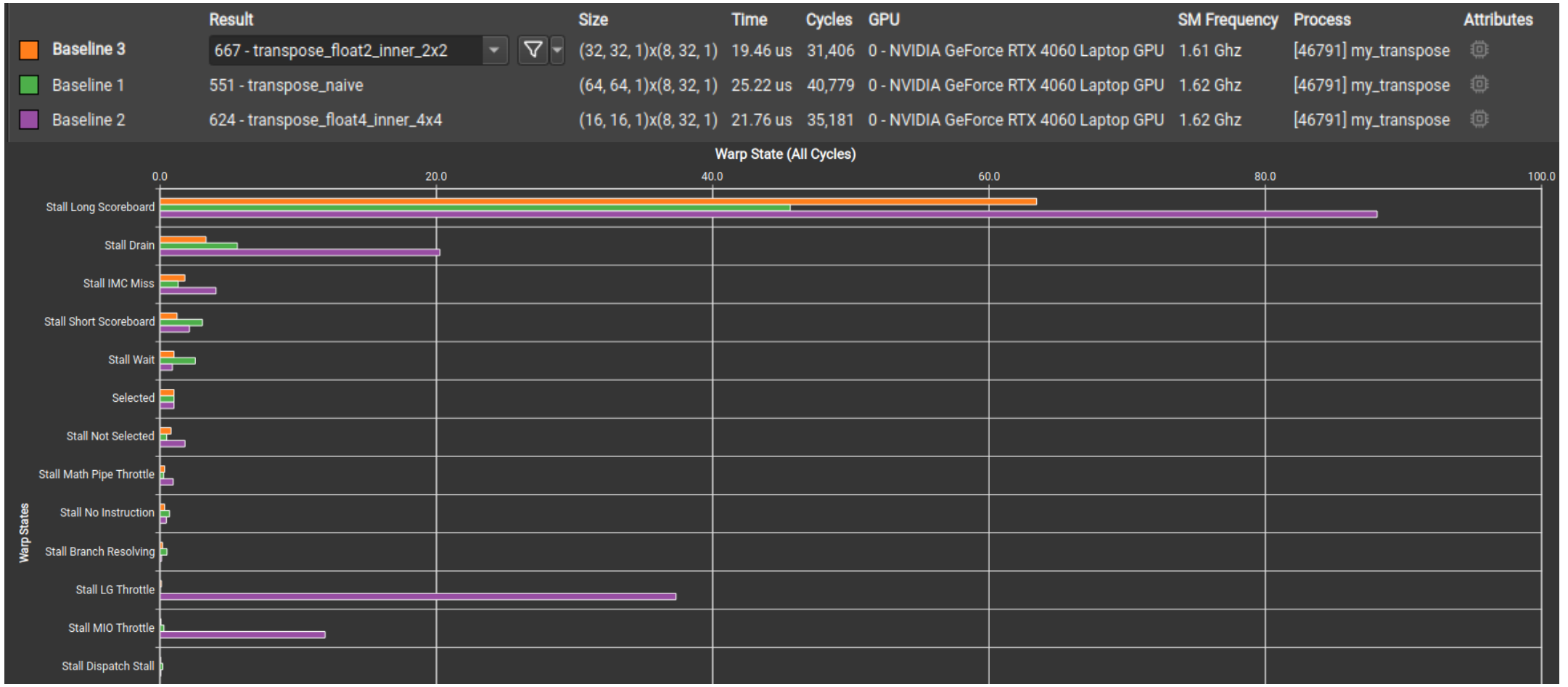
float4版本不同的block排布分析



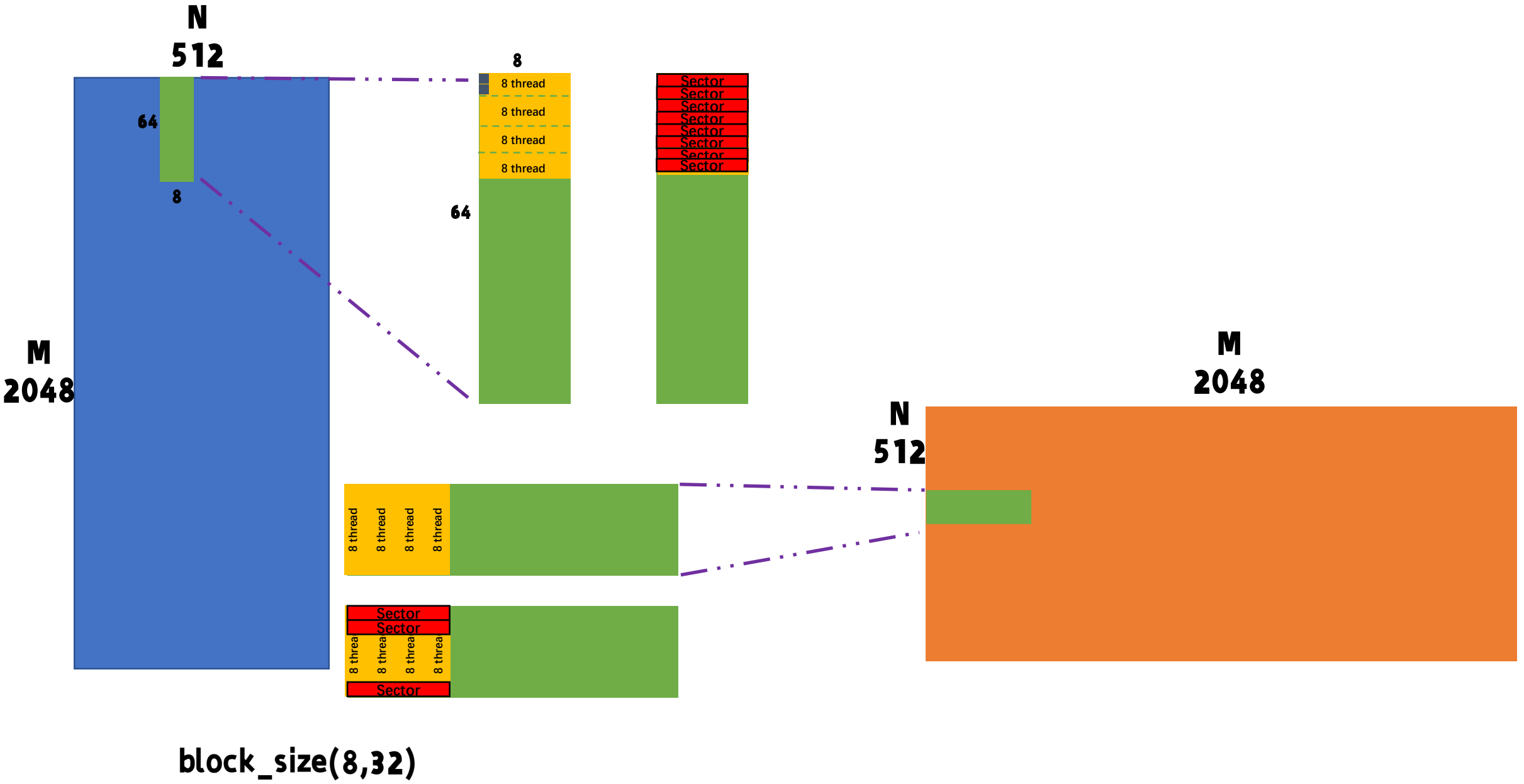
Naive与float4对比分析



Transpose_v3_float2_2x2



Transpose_v4_float2_1x2





ncu与kernel调优指南 Roofline model

- **算力** π：也称为计算平台的**性能上限**，指的是一个计算平台倾尽全力每秒钟所能完成的浮点运算数。单位是 `FLOPS` or `FLOP/s` 。**是一个定值**

π : Maximum FLOPs Per Second

- **带宽** β：也即计算平台的**带宽上限**，指的是一个计算平台倾尽全力每秒所能完成的内存交换量。单位是 `Byte/s` 。**是一个定值**

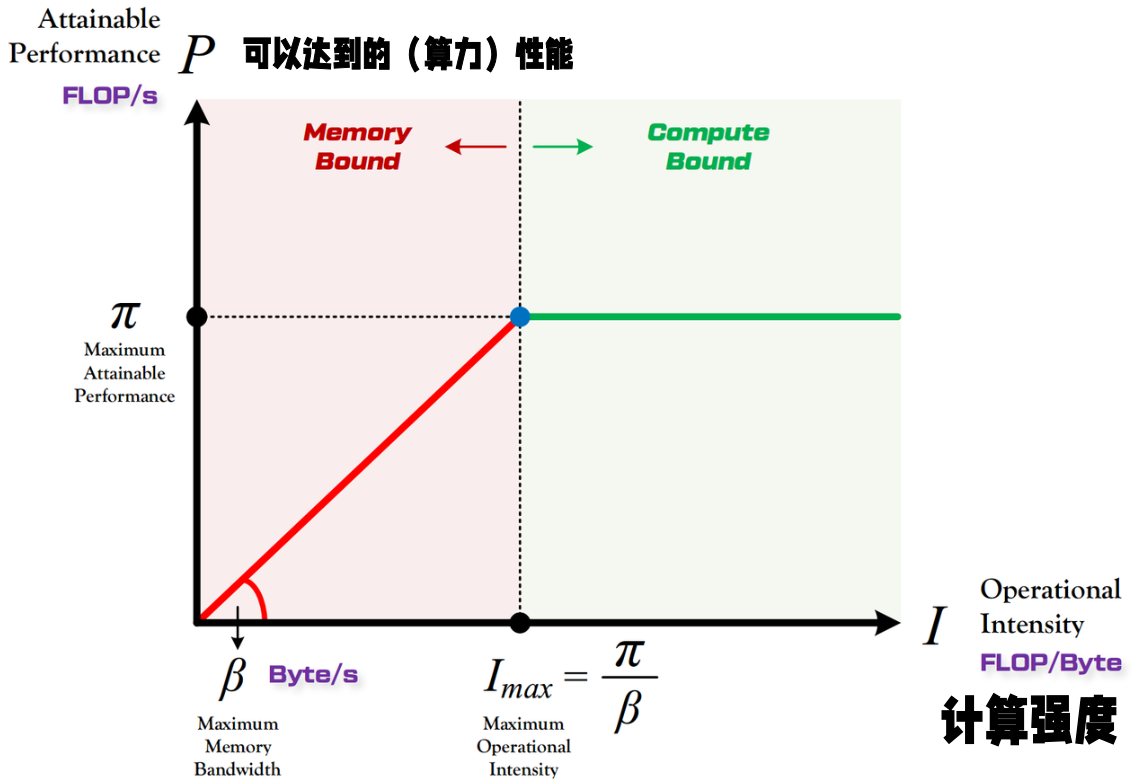
β : Maximum Memory Access Per Second

- **计算强度上限** I_{max} ：两个指标相除即可得到计算平台的**计算强度上限**。它描述的是在这个计算平台上，单位内存交换最多用来进行多少次计算。单位是 `FLOPs/Byte` 。**是一个定值**

$$I_{max} = \frac{\pi}{\beta}$$

- **模型的计算强度** I：由计算量除以访存量就可以得到模型的计算强度，它表示此模型在计算过程中，每 `Byte` 内存交换到底用于进行多少次浮点运算*。单位是 `FLOPs/Byte` 。可以看到，模计算强度越大，其内存使用效率越高。

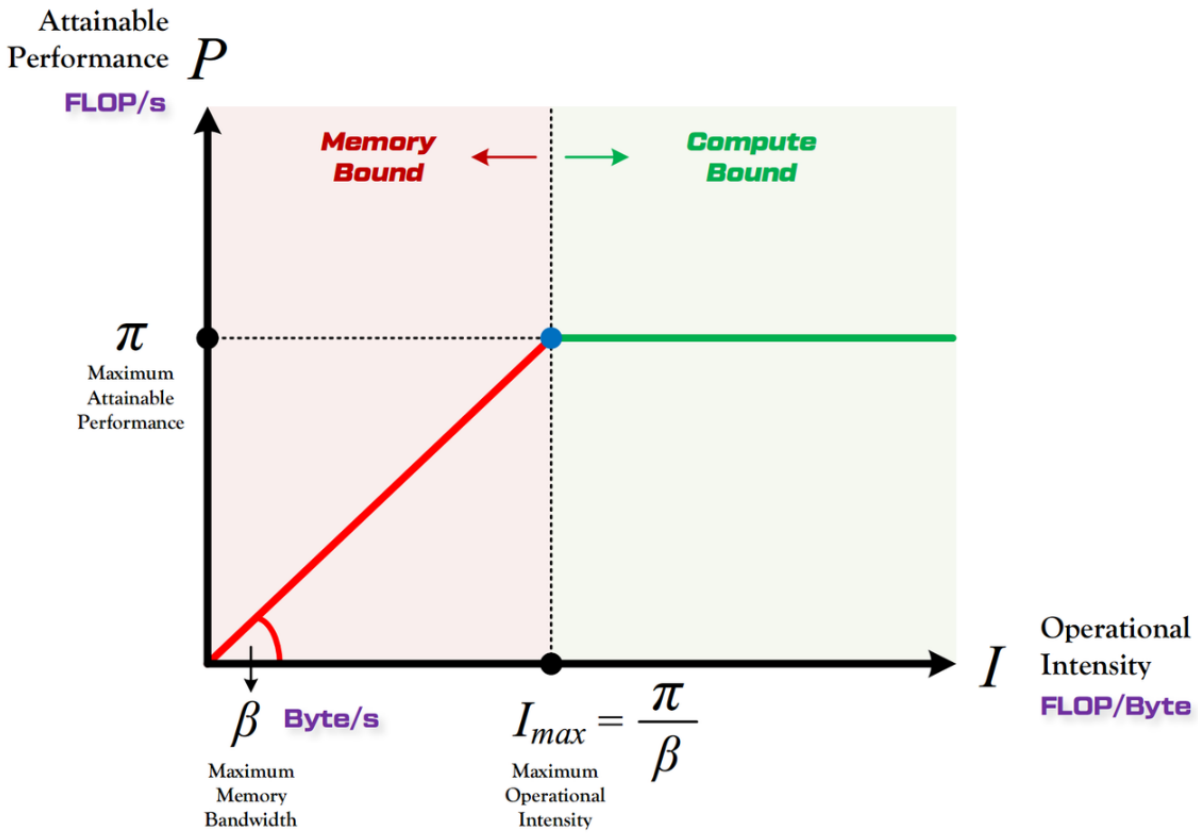
- **模型的理论性能** P：我们最关心的指标，即模型***在计算平台上所能达到的每秒浮点运算次数（理论值）***。单位是 `FLOPS` or `FLOP/s` 。下面我们即将介绍的 Roof-line Model 给出的就是计算这个指标的方法。终于可以进入正题了。



$$P = \begin{cases} \beta \cdot I, & when \ \ I < I_{max} \quad \text{Memory Bound} \\ \pi, & when \ \ I \geqslant I_{max} \quad \text{Compute Bound} \end{cases}$$

当计算强度I小于Imax的时候，模型的理论计算强度是beta*I,但是实际我们写的kernel计算强度一般会小于这个P，我们要努力优化到这个值

[Roofline Model与深度学习模型的性能分析 - 知乎]

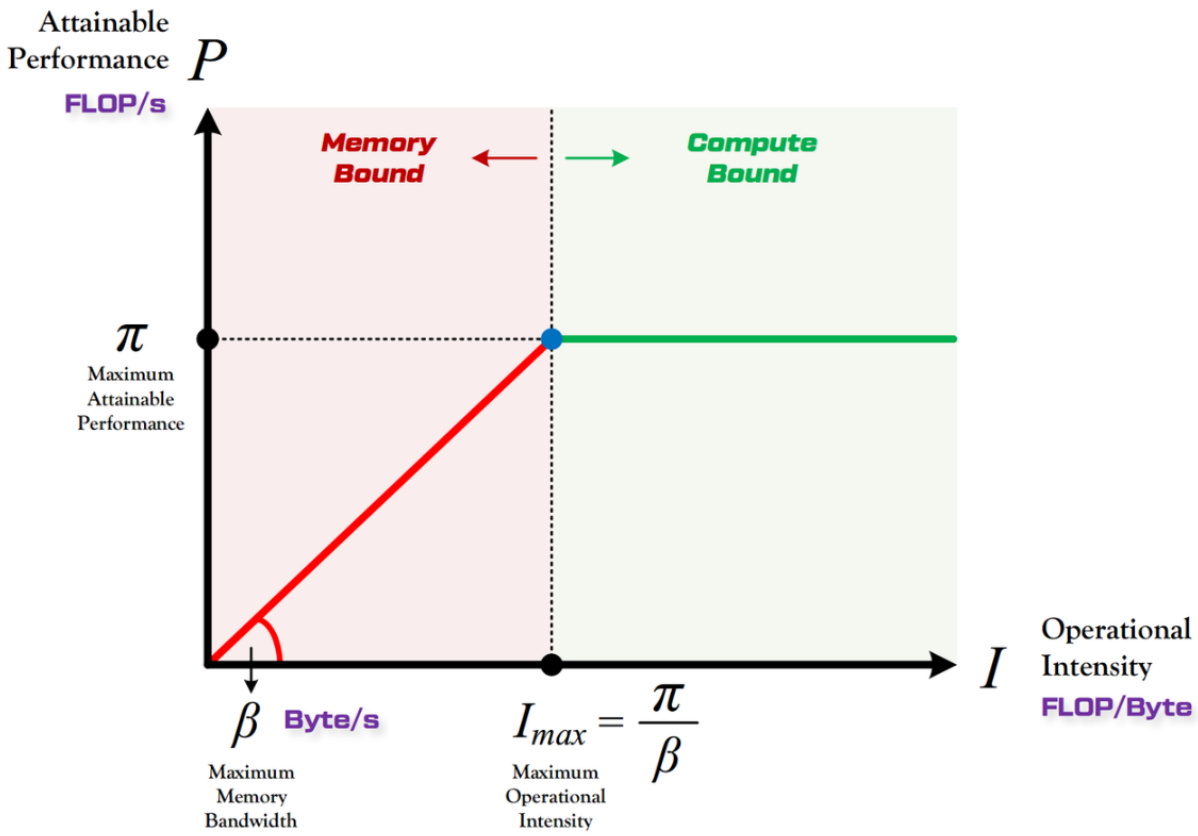


4090显卡

- **峰值算力（π）为 82.58 TFLOPS**
- **带宽上限（β）为 1.01 TB/s**
- **计算强度上限（I_{max}）为 81.76 FLOP/B**

Single Precision Roofline	
Peak Work:	72,802,483,754,512.64
Peak Traffic:	977,559,566,787.00
Arithmetic Intensity [FLOP/byte]:	74.47
Performance [FLOP/s]:	72,802,483,754,512.64

Roofline Model与深度学习模型的性能分析 - 知乎



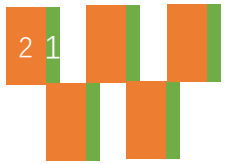
假设一个显卡的

- **峰值算力（π）为 2 FLOPs**
- **带宽上限（β）为 5 B/s**
- **计算强度上限（I_{max}）为 0.4FLOPs/B**

有一个任务(kernel)，访存量为50B，计算量为10次



实际性能：10/15 FLOPs
实际带宽：50/15 B/s
计算强度：0.2

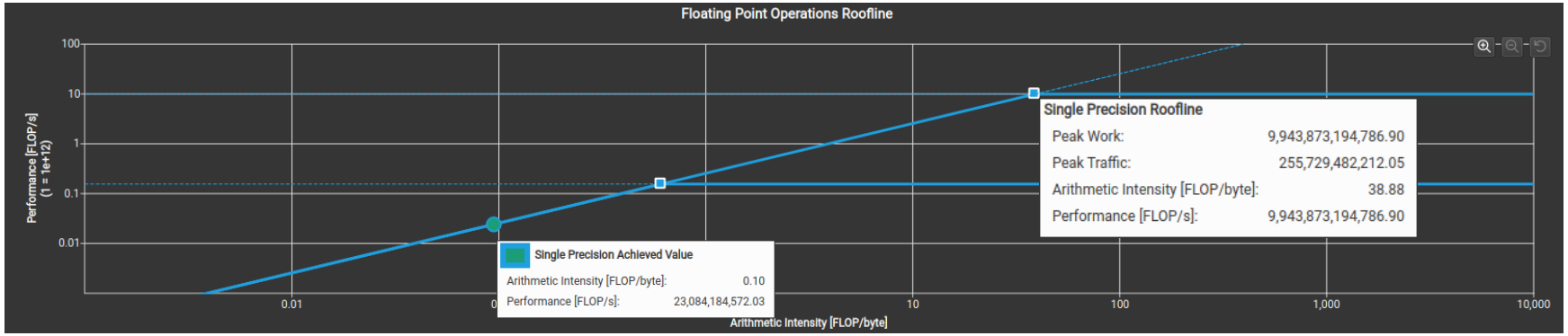


实际性能：10/11 FLOP
实际带宽：50/11 B/s
计算强度：0.2

有一个任务，访存量为10B，计算量为50次



实际性能：50/25.4 FLOP
实际带宽：10/25.4 B/s
计算强度：5



计算强度：0.1
实际算力：23GFLOPS

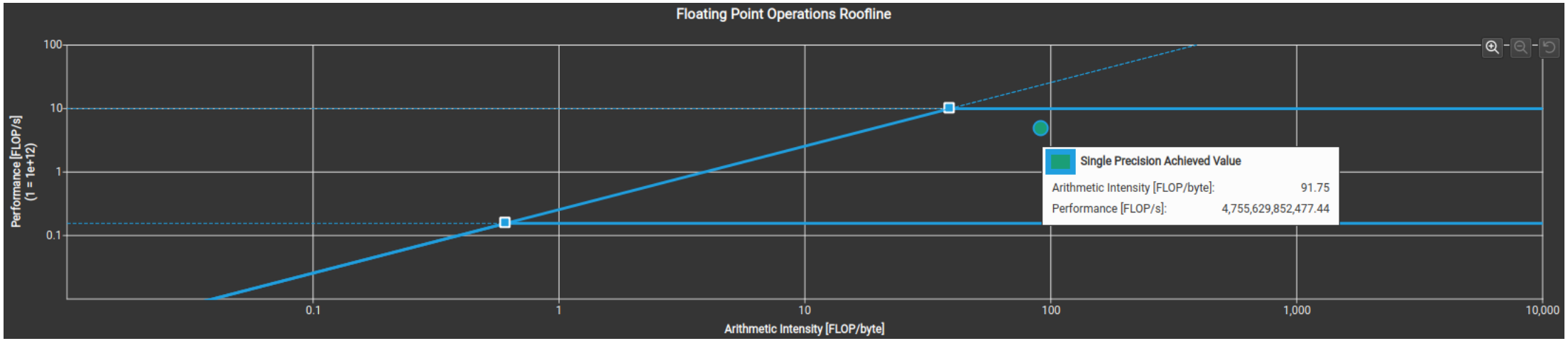
实际的带宽：240GB/s（已经接近理论峰值了，说明这个程序已经倾尽全力在传输数据了，奈何计算的任务太少）

```
void __global__ add1(float*x, float*y, float*z)
{
    int n = threadIdx.x + blockIdx.x * blockDim.x;
    z[n] = x[n] + y[n];
}
```

```cpp
void __global__ add2(float*x, float*y, float*z)
{
  int n = threadIdx.x + blockIdx.x * blockDim.x;
  float a = x[n];
  float b = y[n];
  float c = 0;
  for(int i=0; i<1000; i++)
  {
    c += (a + b);
  }
  z[n] = c;
}
```

计算强度：91.75
实际算力：4.7TFLOPS

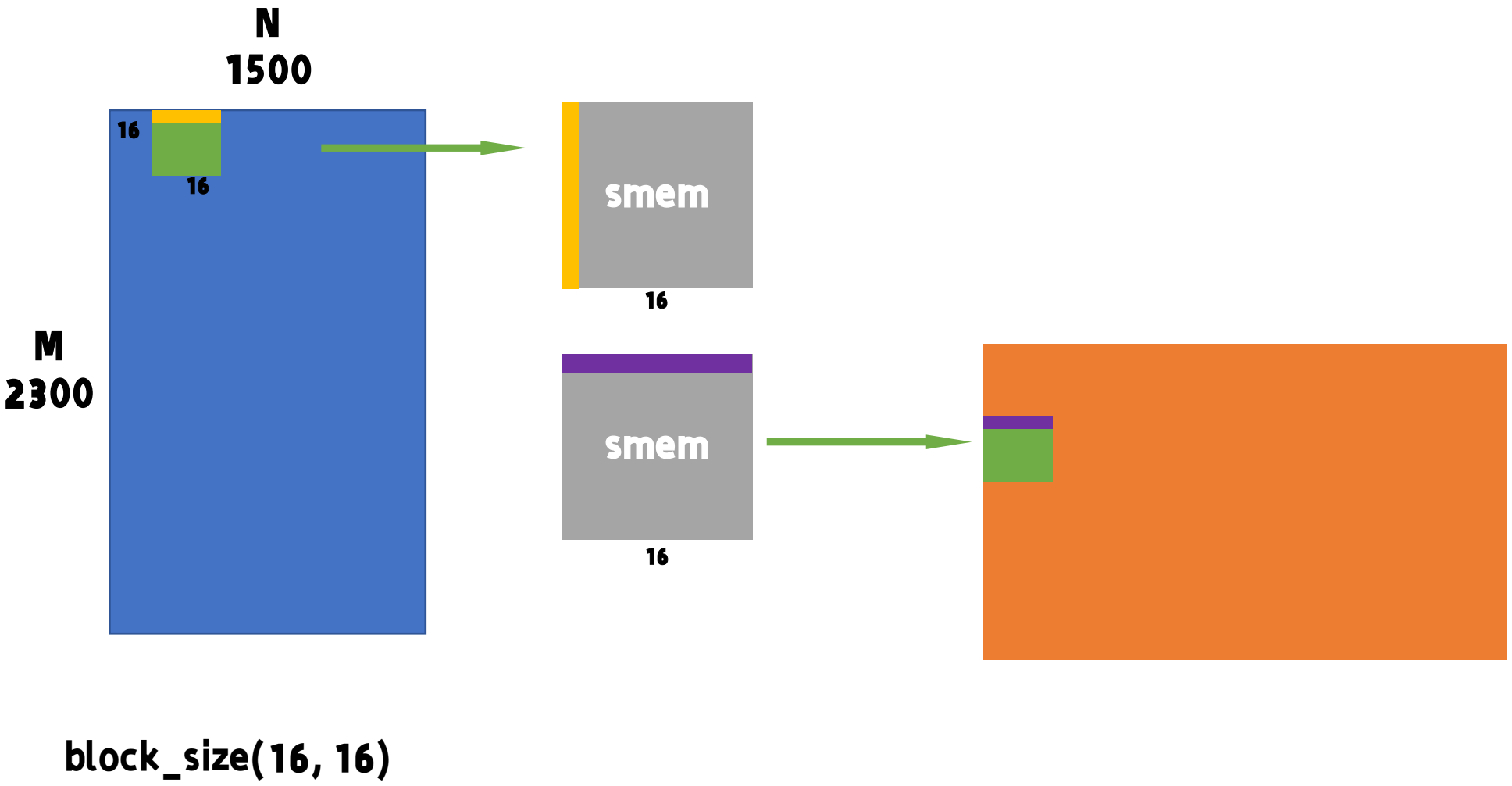
实际的带宽：51GB/s（算力和带宽都没有达到理论值）





ncu与kernel调优指南 Shared Memory

Transpose_v5_shared_memory

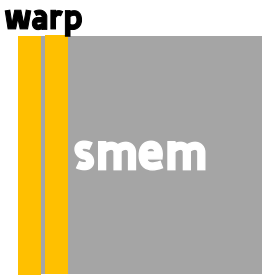


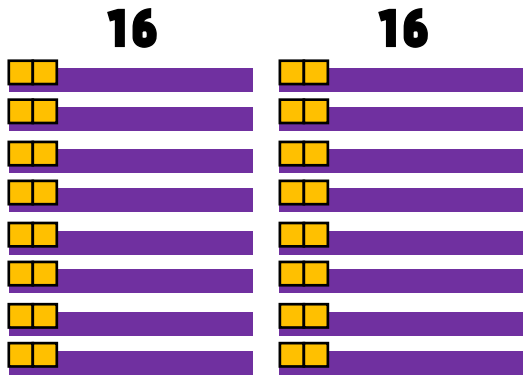
[CUDA 学习笔记] 矩阵转置算子优化_优化矩阵转置-CSDN博客

smem_store





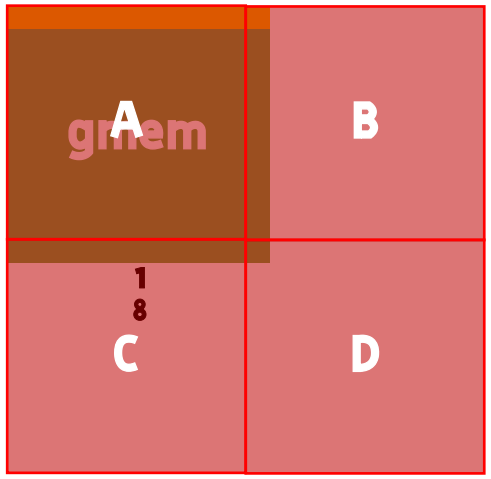


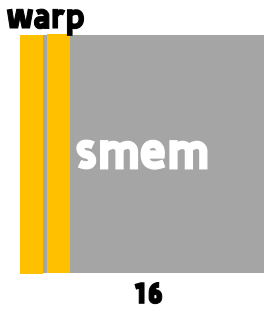


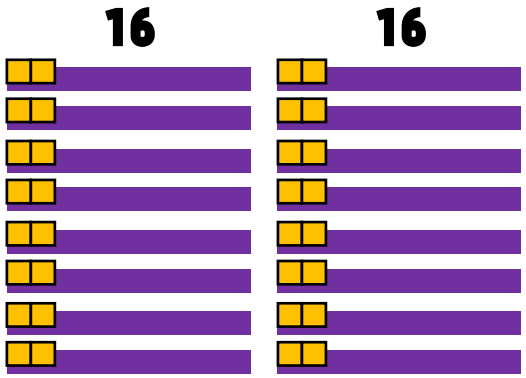
16 * 16 / 32 = 8 inst
8 way-bank
一共是8个warp
Bank Confilcts： 7 * 8 = 56
wavefronts：56 + 8 = 64

Shared Memory					
	Instructions	Requests	Wavefronts	% Peak	Bank Conflicts
Shared Load	8	8	8	0.01	0
Shared Load Matrix	0	0			
Shared Store	8	8	64	0.02	56
Shared Store From Global Load	0	0	0	0	0
Shared Atomic	0	0	0	0	0
Other	-	-	17	0.07	0
Total	16	16	89	0.10	56

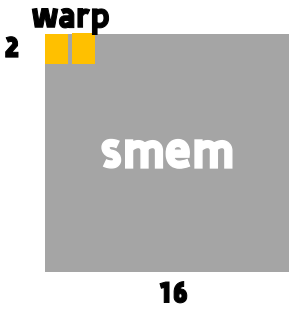
smem_store

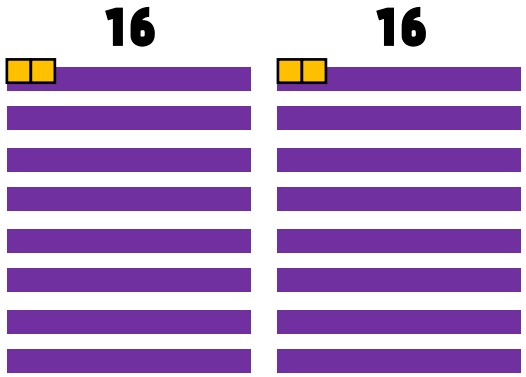






A C





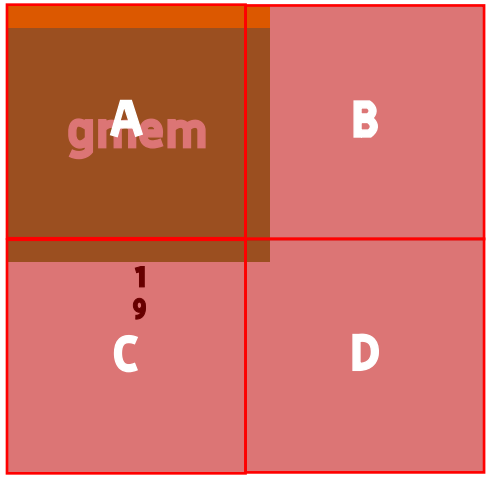
B D

Shared Memory

	Instructions	Requests	Wavefronts	% Peak	Bank Conflicts
Shared Load	18	18	18	0.02	0
Shared Load Matrix	0	0			
Shared Store	18	18	81	0.02	63
Shared Store From Global Load	0	0	0	0	0
Shared Atomic	0	0	0	0	0
Other	-	-	96	0.19	0
Total	36	36	195	0.24	63

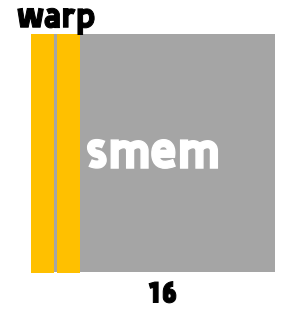
A: 16 * 16 / 32 = 8 inst 7 * 8 = 56 conflicts
B: 16 * 16 / 32 = 8 inst
C: 16 * 2 / 32 = 1 inst 7 * 1 = 7 conflicts
D: 16 * 2 / 32 = 1 inst

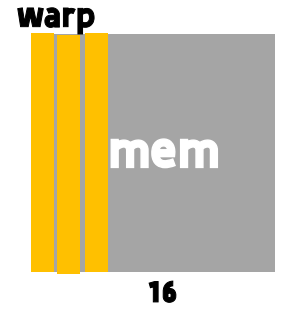
smem_store

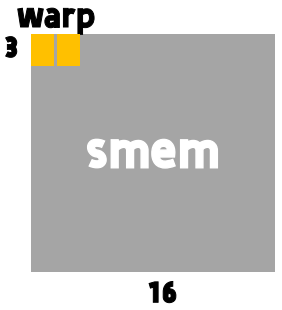


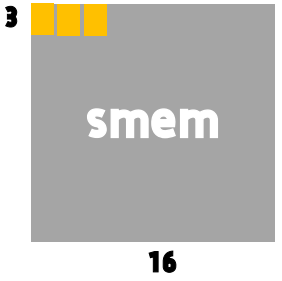
	Shared Memory				
	Instructions	Requests	Wavefronts	% Peak	Bank Conflicts
Shared Load	20	20	20	0.02	0
Shared Load Matrix	0	0			
Shared Store	20	20	100	0.03	80
Shared Store From Global Load	0	0	0	0	0
Shared Atomic	0	0	0	0	0
Other	-	-	92	0.20	0
Total	40	40	212	0.25	80

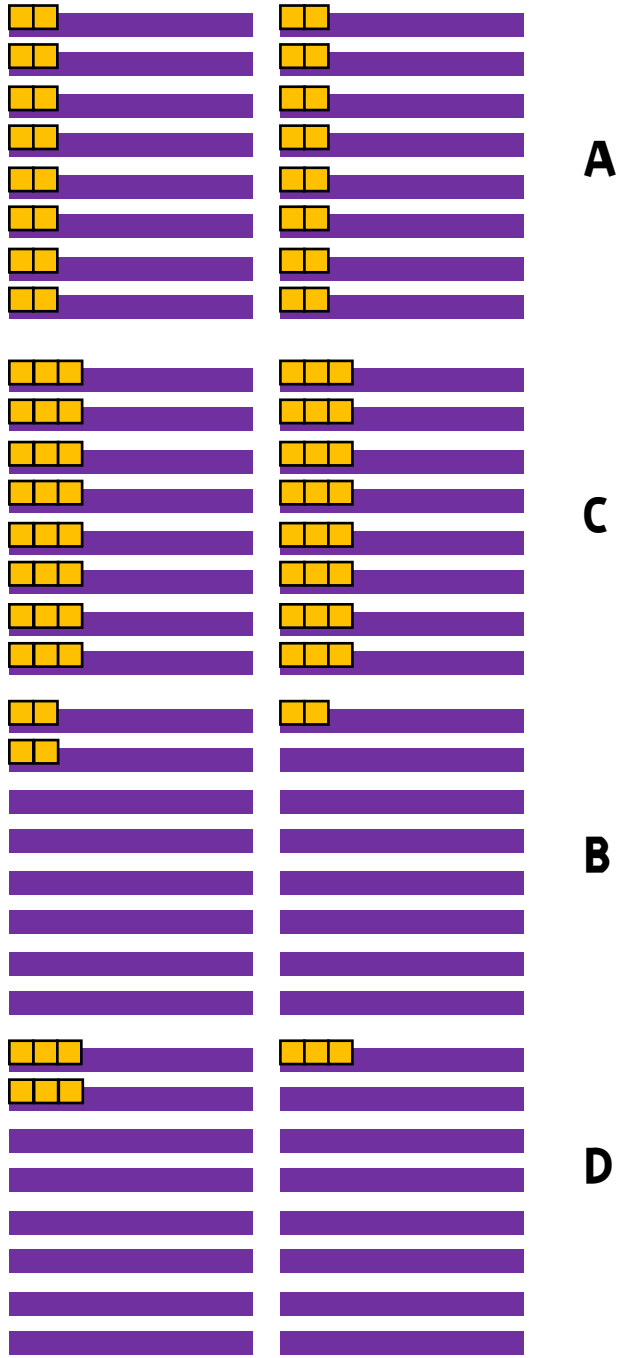
A: 16 * 16 / 32 = 8 inst 7 * 8 = 56 conflicts
B: 16 * 16 / 32 = 8 inst 1 * 8 = 8 conflicts
C: 16 * 4 / 32 = 2 inst 7 * 2 = 14 conflicts
D: 16 * 4 / 32 = 2 inst 1 * 2 = 2 conflicts





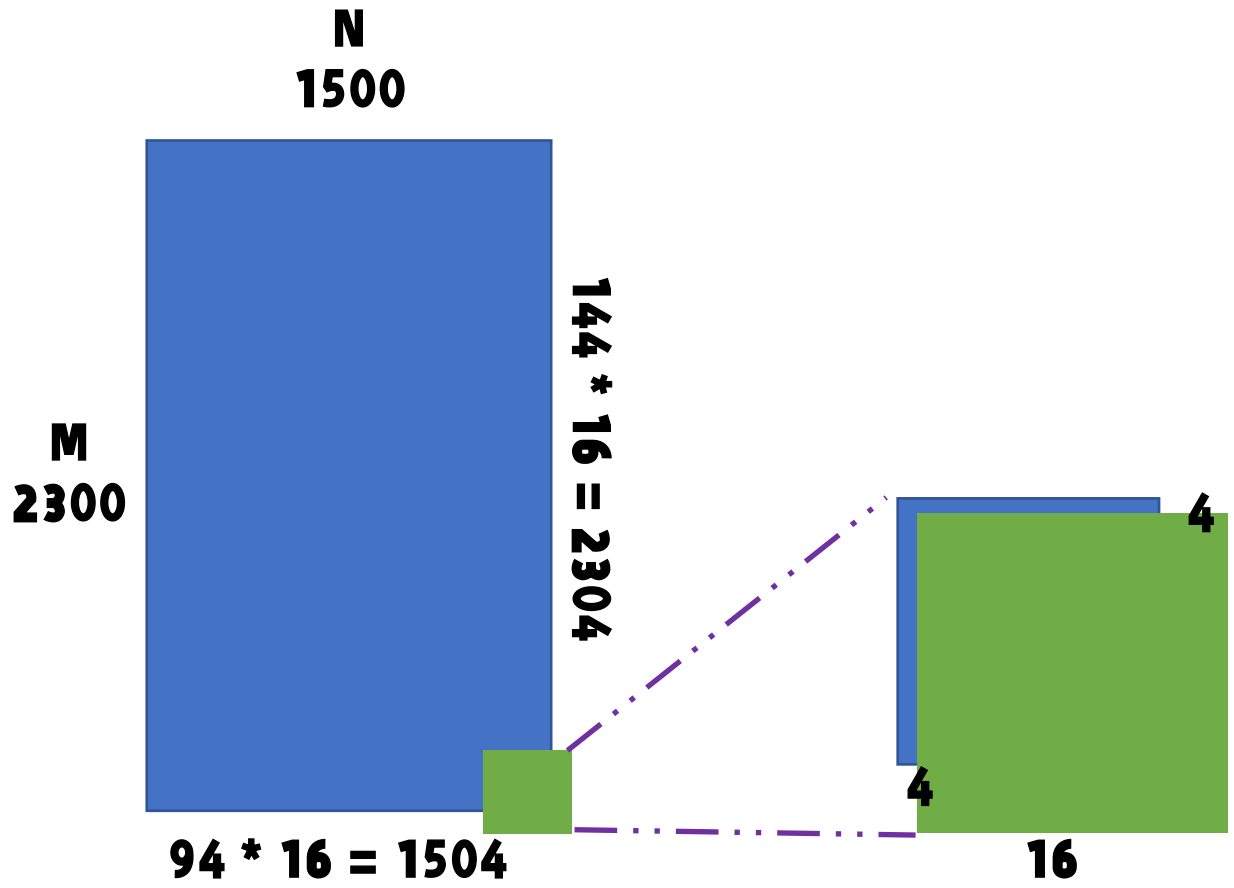


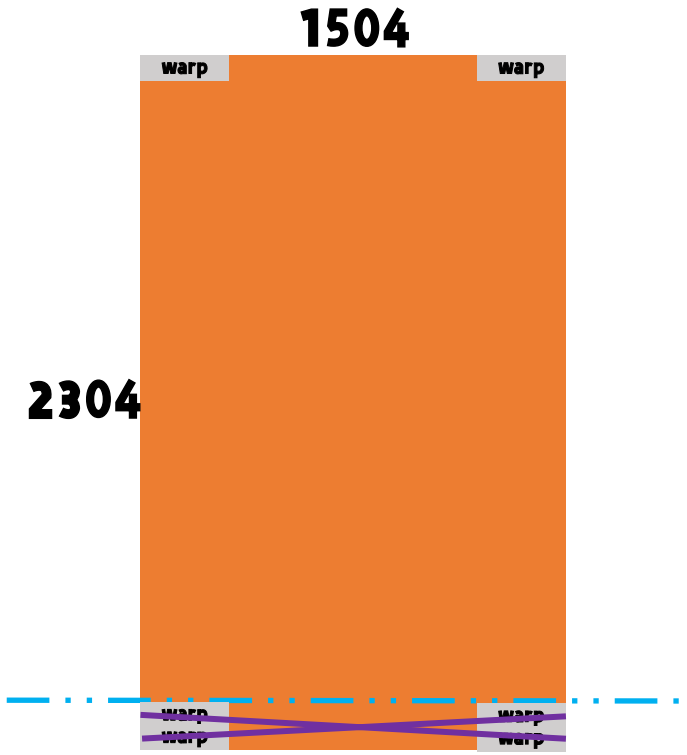




smem_load







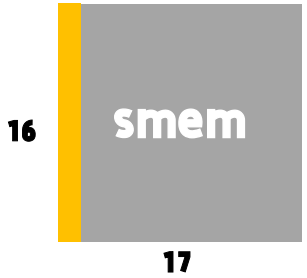
Shared Memory					
	Instructions	Requests	Wavefronts	% Peak	Bank Conflicts
Shared Load	108000	108000	120821	3.49	11308
Shared Load Matrix	0	0			
Shared Store	108100	108100	862500	6.22	754400
Shared Store From Global Load	0	0	0	0	0
Shared Atomic	0	0	0	0	0
Other	-	-	262202	26.22	31711
Total	216100	216100	1245523	35.93	797419

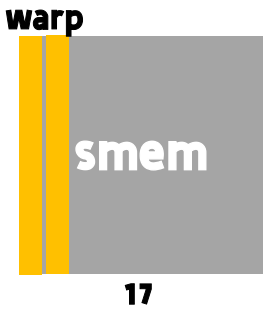
1504 * 2300 / 32 = 108100

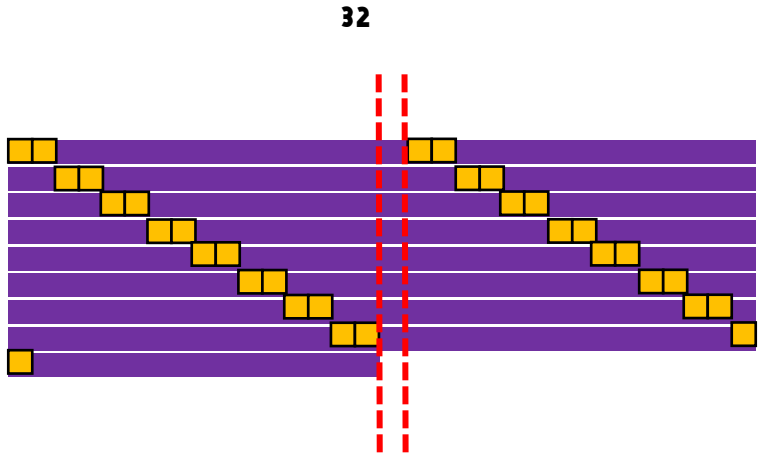
2304 * 1500 / 32 = 108000

smem_store









16 * 16 / 32 = 8 inst
2 way-bank
一共是8个warp
Bank Confilcts： 1 * 8 = 8
wavefronts： 8 + 8 = 16

Shared Memory

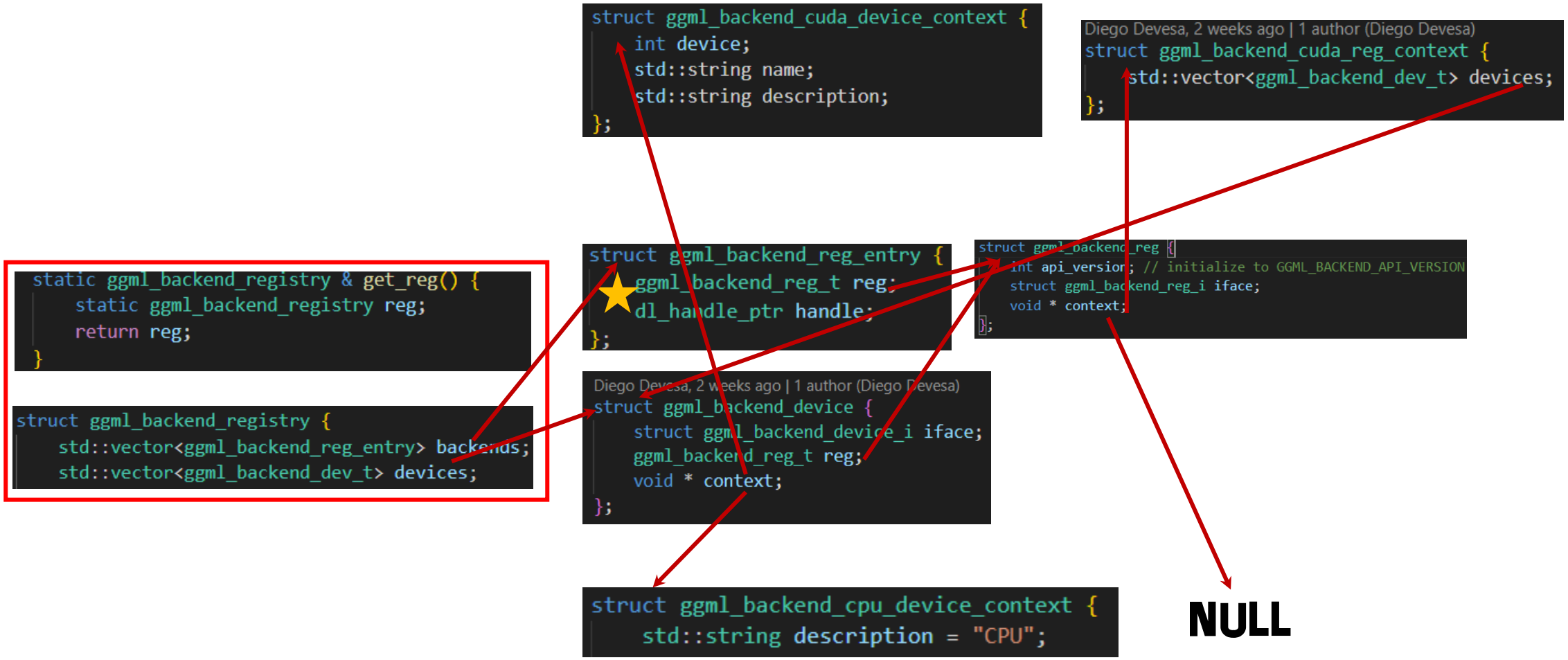
	Instructions	Requests	Wavefronts	% Peak	Bank Conflicts
Shared Load	8	8	16	0.02	8
Shared Load Matrix	0	0			
Shared Store	8	8	16	0.00	8
Shared Store From Global Load	0	0	0	0	0
Shared Atomic	0	0	0	0	0
Other	-	-	17	0.03	0
Total	16	16	49	0.06	16



LLAMA.CPP源码逐行调试带读

1.源码结构解析

llama.cpp_register_backend



ggml_ctx



219个tensor

这个是在构造llama_model_loader ml的时候创建的，ml是临时变量，用完就销毁!

llama_model_loader这个类的对象涵盖了gguf模型文件的全部信息

```cpp
// lists of buffer types used for each layer
using buft_list_t = std::vector<std::pair<ggml_backend_dev_t, ggml_backend_buffer_type_t>>;
```

```cpp
struct ggml_backend_buffer_type {
    struct ggml_backend_buffer_type_i  iface;
    ggml_backend_dev_t device;
    void * context;
};
```

```cpp
llama_model->impl->buft_list_t cpu_buft_list;
```

cpu

```cpp
    /* .context = */ new ggml::cpu::aarch64::extra_buffer_type(),
```

gpu

```cpp
/* .context  = */ nullptr,
```

cpu

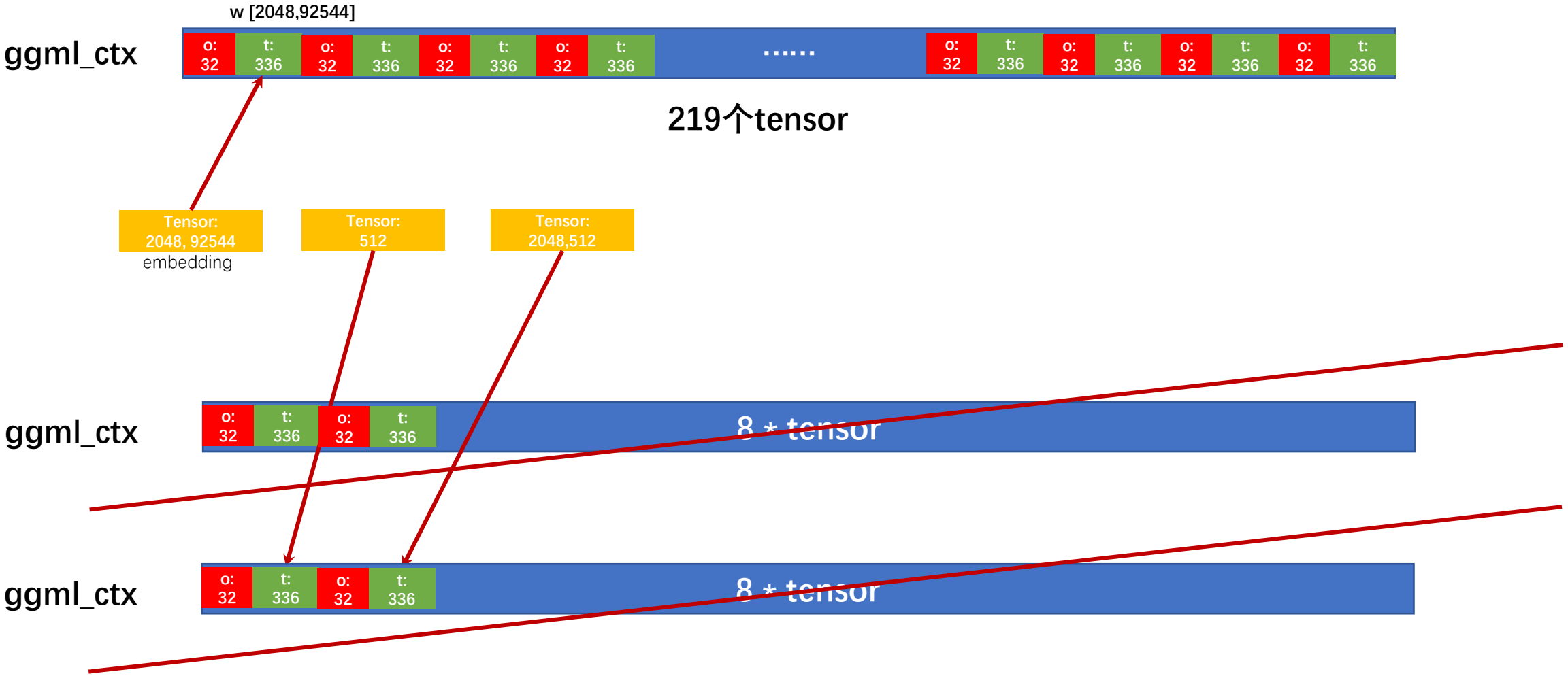
```cpp
/* .context  = */ nullptr,
```

```cpp
llama_model->impl->std::map<ggml_backend_dev_t, buft_list_t> gpu_buft_list;
```

gpu

```cpp
/* .context  = */ new ggml_backend_cuda_buffer_type_context{i, GGML_CUDA_NAME + std::to_string(i)},
```

```cpp
struct ggml_backend_cuda_buffer_type_context {
    int device;
    std::string name;
};
```


w [2048,92544]
ggml_ctx
o: 32
t: 336
.....
219个tensor
Tensor: 2048, 92544
embedding
Tensor: 512
Tensor: 2048,512
ggml_ctx
8 * tensor
ggml_ctx
8 * tensor

ggml_ctx



219个tensor

[2048,92544]



219+1（duplicated output）+ 48(24*2 duplicated rope freq)

[2048,1]



219+1（duplicated output）+ 48(24*2 duplicated rope freq)

[2048,1]



219+1（duplicated output） + 48(24*2 duplicated rope freq)

```
struct ggml_backend_buffer {
    struct ggml_backend_buffer_i  iface;
    ggml_backend_buffer_type_t    buft;
    void * context;
    size_t size;
    enum ggml_backend_buffer_usage usage;
};
```

```
struct ggml_backend_cuda_buffer_context {
    int device;
    void * dev_ptr = nullptr;
    std::string name;

    ggml_backend_cuda_buffer_context(int device, void * dev_ptr) :
        device(device), dev_ptr(dev_ptr),
        name(GGML_CUDA_NAME + std::to_string(device)) {
    }

    ~ggml_backend_cuda_buffer_context() {
        CUDA_CHECK(cudaFree(dev_ptr));
    }
};
```

[2048,92544]



219+1（duplicated output） + 48(24*2 duplicated rope freq)

```
struct ggml_backend_buffer {
    struct ggml_backend_buffer_i  iface;
    ggml_backend_buffer_type_t    buft;
    void * context;
    size_t size;
    enum ggml_backend_buffer_usage usage;
};
```

data

```cpp
struct llama_context {
    llama_context(const llama_model & model) : model(model), t_start_us(model.t_start_us), t_

    const struct llama_model & model;

    struct llama_cparams      cparams;
    struct llama_sbatch       sbatch;  // TODO: revisit if needed
    struct llama_kv_cache     kv_self;
    struct llama_adapter_cvec cvec;

    std::unordered_map<struct llama_adapter_lora *, float> lora;

    std::vector<ggml_backend_ptr>                          backends;
```

```cpp
struct ggml_backend {
    ggml_guid_t guid;
    struct ggml_backend_i iface;
    ggml_backend_dev_t device;
    void * context;
};
```

```cpp
struct ggml_backend_cuda_context {
    int device;
    std::string name;
    cudaEvent_t copy_event = nullptr;

    cudaStream_t streams[GGML_CUDA_MAX_DEVICES][GGML_CUDA_MAX_STREAMS] = { { nullptr } };
    cublasHandle_t cublas_handles[GGML_CUDA_MAX_DEVICES] = {nullptr};

    std::unique_ptr<ggml_cuda_graph> cuda_graph;
```

```cpp
struct ggml_backend_cpu_context {
    int                 n_threads;
    ggml_threadpool_t   threadpool;

    uint8_t *           work_data;
    size_t              work_size;

    ggml_abort_callback abort_callback;
    void *              abort_callback_data;
};
```

构建llama_model总结

llama_model

```
struct ggml_tensor * tok_embd   = nullptr;
struct ggml_tensor * type_embd  = nullptr;
struct ggml_tensor * pos_embd   = nullptr;
struct ggml_tensor * tok_norm   = nullptr;
struct ggml_tensor * tok_norm_b = nullptr;

struct ggml_tensor * output_norm     = nullptr;
struct ggml_tensor * output_norm_b   = nullptr;
struct ggml_tensor * output          = nullptr;
struct ggml_tensor * output_b        = nullptr;
struct ggml_tensor * output_norm_enc = nullptr;

// classifier
struct ggml_tensor * cls       = nullptr;
struct ggml_tensor * cls_b     = nullptr;
struct ggml_tensor * cls_out   = nullptr;
struct ggml_tensor * cls_out_b = nullptr;

struct ggml_tensor * conv1d   = nullptr;
struct ggml_tensor * conv1d_b = nullptr;

std::vector<llama_layer> layers;
private:
    struct impl;
    std::unique_ptr<impl> pimpl;
```

```
// contexts where the model tensors metadata is stored
std::vector<ggml_context_ptr> ctxs;

// the model memory buffers for the tensor data
std::vector<ggml_backend_buffer_ptr> bufs;

buft_list_t cpu_buft_list;
std::map<ggml_backend_dev_t, buft_list_t> gpu_buft_list;

struct layer_dev {
    ggml_backend_dev_t dev;
    buft_list_t * buft_list;
};

layer_dev dev_input = {};
layer_dev dev_output = {};
std::vector<layer_dev> dev_layer;
```

```
llama_model->impl->buft_list_t cpu_buft_list;
```

cpu extra `/* .context = */ new ggml::cpu::aarch64::extra_buffer_type(),`

gpu host `/* .context = */ nullptr,`

cpu `/* .context = */ nullptr,`

llama_model_loader

```
std::map<std::string, struct llama_tensor_weight, weight_name_comparer> weights_map;
std::unordered_map<std::string, struct llama_model_kv_override> kv_overrides;

gguf_context_ptr meta;
std::vector<ggml_context_ptr> contexts;
```



这个是在构造llama_model_loader ml的时候创建的，ml是临时变量，用完就销毁！

llama_model_loader这个类的对象涵盖了gguf模型文件的全部信息

llama_model

```cpp
struct ggml_tensor * tok_embd   = nullptr;
struct ggml_tensor * type_embd  = nullptr;
struct ggml_tensor * pos_embd   = nullptr;
struct ggml_tensor * tok_norm   = nullptr;
struct ggml_tensor * tok_norm_b = nullptr;

struct ggml_tensor * output_norm     = nullptr;
struct ggml_tensor * output_norm_b   = nullptr;
struct ggml_tensor * output          = nullptr;
struct ggml_tensor * output_b        = nullptr;
struct ggml_tensor * output_norm_enc = nullptr;

// classifier
struct ggml_tensor * cls       = nullptr;
struct ggml_tensor * cls_b     = nullptr;
struct ggml_tensor * cls_out   = nullptr;
struct ggml_tensor * cls_out_b = nullptr;

struct ggml_tensor * conv1d   = nullptr;
struct ggml_tensor * conv1d_b = nullptr;

std::vector<llama_layer> layers;
private:
    struct impl;
    std::unique_ptr<impl> pimpl;
```

```cpp
// contexts where the model tensors metadata is stored
std::vector<ggml_context_ptr> ctxs;

// the model memory buffers for the tensor data
std::vector<ggml_backend_buffer_ptr> bufs;

buft_list_t cpu_buft_list;
std::map<ggml_backend_dev_t, buft_list_t> gpu_buft_list;

struct layer_dev {
    ggml_backend_dev_t dev;
    buft_list_t * buft_list;
};

layer_dev dev_input = {};
layer_dev dev_output = {};
std::vector<layer_dev> dev_layer;
```

ne:2048,92544

o: 32 t: 336 cpu 内部只有一个tensor

219 + 1 + n_layer(24) * 2

o: 32 t: 336 gpu 内部有218个tensor

219 + 1 + n_layer(24) * 2

llama_model

```
struct ggml_tensor * tok_embd   = nullptr;
struct ggml_tensor * type_embd  = nullptr;
struct ggml_tensor * pos_embd   = nullptr;
struct ggml_tensor * tok_norm   = nullptr;
struct ggml_tensor * tok_norm_b = nullptr;

struct ggml_tensor * output_norm     = nullptr;
struct ggml_tensor * output_norm_b   = nullptr;
struct ggml_tensor * output          = nullptr;
struct ggml_tensor * output_b        = nullptr;
struct ggml_tensor * output_norm_enc = nullptr;

// classifier
struct ggml_tensor * cls       = nullptr;
struct ggml_tensor * cls_b     = nullptr;
struct ggml_tensor * cls_out   = nullptr;
struct ggml_tensor * cls_out_b = nullptr;

struct ggml_tensor * conv1d   = nullptr;
struct ggml_tensor * conv1d_b = nullptr;

std::vector<llama_layer> layers;
private:
    struct impl;
    std::unique_ptr<impl> pimpl;
```

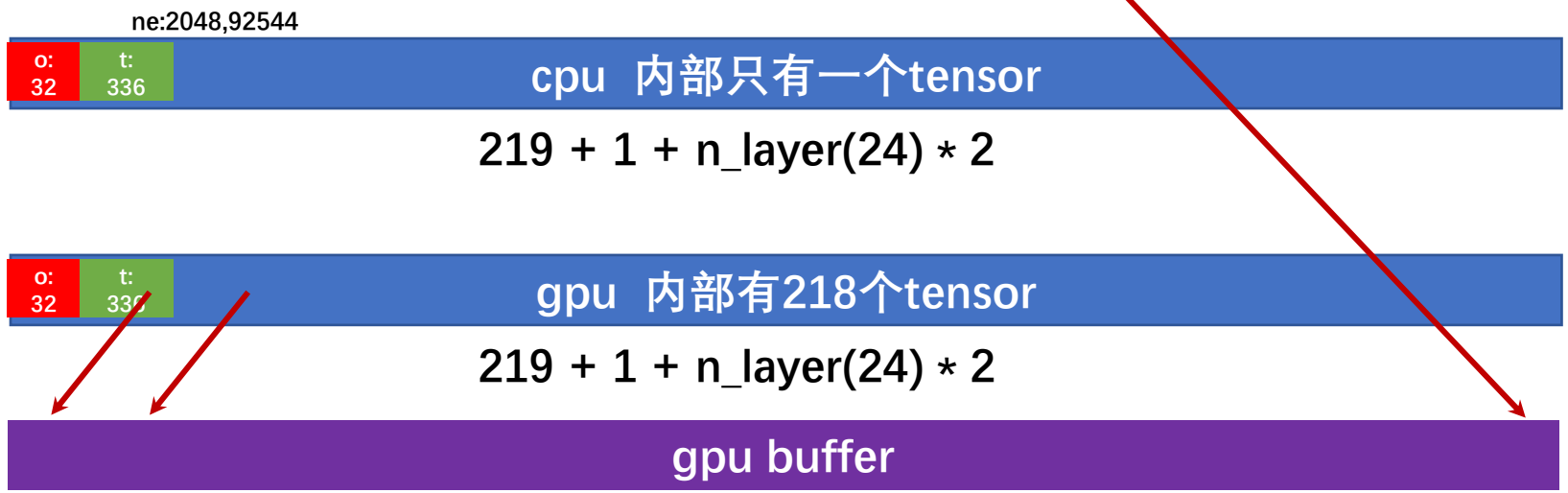
```
// contexts where the model tensors metadata is stored
std::vector<ggml_context_ptr> ctxs;

// the model memory buffers for the tensor data
std::vector<ggml_backend_buffer_ptr> bufs;

buft_list_t cpu_buft_list;
std::map<ggml_backend_dev_t, buft_list_t> gpu_buft_list;

struct layer_dev {
    ggml_backend_dev_t dev;
    buft_list_t * buft_list;
};

layer_dev dev_input = {};
layer_dev dev_output = {};
std::vector<layer_dev> dev_layer;
```



llama_context

```
std::vector<ggml_backend_ptr>                                  backends;
std::vector<std::pair<ggml_backend_t, ggml_backend_set_n_threads_t>> set_n_threads_fns;

ggml_backend_t backend_cpu = nullptr;

ggml_threadpool_t threadpool       = nullptr;
ggml_threadpool_t threadpool_batch = nullptr;
// host buffer for the model output (logits and embeddings)
ggml_backend_buffer_ptr buf_output;
// input tensors
struct ggml_tensor * inp_tokens;          // I32 [n_batch]
struct ggml_tensor * inp_embd;            // F32 [n_embd, n_batch]
struct ggml_tensor * inp_pos;             // I32 [n_batch]
struct ggml_tensor * inp_out_ids;         // I32 [n_outputs]
struct ggml_tensor * inp_KQ_mask;         // F32 [kv_size, n_batch]
struct ggml_tensor * inp_KQ_mask_swa;     // F32 [kv_size, n_batch]
struct ggml_tensor * inp_K_shift;         // I32 [kv_size]
struct ggml_tensor * inp_mean;            // F32 [n_batch, n_batch]
struct ggml_tensor * inp_cls;             // I32 [n_batch]
struct ggml_tensor * inp_s_copy;          // I32 [kv_size]
struct ggml_tensor * inp_s_mask;          // F32 [1, n_kv]
struct ggml_tensor * inp_s_seq;           // I32 [n_kv, n_batch]
struct ggml_tensor * inp_pos_bucket;      // I32 [n_batch|n_kv, n_batch]
struct ggml_tensor * inp_embd_enc;        // F32 [n_embd, n_outputs_enc]
struct ggml_tensor * inp_KQ_mask_cross;   // F32 [n_outputs_enc, n_batch]
struct llama_kv_cache      kv_self;
// memory buffers used to evaluate the model
std::vector<uint8_t>   buf_compute_meta;
ggml_backend_sched_ptr sched;
```

```
struct ggml_backend {
    ggml_guid_t guid;
    struct ggml_backend_i iface;
    ggml_backend_dev_t device;
    void * context;
};
```

```
struct ggml_backend_cuda_context {
    int device;
    std::string name;
    cudaEvent_t copy_event = nullptr;

    cudaStream_t streams[GGML_CUDA_MAX_DEVICES][GGML_CUDA_MAX_STREAMS] = { { nullptr } };
    cublasHandle_t cublas_handles[GGML_CUDA_MAX_DEVICES] = {nullptr};

    std::unique_ptr<ggml_cuda_graph> cuda_graph;
```

```
struct ggml_backend_cpu_context {
    int                 n_threads;
    ggml_threadpool_t   threadpool;

    uint8_t *           work_data;
    size_t              work_size;

    ggml_abort_callback abort_callback;
    void *              abort_callback_data;
};
```

```
struct llama_kv_cache {
    bool has_shift = false;
    bool do_defrag = false;
    bool recurrent = false; // with recurrent sta
    bool v_trans   = true;  // the value tensor i
    bool can_shift = false;

    // Note: The value of head isn't only used to
    // for a free KV slot. llama_decode_internal
    // cannot be freely changed after a slot has
    uint32_t head = 0;
    uint32_t size = 0;
    uint32_t used = 0; // used cells (i.e. at lea

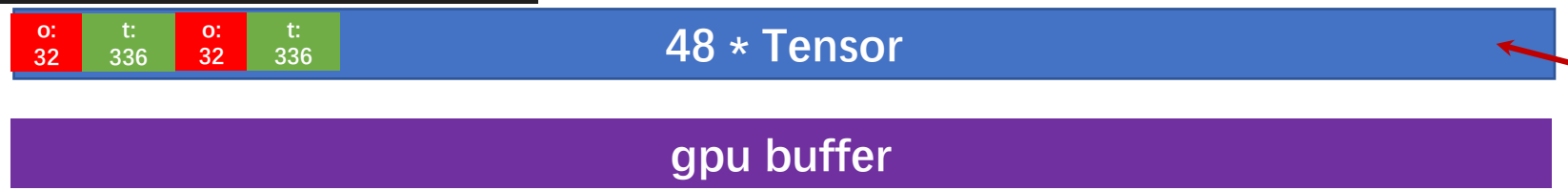
    // computed before each graph build
    uint32_t n = 0;

    ggml_type type_k = GGML_TYPE_F16;
    ggml_type type_v = GGML_TYPE_F16;

    std::vector<llama_kv_cell> cells;

    std::vector<struct ggml_tensor *> k_l; // per
    std::vector<struct ggml_tensor *> v_l;

    std::vector<ggml_context_ptr> ctxs;
    std::vector<ggml_backend_buffer_ptr> bufs;
```



ggml_backend_sched

```
ggml_backend_t backends[GGML_SCHED_MAX_BACKENDS];
ggml_backend_buffer_type_t bufts[GGML_SCHED_MAX_BACKENDS];
ggml_gallocr_t galloc;

// hash map of the nodes in the graph
struct ggml_hash_set  hash_set;
int                 * hv_tensor_backend_ids; // [hash_set.size]
struct ggml_tensor ** hv_tensor_copies;      // [hash_set.size][n_backends][n_copies]

int * node_backend_ids; // [graph_size]
int * leaf_backend_ids; // [graph_size]

int * prev_node_backend_ids; // [graph_size]
int * prev_leaf_backend_ids; // [graph_size]

// copy of the graph with modified inputs
struct ggml_cgraph graph;

// graph splits
struct ggml_backend_sched_split * splits;
int n_splits;
int splits_capacity;

// pipeline parallelism support
int n_copies;
int cur_copy;
ggml_backend_event_t events[GGML_SCHED_MAX_BACKENDS][GGML_SCHED_MAX_COPIES];
struct ggml_tensor * graph_inputs[GGML_SCHED_MAX_SPLIT_INPUTS];
int n_graph_inputs;

struct ggml_context * ctx;
char * context_buffer;
size_t context_buffer_size;
```

```
{...}
  size = 8209
> used = 0x55555649fad0
> keys = 0x55555616cac0
```

8192 + 8192*10*2

8192*10*2*tensor + cgraph

```
struct ggml_hash_set {
    size_t size;
    ggml_bitset_t * used;
    struct ggml_tensor ** keys;
};
```

```
struct ggml_cgraph {
    int size;    // maximum number of nodes
    int n_nodes; // number of nodes current
    int n_leafs; // number of leafs current

    struct ggml_tensor ** nodes;     // ten
    struct ggml_tensor ** grads;     // the
    struct ggml_tensor ** grad_accs; // acc
    struct ggml_tensor ** leafs;     // ten

    struct ggml_hash_set visited_hash_set;

    enum ggml_cgraph_eval_order order;
};
```

```
struct ggml_backend_sched_split {
    int                   backend_id;
    int                   i_start;
    int                   i_end;
    struct ggml_tensor *  inputs[GGML_SCHED_MAX_SPLIT_INPUTS];
    int                   n_inputs;
    // graph view of this split
    struct ggml_cgraph    graph;
};
```

ggml_backend_sched

```
ggml_backend_t backends[GGML_SCHED_MAX_BACKENDS];
ggml_backend_buffer_type_t bufts[GGML_SCHED_MAX_BACKENDS];
ggml_gallocr_t galloc;

// hash map of the nodes in the graph
struct ggml_hash_set  hash_set;
int                 * hv_tensor_backend_ids; // [hash_set.size]
struct ggml_tensor ** hv_tensor_copies;      // [hash_set.size][n_backends][n_copies]

int * node_backend_ids; // [graph_size]
int * leaf_backend_ids; // [graph_size]

int * prev_node_backend_ids; // [graph_size]
int * prev_leaf_backend_ids; // [graph_size]

// copy of the graph with modified inputs
struct ggml_cgraph graph;

// graph splits
struct ggml_backend_sched_split * splits;
int n_splits;
int splits_capacity;

// pipeline parallelism support
int n_copies;
int cur_copy;
ggml_backend_event_t events[GGML_SCHED_MAX_BACKENDS][GGML_SCHED_MAX_COPIES];
struct ggml_tensor * graph_inputs[GGML_SCHED_MAX_SPLIT_INPUTS];
int n_graph_inputs;

struct ggml_context * ctx;
char * context_buffer;
size_t context_buffer_size;
```

8192 + 8192*10*2

```
struct ggml_gallocr {
    ggml_backend_buffer_type_t * bufts; // [n_buffers]
    ggml_backend_buffer_t * buffers; // [n_buffers]
    struct ggml_dyn_tallocr ** buf_tallocs; // [n_buffers]
    int n_buffers;

    struct ggml_hash_set hash_set;
    struct hash_node * hash_values; // [hash_set.size]

    struct node_alloc * node_allocs; // [n_nodes]
    int n_nodes;

    struct leaf_alloc * leaf_allocs; // [n_leafs]
    int n_leafs;
};
```

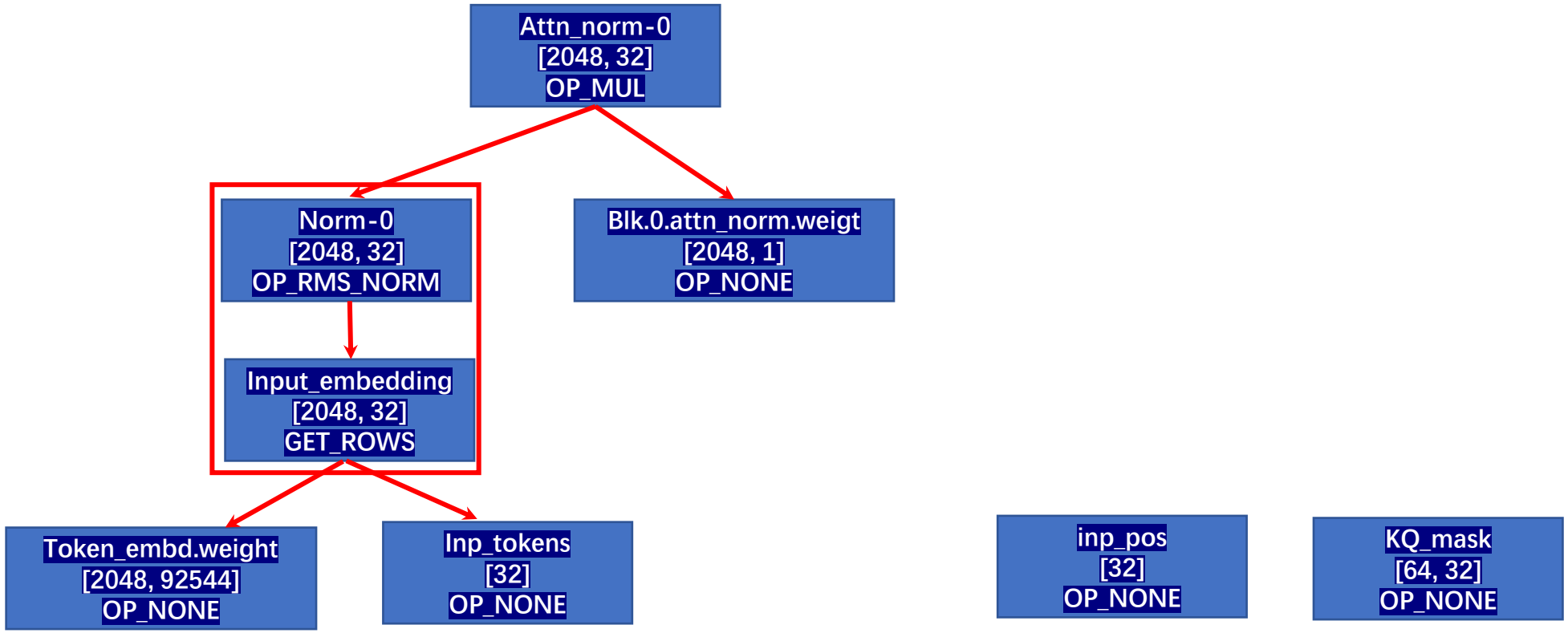
```
struct leaf_alloc {
    struct tensor_alloc leaf;
};

struct node_alloc {
    struct tensor_alloc dst;
    struct tensor_alloc src[GGML_MAX_SRC];
};
```

```
struct hash_node {
    int n_children;
    int n_views;
    int buffer_id;
    size_t offset; // off
    bool allocated;
};
```

```
struct ggml_dyn_tallocr {
    size_t alignment;
    int n_free_blocks;
    struct free_block free_blocks[MAX_FREE_BLOCKS];
    size_t max_size;
};
```

```
struct free_block {
    size_t offset;
    size_t size;
};
```

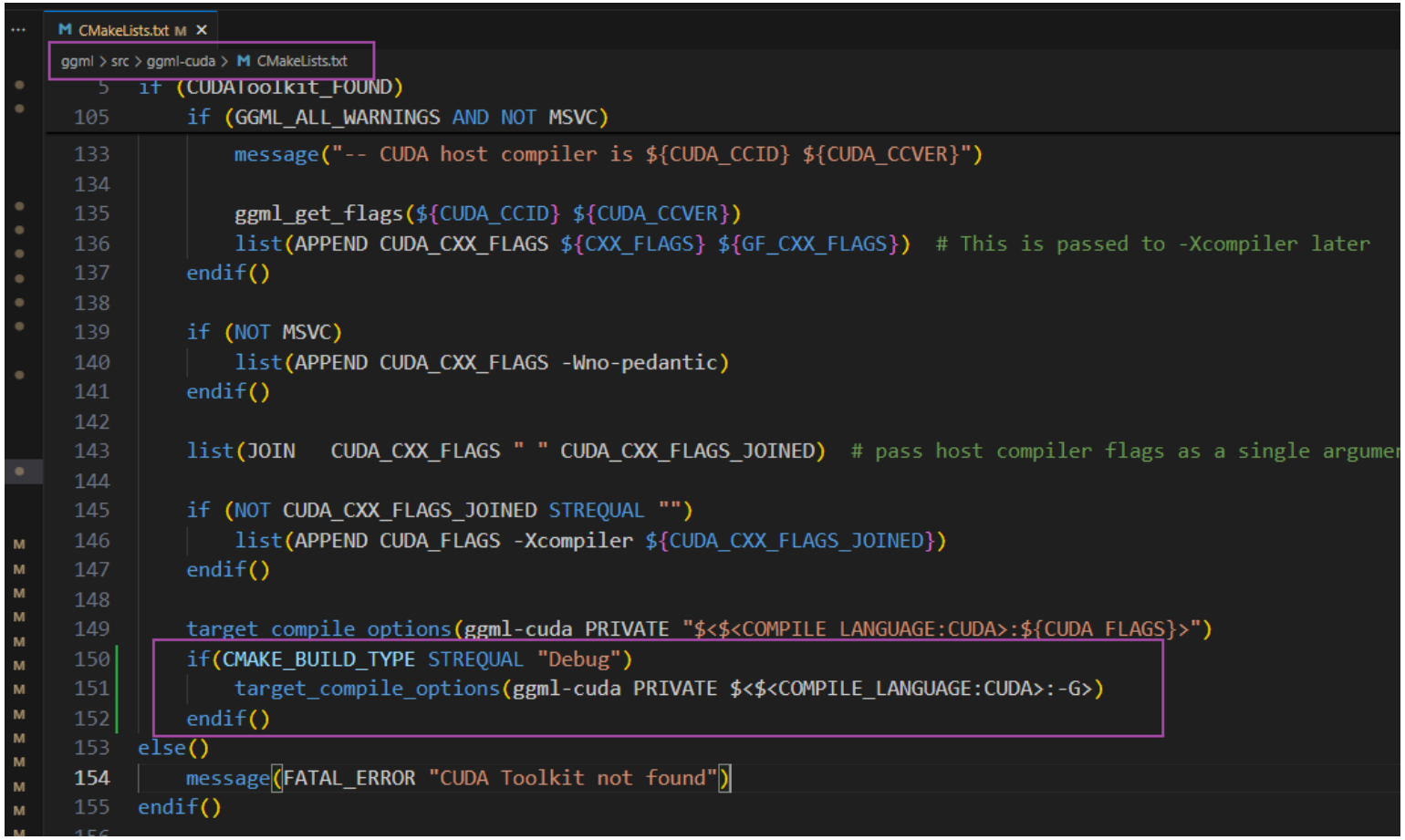

Attn_norm-0
[2048, 32]
OP_MUL
Norm-0
[2048, 32]
OP_RMS_NORM
Blk.0.attn_norm.weigt
[2048, 1]
OP_NONE
Input_embedding
[2048, 32]
GET_ROWS
Token_embd.weight
[2048, 92544]
OP_NONE
Inp_tokens
[32]
OP_NONE
inp_pos
[32]
OP_NONE
KQ_mask
[64, 32]
OP_NONE



LLAMA.CPP源码逐行调试带读

2.算子源码解析

WSL2搭建调试环境


CMakeLists.txt M
ggml > src > ggml-cuda > CMakeLists.txt
if (CUDAToolkit_FOUND)
if (GGML_ALL_WARNINGS AND NOT MSVC)
message("-- CUDA host compiler is ${CUDA_CCID} ${CUDA_CCVER}")
ggml_get_flags(${CUDA_CCID} ${CUDA_CCVER})
list(APPEND CUDA_CXX_FLAGS ${CXX_FLAGS} ${GF_CXX_FLAGS}) # This is passed to -Xcompiler later
endif()
if (NOT MSVC)
list(APPEND CUDA_CXX_FLAGS -Wno-pedantic)
endif()
list(JOIN CUDA_CXX_FLAGS " " CUDA_CXX_FLAGS_JOINED) # pass host compiler flags as a single argumer
if (NOT CUDA_CXX_FLAGS_JOINED STREQUAL "")
list(APPEND CUDA_FLAGS -Xcompiler ${CUDA_CXX_FLAGS_JOINED})
endif()
target_compile_options(ggml-cuda PRIVATE "$<$<COMPILE_LANGUAGE:CUDA>:${CUDA_FLAGS}>")
if(CMAKE_BUILD_TYPE STREQUAL "Debug")
target_compile_options(ggml-cuda PRIVATE $<$<COMPILE_LANGUAGE:CUDA>:-G>)
endif()
else()
message(FATAL_ERROR "CUDA Toolkit not found")
endif()

GET_ROWS 算子

```
node = 0x555557028b30
  type = GGML_TYPE_F32
> buffer = 0x555556237170
ne = [4]
  0 = 2048
  1 = 5
  2 = 1
  3 = 1
> nb = [4]
  op = GGML_OP_GET_ROWS
> op_params = [16]
  flags = 0
> src = [10]
> view_src = 0x0
  view_offs = 0
  data = 0x204c5c780
> name = [64]
  extra = 0x0
> padding = [8]
```

```
0 = 0x555556930ea0
  type = GGML_TYPE_F32
> buffer = 0x555556949bd0
ne = [4]
  0 = 2048
  1 = 92544
  2 = 1
  3 = 1
> nb = [4]
  op = GGML_OP_NONE
```

```
1 = 0x5555570289c0
  type = GGML_TYPE_I32
> buffer = 0x555556237170
ne = [4]
  0 = 5
  1 = 1
  2 = 1
  3 = 1
> nb = [4]
  op = GGML_OP_NONE
```

LLM_int8.py | InternLM_HF_demo.py | modeling_internlm2.py | _contextlib.py | utils.py

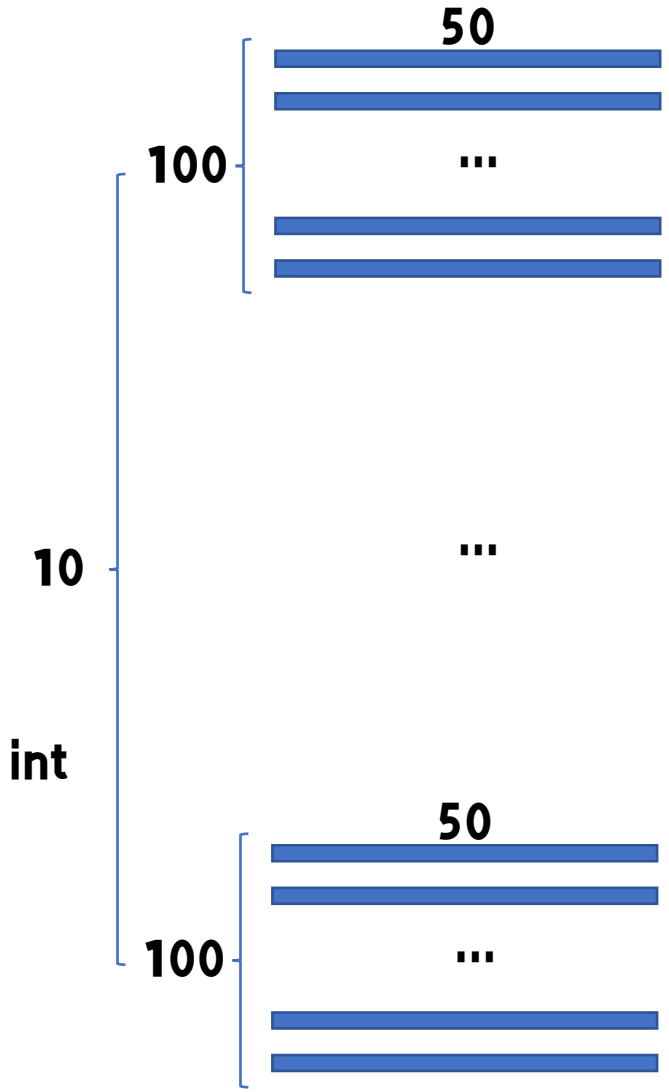
```
946         use_cache = use_cache if use_cache is not None else self.config.use_cache
947         return_dict = return_dict if return_dict is not None else self.config.use_retur
948
949         if (input_ids is None) ^ (inputs_embeds is not None):
950             raise ValueError(
951                 "You cannot specify both input_ids and inputs_embeds at the same time,
952             )
953
954         if self.gradient_checkpointing and self.training and use_cache:
955             logger.warning_once(
956                 "`use_cache=True` is incompatible with gradient checkpointing. Setting
957             )
958             use_cache = False
959
960         if inputs_embeds is None:
                inputs_embeds = self.tok_embeddings(input_ids)
```

```
result = {Tensor: (1, 5)} tensor([[   1, 9843,  983,  963,  505]], device='cuda:0')
```

```
result = {Embedding} Embedding(92544, 2048, padding_idx=2)
```

```
result = {Tensor: (1, 5, 2048)} tensor([[[ 2.1458e-04, -4.7493e-04,  3.9673e-
```

GET_ROWS 算子



pytorch【10, 100, 50】
Value = a[5][7][24]

llama.cpp【50, 100, 10, 1】【4, 200, 20000, 200000】
Value = *(int32_t*)((char*) a+24*4+7*200+5*20000)

想获取第25374个元素，其对应的各个维度的索引分别是多少？

25374/(50*100) = 5 (ne2_index)
(25374 - 5*50*100)/50 = 7 (ne1_index)
25374 - 5*50*100 - 7*50 = 24(ne0_index)

想获取第25374个元素的值
(int32_t)((char*)src->data + 5*20000 + 7*200 + 24*4)

GET_ROWS 算子

```
const int64_t ne00 = (src0)->ne[0];
(void) (ne00);
constint64_t ne01 = (src0)->ne[1];
(void) (ne01);
constint64_t ne02 = (src0)->ne[2];
(void) (ne02);
constint64_t ne03 = (src0)->ne[3];
(void) (ne03);

constsize_t nb00 = (src0)->nb[0];
(void) (nb00);
constsize_t nb01 = (src0)->nb[1];
(void) (nb01);
constsize_t nb02 = (src0)->nb[2];
(void) (nb02);
constsize_t nb03 = (src0)->nb[3];
(void) (nb03);
```

```
const int64_t ne10 = (src1)->ne[0];
(void) (ne10);
constint64_t ne11 = (src1)->ne[1];
(void) (ne11);
constint64_t ne12 = (src1)->ne[2];
(void) (ne12);
constint64_t ne13 = (src1)->ne[3];
(void) (ne13);

constsize_t nb10 = (src1)->nb[0];
(void) (nb10);
constsize_t nb11 = (src1)->nb[1];
(void) (nb11);
constsize_t nb12 = (src1)->nb[2];
(void) (nb12);
constsize_t nb13 = (src1)->nb[3];
(void) (nb13);
```

```
const int64_t ne0 = (dst)->ne[0];
(void) (ne0);
const int64_t ne1 = (dst)->ne[1];
(void) (ne1);
const int64_t ne2 = (dst)->ne[2];
(void) (ne2);
const int64_t ne3 = (dst)->ne[3];
(void) (ne3);

const size_t nb0 = (dst)->nb[0];
(void) (nb0);
const size_t nb1 = (dst)->nb[1];
(void) (nb1);
const size_t nb2 = (dst)->nb[2];
(void) (nb2);
const size_t nb3 = (dst)->nb[3];
(void) (nb3);
```

GET_ROWS 算子

```c
static void ggml_compute_forward_get_rows_f32(
        const struct ggml_compute_params * params,
              struct ggml_tensor * dst) {

    const struct ggml_tensor * src0 = dst->src[0];
    const struct ggml_tensor * src1 = dst->src[1];

    GGML_TENSOR_BINARY_OP_LOCALS

    const int64_t nc = ne00;
    const int64_t nr = ggml_nelements(src1);

    assert(ne0  == nc);
    assert(ne02 == ne11);
    assert(nb00 == sizeof(float));
    assert(ggml_nrows(dst) == nr);

    const int ith = params->ith;
    const int nth = params->nth;

    // rows per thread
    const int dr = (nr + nth - 1)/nth;

    // row range for this thread
    const int ir0 = dr*ith;
    const int ir1 = MIN(ir0 + dr, nr);

    for (int64_t i = ir0; i < ir1; ++i) {
        const int64_t i12 = i/(ne11*ne10);
        const int64_t i11 = (i - i12*ne11*ne10)/ne10;
        const int64_t i10 = (i - i12*ne11*ne10 - i11*ne10);
        const int64_t i01 = *(int32_t *) ((char *) src1->data + i10*nb10 + i11*nb11 + i12*nb12);

        GGML_ASSERT(i01 >= 0 && i01 < ne01);

        ggml_vec_cpy_f32(nc,
                (float *) ((char *)  dst->data + i10*nb1  + i11*nb2  + i12*nb3),
                (float *) ((char *) src0->data + i01*nb01 + i11*nb02 + i12*nb03));
    }
}
```

用于控制多线程

src0 [2048, 92544, 1, 1]
src1 [5, 1, 1, 1]
dst [2048, 5, 1, 1]

for (int i = ir0; i < ir1; i++)

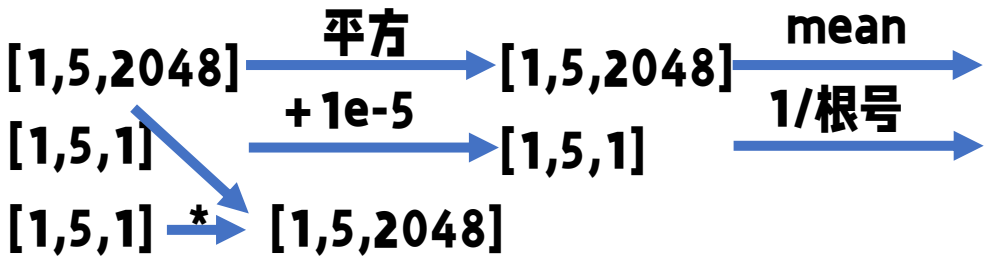
- 从src1中取第i个数，记为index
- 从src0中第1个维度取第index个数据，记为data
- 将data写入dst

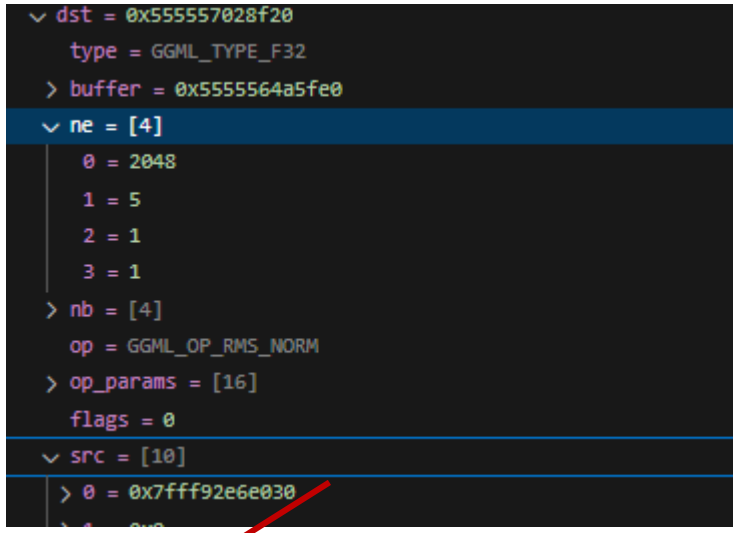
RMS_NORM算子

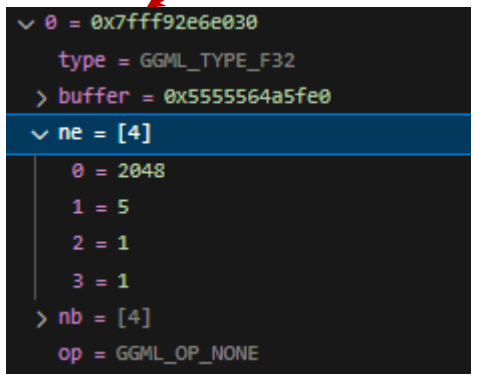
$$RMSNorm: y = \frac{x}{\sqrt{Mean(x^2)+\epsilon}} * \gamma$$

2048

$$Mean(x^2) = \frac{1}{N}\sum_{i=1}^{N} x_i^2$$







RMS_NORM算子

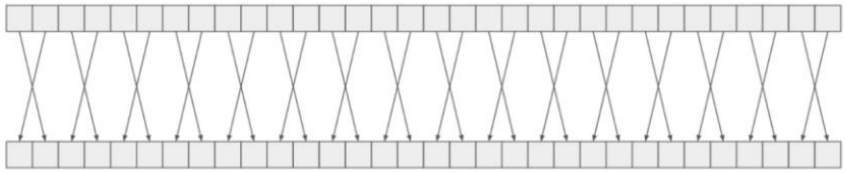
$$RMSNorm: y = \frac{x}{\sqrt{Mean(x^2)+\epsilon}} * \gamma$$

2048

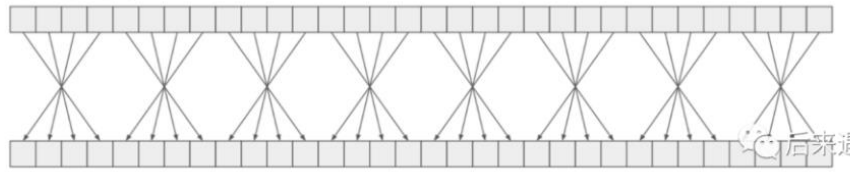
$$Mean(x^2) = \frac{1}{N}\sum_{i=1}^{N} x_i^2$$

Warp Broadcast

```
T __shfl_xor_sync(unsigned mask, T var, int laneMask, int width=warpSize);
int y = __shfl_xor_sync(0xffffffff,x, 1);
```



```
int y = __shfl_ xor_sync(0xffffffff,x, 3);
```



后浪遇见AI

2048

block(1024 thread)

WARP

16

8

4

2

1

1024 NUMS

WARP

RMS_NORM算子

```cpp
template <int block_size>
static __global__ void rms_norm_f32(const float * x, float * dst, const int ncols, const float eps) {
    const int row = blockIdx.x*blockDim.y + threadIdx.y;
    const int tid = threadIdx.x;

    float tmp = 0.0f; // partial sum for thread in warp

    for (int col = tid; col < ncols; col += block_size) {
        const float xi = x[row*ncols + col];
        tmp += xi * xi;
    }

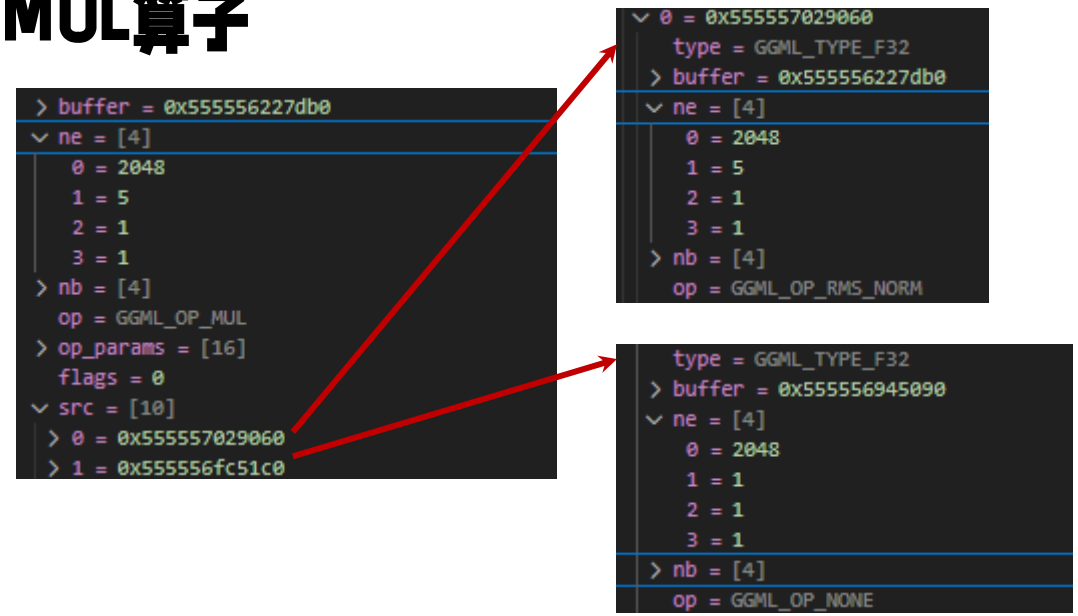
    // sum up partial sums
    tmp = warp_reduce_sum(tmp);
    if (block_size > WARP_SIZE) {
        __shared__ float s_sum[32];
        int warp_id = threadIdx.x / WARP_SIZE;
        int lane_id = threadIdx.x % WARP_SIZE;
        if (lane_id == 0) {
            s_sum[warp_id] = tmp;
        }
        __syncthreads();
        tmp = s_sum[lane_id];
        tmp = warp_reduce_sum(tmp);
    }

    const float mean = tmp / ncols;
    const float scale = rsqrtf(mean + eps);

    for (int col = tid; col < ncols; col += block_size) {
        dst[row*ncols + col] = scale * x[row*ncols + col];
    }
}
```

无论Tensor的维度是多少都会变成1024，而下面的代码就是为了处理1024个数的规约

MUL算子



02 广播的规则

Numpy的广播遵循一组严格的规则，设定这组规则是为了决定两个数组之间的操作，其规则如下：

规则1:如果两个数组的维度不相同，那么小维度数组的形状将会在最左边补1.

Llama.cpp不需要补1，因为本身就是1

规则2:如果两个数组的形状在任何一个维度上都不匹配，那么数组的形状会沿着维度为1的维度拓展以匹配另外一个数组形状。

不可以扩展，因为没有为其预留扩展的空间

规则3:如果两个数组的形状在任何一个维度上都不匹配并且没有任何一个维度等于1，那么会引发异常。

[Python入门教程(五):Numpy计算之广播]

MUL算子

```
// collapse dimensions until first broadcast dimension
int64_t cne[] = {ne0, ne1, ne2, ne3};
int64_t cne0[] = {ne00, ne01, ne02, ne03};
int64_t cne1[] = {ne10, ne11, ne12, ne13};

size_t cnb[] = {nb0, nb1, nb2, nb3};
size_t cnb0[] = {nb00, nb01, nb02, nb03};
size_t cnb1[] = {nb10, nb11, nb12, nb13};

auto collapse = [](int64_t cne[]) {
    cne[0] *= cne[1];
    cne[1] = cne[2];
    cne[2] = cne[3];
    cne[3] = 1;
};

auto collapse_nb = [](size_t cnb[], const int64_t cne[]) {
    cnb[1] *= cne[1];
    cnb[2] *= cne[2];
    cnb[3] *= cne[3];
};

if (ggml_is_contiguous(src0) && ggml_is_contiguous(src1) && ggml_is_contiguous(dst)) {
    for (int i = 0; i < 4; i++) {
        if (nr[i] != 1) {
            break;
        }
        if (i > 0) {
            collapse_nb(cnb, cne);
            collapse_nb(cnb0, cne0);
            collapse_nb(cnb1, cne1);
            collapse(cne);
            collapse(cne0);
            collapse(cne1);
        }
    }
}
```

折叠维度直到第一个广播维度

[5,2,3,1] * [5,2,1,1]

↓

[10,3,1,1] * [10,1,1,1]

[5,2,3,5] * [5,2,3,1]

↓

[30,5,1,1] * [30,1,1,1]

[5,2,3,5] * [5,2,1,1]

↓

[10,3,5,1] * [10,1,1,1]

MUL算子

```cpp
template<float (*bin_op)(const float, const float), typename src0_t, typename src1_t, typename dst_t>
static __global__ void k_bin_bcast(const src0_t * src0, const src1_t * src1, dst_t * dst,
        int ne0, int ne1, int ne2, int ne3,
        int ne10, int ne11, int ne12, int ne13,
        /*int s0, */ int s1,  int s2,  int s3,
        /*int s00,*/ int s01, int s02, int s03,
        /*int s10,*/ int s11, int s12, int s13) {
    const int i0s = blockDim.x*blockIdx.x + threadIdx.x;
    const int i1 = (blockDim.y*blockIdx.y + threadIdx.y);
    const int i2 = (blockDim.z*blockIdx.z + threadIdx.z) / ne3;
    const int i3 = (blockDim.z*blockIdx.z + threadIdx.z) % ne3;

    if (i0s >= ne0 || i1 >= ne1 || i2 >= ne2 || i3 >= ne3) {
        return;
    }

    const int i11 = i1 % ne11;
    const int i12 = i2 % ne12;
    const int i13 = i3 % ne13;

    const size_t i_src0 =  i3*s03 +  i2*s02 +  i1*s01;
    const size_t i_src1 = i13*s13 + i12*s12 + i11*s11;
    const size_t i_dst  =  i3*s3  +  i2*s2  +  i1*s1;

    const src0_t * src0_row = src0 + i_src0;
    const src1_t * src1_row = src1 + i_src1;
    dst_t * dst_row = dst + i_dst;

    for (int i0 = i0s; i0 < ne0; i0 += blockDim.x*gridDim.x) {
        const int i10 = i0 % ne10;
        dst_row[i0] = (dst_t)bin_op(src0 ? (float)src0_row[i0] : 0.0f, (float)src1_row[i10]);
    }
}
```

block和grid有3维，但是tensor有4维

保证src1只取第一行（只有一行）

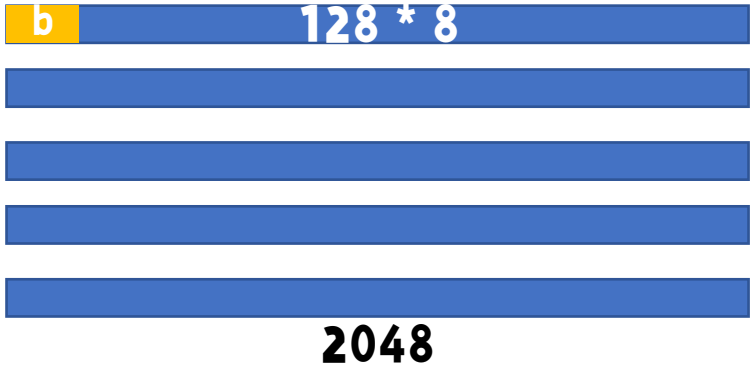
每个block对应哪一行

```
> buffer = 0x555556227db0
v ne = [4]
    0 = 2048
    1 = 5
    2 = 1
    3 = 1
> nb = [4]
  op = GGML_OP_MUL
> op_params = [16]
  flags = 0
v src = [10]
  > 0 = 0x555557029060
  > 1 = 0x555556fc51c0
```

```
v 0 = 0x555557029060
    type = GGML_TYPE_F32
  > buffer = 0x555556227db0
v ne = [4]
    0 = 2048
    1 = 5
    2 = 1
    3 = 1
  > nb = [4]
    op = GGML_OP_RMS_NORM
```

```
    type = GGML_TYPE_F32
  > buffer = 0x555556945090
v ne = [4]
    0 = 2048
    1 = 1
    2 = 1
    3 = 1
  > nb = [4]
    op = GGML_OP_NONE
```

block_size [128, 1, 1]
grid_size [8, 5, 1]



MAT_MUL算子

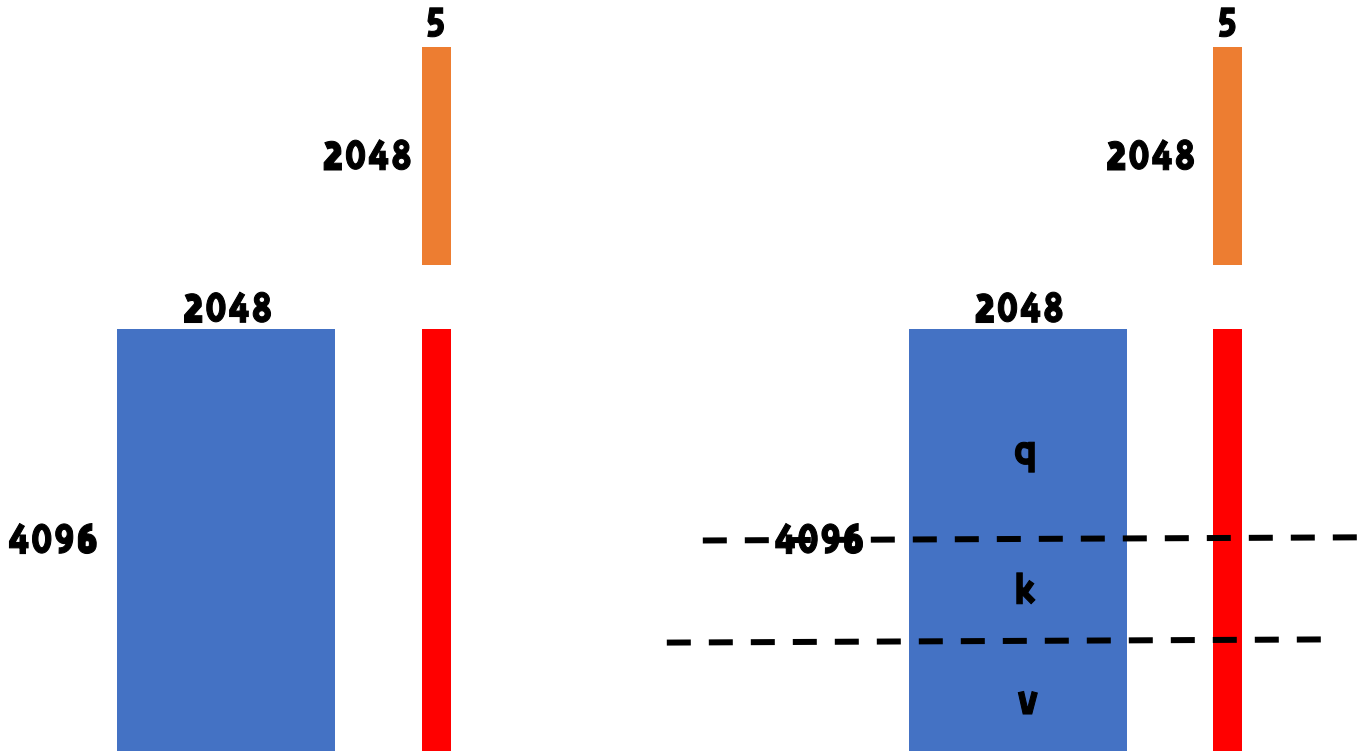
```
> buffer = 0x55555694c590
ne = [4]
  0 = 2048
  1 = 5
  2 = 1
  3 = 1
> nb = [4]
  op = GGML_OP_MUL_MAT
```

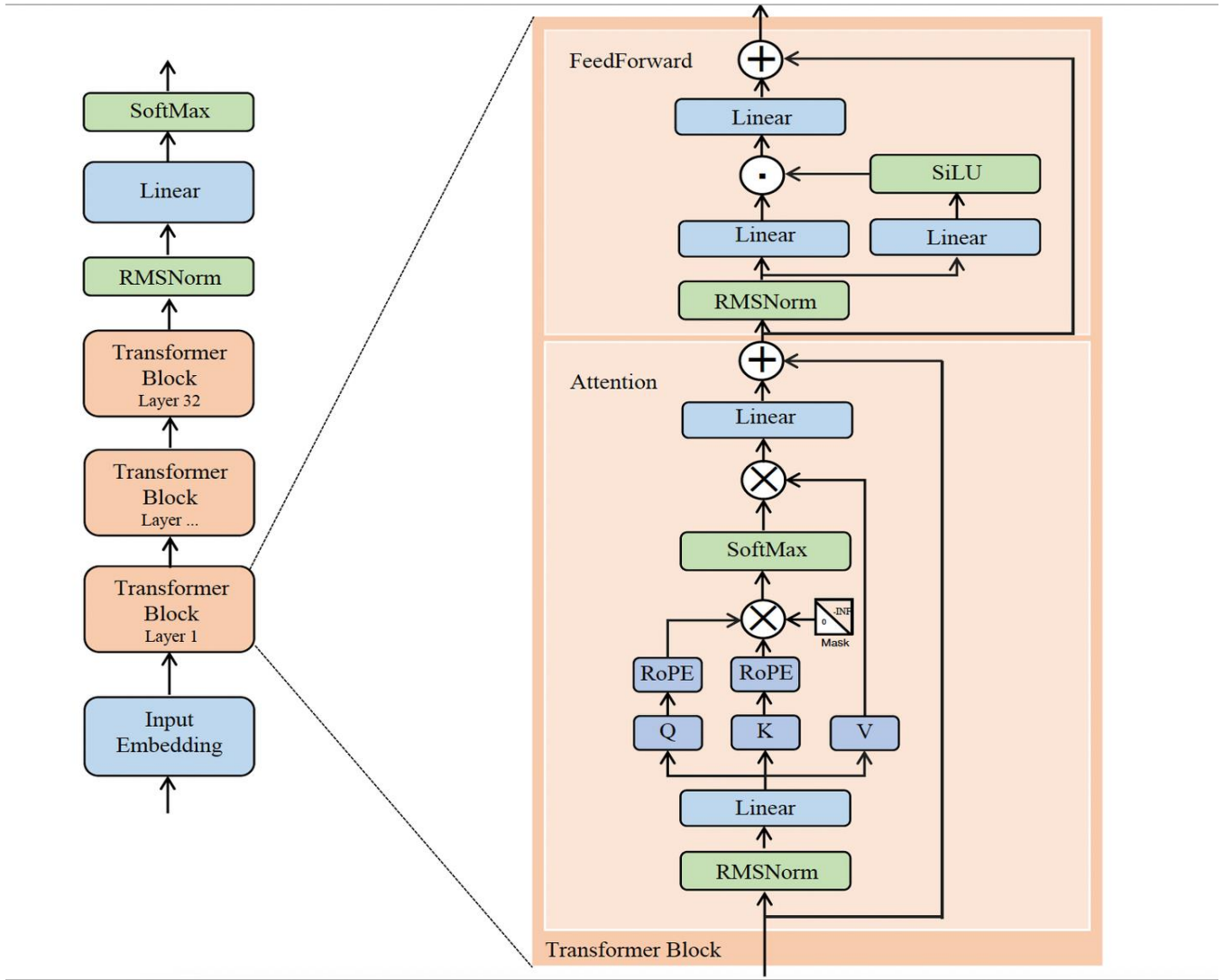
```
> buffer = 0x555556fe6950
ne = [4]
  0 = 2048
  1 = 2048
  2 = 1
  3 = 1
> nb = [4]
  op = GGML_OP_NONE
```

```
> buffer = 0x55555694c590
ne = [4]
  0 = 2048
  1 = 5
  2 = 1
  3 = 1
> nb = [4]
  op = GGML_OP_MUL
```

transformers InternLM2-1.8B
input: Hello my name is -> [1, 9843, 983, 963, 505]

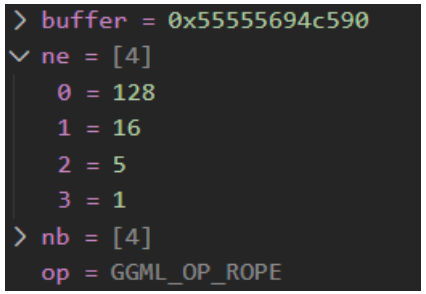
input[1, 5] → embedding_weight[92544, 2048] → [1, 5, 2048] → RMSNorm → [1, 5, 2048] → linear[2048, 4096] → [1, 5, 4096]

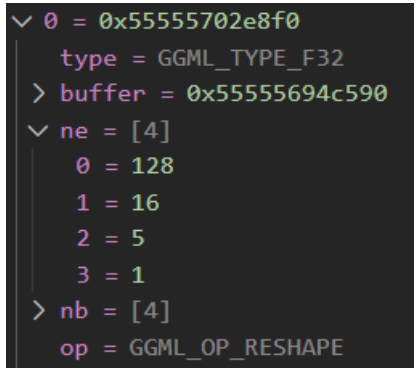


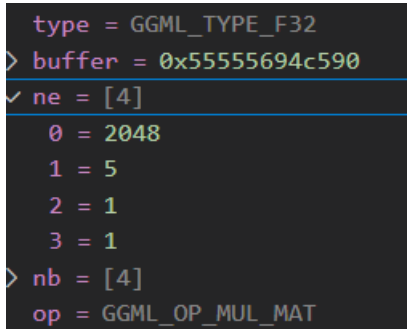

SoftMax
Linear
RMSNorm
Transformer Block Layer 32
Transformer Block Layer ...
Transformer Block Layer 1
Input Embedding
FeedForward
Linear
SiLU
Linear
Linear
RMSNorm
Attention
Linear
SoftMax
-INF
0
Mask
RoPE
RoPE
Q
K
V
Linear
RMSNorm
Transformer Block

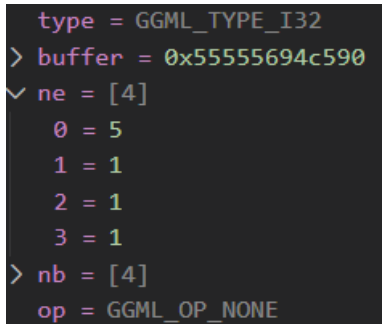
Llama 2详解 - 知乎

ROPE算子









0,1,2,3,4

transformers InternLM2-1.8B

input: Hello my name is -> [1, 9843, 983, 963, 505]

input[1, 5] → embedding_weight[92544, 2048] → [1, 5, 2048] → RMSNorm → [1, 5, 2048] → linear[2048, 4096] → [1, 5, 4096] →rearrange → [1, 5, 8, 4, 128] →

- [..., :2, :] → q [1, 5, 8, 2, 128] → [1, 5, 16, 128] → [1, 16, 5, 128] → rope(维度不变)
- [..., -2, :] → k [1, 5, 8, 1, 128] → [1, 5, 8, 128] → [1, 8, 5, 128] → rope(维度不变)
- [..., -1, :] → v [1, 5, 8, 1, 128] → [1, 5, 8, 128] → [1, 8, 5, 128]

ROPE算子

$$\left[\ h_0 cosmf_0 - h_1 sinmf_0\ ,\ h_0 sinmf_0 + h_1 cosmf_0\ ,\ h_2 cosmf_1 - h_3 sinmf_1, h_2 sinmf_1 + h_3 cosmf_1,\ \ldots, h_{d-2} cosmf_{\frac{d}{2}-1} - h_{d-1} sinmf_{\frac{d}{2}-1}, h_{d-2} sinmf_{\frac{d}{2}-1} + h_{d-1} cosmf_{\frac{d}{2}-1}\ \right] \qquad f_i = \frac{1}{10000^{\frac{2i}{d}}}$$

$$\left[h_0, h_1, h_2, \ldots, h_{d-2}, h_{d-1}\right] \otimes \left[cosmf_0, cosmf_0, cosmf_1, cosmf_1 \ldots, cosmf_{\frac{d}{2}-1}, cosmf_{\frac{d}{2}-1}\right] + \left[-h_1, h_0, -h_3, h_2 \ldots, -h_{d-1}, h_{d-2}\right] \otimes \left[sinmf_0, sinmf_0, sinmf_1, sinmf_1 \ldots, sinmf_{\frac{d}{2}-1}, sinmf_{\frac{d}{2}-1}\right]$$

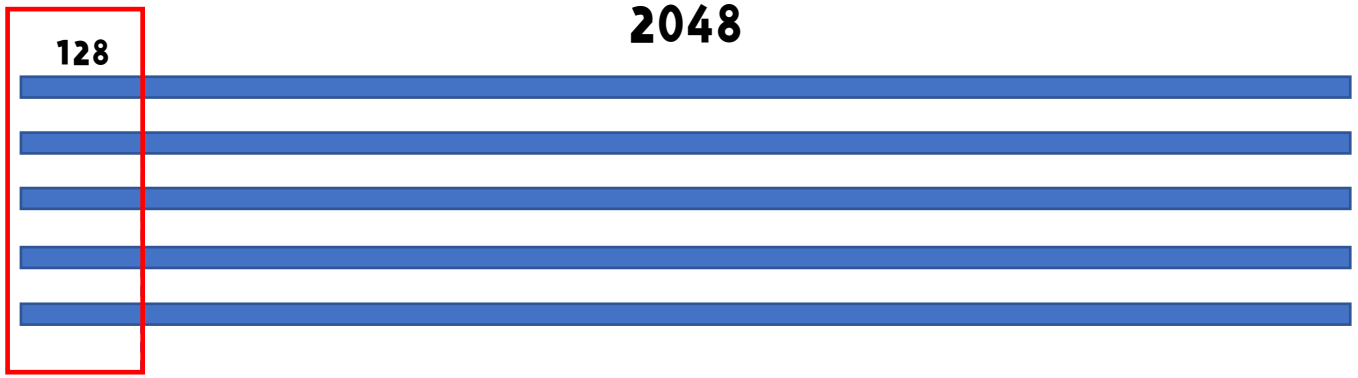
5

$$[h_0, h_1, h_2, \ldots h_{127}, h_{128}] \otimes [cos0f_0, cos0f_0, cos0f_1, cos0f_1, \ldots cos0f_{63}, cos0f_{63}] + [-h_1, h_0, -h_3, \ldots -h_{128}, h_{127}] \otimes [sin0f_0, sin0f_0, sin0f_1, sin0f_1, \ldots sin0f_{63}, sin0f_{63}]$$

$$[h_0, h_1, h_2, \ldots h_{127}, h_{128}] \otimes [cos1f_0, cos1f_0, cos1f_1, cos1f_1, \ldots cos1f_{63}, cos1f_{63}] + [-h_1, h_0, -h_3, \ldots -h_{128}, h_{127}] \otimes [sin1f_0, sin1f_0, sin1f_1, sin1f_1, \ldots sin1f_{63}, sin1f_{63}]$$

$$[h_0, h_1, h_2, \ldots h_{127}, h_{128}] \otimes [cos4f_0, cos4f_0, cos4f_1, cos4f_1, \ldots cos4f_{63}, cos4f_{63}] + [-h_1, h_0, -h_3, \ldots -h_{128}, h_{127}] \otimes [sin4f_0, sin4f_0, sin4f_1, sin4f_1, \ldots sin4f_{63}, sin4f_{63}]$$

5



$$f_0 = 1/10000^{2*0/128}$$

$$f_1 = 1/10000^{2*1/128}$$

$$f_{63} = 1/10000^{2*63/128}$$

64个数

ROPE算子

```
> buffer = 0x55555694c590
v ne = [4]
    0 = 128
    1 = 16
    2 = 5
    3 = 1
> nb = [4]
  op = GGML_OP_ROPE
```

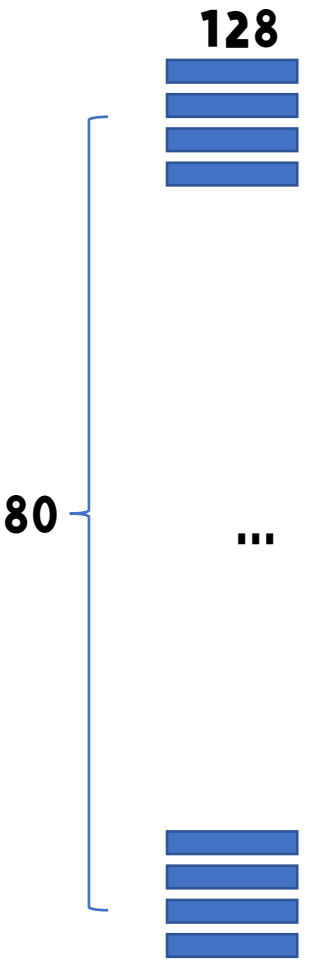
```
v 0 = 0x55555702e8f0
    type = GGML_TYPE_F32
  > buffer = 0x55555694c590
  v ne = [4]
      0 = 128
      1 = 16
      2 = 5
      3 = 1
  > nb = [4]
    op = GGML_OP_RESHAPE
```

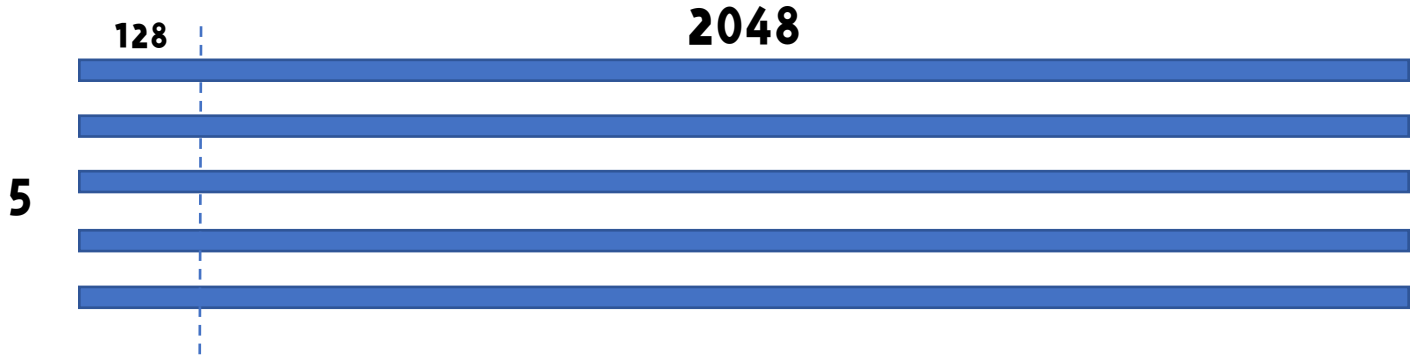
```
  type = GGML_TYPE_I32
> buffer = 0x55555694c590
v ne = [4]
    0 = 5
    1 = 1
    2 = 1
    3 = 1
> nb = [4]
  op = GGML_OP_NONE
```

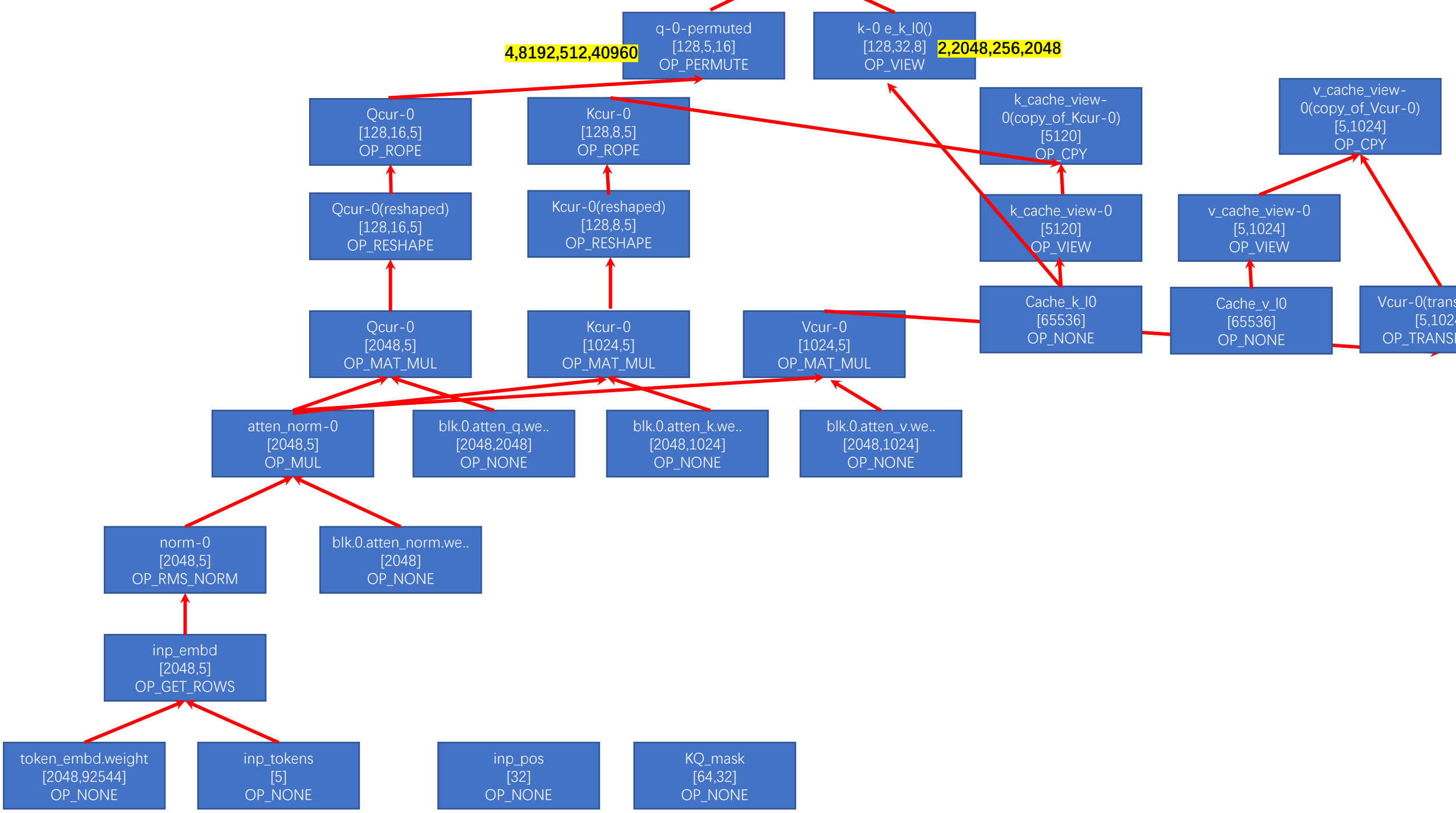
```
  type = GGML_TYPE_F32
> buffer = 0x55555694c590
v ne = [4]
    0 = 2048
    1 = 5
    2 = 1
    3 = 1
> nb = [4]
  op = GGML_OP_MUL_MAT
```

gridDim [80,1,1]
blockDim [1,256,1]

gridDim [80,1,1]
blockDim [1,64,1]






q-0-permuted
[128,5,16]
OP_PERMUTE
4,8192,512,40960
k-0 e_k_l0()
[128,32,8]
OP_VIEW
2,2048,256,2048
Qcur-0
[128,16,5]
OP_ROPE
Kcur-0
[128,8,5]
OP_ROPE
k_cache_view-
0(copy_of_Kcur-0)
[5120]
OP_CPY
v_cache_view-
0(copy_of_Vcur-0)
[5,1024]
OP_CPY
Qcur-0(reshaped)
[128,16,5]
OP_RESHAPE
Kcur-0(reshaped)
[128,8,5]
OP_RESHAPE
k_cache_view-0
[5120]
OP_VIEW
v_cache_view-0
[5,1024]
OP_VIEW
Qcur-0
[2048,5]
OP_MAT_MUL
Kcur-0
[1024,5]
OP_MAT_MUL
Vcur-0
[1024,5]
OP_MAT_MUL
Cache_k_l0
[65536]
OP_NONE
Cache_v_l0
[65536]
OP_NONE
Vcur-0(trans
[5,102
OP_TRANSP
atten_norm-0
[2048,5]
OP_MUL
blk.0.atten_q.we..
[2048,2048]
OP_NONE
blk.0.atten_k.we..
[2048,1024]
OP_NONE
blk.0.atten_v.we..
[2048,1024]
OP_NONE
norm-0
[2048,5]
OP_RMS_NORM
blk.0.atten_norm.we..
[2048]
OP_NONE
inp_embd
[2048,5]
OP_GET_ROWS
token_embd.weight
[2048,92544]
OP_NONE
inp_tokens
[5]
OP_NONE
inp_pos
[32]
OP_NONE
KQ_mask
[64,32]
OP_NONE

1. 两者相同之处

view()和 reshape()在pytorch 中都可以用来重新调整 tensor 的形状。

2. 两者不同之处

1). view()产生的tensor总是和原来的tensor共享一份相同的数据，而reshape()在新形状满足一定条件时会共享相同一份数据，否则会复制一份新的数据。

2). 两者对于原始tensor的连续性要求不同。reshape()不管tensor是否是连续的，都能成功改变形状。而view()对于不连续的tensor()，需要新形状 shape 满足一定条件才能成功改变形状，否则会报错。 transpose, permute 等操作会改变 tensor的连续性，在新形状shape不满足一定的情况下会报错。(注：有的人说view() 函数 只用于连续的tensor，事实上这是不准确的，可看下面解释)

3. 使用指南

如果你想要新的tensor和旧的tensor始终共享一份数据，使用view()

若只是单纯的改变形状，不要求共享数据，reshape()不容易出错

大部分情况下，了解以上即足够了，如果想要知道：

> view(shape)在新形状shape满足哪些情况下才能成功改变形状
> view和reshape的工作机制

View与reshape

torch.transpose()和torch.permute()

两者作用相似，都是用于交换不同维度的内容。但其中 `torch.transpose()` 是交换指定的两个维度的内容， `permute()` 则可以一次性交换多个维度。具体情况如code所示：

Permute与transpose

[Pytorch: view()和reshape()的区别？他们与continues()的关系是什么？_view和reshape的区别-CSDN博客]

[torch.view()、transpose()和permute()的联系与区别_torch.transpose-CSDN博客]

Reshape算子

```
if (ggml_is_empty(node) || node->op == GGML_OP_RESHAPE || node->op == GGML_OP_TRANSPOSE || node->op == GGML_OP_VIEW || node->op == GGML_OP_PERMUTE || node->op == GGML_OP_NONE) {
    continue;
}
```

```
struct ggml_tensor * ggml_reshape_3d(
        struct ggml_context * ctx,
        struct ggml_tensor  * a,
        int64_t               ne0,
        int64_t               ne1,
        int64_t               ne2) {
    GGML_ASSERT(ggml_is_contiguous(a));
    GGML_ASSERT(ggml_nelements(a) == ne0*ne1*ne2);

    const int64_t ne[3] = { ne0, ne1, ne2 };
    struct ggml_tensor * result = ggml_new_tensor_impl(ctx, a->type, 3, ne, a, 0);
    ggml_format_name(result, "%s (reshaped)", a->name);

    result->op     = GGML_OP_RESHAPE;
    result->src[0] = a;

    return result;
}
```

Reshape算子

View算子

```
if (ggml_is_empty(node) || node->op == GGML_OP_RESHAPE || node->op == GGML_OP_TRANSPOSE || node->op == GGML_OP_VIEW || node->op == GGML_OP_PERMUTE || node->op == GGML_OP_NONE) {
    continue;
}
```

```
struct ggml_tensor * ggml_view_1d(
        struct ggml_context * ctx,
        struct ggml_tensor  * a,
        int64_t               ne0,
        size_t                offset) {
    struct ggml_tensor * result = ggml_view_impl(ctx, a, 1, &ne0, offset);

    return result;
}
```

View算子

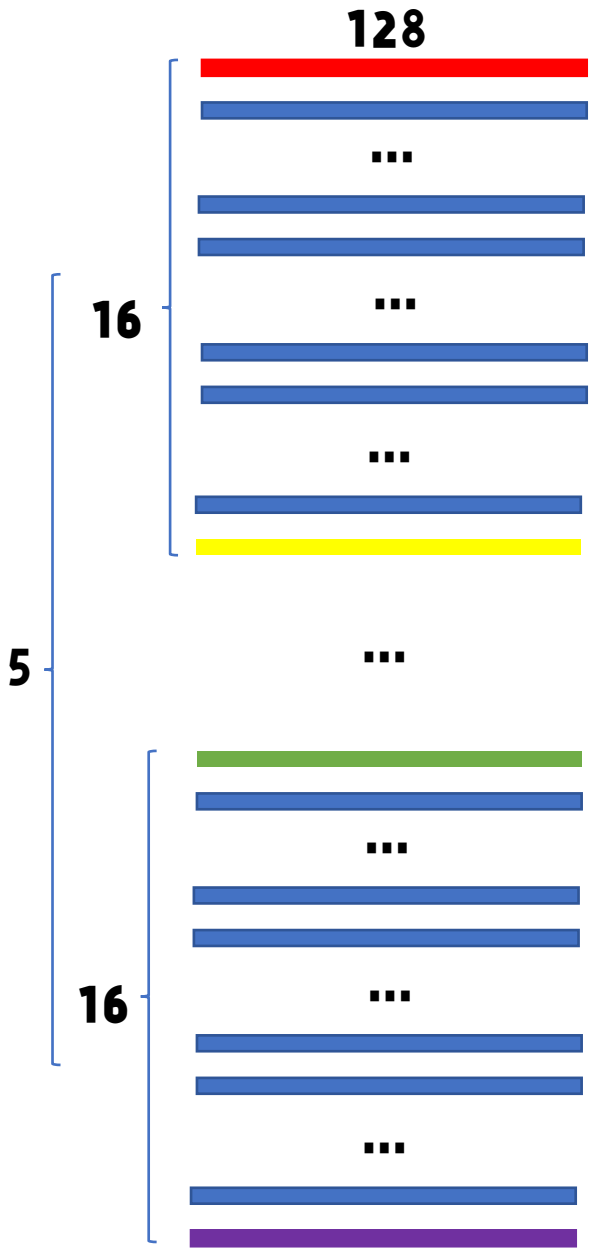
```
static struct ggml_tensor * ggml_view_impl(
        struct ggml_context * ctx,
        struct ggml_tensor  * a,
        int                   n_dims,
        const int64_t       * ne,
        size_t                offset) {
    struct ggml_tensor * result = ggml_new_tensor_impl(ctx, a->type, n_dims, ne, a, offset);
    ggml_format_name(result, "%s (view)", a->name);

    ggml_set_op_params(result, &offset, sizeof(offset));

    result->op     = GGML_OP_VIEW;
    result->src[0] = a;

    return result;
}
```

Permute算子

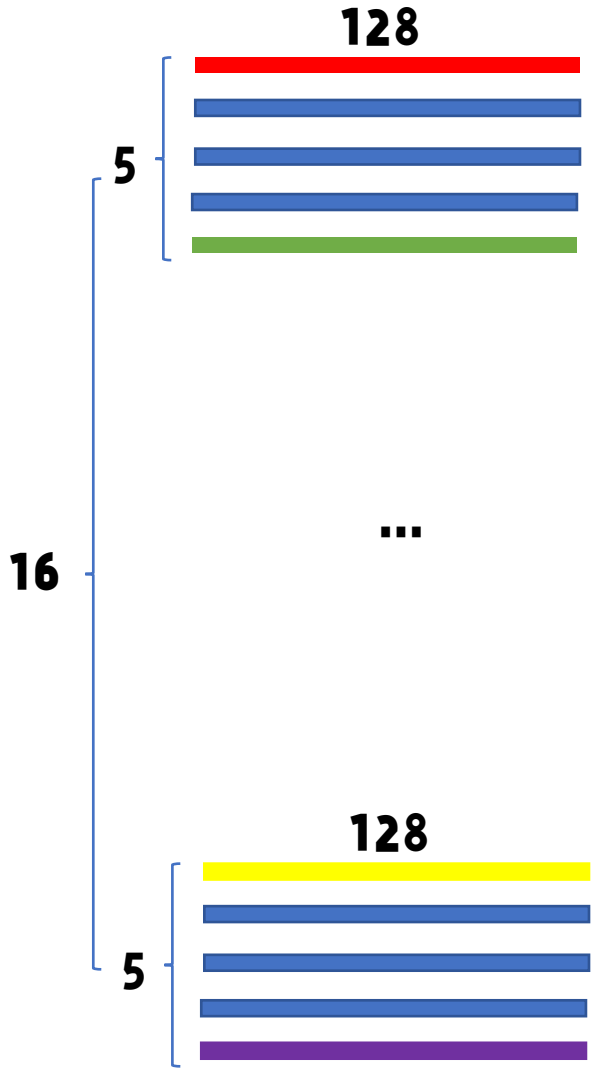


[128,16,5]
[4,512,8192,40960]



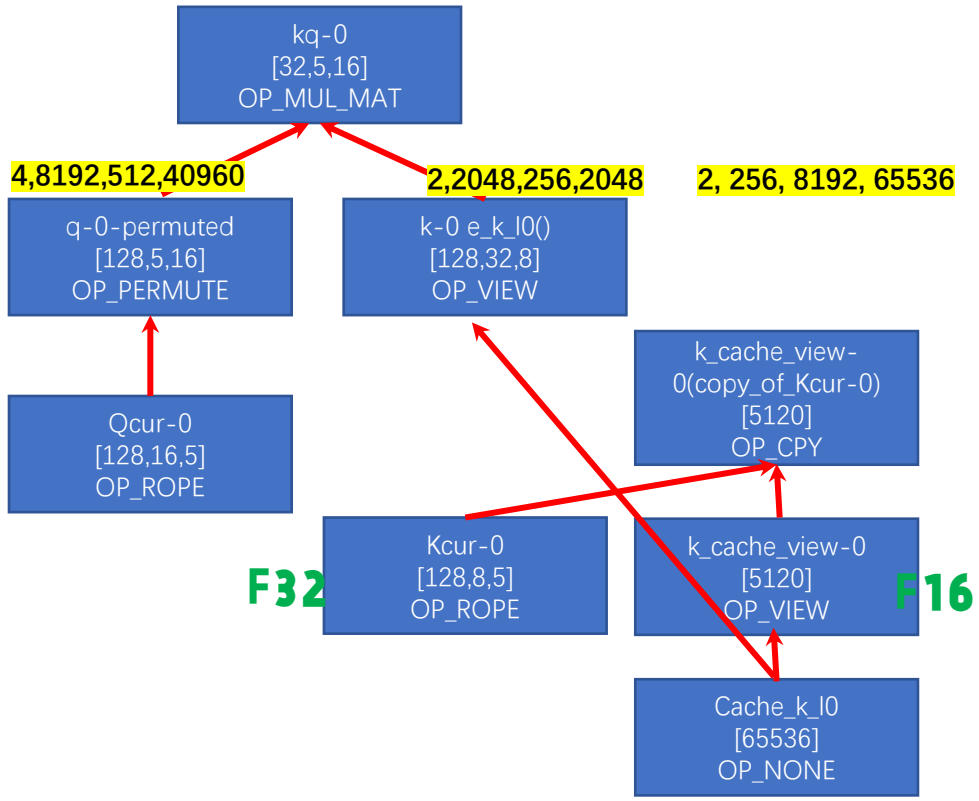
[128,5,16]
[4, 8192,512,40960]





[128,5,16]
[4,512,2560,40960]

CPY算子




```cpp
template <cpy_kernel_t cpy_1>
static __global__ void cpy_f32_f16(const char * cx, char * cdst, const int ne,
                                   const int ne00, const int ne01, const int ne02, const int nb00, const int nb01, const int nb02,
                                   const int nb03, const int ne10, const int ne11, const int ne12, const int nb10, const int nb11,
                                   const int nb12, const int nb13) {
    const int64_t i = blockDim.x*blockIdx.x + threadIdx.x;

    if (i >= ne) {
        return;
    }

    // determine indices i03/i13, i02/i12, i01/i11, i00/i10 as a function of index i of flattened tensor
    // then combine those indices with the corresponding byte offsets to get the total offsets
    const int64_t i03 = i/(ne00 * ne01 * ne02);
    const int64_t i02 = (i - i03*ne00*ne01*ne02 )/ (ne00*ne01);
    const int64_t i01 = (i - i03*ne00*ne01*ne02  -  i02*ne01*ne00) / ne00;
    const int64_t i00 = i - i03*ne00*ne01*ne02 - i02*ne01*ne00 - i01*ne00;
    const int64_t x_offset = i00*nb00 + i01*nb01 + i02*nb02 + i03 * nb03;

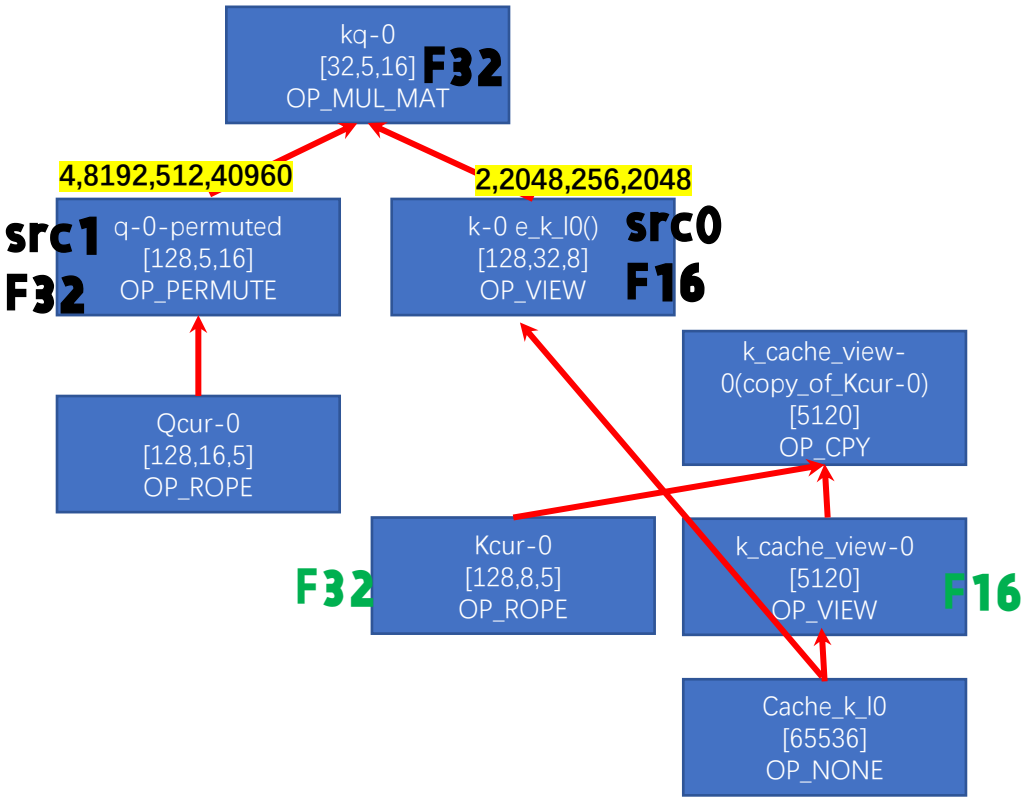
    const int64_t i13 = i/(ne10 * ne11 * ne12);
    const int64_t i12 = (i - i13*ne10*ne11*ne12) / (ne10*ne11);
    const int64_t i11 = (i - i13*ne10*ne11*ne12 - i12*ne10*ne11) / ne10;
    const int64_t i10 = i - i13*ne10*ne11*ne12 - i12*ne10*ne11 - i11*ne10;
    const int64_t dst_offset = i10*nb10 + i11*nb11 + i12*nb12 + i13 * nb13;

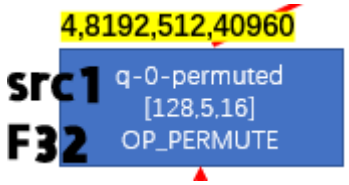
    cpy_1(cx + x_offset, cdst + dst_offset);
}
```

Block_num:80
Block_dim:64

此处的写法看起来麻烦，实际上为了兼顾transpose和permute算子的cpy，必须这么写。

BATCH_MAT_MUL算子



1. 

→ F16
   - 要考虑存在哪？
   - 多线程每个线程转换一个数

2. Src0与src1数据格式转换

```
ggml_cuda_pool_alloc<const void *> ptrs_src(ctx.pool(), 2*ne23);
ggml_cuda_pool_alloc<      void *> ptrs_dst(ctx.pool(), 1*ne23);
```

3. 执行批量矩阵乘法

input[1, 5] → embedding_weight[92544, 2048] → [1, 5, 2048] → RMSNorm → [1, 5, 2048] → linear[2048, 4096] → [1, 5, 4096]
→rearrange → [1, 5, 8, 4, 128] →
- [..., :2, :] → q [1, 5, 8, 2, 128] → [1, 5, 16, 128] → [1, 16, 5, 128] → rope(维度不变)
- [..., -2, :] → k [1, 5, 8, 1, 128] → [1, 5, 8, 128] → [1, 8, 5, 128] → rope(维度不变)
- [..., -1, :] → v [1, 5, 8, 1, 128] → [1, 5, 8, 128] → [1, 8, 5, 128]

repeat→[1,16,5,128]→*q→[1,16,5,5]→/根号128→[1,16,5,5]

BATCH_MAT_MUL算子

函数原型

收起 ^

C

```
cublasStatus_t cublasGemmBatchedEx(
    cublasHandle_t handle,
    cublasOperation_t transa,
    cublasOperation_t transb,
    int m,
    int n,
    int k,
    const void* alpha,
    const void* const* Aarray,
    cudaDataType Atype,
    int lda,
    const void* const* Barray,
    cudaDataType Btype,
    int ldb,
    const void* beta,
    void* const* Carray,
    cudaDataType Ctype,
    int ldc,
    int batchCount,
    cudaDataType computeType,
    cublasGemmAlgo_t algo
);
```

1. `cublasHandle_t handle`：
   - cuBLAS 库的句柄，用于管理 cuBLAS 上下文和资源。在调用该函数之前，需要使用 `cublasCreate` 函数创建一个有效的句柄。
2. `cublasOperation_t transa` 和 `cublasOperation_t transb`：
   - 分别指定矩阵 `A` 和矩阵 `B` 是否需要进行转置操作。取值可以是 `CUBLAS_OP_N`（不转置）、`CUBLAS_OP_T`（转置）或 `CUBLAS_OP_C`（共轭转置）。
3. `int m`、`int n` 和 `int k`：
   - `m` 表示矩阵 `C` 和矩阵 `A` 的行数（经过转置操作后）。
   - `n` 表示矩阵 `C` 和矩阵 `B` 的列数（经过转置操作后）。
   - `k` 表示矩阵 `A` 的列数和矩阵 `B` 的行数（经过转置操作后）。
4. `const void* alpha`：
   - 标量因子，用于缩放矩阵乘法的结果。其数据类型需要与 `computeType` 一致。
5. `const void* const* Aarray`：
   - 指向一批矩阵 `A` 的指针数组。每个指针指向一个矩阵 `A` 的起始地址。
6. `cudaDataType Atype`：
   - 矩阵 `A` 的数据类型，例如 `CUDA_R_32F` 表示单精度浮点数。
7. `int lda`：
   - 矩阵 `A` 的 leading dimension，即矩阵 `A` 的每一行的元素个数（可能大于实际的列数）。
8. `const void* const* Barray`：
   - 指向一批矩阵 `B` 的指针数组。每个指针指向一个矩阵 `B` 的起始地址。
9. `cudaDataType Btype`：
   - 矩阵 `B` 的数据类型。
10. `int ldb`：
    - 矩阵 `B` 的 leading dimension。
11. `const void* beta`：
    - 标量因子，用于缩放矩阵 `C` 的原始值。其数据类型需要与 `computeType` 一致。
12. `void* const* Carray`：
    - 指向一批矩阵 `C` 的指针数组。每个指针指向一个矩阵 `C` 的起始地址，同时也是存储计算结果的位置。
13. `cudaDataType Ctype`：
    - 矩阵 `C` 的数据类型。
14. `int ldc`：
    - 矩阵 `C` 的 leading dimension。
15. `int batchCount`：
    - 批量矩阵乘法的批次数，即 `Aarray`、`Barray` 和 `Carray` 中矩阵的数量。
16. `cudaDataType computeType`：
    - 计算数据类型，指定在计算过程中使用的数据类型。
17. `cublasGemmAlgo_t algo`：
    - 矩阵乘法算法的选择。可以选择不同的算法以平衡性能和精度。

SOFT_MAX算子

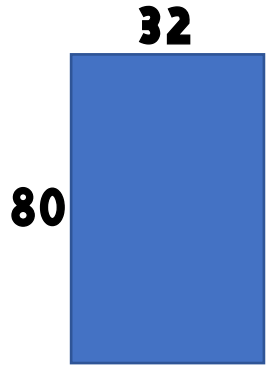
```
type = GGML_TYPE_F32
> buffer = 0x555556208120
ne = [4]
  0 = 32
  1 = 5
  2 = 16
  3 = 1
> nb = [4]
op = GGML_OP_SOFT_MAX
```

```
type = GGML_TYPE_F32
> buffer = 0x555556208120
ne = [4]
  0 = 32
  1 = 5
  2 = 16
  3 = 1
> nb = [4]
op = GGML_OP_MUL_MAT
```

```
type = GGML_TYPE_F32
> buffer = 0x555556208120
ne = [4]
  0 = 32
  1 = 32
  2 = 1
  3 = 1
> nb = [4]
op = GGML_OP_NONE
```

```
tensor([[[[     0., -65504., -65504., -65504., -65504.],
          [     0.,      0., -65504., -65504., -65504.],
          [     0.,      0.,      0., -65504., -65504.],
          [     0.,      0.,      0.,      0., -65504.],
          [     0.,      0.,      0.,      0.,      0.]]],
        device='cuda:0',
       dtype=torch.float16)
```

block_num: [80,1,1]
block_size:[32,1,1]



$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}} \quad \text{for } i = 1, 2, \cdots, n$$

```python
def softmax(z):
    """
    计算输入向量 z 的 Softmax 值。
    :param z: 输入的实数向量，可以是一维数组或多维数组（按最后一个轴计算）
    :return: Softmax 转换后的概率分布向量
    """
    # 为了数值稳定性，减去向量中的最大值
    exp_z = np.exp(z - np.max(z, axis=-1, keepdims=True))
    return exp_z / np.sum(exp_z, axis=-1, keepdims=True)
```

input[1, 5] → embedding_weight[92544, 2048] → [1, 5, 2048] → RMSNorm → [1, 5, 2048] → linear[2048, 4096] → [1, 5, 4096]
→rearrange → [1, 5, 8, 4, 128] →
- [..., :2, :] → q [1, 5, 8, 2, 128] → [1, 5, 16, 128] → [1, 16, 5, 128] → rope(维度不变)
- [..., -2, :] → k [1, 5, 8, 1, 128] → [1, 5, 8, 128] → [1, 8, 5, 128] → rope(维度不变)
- [..., -1, :] → v [1, 5, 8, 1, 128] → [1, 5, 8, 128] → [1, 8, 5, 128]

repeat→[1,16,5,128]→*q→[1,16,5,5]→/根号128→[1,16,5,5] →+mask→softmax→ [1,16,5,5]

transformers InternLM2-1.8B

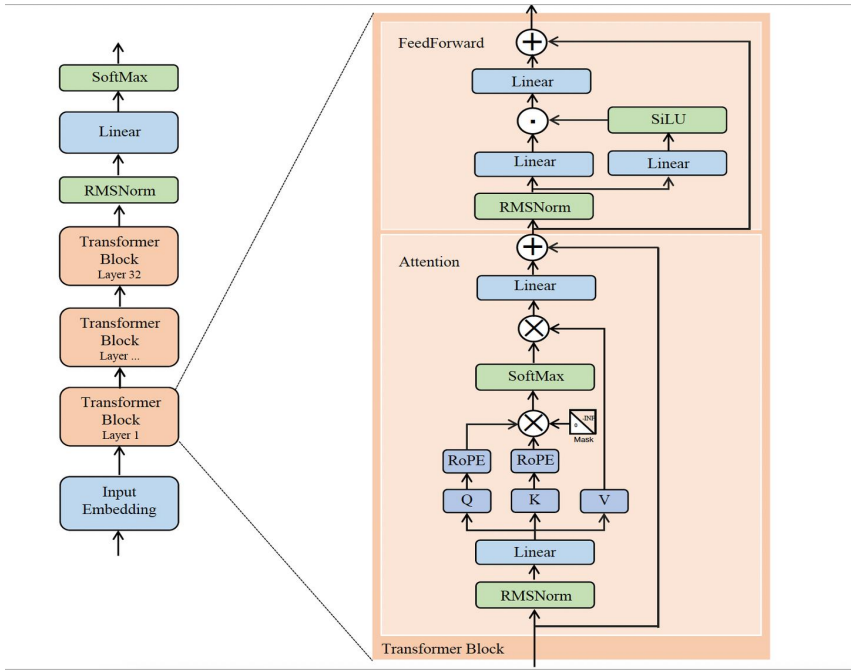
input: Hello my name is -> [1, 9843, 983, 963, 505]

input[1, 5] → embedding_weight[92544, 2048] → [1, 5, 2048] → RMSNorm → [1, 5, 2048] → linear[2048, 4096] → [1, 5, 4096] →
rearrange → [1, 5, 8, 4, 128] →
[..., :2, :] → q [1, 5, 8, 2, 128] → [1, 5, 16, 128] → [1, 16, 5, 128] → rope(维度不变)
[..., -2, :] → k [1, 5, 8, 1, 128] → [1, 5, 8, 128] → [1, 8, 5, 128] → rope(维度不变)
[..., -1, :] → v [1, 5, 8, 1, 128] → [1, 5, 8, 128] → [1, 8, 5, 128]→repeat→[1,16,5,128]

repeat→[1,16,5,128]→*q→[1,16,5,5]→/根号128→[1,16,5,5]→+mask→[1,16,5,5]→*v→[1,16,5,128]→transpose→
[1,5,16,128]→reshape→[1,5,2048]→linear→[1,5,2048]→+ →[1,5,2048]→RMSNorm→[1,5,2048]→linear→[1,5,8192]→

linear→[1,5,8192] →silu

*→[1,5,8192]→linear→[1,5,2048]→+ →[1,5,2048]



CONT算子 (contiguous)

dst

```
type = GGML_TYPE_F32
> buffer = 0x5555566556d0
v ne = [4]
    0 = 2048
    1 = 5
    2 = 1
    3 = 1
v nb = [4]
    0 = 4
    1 = 8192
    2 = 40960
    3 = 40960
op = GGML_OP_CONT
```

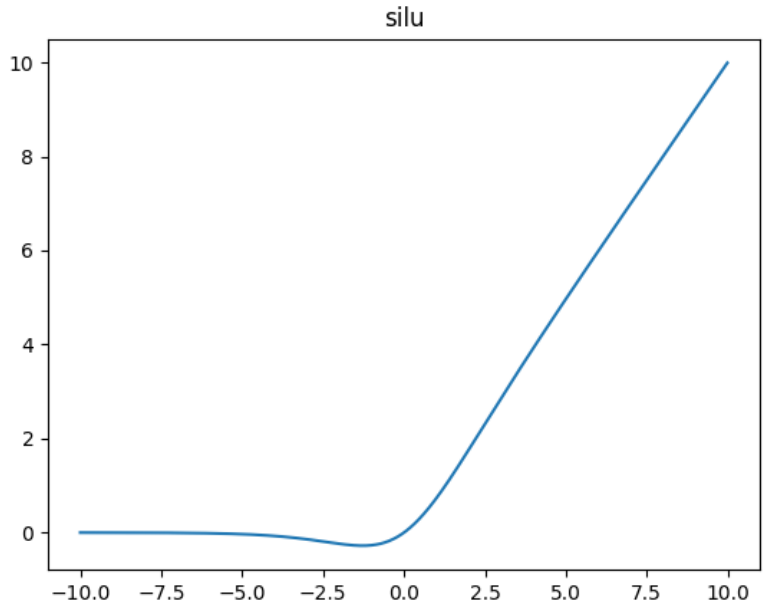
src

```
v 0 = 0x555557023930
    type = GGML_TYPE_F32
  > buffer = 0x5555566556d0
  v ne = [4]
      0 = 128
      1 = 16
      2 = 5
      3 = 1
  v nb = [4]
      0 = 4
      1 = 2560
      2 = 512
      3 = 40960
    op = GGML_OP_PERMUTE
```

[1,5,16,128]→reshape→[1,5,2048]

SILU算子

$$silu(x) = x * sigmoid(x) = \frac{x}{1 + e^{-x}}$$



[1,5,8192]→silu



LLAMA.CPP源码逐行调试带读

3. 8bit量化算子解析

Pytorch端的量化部分

group: 92544 // 16 or 1 = 5784

2048→2176: 2048//32*34 = 2176最后的输出：92544 * 2176个uint8

```
GGML_QUANT_SIZES: dict[GGMLQuantizationType, tuple[int, int]] = {
    GGMLQuantizationType.F32:     (1, 4),
    GGMLQuantizationType.F16:     (1, 2),
    GGMLQuantizationType.Q4_0:    (32, 2 + 16),
    GGMLQuantizationType.Q4_1:    (32, 2 + 2 + 16),
    GGMLQuantizationType.Q5_0:    (32, 2 + 4 + 16),
    GGMLQuantizationType.Q5_1:    (32, 2 + 2 + 4 + 16),
    GGMLQuantizationType.Q8_0:    (32, 2 + 32),
    GGMLQuantizationType.Q8_1:    (32, 4 + 4 + 32)
    GGMLQuantizationType.Q2_K:    (256, 2 + 2 + QK_K // 16 + QK_K // 4)
```

block_size, type_size

每个组 16*2048 reshape 成 1024 * 32

执行

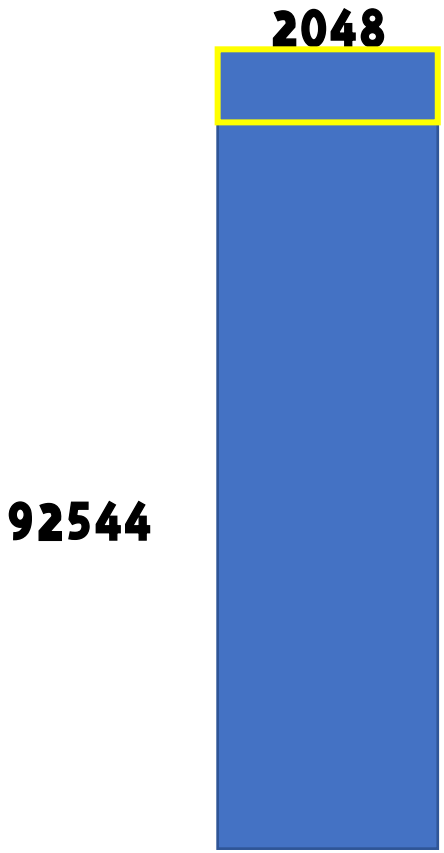
```
def quantize_blocks(cls, blocks: np.ndarray) -> np.ndarray:

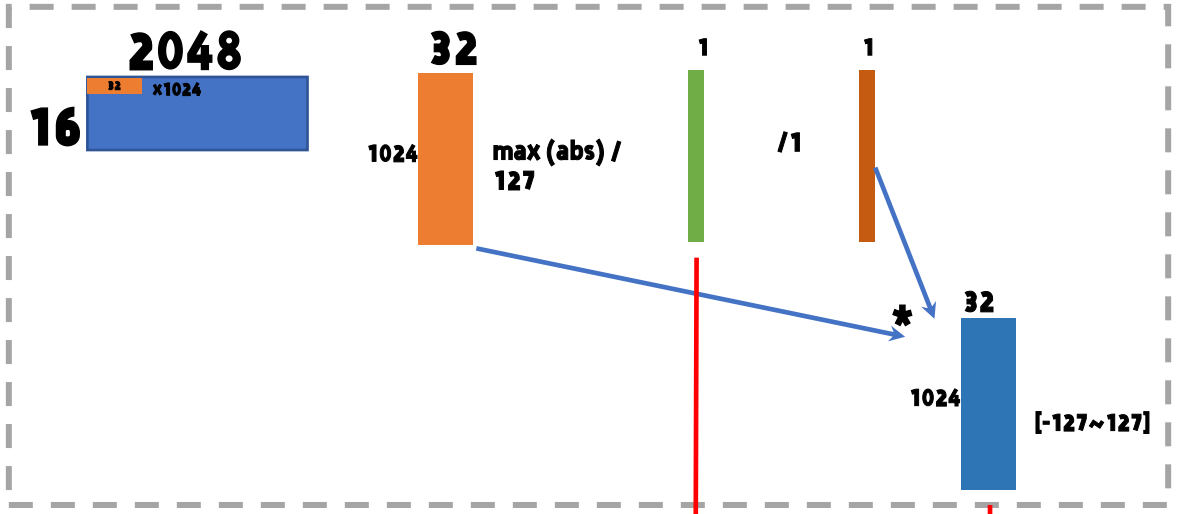
    d = abs(blocks).max(axis=1, keepdims=True) / 127
    with np.errstate(divide="ignore"):
        id = np.where(d == 0, 0, 1 / d)
    qs = np_roundf(blocks * id)

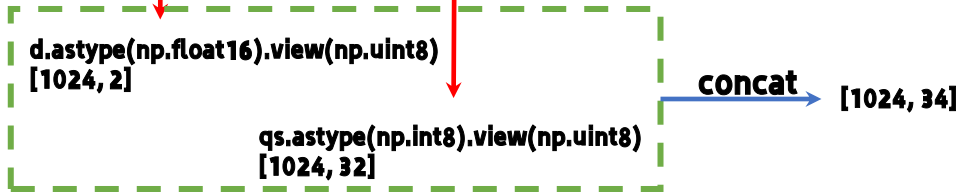
    # (n_blocks, 2)
    d = d.astype(np.float16).view(np.uint8)
    # (n_blocks, block_size)
    qs = qs.astype(np.int8).view(np.uint8)

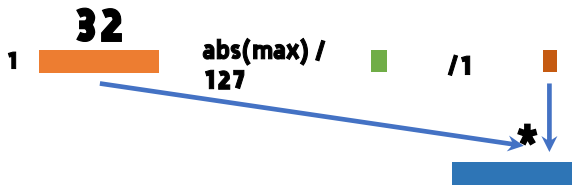
    return np.concatenate( arrays: [d, qs], axis=1)
```

Pytorch端的量化部分









$$s = \frac{\max(abs(f))}{127}$$

$$q = f * 1/s \qquad f = q * s$$

GET_ROWS算子

```
dst = 0x55555702ebc0
  type = GGML_TYPE_F32
> buffer = 0x55555622ea50
v ne = [4]
    0 = 2048
    1 = 5
    2 = 1
    3 = 1
> nb = [4]
  op = GGML_OP_GET_ROWS
```

```
v 0 = 0x5555569307e0
  type = GGML_TYPE_Q8_0
> buffer = 0x55555694c510
v ne = [4]
    0 = 2048
    1 = 92544
    2 = 1
    3 = 1
> nb = [4]
  op = GGML_OP_NONE
```

```
v 1 = 0x55555702ea50
  type = GGML_TYPE_I32
> buffer = 0x55555622ea50
v ne = [4]
    0 = 5
    1 = 1
    2 = 1
    3 = 1
> nb = [4]
  op = GGML_OP_NONE
```

```c
void dequantize_row_q8_0(const block_q8_0 * restrict x, float * restrict y, int64_t k) {
    static const int qk = QK8_0;

    assert(k % qk == 0);

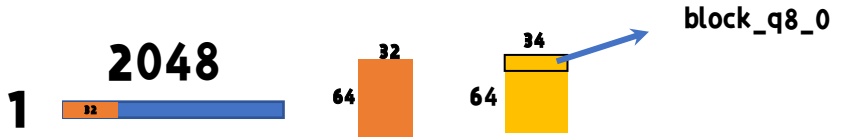
    const int nb = k / qk;

    for (int i = 0; i < nb; i++) {
        const float d = GGML_FP16_TO_FP32(x[i].d);

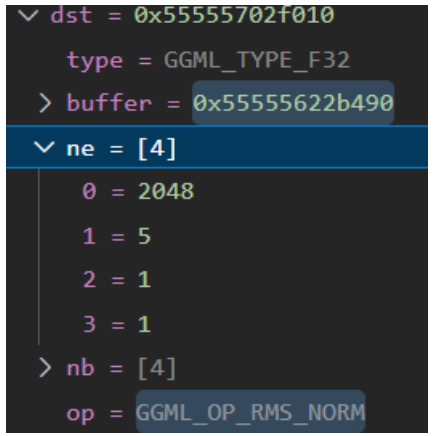
        for (int j = 0; j < qk; ++j) {
            y[i*qk + j] = x[i].qs[j]*d;
        }
    }
}
```

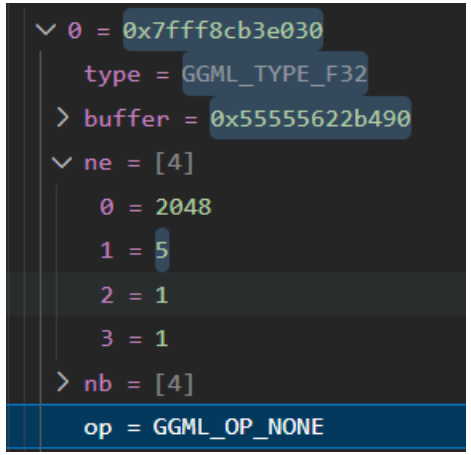
这里面只会反量化一行
也就是2048个数据

```c
#define QK8_0 32
typedef struct {
    ggml_half d;       // delta
    int8_t  qs[QK8_0]; // quants
} block_q8_0;
```

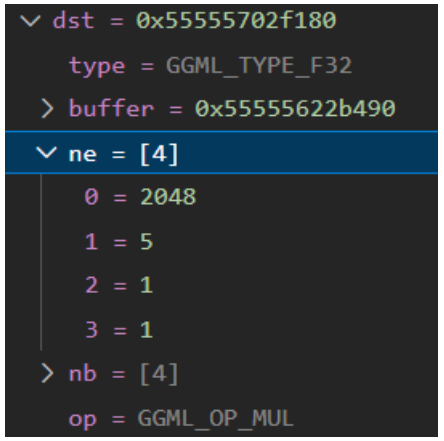


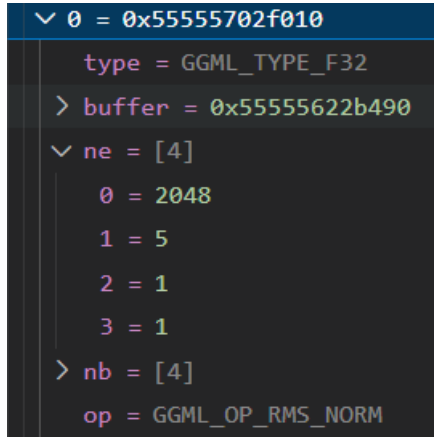
RMS_NORM算子（未量化）

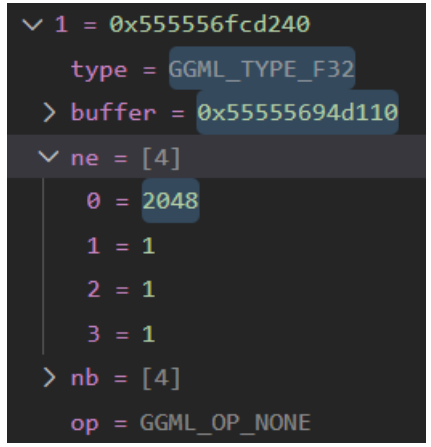




MUL算子（未量化）







MUL_MAT算子

```
dst = 0x55555702f2f0
  type = GGML_TYPE_F32
> buffer = 0x55555622b490
v ne = [4]
    0 = 2048
    1 = 5
    2 = 1
    3 = 1
> nb = [4]
  op = GGML_OP_MUL_MAT
```

```
v 0 = 0x555556fcd3b0
    type = GGML_TYPE_Q8_0
  > buffer = 0x55555694d110
  v ne = [4]
      0 = 2048
      1 = 2048
      2 = 1
      3 = 1
  > nb = [4]
    op = GGML_OP_NONE
```

```
  type = GGML_TYPE_F32
> buffer = 0x55555622b490
v ne = [4]
    0 = 2048
    1 = 5
    2 = 1
    3 = 1
> nb = [4]
  op = GGML_OP_MUL
```

```c
#define QK8_1 32
typedef struct {
    union {
        struct {
            ggml_half d; // delta
            ggml_half s; // d * sum(qs[i])
        } GGML_COMMON_AGGR_S;
        ggml_half2 ds;
    } GGML_COMMON_AGGR_U;
    int8_t qs[QK8_1]; // quants
} block_q8_1;
static_assert(sizeof(block_q8_1) == 2*sizeof(ggml_half) + QK8_1, "wrong q8_1 block size/padding");
```

要对这部分进行量化
每32个float数要用32个int8
搭配1个int32来表示

MUL_MAT算子

grid_dim:[8,5,1]
block_dim:[256,1,1]
每个线程都要量化一个数

```cpp
static __global__ void quantize_q8_1(const float * __restrict__ x, void * __restrict__ vy, const int64_t kx, const int64_t kx0_padded) {
    const int64_t ix0 = (int64_t)blockDim.x*blockIdx.x + threadIdx.x;

    if (ix0 >= kx0_padded) {
        return;
    }

    const int64_t ix1 = blockIdx.y;

    const int64_t i_padded = ix1*kx0_padded + ix0;

    block_q8_1 * y = (block_q8_1 *) vy;

    const int64_t ib = i_padded / QK8_1; // block index
    const int64_t iqs = i_padded % QK8_1; // quant index

    const float xi = ix0 < kx ? x[ix1*kx + ix0] : 0.0f;
    float amax = fabsf(xi);
    float sum = xi;

    amax = warp_reduce_max(amax);
    sum = warp_reduce_sum(sum);

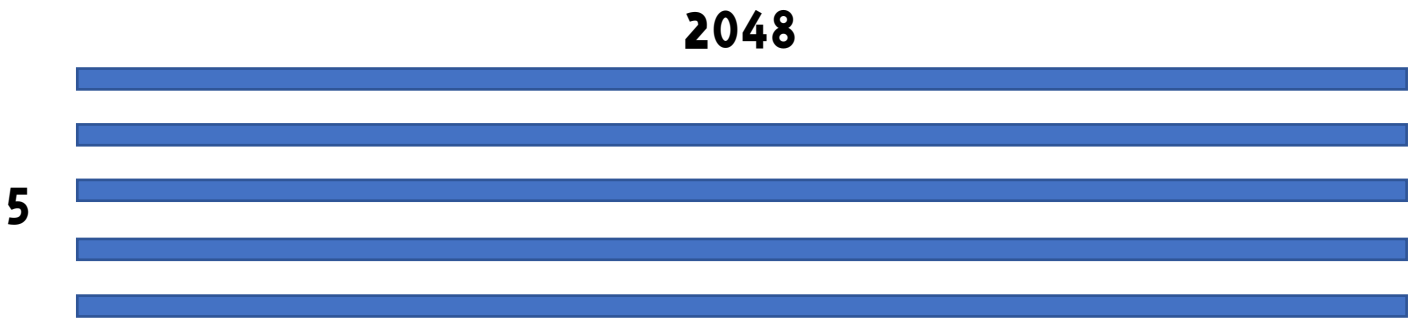
    const float d = amax / 127;
    const int8_t q = amax == 0.0f ? 0 : roundf(xi / d);

    y[ib].qs[iqs] = q;

    if (iqs > 0) {
        return;
    }

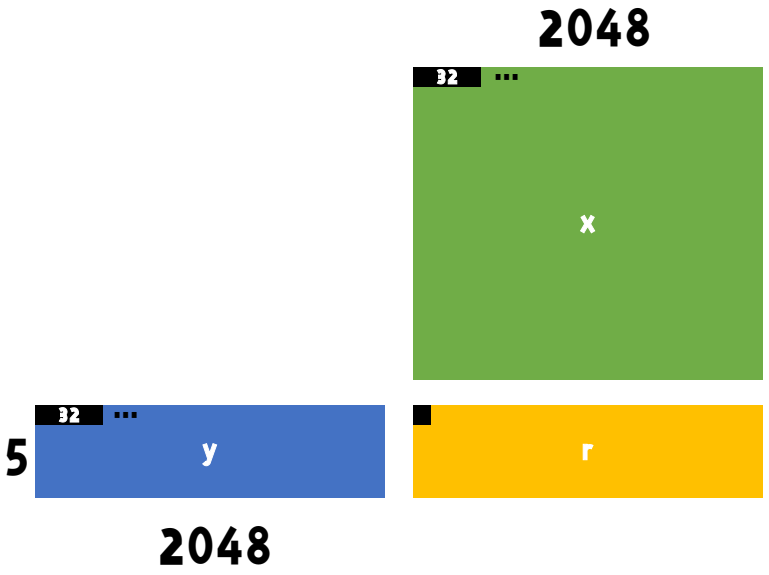
    reinterpret_cast<half&>(y[ib].ds.x) = d;
    reinterpret_cast<half&>(y[ib].ds.y) = sum;
}
```

```cpp
#define QK8_1 32
typedef struct {
    union {
        struct {
            ggml_half d; // delta
            ggml_half s; // d * sum(qs[i])
        } GGML_COMMON_AGGR_S;
        ggml_half2 ds;
    } GGML_COMMON_AGGR_U;
    int8_t qs[QK8_1]; // quants
} block_q8_1;
static_assert(sizeof(block_q8_1) == 2*sizeof(ggml_half) + QK8_1, "wrong q8_1 block size/padding");
```



2048 * 5 / 32 = 320 个 block_q8_1

MUL_MAT算子



grid_dim:[1024,1,1]
Block_dim:[32,2,1]
~~看起来是每32个thread负责1列的计算~~

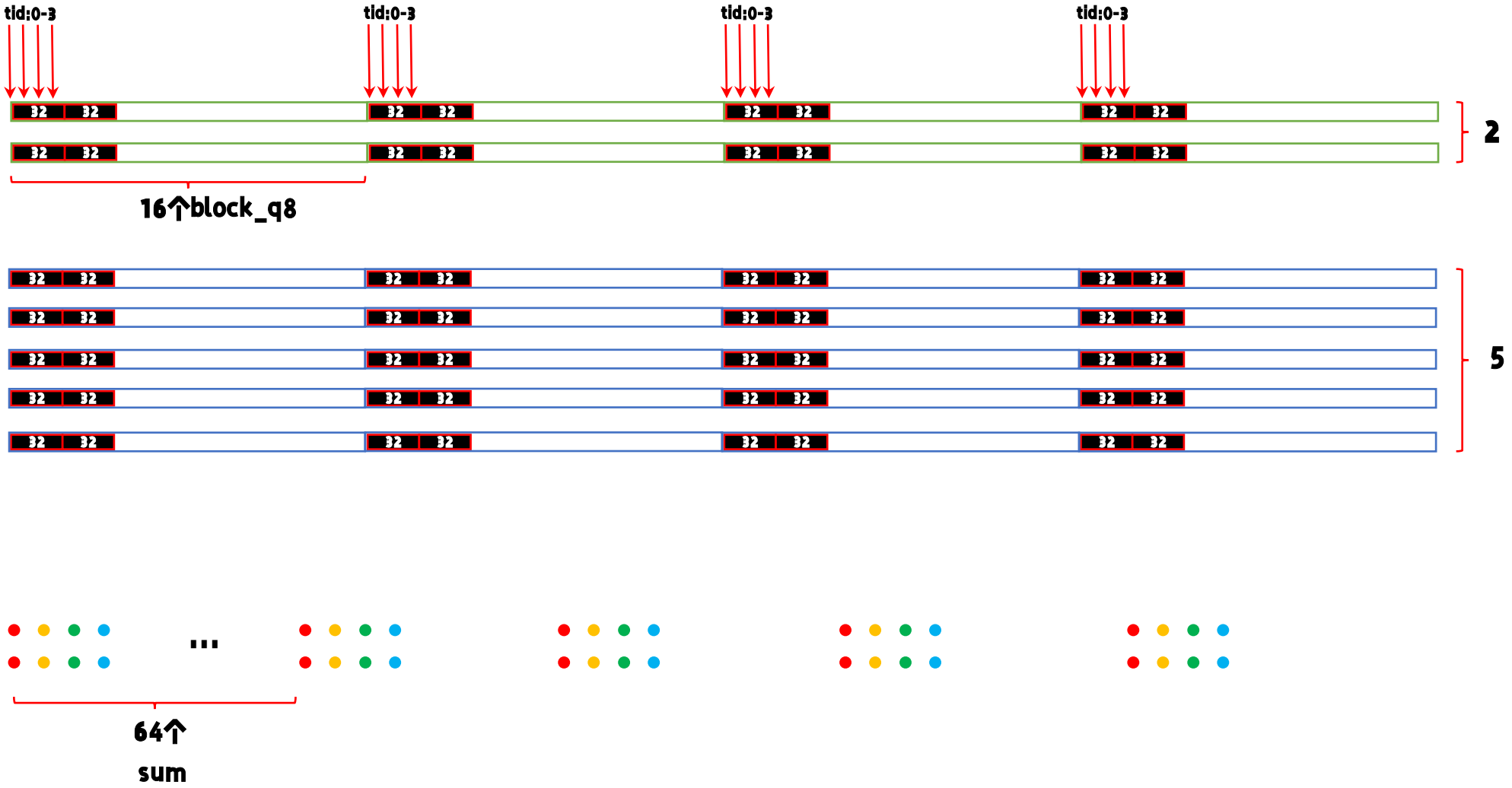
$$r_{(0,0)} = x_{(0,0)}y_{(0,0)} + x_{(0,1)}y_{(0,1)} + \cdots + x_{(0,2048)}y_{(0,2048)}$$

$$x_{(0,0)}y_{(0,0)} = q_x_{(0,0)}s_x_{[0\sim31]}q_y_{(0,0)}s_y_{[0\sim31]}$$

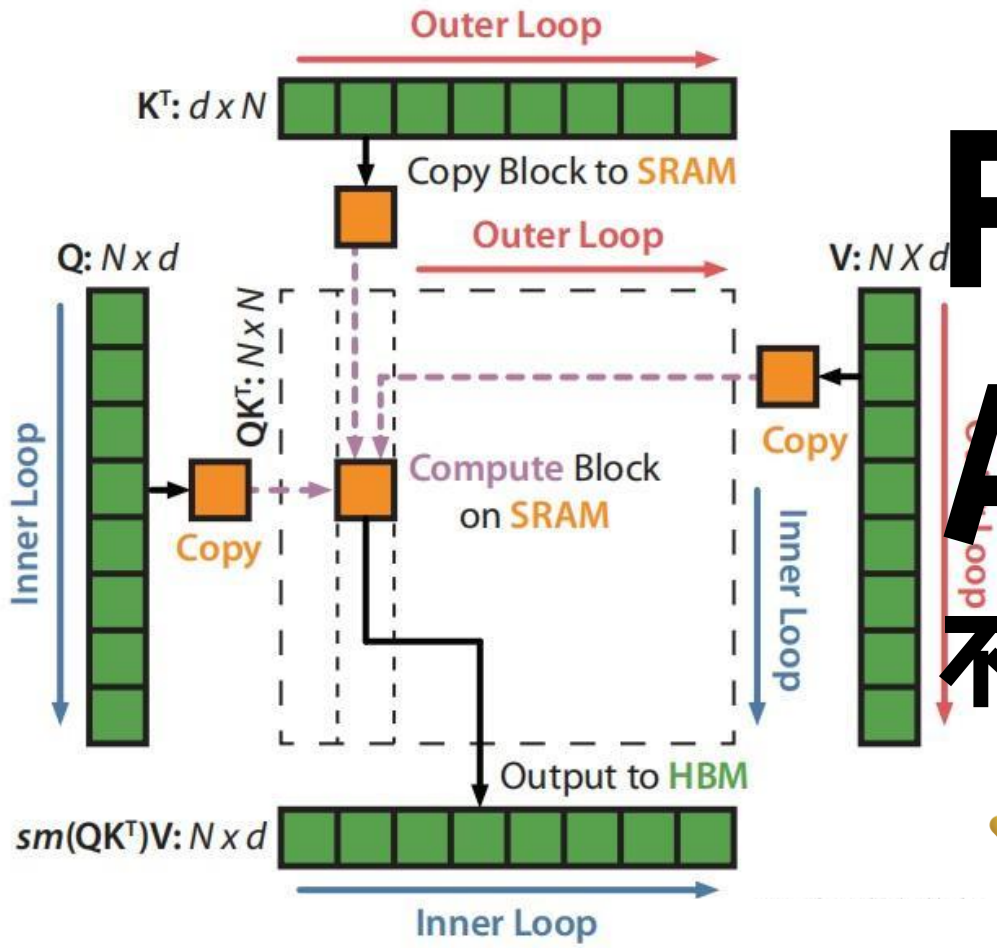
$$x_{(0,1)}y_{(0,1)} = q_x_{(0,1)}s_x_{[0\sim31]}q_y_{(0,1)}s_y_{[0\sim31]}$$

$$r_{(0,0)} = s_x_{[0\sim31]}s_y_{[0\sim31]}(q_x_{(0,0)}q_y_{(0,0)} + \cdots + q_x_{(0,31)}q_y_{(0,31)}) + \cdots$$
$$+ s_x_{[2016\sim2047]}s_y_{[2016\sim2047]}(q_x_{(0,2016)}q_y_{(0,2016)} + \cdots + q_x_{(0,2047)}q_y_{(0,2047)})$$

MUL_MAT算子



per_block



Flash Attention

不止Flash Attention

⭐ 学习过程详解 ⭐

1.寻找切入点

- 从llama.cpp中寻找代码，找到了但是好像实现有点过于复杂，我的目标是学习这种思想，抓住核心目的之后需要让其他因素尽可能简化
- https://github.com/tspeterkim/flash-attention-minimal
- cuda写的是python的插件，无法debug

```python
import math

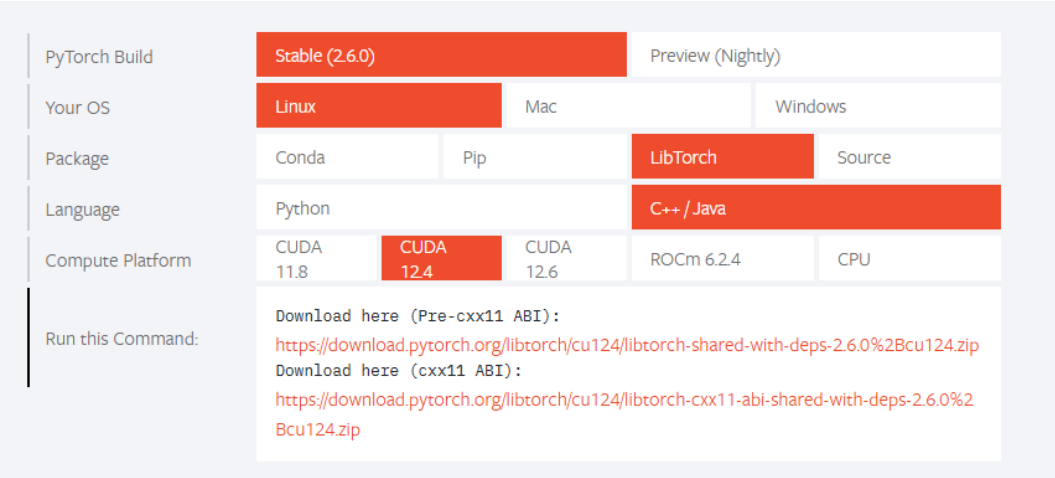
import torch
from torch.nn import functional as F
from torch.utils.cpp_extension import load

# Load the CUDA kernel as a python module
minimal_attn = load(name='minimal_attn', sources=['main.cpp', 'flash.cu'], extra_cuda_cflags=['-O2'])
```

- Llama.cpp由于所有东西都是自己实现的，所以单纯想把某部分代码拿出来是有难度的，所以需要找一个库能够无缝衔接所有的cuda算子。libtorch

2.Libtorch开发环境搭建

- 搭建一个新环境（库），不要上来就去百度，一定是先在官网上自己找




```
cmake_minimum_required(VERSION 3.18 FATAL_ERROR)
project(example-app)

find_package(Torch REQUIRED)
set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} ${TORCH_CXX_FLAGS}")

add_executable(example-app example-app.cpp)
target_link_libraries(example-app "${TORCH_LIBRARIES}")
set_property(TARGET example-app PROPERTY CXX_STANDARD 17)

# The following code block is suggested to be used on Windows.
# According to https://github.com/pytorch/pytorch/issues/25457,
# the DLLs need to be copied to avoid memory errors.
if (MSVC)
  file(GLOB TORCH_DLLS "${TORCH_INSTALL_PREFIX}/lib/*.dll")
  add_custom_command(TARGET example-app
                     POST_BUILD
                     COMMAND ${CMAKE_COMMAND} -E copy_if_different
                     ${TORCH_DLLS}
                     $<TARGET_FILE_DIR:example-app>)
endif (MSVC)
```

The implementation of our example will simply create a new *torch::Tensor* and print it:

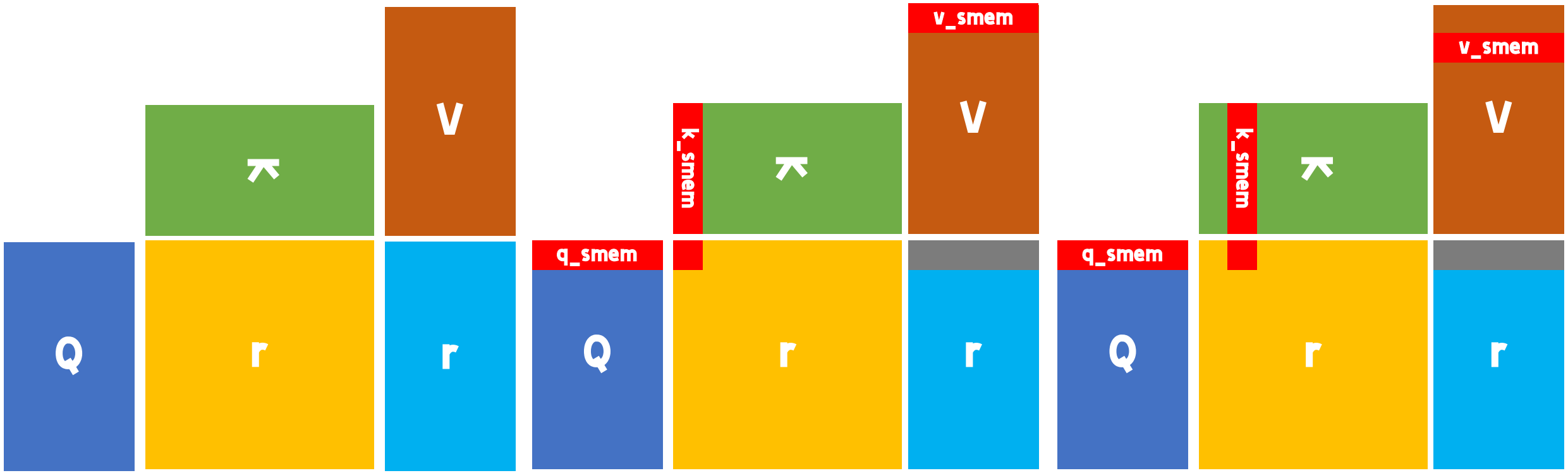
```
#include <torch/torch.h>
#include <iostream>

int main() {
  torch::Tensor tensor = torch::rand({2, 3});
  std::cout << tensor << std::endl;
}
```

- [Installing C++ Distributions of PyTorch — PyTorch main documentation]

3.Flash Atten分块乘法

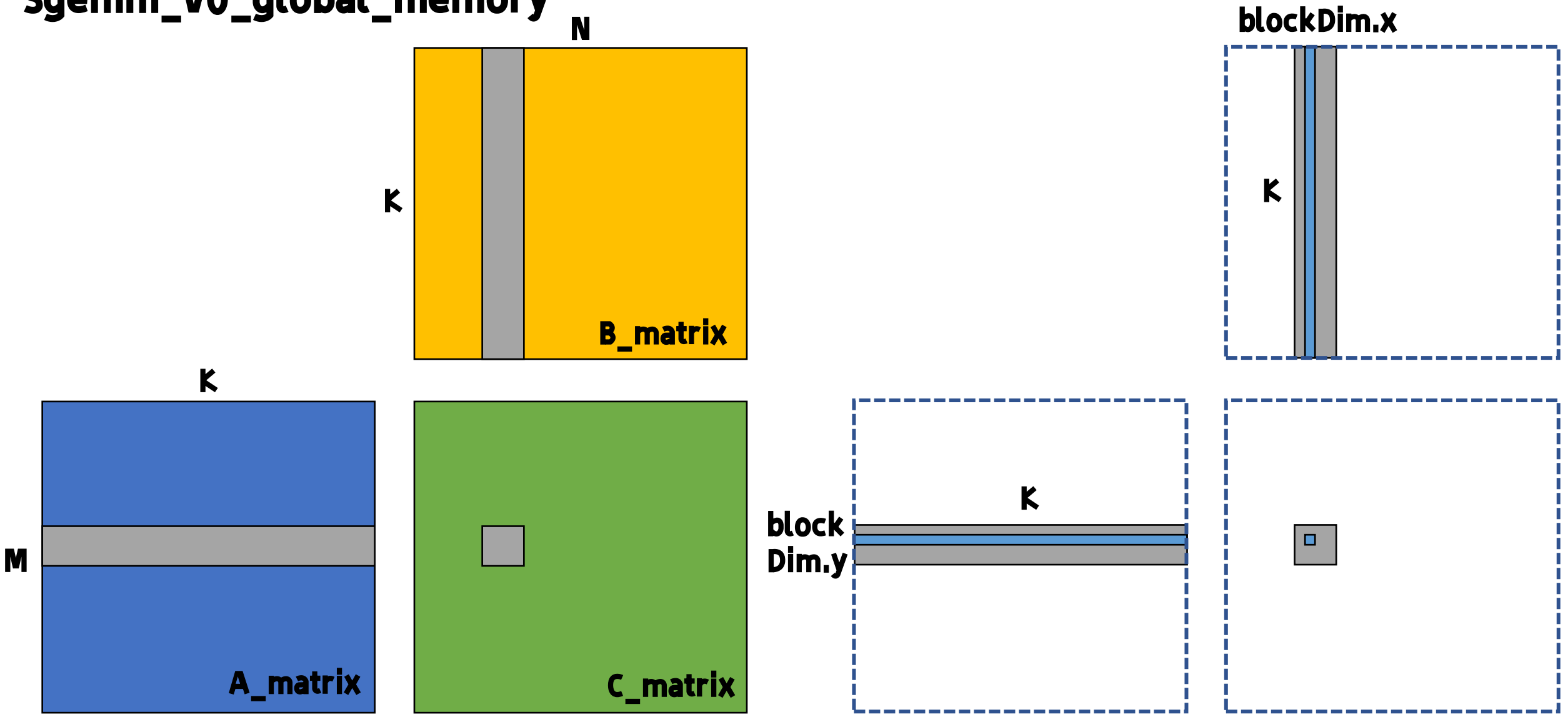
$$Attention(Q,K,V) = Softmax(\frac{QK^\top}{\sqrt{d_k}})V$$



Flash Attention 为什么那么快？原理讲解_哔哩哔哩_bilibili

神经网络-量化与部署，进阶之路迟早要越过的大山_哔哩哔哩_bilibili

Sgemm_v0_global_memory

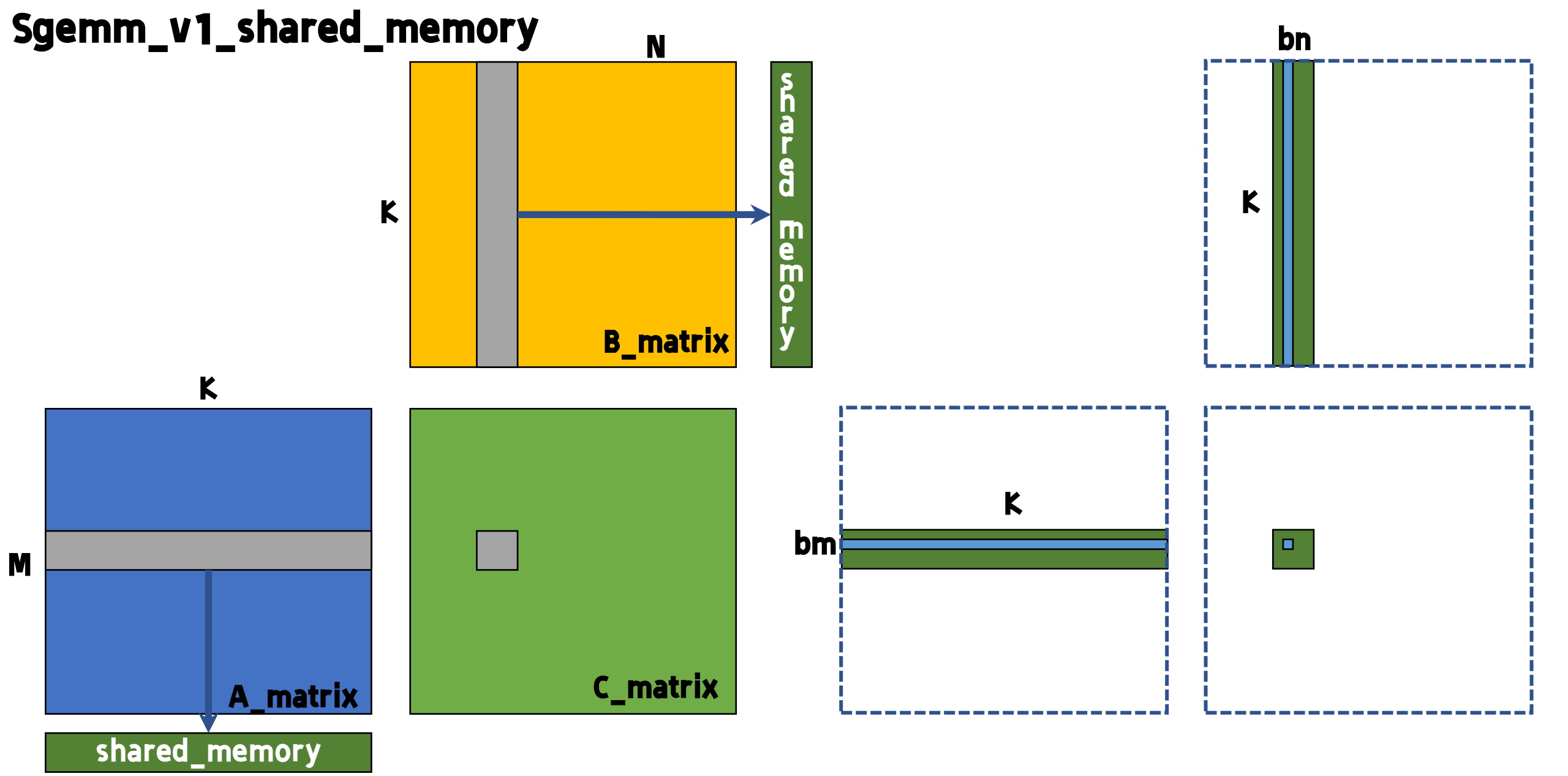


global_memory

- read:2MNK
- write:MN

深入浅出GPU优化系列：GEMM优化（一） - 知乎

Sgemm_v1_shared_memory



global_memory

read:bm*K*(N/bn)*(M/bm)+bn*K*(N/bn)*(M/bm)=KNM/bn+KNM/bm=KNM(1/bn+1/bm)

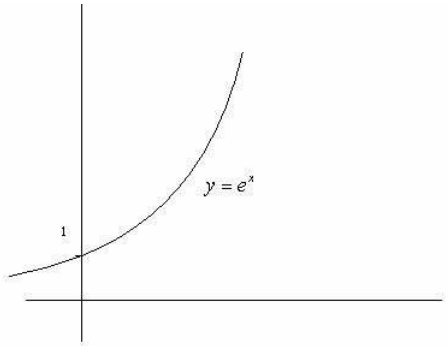
write:MN

CUDA实现矩阵乘法的8种优化策略编程介绍_哔哩哔哩_bilibili

4.Naive / safe Softmax

Algorithm 1 Naive softmax

$d_0 \leftarrow 0$
for $j \leftarrow 1, V$ **do**
    $d_j \leftarrow d_{j-1} + e^{x_j}$
end for
for $i \leftarrow 1, V$ **do**
    $y_i \leftarrow \frac{e^{x_i}}{d_V}$
end for



由于在实际的计算中，指数计算exp存在不稳定性，比如数值容易溢出，超过一定范围计算精度会下降等问题

Algorithm 2 Safe softmax

$m_0 \leftarrow -\infty$
for $k \leftarrow 1, V$ **do**
    $m_k \leftarrow \max(m_{k-1}, x_k)$
end for
$d_0 \leftarrow 0$
for $j \leftarrow 1, V$ **do**
    $d_j \leftarrow d_{j-1} + e^{x_j - m_V}$
end for
for $i \leftarrow 1, V$ **do**
    $y_i \leftarrow \frac{e^{x_i - m_V}}{d_V}$
end for

与Naive版本相比，不会出现数值溢出，但是计算复杂度高了一些。

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}} \quad \text{for } i = 1, 2, \cdots, n$$

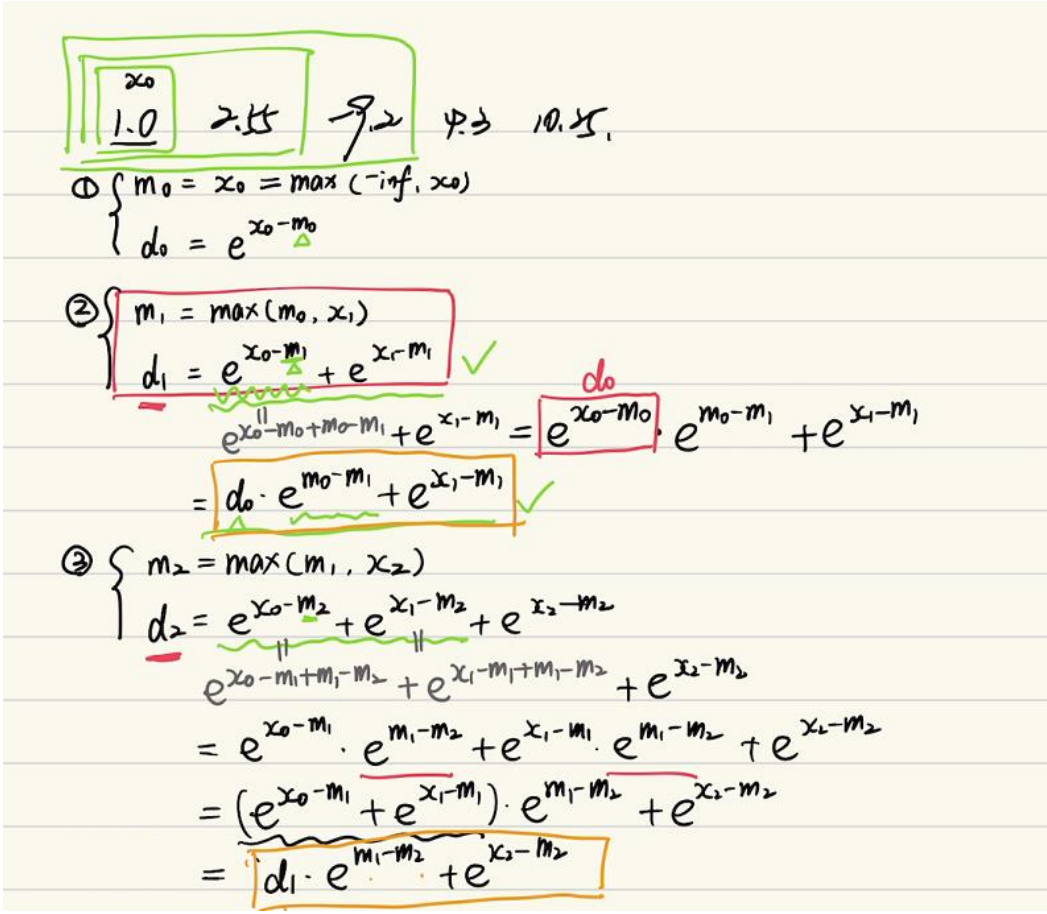
[一心二用的Online Softmax - 知乎]

5.Online Softmax

Algorithm 3 Safe softmax with online normalizer calculation

$m_0 \leftarrow -\infty$
$d_0 \leftarrow 0$
for $j \leftarrow 1, V$ **do**
  $m_j \leftarrow \max(m_{j-1}, x_j)$
  $d_j \leftarrow d_{j-1} \times e^{m_{j-1}-m_j} + e^{x_j-m_j}$
end for
for $i \leftarrow 1, V$ **do**
  $y_i \leftarrow \frac{e^{x_i-m_V}}{d_V}$
end for

核心目标：求max和求sum能不能只遍历1次



[一心二用的Online Softmax - 知乎]

6. Online Softmax * value

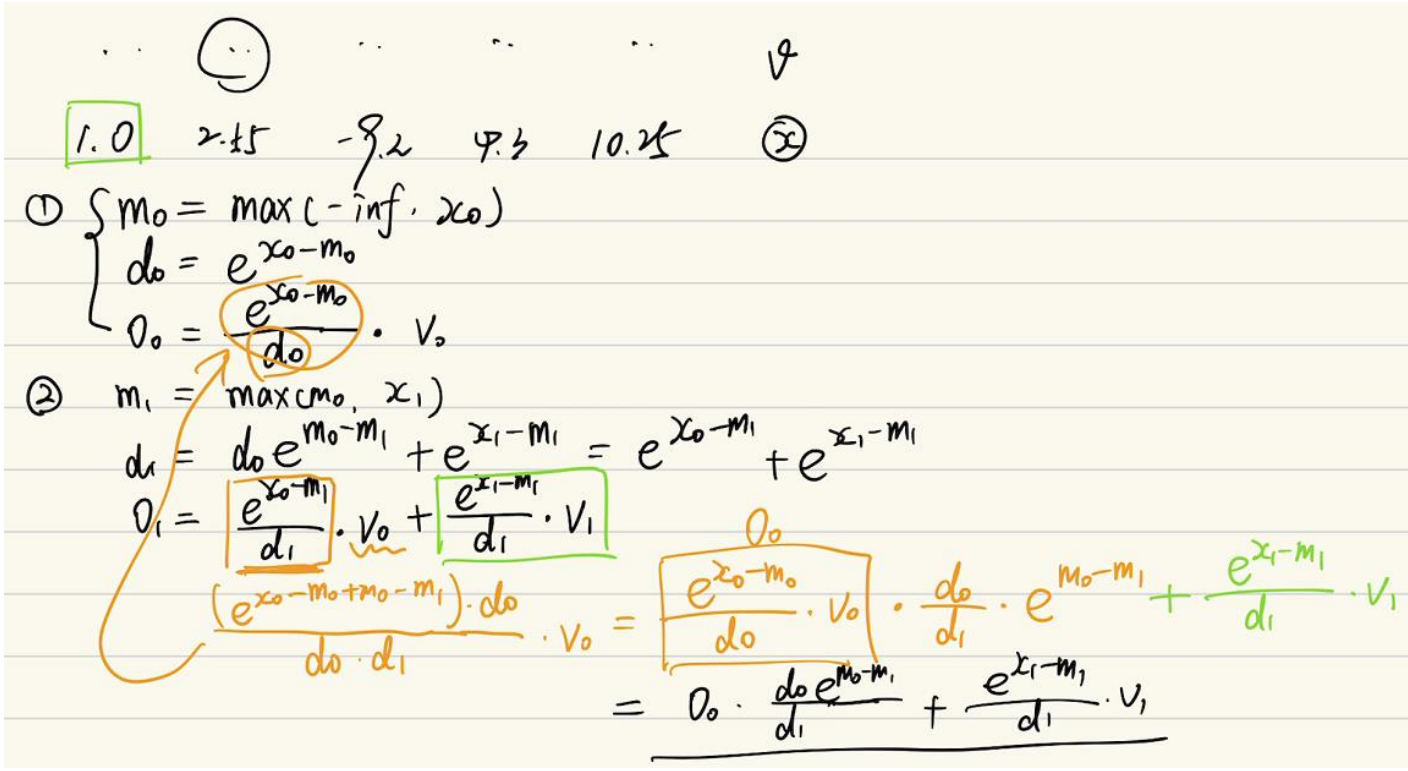
Algorithm 3 Safe softmax with online normalizer calculation

1: $m_0 \leftarrow -\infty$
2: $d_0 \leftarrow 0$
3: **for** $j \leftarrow 1, V$ **do**
4: $\quad m_j \leftarrow \max(m_{j-1}, x_j)$
5: $\quad d_j \leftarrow d_{j-1} \times e^{m_{j-1}-m_j} + e^{x_j-m_j}$
6: **end for**
7: **for** $i \leftarrow 1, V$ **do**
8: $\quad y_i \leftarrow \frac{e^{x_i-m_V}}{d_V} v_j$
9: **end for**



1: $m_0 \leftarrow -\infty$
2: $d_0 \leftarrow 0$
3: **for** $j \leftarrow 1, V$ **do**
4: $\quad m_j \leftarrow \max(m_{j-1}, x_j)$
5: $\quad d_j \leftarrow d_{j-1} \times e^{m_{j-1}-m_j} + e^{x_j-m_j}$

$$o_j \leftarrow o_{j-1} \frac{d_{j-1} e^{m_{j-1}-m_j}}{d_j} + \frac{e^{x_j-m_j}}{d_j} v_j$$



7. Cuda算子详解

Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M.

Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min\left(\lceil \frac{M}{4d} \rceil, d\right)$.

Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.

Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \ldots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \ldots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \ldots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.

Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \ldots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \ldots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \ldots, m_{T_r}$ of size B_r each.

for $1 \le j \le T_c$ **do**

  Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.

  for $1 \le i \le T_r$ **do**

    Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.

    On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.

    On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.

    On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.

    Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1}(\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.

    Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.

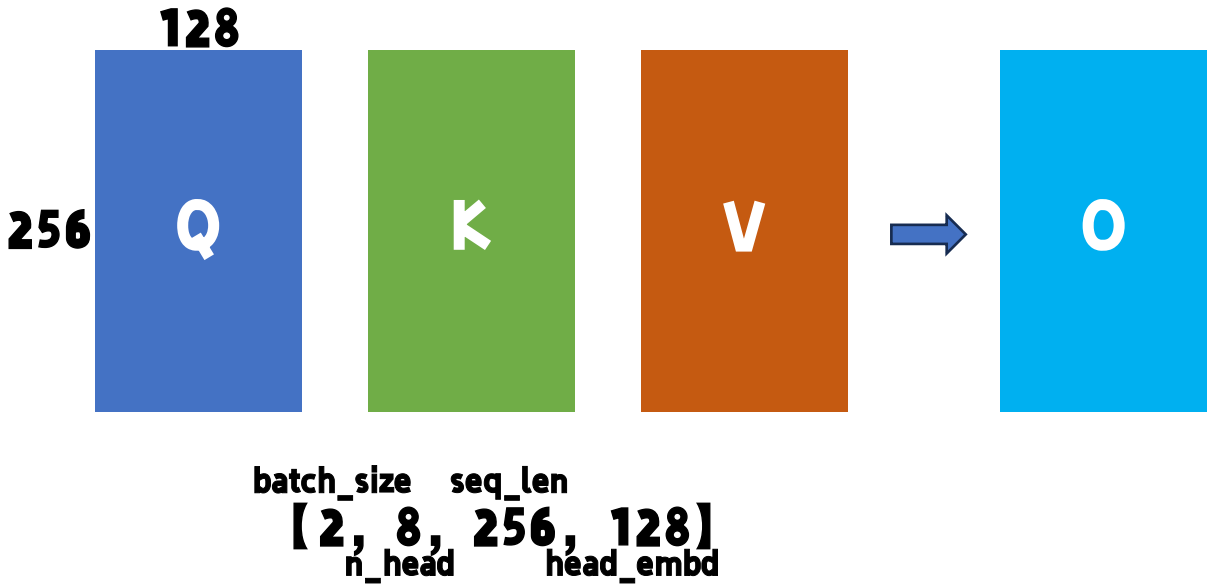
  end for

end for

Return $\mathbf{O}$.

知乎 @Alan小分享

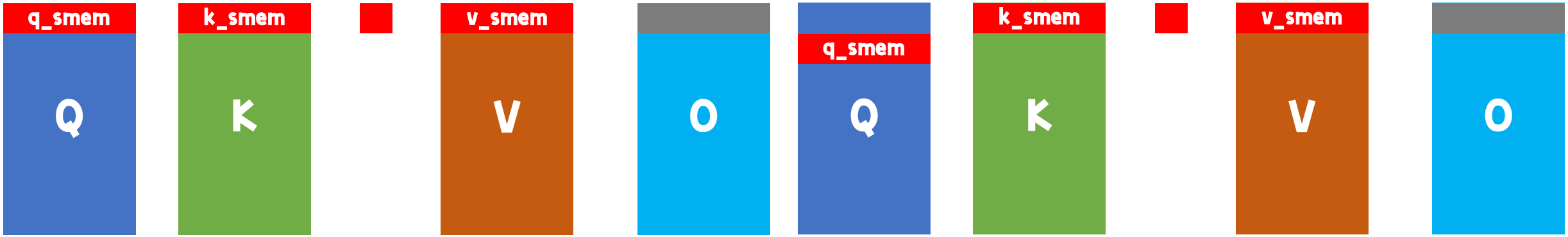
7. Cuda算子详解



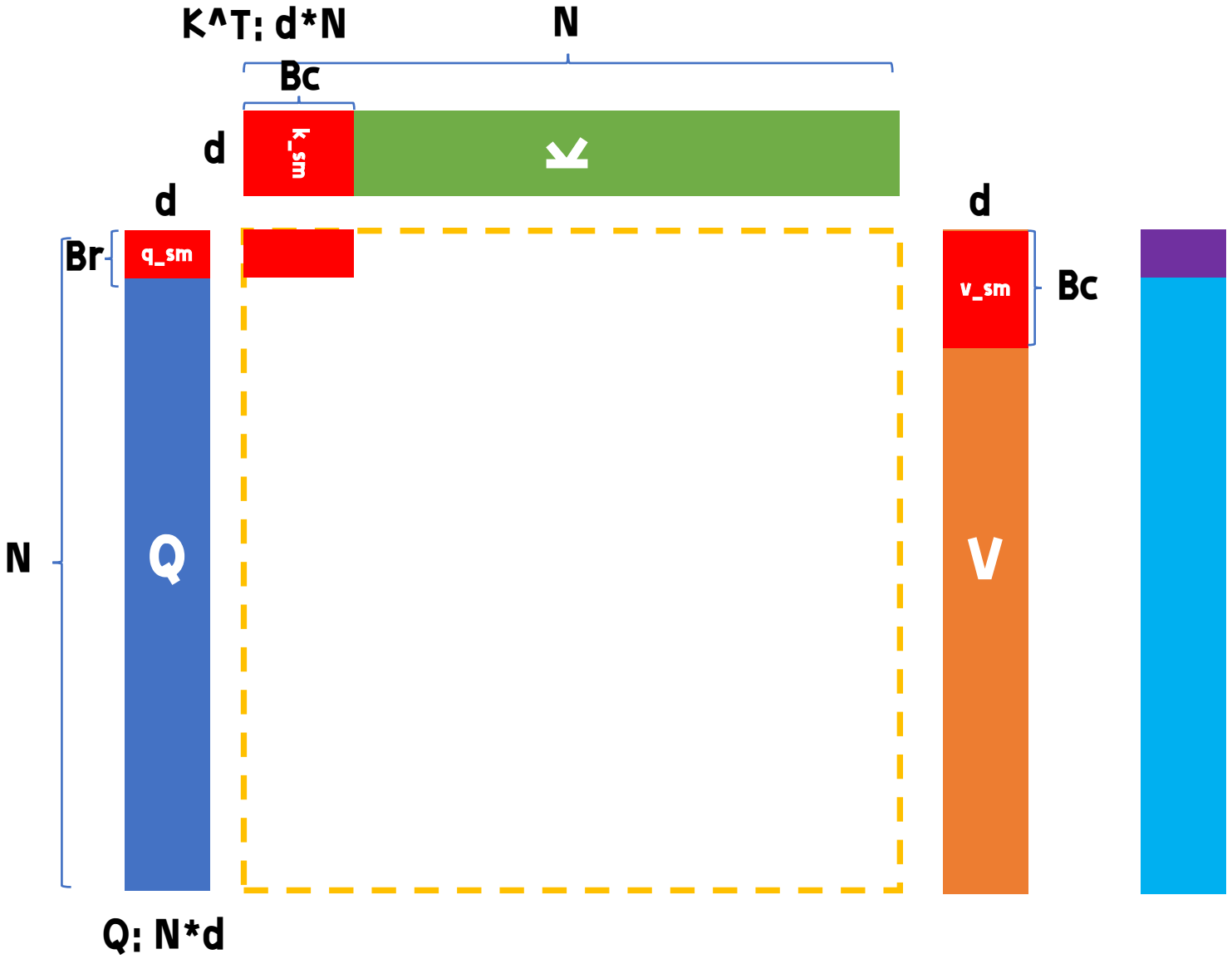
Grid:[2,8,1]
Block:[32,1,1]
sram:3*128*32*4+32*32*4

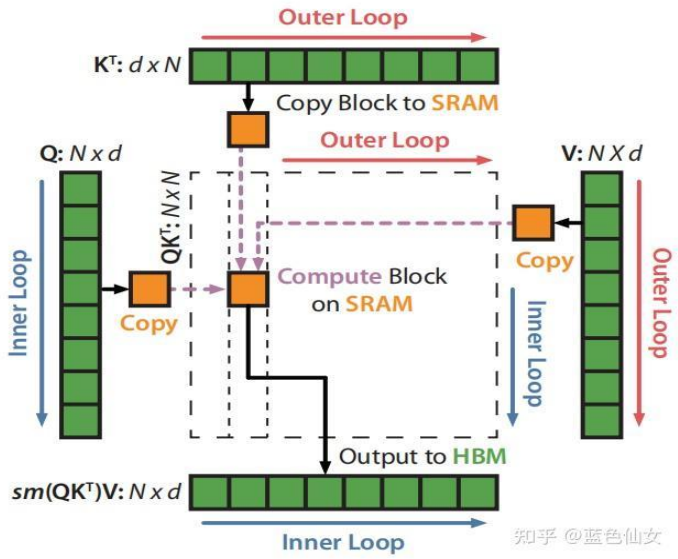
l: [2,8,256]
m: [2,8,256]





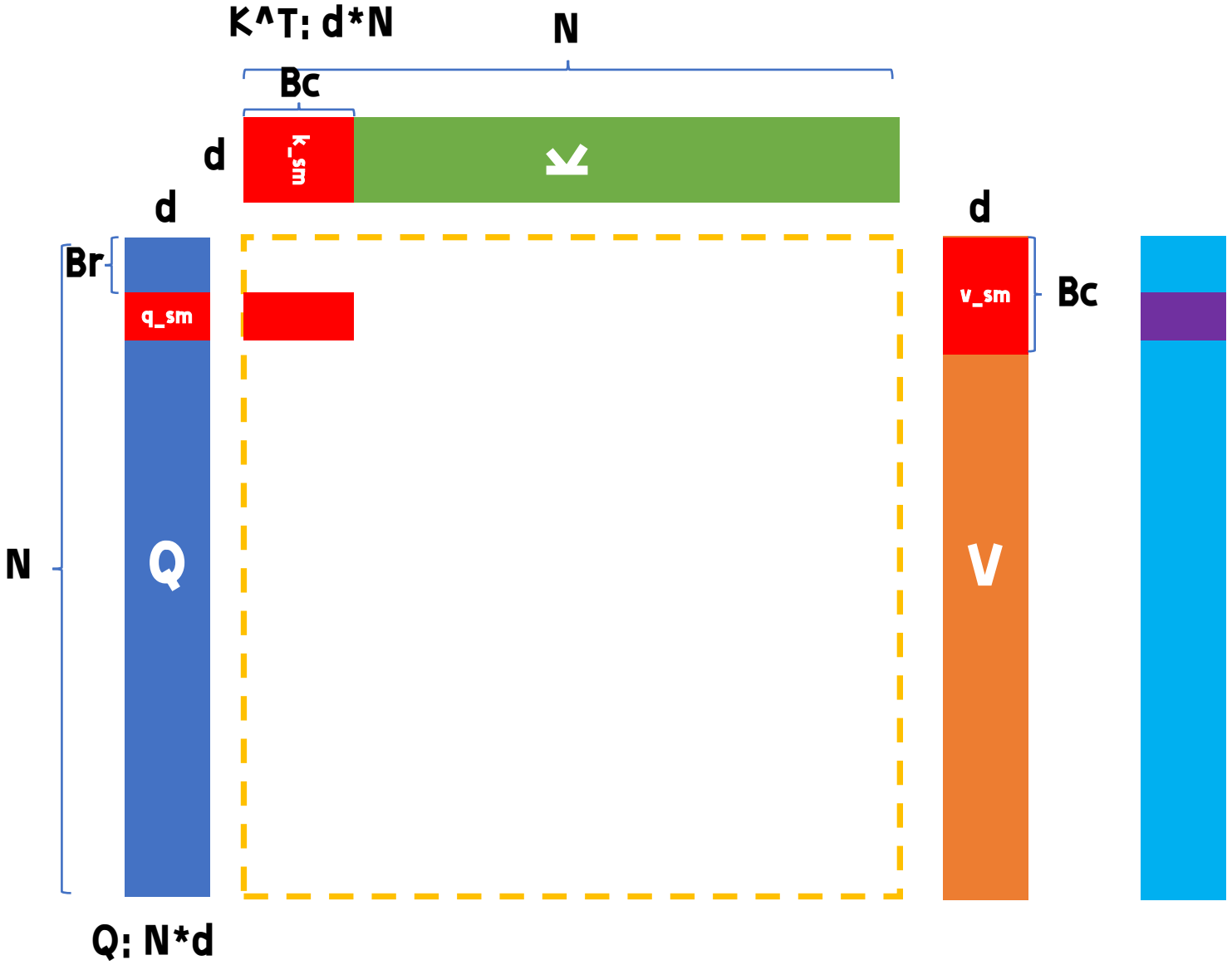
7. Cuda算子详解

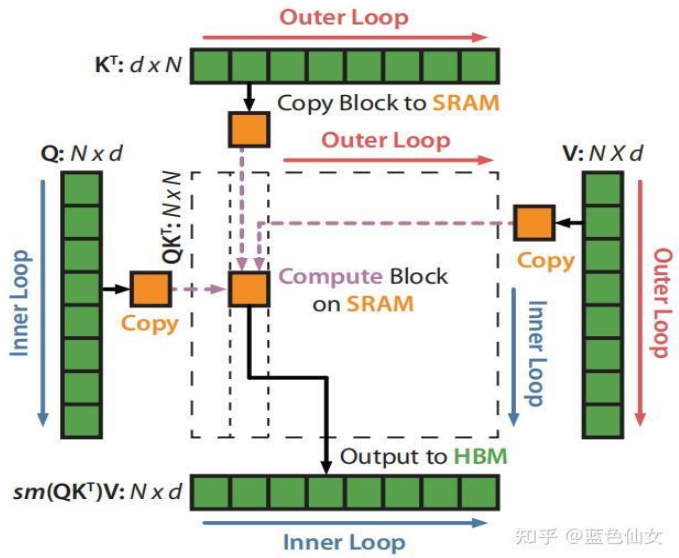




[FlashAttention v1、v2 - 公式推导 && 算法讲解 - 知乎]

7. Cuda算子详解

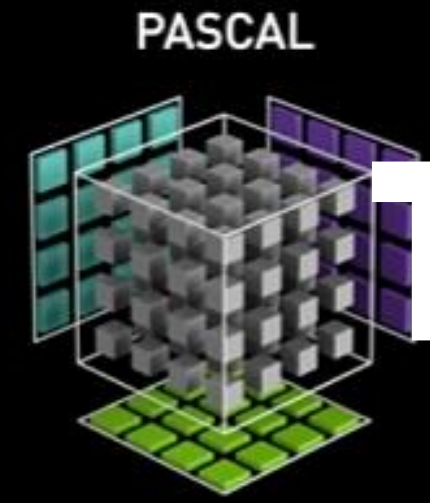


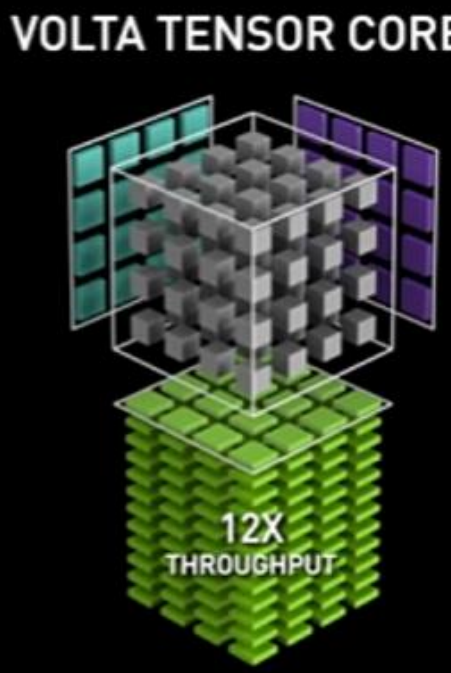


[FlashAttention v1、v2 - 公式推导 && 算法讲解 - 知乎]

8.疑惑与思考

- 当前我自己写的cuda算子总是我把弄的心力交瘁
  - 面临大量的索引计算个人感觉很容易出错
  - 完全不会函数封装，导致代码没办法复用
  - 写的时候会不自觉的想到：访存合并；边界判断；如何统一不同线程数的代码
- 优秀的开源一定要考虑到这些
  - vLLM：使用的是triton
  - Llama.cpp：使用了tensor core
  - 网上其他开源：使用了cutlass





Tensor Core 加速hgemm 实战教程



代码参考

- xlite-dev/CUDA-Learn-Notes: 📚200+ Tensor/CUDA Cores Kernels, ⚡flash-attn-mma, ⚡hgemm with WMMA, MMA and CuTe (98%~100% TFLOPS of cuBLAS/FA2 🎉🎉).

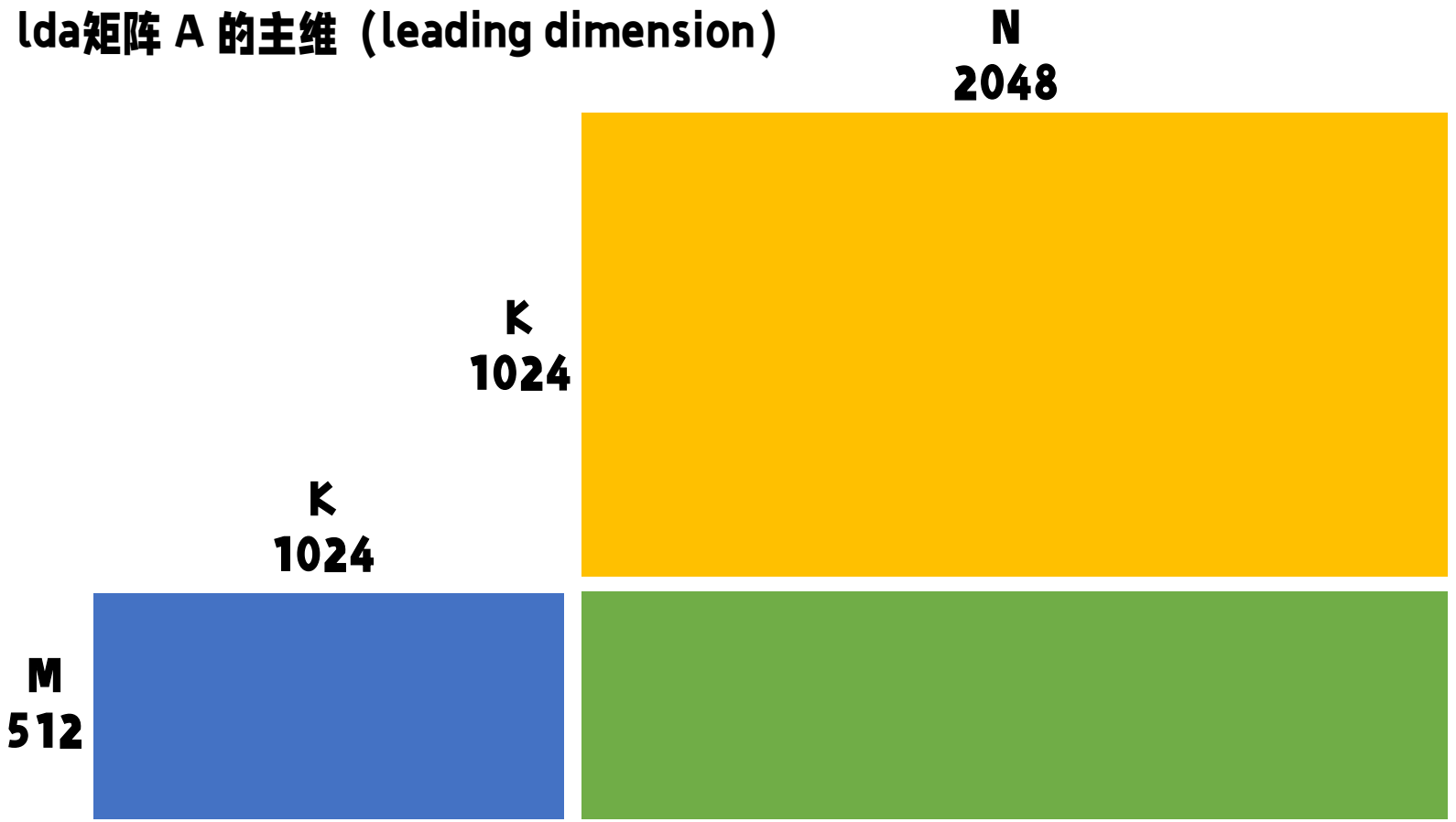
- Bruce-Lee-LY/cuda_hgemm: Several optimization methods of half-precision general matrix multiplication (HGEMM) using tensor core with WMMA API and MMA PTX instructio0n.

cublasGemmEx函数解析

```
cublasStatus_t cublasGemmEx(
    cublasHandle_t handle,
    cublasOperation_t transa,
    cublasOperation_t transb,
    int m,
    int n,
    int k,
    const void *alpha,
    const void *A,
    cudaDataType_t Atype,
    int lda,
    const void *B,
    cudaDataType_t Btype,
    int ldb,
    const void *beta,
    void *C,
    cudaDataType_t Ctype,
    int ldc,
    cudaDataType_t computeType,
    cublasGemmAlgo_t algo
);
```

$$C = \alpha \cdot op(A) \cdot op(B) + \beta \cdot C$$

lda矩阵 A 的主维（leading dimension）



[有关CUBLAS中的矩阵乘法函数 - 爨爨爨好 - 博客园]

cublasGemmEx函数解析

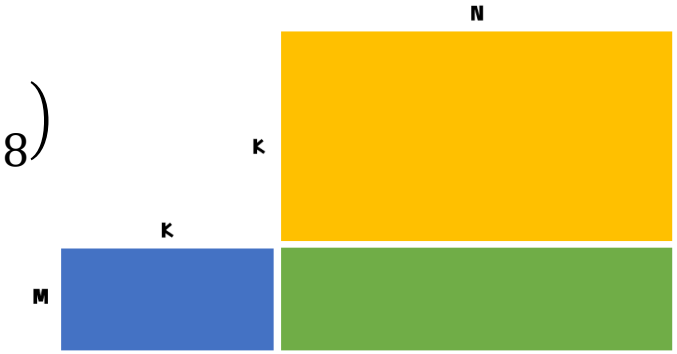
cublas是按列优先的原则存储矩阵信息的

错误写法

```
cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, m, n, k, &alpha, *A, m, *B, k, &beta, *C, m);
```

m=2, n=4, k=3

$$A=\begin{pmatrix}1 & 2 & 3\\ 4 & 5 & 6\end{pmatrix} \quad B=\begin{pmatrix}1 & 2 & 3 & 4\\ 5 & 6 & 7 & 8\\ 9 & 10 & 11 & 12\end{pmatrix} \quad C=AB=\begin{pmatrix}38 & 44 & 50 & 56\\ 83 & 98 & 113 & 128\end{pmatrix}$$



输入$A=\begin{pmatrix}1 & 2 & 3\\ 4 & 5 & 6\end{pmatrix}$ 内存排布 $\{1\ 2\ 3\ 4\ 5\ 6\}$ lda=m=2 列主序 $A_1=\begin{pmatrix}1 & 3 & 5\\ 2 & 4 & 6\end{pmatrix}$

输入$B=\begin{pmatrix}1 & 2 & 3 & 4\\ 5 & 6 & 7 & 8\\ 9 & 10 & 11 & 12\end{pmatrix}$ 内存排布 $\{1\ 2\ 3\ 4\ 5\ 6\ \dots\ 11\ 12\}$ ldb=k=3 列主序 $B_1=\begin{pmatrix}1 & 4 & 7 & 10\\ 2 & 5 & 8 & 11\\ 3 & 6 & 9 & 12\end{pmatrix}$

$C_1=A_1B_1=\begin{pmatrix}22 & 29 & 76 & 103\\ 28 & 64 & 100 & 136\end{pmatrix}$ 内存排布 $\{22\ 28\ 29\ 64\ \dots\ 103\ 136\}$ ldc=m=2 行主序 $\begin{pmatrix}22 & 28\\ 29 & 64\\ 76 & 100\\ 103 & 136\end{pmatrix}$ 解析成 $\begin{pmatrix}22 & 28 & 29 & 64\\ 76 & 100 & 103 & 136\end{pmatrix}$

有关CUBLAS中的矩阵乘法函数 - 爨爨爨好 - 博客园

cublasGemmEx函数解析

cublas是按列优先的原则存储矩阵信息的

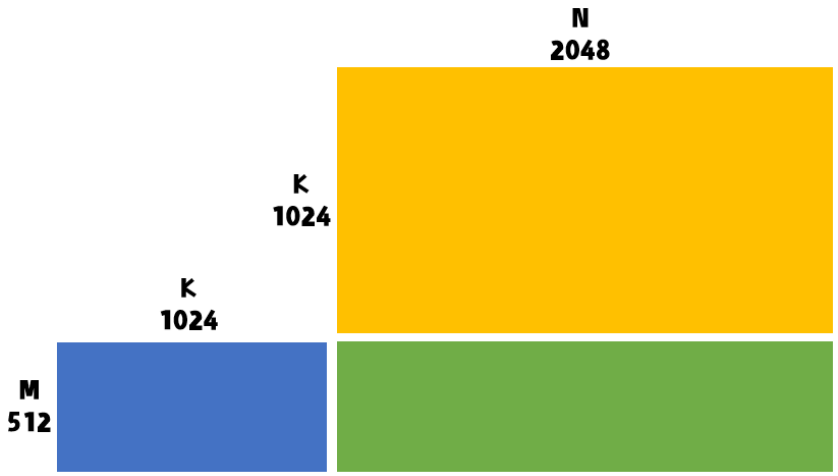
```
auto m_C_t = std::make_shared<Matrix>(m_M, m_N, "Matrix C T");
cublasGemmEx(handle,
             CUBLAS_OP_T,
             CUBLAS_OP_T,
             M, N, K,
             &alpha,
             A, CUDA_R_16F, K,
             B, CUDA_R_16F, N,
             &beta,
             m_C_t->getDevPtr(), CUDA_R_16F, M,
             CUBLAS_COMPUTE_16F,
             CUBLAS_GEMM_DEFAULT_TENSOR_OP);

dim3 block_size(32, 8);
dim3 grid_size((M - 1) / block_size.x + 1, (N - 1) / block_size.y + 1);
transpose_naive<<<grid_size, block_size>>>(m_C_t->getDevPtr(), C, N, M);
```

先转置

获得的结果是C的转置

再将C转置回来



有关CUBLAS中的矩阵乘法函数 - 爨爨爨好 - 博客园

cublasGemmEx函数解析

m=2, n=4, k=3

$$A=\begin{pmatrix}1 & 2 & 3\\ 4 & 5 & 6\end{pmatrix} \qquad B=\begin{pmatrix}1 & 2 & 3 & 4\\ 5 & 6 & 7 & 8\\ 9 & 10 & 11 & 12\end{pmatrix} \qquad C=AB=\begin{pmatrix}38 & 44 & 50 & 56\\ 83 & 98 & 113 & 128\end{pmatrix}$$

输入 $A=\begin{pmatrix}1 & 2 & 3\\ 4 & 5 & 6\end{pmatrix}$ **内存排布** $\{1\ 2\ 3\ 4\ 5\ 6\}$ **lda=k=3 列主序** $A_1=\begin{pmatrix}1 & 4\\ 2 & 5\\ 3 & 6\end{pmatrix}$ **转置** $A_2=\begin{pmatrix}1 & 2 & 3\\ 4 & 5 & 6\end{pmatrix}$

输入 $B=\begin{pmatrix}1 & 2 & 3 & 4\\ 5 & 6 & 7 & 8\\ 9 & 10 & 11 & 12\end{pmatrix}$ **内存排布** $\{1\ 2\ 3\ 4\ 5\ 6\ \dots\ 11\ 12\}$ **ldb=n=4 列主序** $B_1=\begin{pmatrix}1 & 5 & 9\\ 2 & 6 & 10\\ 3 & 7 & 11\\ 4 & 8 & 12\end{pmatrix}$ **转置** $B_2=\begin{pmatrix}1 & 2 & 3 & 4\\ 5 & 6 & 7 & 8\\ 9 & 10 & 11 & 12\end{pmatrix}$

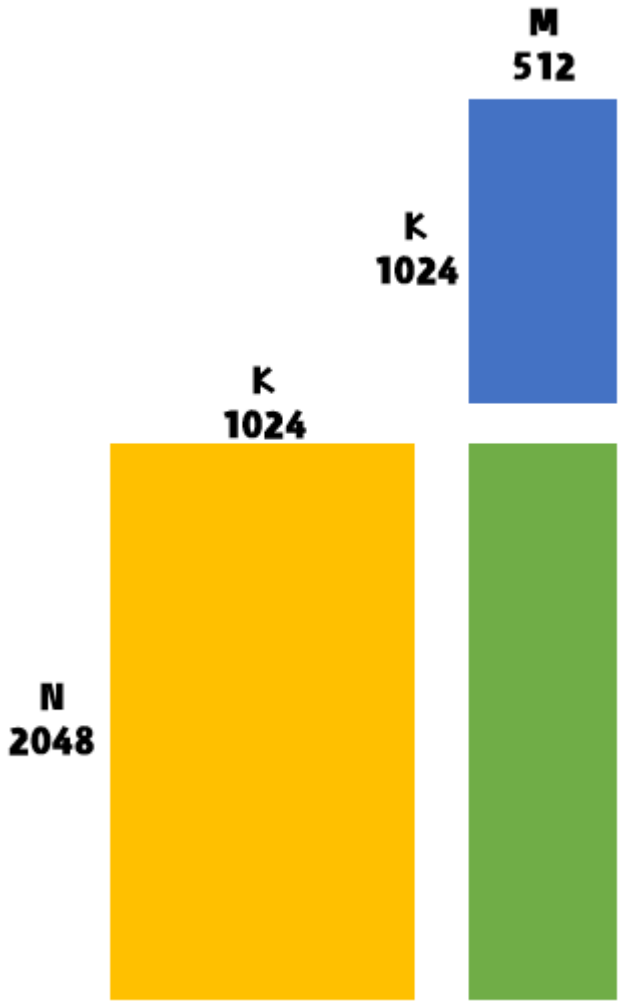
$C_2=A_2B_2=\begin{pmatrix}38 & 44 & 50 & 56\\ 83 & 98 & 113 & 128\end{pmatrix}$ **内存排布** $\{38\ 83\ 44\ 98\ \dots\ 56\ 128\}$ **ldc=m=2 行主序** $\begin{pmatrix}38 & 83\\ 44 & 98\\ 50 & 113\\ 56 & 128\end{pmatrix}$

[有关CUBLAS中的矩阵乘法函数 - 爨爨爨好 - 博客园]

cublasGemmEx函数解析

cublas是按列优先的原则存储矩阵信息的
根据C.T = (AB).T=B.T A.T可得如下

```
HGEMM_CHECK_CUBLAS_ERROR(cublasGemmEx(handle,
                                      CUBLAS_OP_N,
                                      CUBLAS_OP_N,
                                      N, M, K,
                                      &alpha,
                                      B, CUDA_R_16F, N,
                                      A, CUDA_R_16F, K,
                                      &beta,
                                      C, CUDA_R_16F, N,
                                      CUBLAS_COMPUTE_16F,
                                      CUBLAS_GEMM_DEFAULT_TENSOR_OP));
```



有关CUBLAS中的矩阵乘法函数 - 爨爨爨好 - 博客园

Tensor Core详解之wmma

10.24. Warp Matrix Functions

WMMA（Warp Matrix Multiply - Accumulate）

C++ warp matrix operations leverage Tensor Cores to accelerate matrix problems of the form D=A*B+C. These operations are supported on mixed-precision floating point data for devices of compute capability 7.0 or higher. This requires co-operation from all threads in a *warp*. In addition, these operations are allowed in conditional code only if the condition evaluates identically across the entire *warp*, otherwise the code execution is likely to hang.

C++ 线程束矩阵操作利用张量核心加速形式为 D=A*B+C 的矩阵问题。这些操作支持计算能力 7.0 或更高版本的混合精度浮点数据。这需要线程束中所有线程的协作。此外，如果条件在整个线程束中评估结果相同，则这些操作才允许在条件代码中执行，否则代码执行可能会挂起。

所有以下函数和类型均定义在命名空间 nvcuda::wmma 中

```
template<typename Use, int m, int n, int k, typename T, typename Layout=void> class
↪fragment;

void load_matrix_sync(fragment<...> &a, const T* mptr, unsigned ldm);
void load_matrix_sync(fragment<...> &a, const T* mptr, unsigned ldm, layout_t layout);
void store_matrix_sync(T* mptr, const fragment<...> &a, unsigned ldm, layout_t
↪layout);
void fill_fragment(fragment<...> &a, const T& v);
void mma_sync(fragment<...> &d, const fragment<...> &a, const fragment<...> &b, const
↪fragment<...> &c, bool satf=false);
```

Tensor Core详解之wmma

10.24.7. Example

The following code implements a 16x16x16 matrix multiplication in a single warp.

```
#include <mma.h>
using namespace nvcuda;

__global__ void wmma_ker(half *a, half *b, float *c) {
   // Declare the fragments
   wmma::fragment<wmma::matrix_a, 16, 16, 16, half, wmma::col_major> a_frag;
   wmma::fragment<wmma::matrix_b, 16, 16, 16, half, wmma::row_major> b_frag;
   wmma::fragment<wmma::accumulator, 16, 16, 16, float> c_frag;

   // Initialize the output to zero
   wmma::fill_fragment(c_frag, 0.0f);

   // Load the inputs
   wmma::load_matrix_sync(a_frag, a, 16);
   wmma::load_matrix_sync(b_frag, b, 16);

   // Perform the matrix multiplication
   wmma::mma_sync(c_frag, a_frag, b_frag, c_frag);

   // Store the output
   wmma::store_matrix_sync(c, c_frag, 16, wmma::mem_row_major);
}
```

fragment

```
template<typename Use, int m, int n, int k, typename T, typename Layout=void> class
↪fragment;
```

An overloaded class containing a section of a matrix distributed across all threads in the warp. The mapping of matrix elements into `fragment` internal storage is unspecified and subject to change in future architectures.

Only certain combinations of template arguments are allowed. The first template parameter specifies how the fragment will participate in the matrix operation. Acceptable values for Use are:

- `matrix_a` when the fragment is used as the first multiplicand, A,
- `matrix_b` when the fragment is used as the second multiplicand, B, or
- `accumulator` when the fragment is used as the source or destination accumulators (C or D, respectively).

当片段用作第一个乘数A时使用matrix_a
当片段用作第二个乘数B时使用matrix_b
当片段用作源或目标累加器（C 或 D）时使用accumulator

The `m`, `n` and `k` sizes describe the shape of the warp-wide matrix tiles participating in the multiply-accumulate operation. The dimension of each tile depends on its role. For `matrix_a` the tile takes dimension `m x k`; for `matrix_b` the dimension is `k x n`, and `accumulator` tiles are `m x n`.

The data type, `T`, may be `double`, `float`, `__half`, `__nv_bfloat16`, `char`, or `unsigned char` for multiplicands and `double`, `float`, `int`, or `__half` for accumulators. As documented in *Element Types and Matrix Sizes*, limited combinations of accumulator and multiplicand types are supported. The Layout parameter must be specified for `matrix_a` and `matrix_b` fragments. `row_major` or `col_major` indicate that elements within a matrix row or column are contiguous in memory, respectively. The `Layout` parameter for an `accumulator` matrix should retain the default value of `void`. A row or column layout is specified only when the accumulator is loaded or stored as described below.

对于 matrix_a 和 matrix_b 片段，必须指定 Layout 参数。row_major 或 col_major 表示矩阵行或列中的元素在内存中是连续的。累加器矩阵的 Layout 参数应保留默认值 void

fragment

```
template<typename Use, int m, int n, int k, typename T, typename Layout=void> class
↪fragment;
```

Matrix A	Matrix B	Accumulator	Matrix Size (m-n-k)
__half	__half	float	16x16x16
__half	__half	float	32x8x16
__half	__half	float	8x32x16
__half	__half	__half	16x16x16
__half	__half	__half	32x8x16
__half	__half	__half	8x32x16
unsigned char	unsigned char	int	16x16x16
unsigned char	unsigned char	int	32x8x16
unsigned char	unsigned char	int	8x32x16
signed char	signed char	int	16x16x16
signed char	signed char	int	32x8x16
signed char	signed char	int	8x32x16

Matrix A	Matrix B	Accumulator	Matrix Size (m-n-k)
__nv_bfloat16	__nv_bfloat16	float	16x16x16
__nv_bfloat16	__nv_bfloat16	float	32x8x16
__nv_bfloat16	__nv_bfloat16	float	8x32x16
precision::tf32	precision::tf32	float	16x16x8

Matrix A	Matrix B	Accumulator	Matrix Size (m-n-k)
double	double	double	8x8x4

Matrix A	Matrix B	Accumulator	Matrix Size (m-n-k)
precision::u4	precision::u4	int	8x8x32
precision::s4	precision::s4	int	8x8x32
precision::b1	precision::b1	int	8x8x128

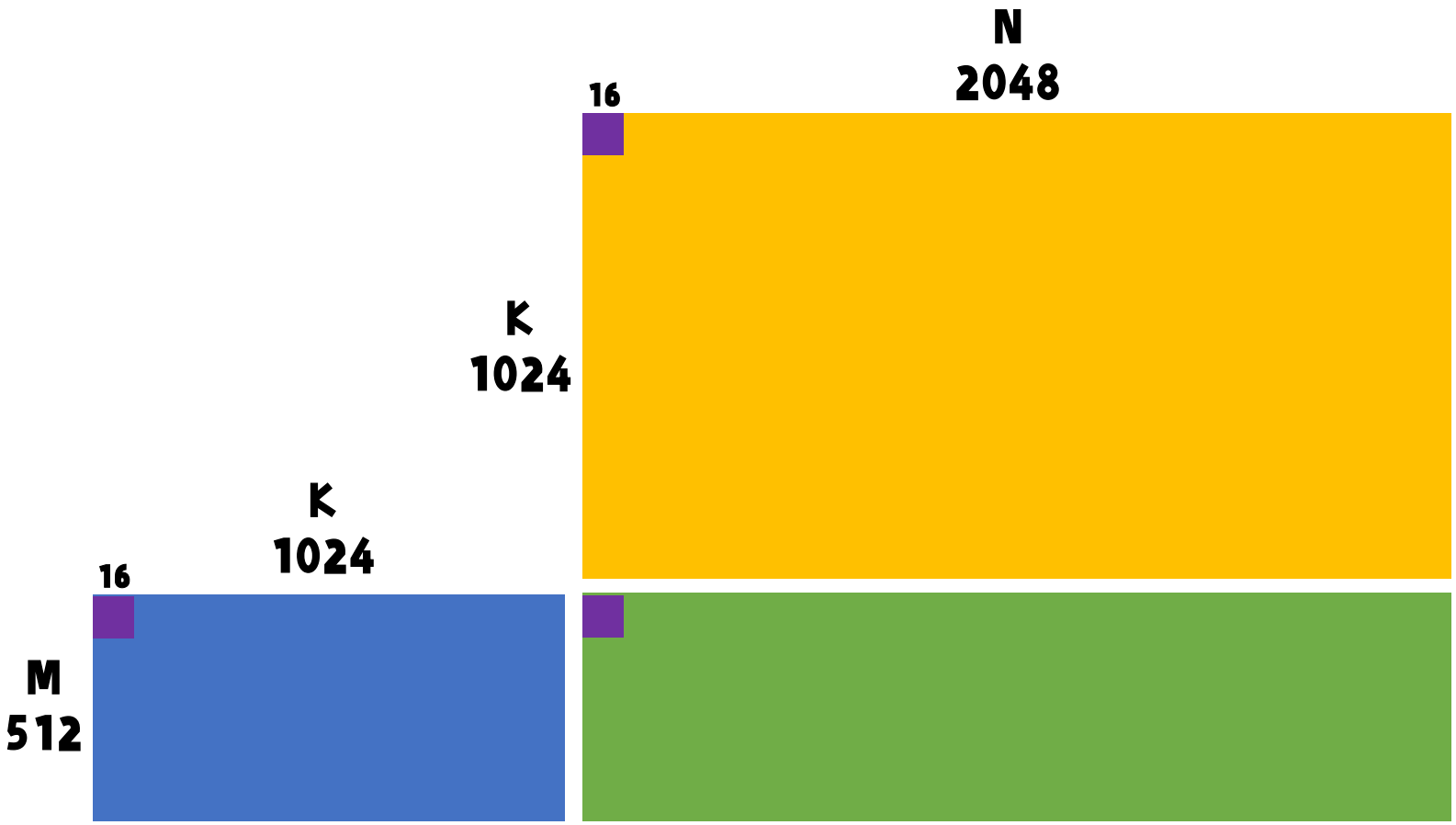
load_matrix_sync

```
void load_matrix_sync(fragment<...> &a, const T* mptr, unsigned ldm);
void load_matrix_sync(fragment<...> &a, const T* mptr, unsigned ldm, layout_t layout);
```

Waits until all warp lanes have arrived at load_matrix_sync and then loads the matrix fragment a from memory. `mptr` must be a 256-bit aligned pointer pointing to the first element of the matrix in memory. `ldm` describes the stride in elements between consecutive rows (for row major layout) or columns (for column major layout) and must be a multiple of 8 for `__half` element type or multiple of 4 for `float` element type. (i.e., multiple of 16 bytes in both cases). If the fragment is an `accumulator`, the `layout` argument must be specified as either `mem_row_major` or `mem_col_major`. For `matrix_a` and `matrix_b` fragments, the layout is inferred from the fragment's `layout` parameter. The values of `mptr`, `ldm`, `layout` and all template parameters for a must be the same for all threads in the warp. This function must be called by all threads in the warp, or the result is undefined.

等待所有 warp 通道到达 load_matrix_sync，然后从内存中加载矩阵片段 a。mptr 必须是一个 256 位对齐的指针，指向内存中矩阵的第一个元素。ldm 描述了连续行（对于行主序布局）或列（对于列主序布局）之间的元素步长，对于 __half 元素类型必须是 8 的倍数，对于 float 元素类型必须是 4 的倍数。（即两种情况下都是 16 字节的倍数）。如果片段是累加器，则必须将布局参数指定为 mem_row_major 或 mem_col_major。对于 matrix_a 和 matrix_b 片段，布局从片段的布局参数中推断。mptr、ldm、layout 以及 a 的所有模板参数的值必须对所有 warp 中的线程相同。此函数必须由 warp 中的所有线程调用，否则结果未定义。

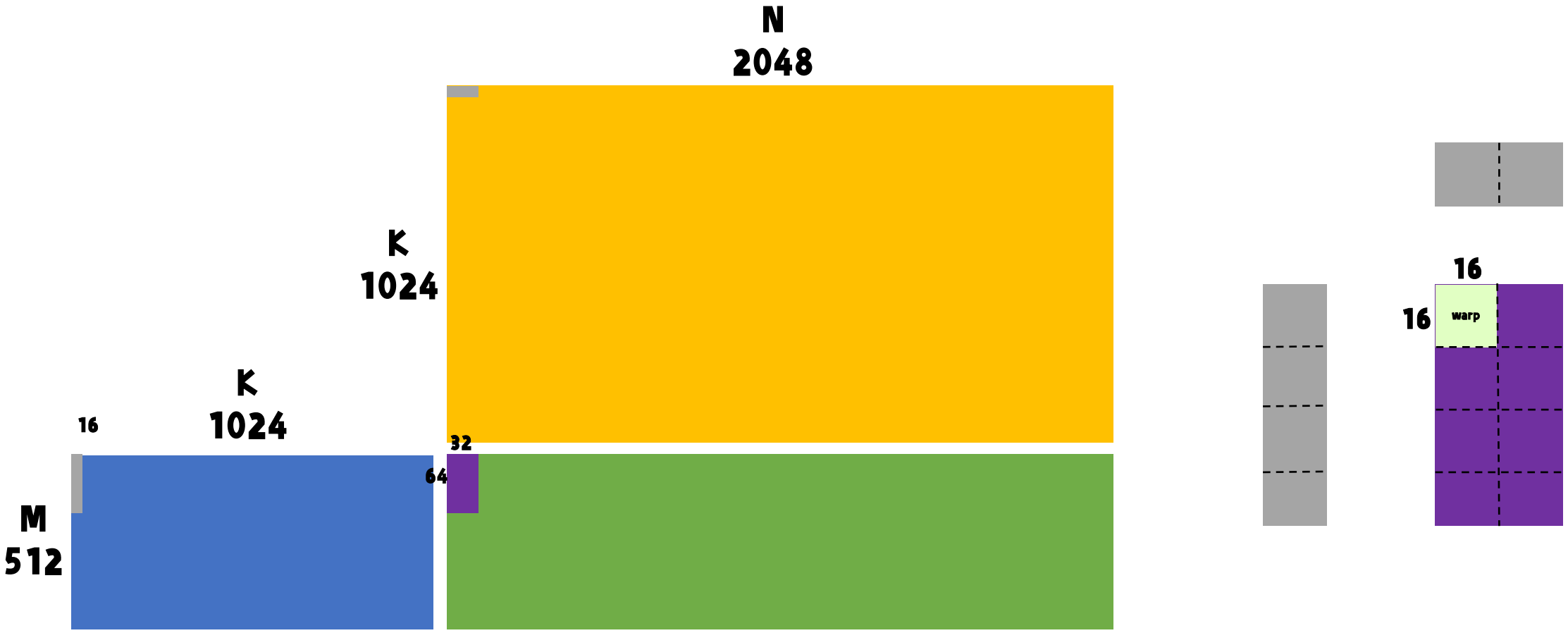
hgemm_v1_wmma_m16n16k16_naive_kernel



block_size:32
grid_size:128,32,1

2048/128=16
512/32=16

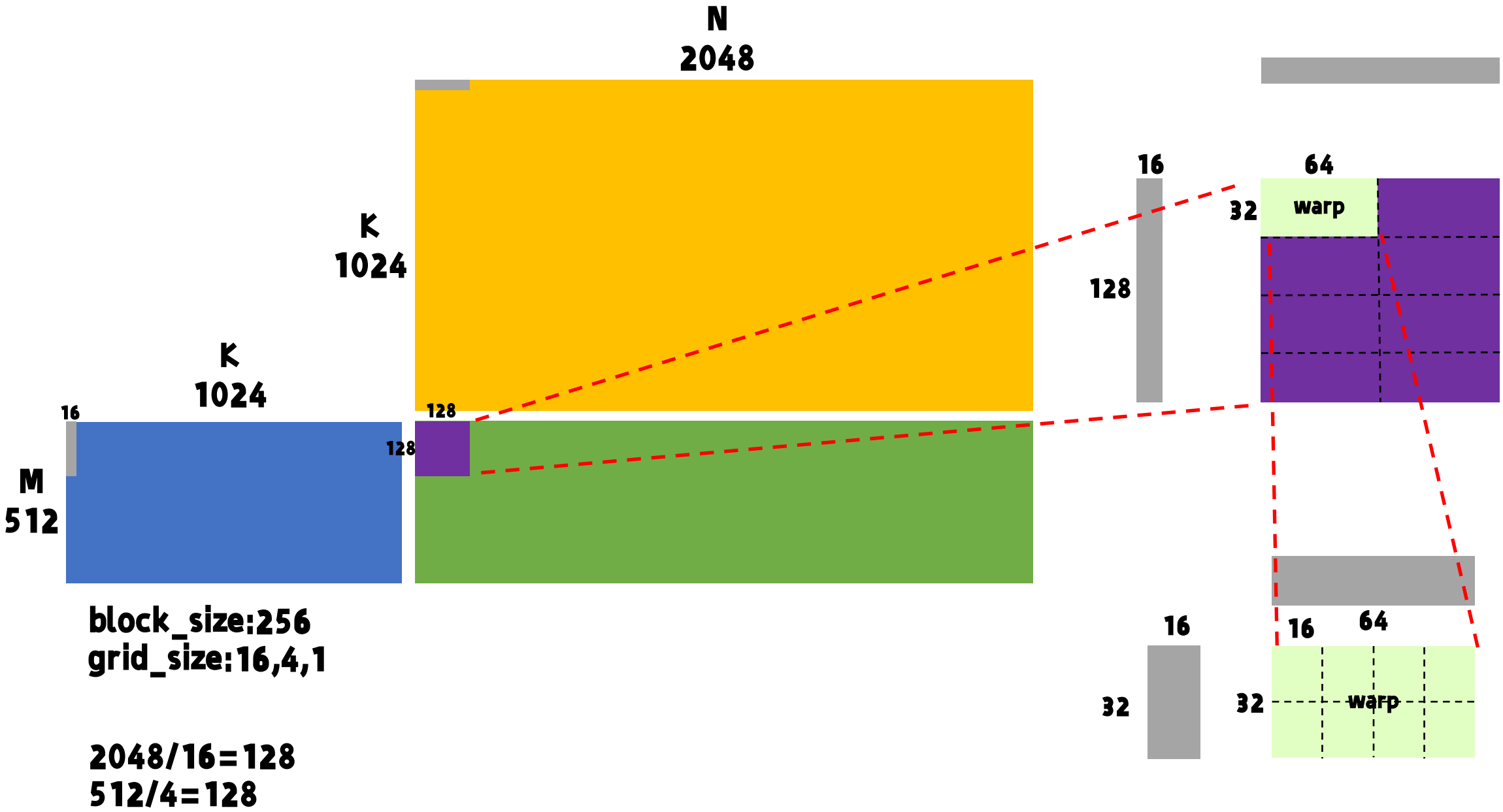
hgemm_v2_wmma_m16n16k16_mma4x2_kernel



block_size:256
grid_size:64,8,1

2048/64=32
512/8=64

hgemm_v3_wmma_m16n16k16_mma4x2_warp2x4_kernel



block_size:256
grid_size:16,4,1

2048/16=128
512/4=128

hgemm_v4_wmma_m16n16k16_mma4x2_warp2x4_dbuf_async_kernel

- **如何在cuda中内嵌ptx指令**

```
#define CP_ASYNC_CG(dst, src, Bytes) \
   asm volatile("cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" ::"r"(dst), "l"(src), "n"(Bytes))
#define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n" ::)
#define CP_ASYNC_WAIT_GROUP(N) asm volatile("cp.async.wait_group %0;\n" ::"n"(N))
```

- cp.async.cg.shared.global.L2::128B

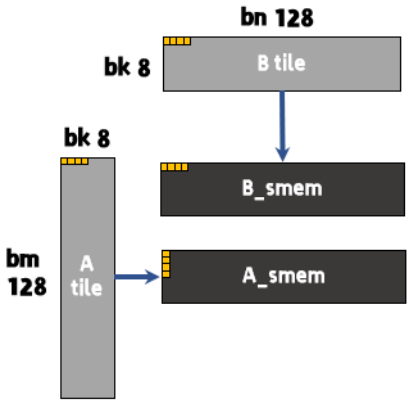
  这是一个异步拷贝指令，用于从全局内存（global）异步拷贝数据到共享内存（shared），同时通过 L2 缓存并以 128 字节的缓存行粒度进行操作。

- cp.async.commit_group

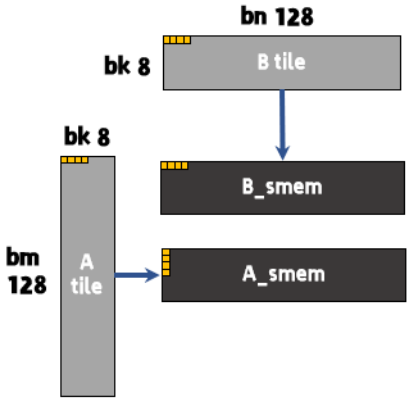
  提交一个 异步拷贝操作组（async-copy group），表示当前所有未提交的 cp.async 操作（如 cp.async.cg）正式进入执行队列。

- cp.async.wait_group %0

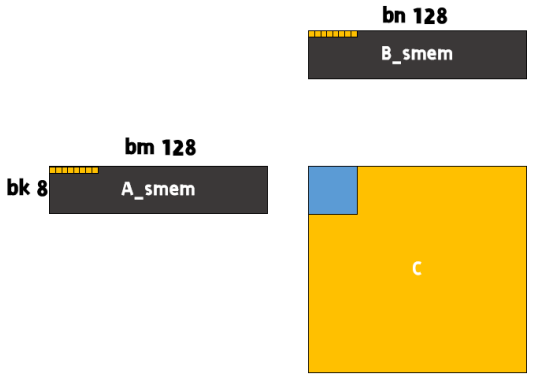
  等待一个 已提交的异步拷贝操作组 完成。类似于 CUDA 的 __syncthreads()，但仅针对异步拷贝操作，而不是全部线程同步。



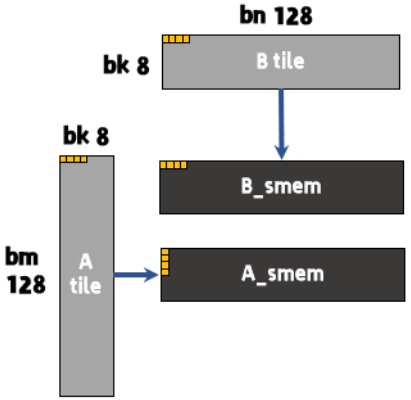
shared_0



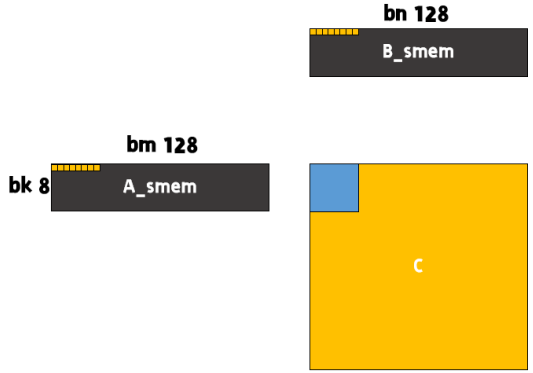
shared_1



shared_0 → reg

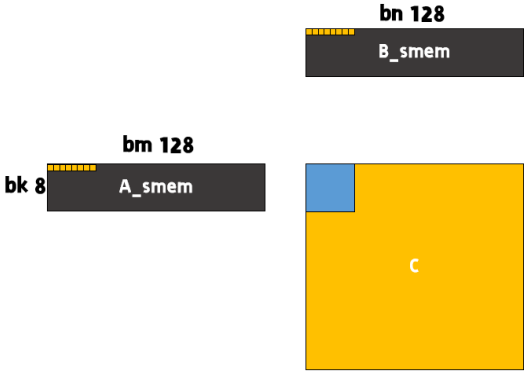


shared_0



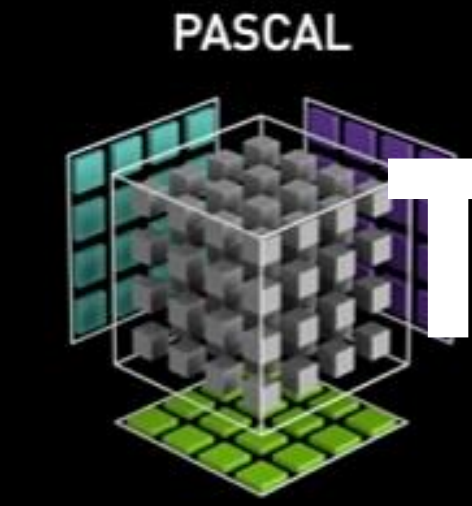
shared_1 → reg

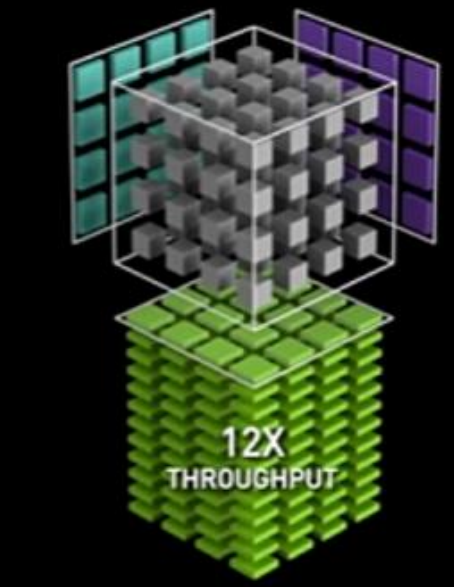
.....



shared_x → reg

Sgemm_v8_double_buffer





Tensor Core MMA 分析 Bank conflict 解決 &swizzle



Tensor Core详解之mma

PTX provides two ways to perform matrix multiply-and-accumulate computation:

- Using `wmma` instructions:
  - This warp-level computation is performed collectively by all threads in the warp as follows:
    - Load matrices A, B and C from memory into registers using the `wmma.load` operation. When the operation completes, the destination registers in each thread hold a fragment of the loaded matrix.
    - Perform the matrix multiply and accumulate operation using the `wmma.mma` operation on the loaded matrices. When the operation completes, the destination registers in each thread hold a fragment of the result matrix returned by the `wmma.mma` operation.
    - Store result Matrix D back to memory using the `wmma.store` operation. Alternately, result matrix D can also be used as argument C for a subsequent `wmma.mma` operation.

    The `wmma.load` and `wmma.store` instructions implicitly handle the organization of matrix elements when loading the input matrices from memory for the `wmma.mma` operation and when storing the result back to memory.
- Using `mma` instruction:
  - Similar to `wmma`, `mma` also requires computation to be performed collectively by all threads in the warp however distribution of matrix elements across different threads in warp needs to be done explicitly before invoking the `mma` operation. The `mma` instruction supports both dense as well as sparse matrix A. The sparse variant can be used when A is a structured sparse matrix as described in *Sparse matrix storage*.

PTX 提供了两种执行矩阵乘累加计算的方法：

使用 wmma 指令：

这种线程束级别的计算由线程束中的所有线程协同执行，步骤如下：

- 使用`wmma.load`操作将矩阵 A、B 和 C 从内存加载到寄存器中。操作完成后，每个线程中的目标寄存器会保存所加载矩阵的一个片段。
- 对已加载的矩阵使用`wmma.mma`操作执行矩阵乘累加运算。操作完成后，每个线程中的目标寄存器会保存由`wmma.mma`操作返回的结果矩阵的一个片段。
- 使用`wmma.store`操作将结果矩阵 D 存储回内存。或者，结果矩阵 D 也可以作为后续`wmma.mma`操作的参数 C。

  `wmma.load`和`wmma.store`指令在为`wmma.mma`操作从内存加载输入矩阵以及将结果存储回内存时，会隐式处理矩阵元素的组织方式。

使用 mma 指令：

与 wmma 类似，mma 也要求由线程束中的所有线程协同执行计算，不过在线程束中不同线程间分配矩阵元素，需要在调用 mma 操作前显式完成。mma 指令支持稠密矩阵和稀疏矩阵。当矩阵 A 是结构化稀疏矩阵（如 A. 中所述的稀疏矩阵存储方式）时，可以使用稀疏矩阵变体。

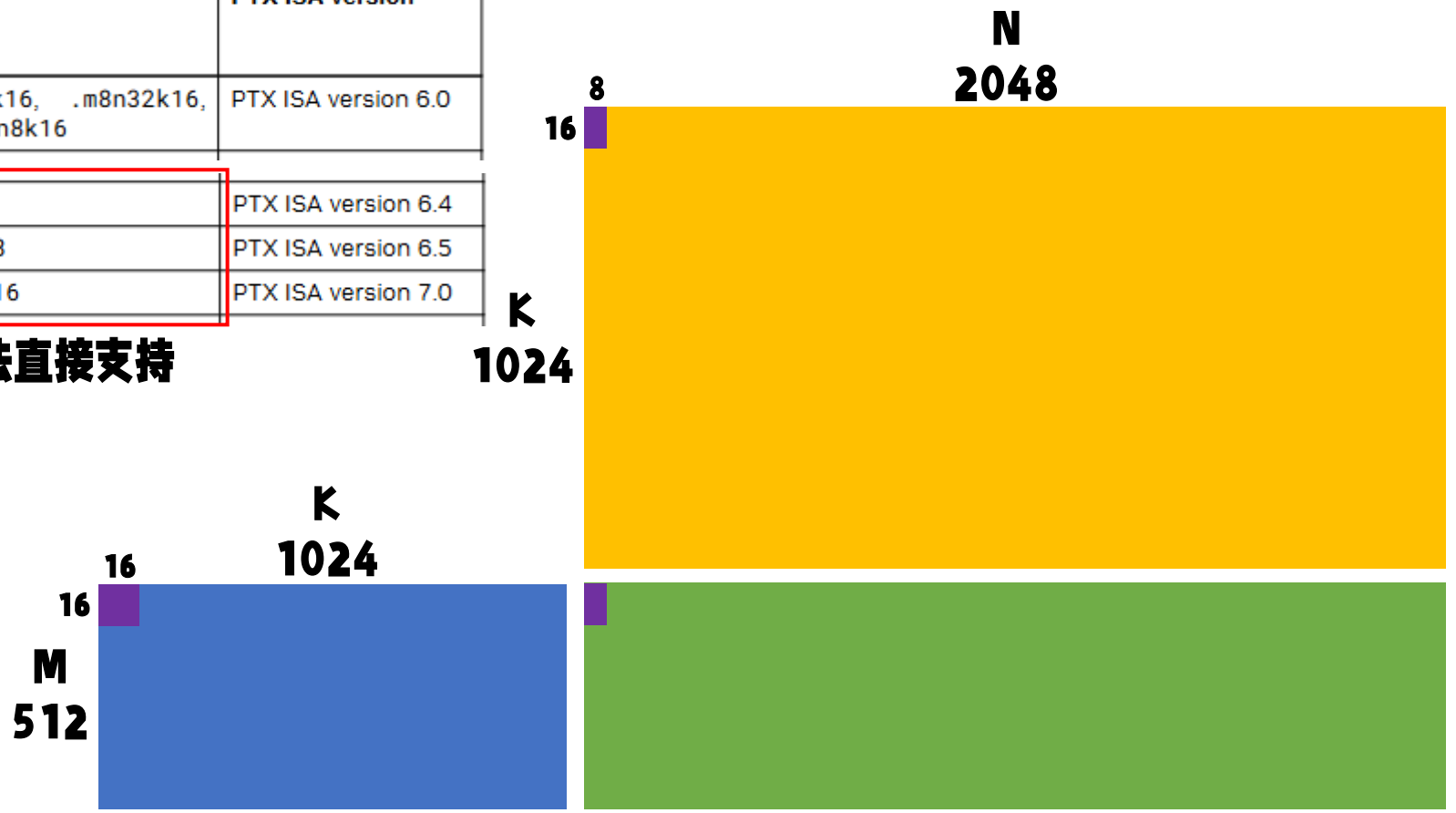
Tensor Core详解之mma

hgemm_mma_m16n8k16_naive_kernel

- **为什么mma版本的naive不用m16n16k16**

In-struc-tion	Scale	Sparsity	Multiplicand Data-type	Shape	PTX ISA version
wmma	NA	Dense	Floating-point - .f16	.m16n16k16, .m8n32k16, and .m32n8k16	PTX ISA version 6.0
mma		Dense	Floating-point - .f16	.m8n8k4	PTX ISA version 6.4
				.m16n8k8	PTX ISA version 6.5
				.m16n8k16	PTX ISA version 7.0

无法直接支持



Tensor Core详解之mma

hgemm_mma_m16n8k16_naive_kernel

ldmatrix

Collectively load one or more matrices from shared memory for mma instruction

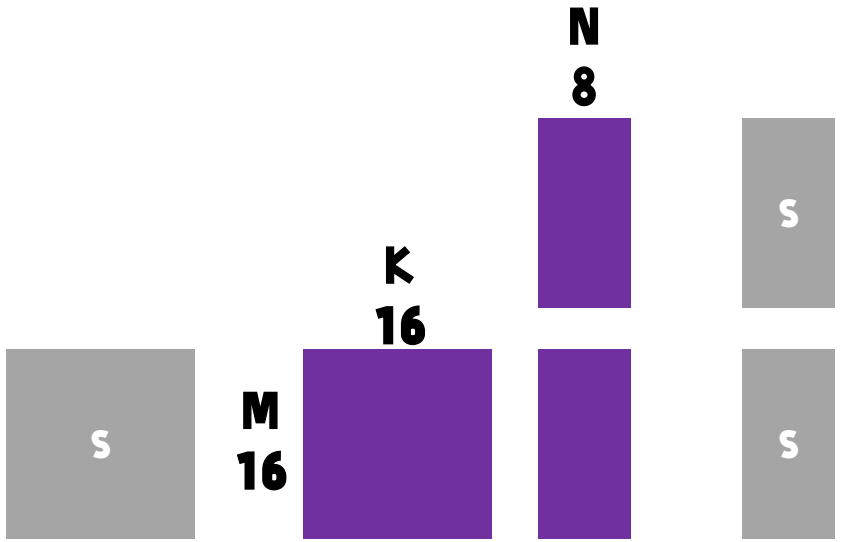
为 mma 指令从共享内存中集体加载一个或多个矩阵

Syntax

```
ldmatrix.sync.aligned.shape.num{.trans}{.ss}.type r, [p];

ldmatrix.sync.aligned.m8n16.num{.ss}.dst_fmt.src_fmt        r, [p];
ldmatrix.sync.aligned.m16n16.num.trans{.ss}.dst_fmt.src_fmt r, [p];

.shape   = {.m8n8, .m16n16};
.num     = {.x1, .x2, .x4};
.ss      = {.shared{::cta}};
.type    = {.b16, .b8};
.dst_fmt = { .b8x16 };
.src_fmt = { .b6x16_p32, .b4x16_p64 };
```



Following table shows the matrix load case for each .shape.

.shape	Matrix shape	Element size
.m8n8	8x8	16-bit
.m16n16	16x16	8-bit or 6-bit or 4-bit
.m8n16	8x16	6-bit or 4-bit

- 为什么数据要先搬到smem

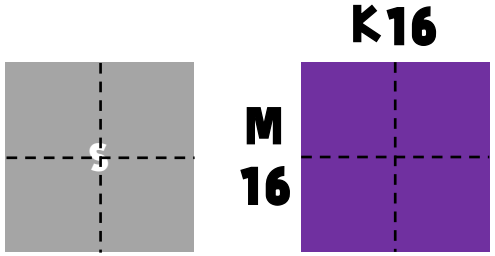
```
uint32_t RA[4];
uint32_t load_smem_a_ptr = __cvta_generic_to_shared(&s_a[lane_id % 16][(lane_id / 16) * 8]);
LDMATRIX_X4(RA[0], RA[1], RA[2], RA[3], load_smem_a_ptr);

#define LDMATRIX_X4(R0, R1, R2, R3, addr)                       \
  asm volatile("ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n" \
               : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3)         \
               : "r"(addr))
```

- 为什么这里是m8n8

Tensor Core详解之mma

hgemm_mma_m16n8k16_naive_kernel



Consecutive instances of row need not be stored contiguously in memory. The eight addresses required for each matrix are provided by eight threads, depending upon the value of .num as shown in the following table. Each address corresponds to the start of a matrix row. Addresses addr0–addr7 correspond to the rows of the first matrix, addresses addr8–addr15 correspond to the rows of the second matrix, and so on.

连续的 row 实例不需要在内存中连续存储。每个矩阵所需的八个地址由八个线程提供，具体取决于.num 的值，如下表所示。每个地址对应于矩阵行开始。地址 addr0–addr7 对应于第一个矩阵的行，地址 addr8–addr15 对应于第二个矩阵的行，依此类推。

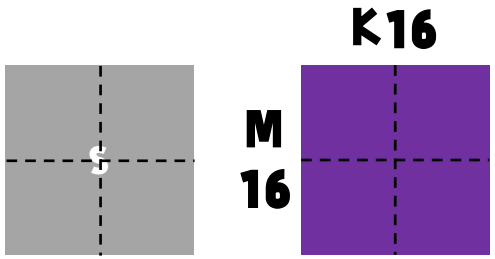
.num	Threads 0–7	Threads 8–15	Threads 16–23	Threads 24–31
.x1	addr0–addr7	–	–	–
.x2	addr0–addr7	addr8–addr15	–	–
.x4	addr0–addr7	addr8–addr15	addr16–addr23	addr24–addr31

```
uint32_t RA[4];
uint32_t load_smem_a_ptr = __cvta_generic_to_shared(&s_a[lane_id % 16][(lane_id / 16) * 8]);
LDMATRIX_X4(RA[0], RA[1], RA[2], RA[3], load_smem_a_ptr);

#define LDMATRIX_X4(R0, R1, R2, R3, addr)                                \
  asm volatile("ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n" \
          : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3)                       \
          : "r"(addr))
```

Tensor Core详解之mma

hgemm_mma_m16n8k16_naive_kernel



Row\Col	0	1	2	3	4	5	6	7
0	T0:r	T0:r	T1:r	T1:r	T2:r	T2:r	T3:r	T3:r
1	T4:r	T4:r	T5:r	T5:r	T6:r	T6:r	T7:r	T7:r
2	T8:r	T8:r	T9:r	T9:r	T10:r	T10:r	T11:r	T11:r
3	T12:r	T12:r	T13:r	T13:r	T14:r	T14:r	T15:r	T15:r
4	T16:r	T16:r	T17:r	T17:r	T18:r	T18:r	T19:r	T19:r
5	T20:r	T20:r	T21:r	T21:r	T22:r	T22:r	T23:r	T23:r
6	T24:r	T24:r	T25:r	T25:r	T26:r	T26:r	T27:r	T27:r
7	T28:r	T28:r	T29:r	T29:r	T30:r	T30:r	T31:r	T31:r

Figure 103: ldmatrix fragment layout for one 8x8 Matrix with 16-bit elements

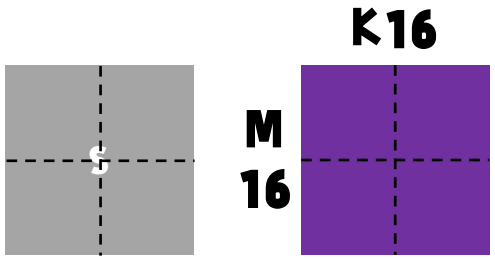
.x1 一个warp(32个Thread)搬运8*8个half

```
uint32_t RA[4];
uint32_t load_smem_a_ptr = __cvta_generic_to_shared(&s_a[lane_id % 16][(lane_id / 16) * 8]);
LDMATRIX_X4(RA[0], RA[1], RA[2], RA[3], load_smem_a_ptr);

#define LDMATRIX_X4(R0, R1, R2, R3, addr)                                \
  asm volatile("ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n" \
               : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3)                  \
               : "r"(addr))
```

Tensor Core详解之mma

hgemm_mma_m16n8k16_naive_kernel



R0

Row\Col	0	1	2	3	4	5	6	7
0	T0:r	T0:r	T1:r	T1:r	T2:r	T2:r	T3:r	T3:r
1	T4:r	T4:r	T5:r	T5:r	T6:r	T6:r	T7:r	T7:r
2	T8:r	T8:r	T9:r	T9:r	T10:r	T10:r	T11:r	T11:r
3	T12:r	T12:r	T13:r	T13:r	T14:r	T14:r	T15:r	T15:r
4	T16:r	T16:r	T17:r	T17:r	T18:r	T18:r	T19:r	T19:r
5	T20:r	T20:r	T21:r	T21:r	T22:r	T22:r	T23:r	T23:r
6	T24:r	T24:r	T25:r	T25:r	T26:r	T26:r	T27:r	T27:r
7	T28:r	T28:r	T29:r	T29:r	T30:r	T30:r	T31:r	T31:r

R2

Row\Col	0	1	2	3	4	5	6	7
0	T0:r	T0:r	T1:r	T1:r	T2:r	T2:r	T3:r	T3:r
1	T4:r	T4:r	T5:r	T5:r	T6:r	T6:r	T7:r	T7:r
2	T8:r	T8:r	T9:r	T9:r	T10:r	T10:r	T11:r	T11:r
3	T12:r	T12:r	T13:r	T13:r	T14:r	T14:r	T15:r	T15:r
4	T16:r	T16:r	T17:r	T17:r	T18:r	T18:r	T19:r	T19:r
5	T20:r	T20:r	T21:r	T21:r	T22:r	T22:r	T23:r	T23:r
6	T24:r	T24:r	T25:r	T25:r	T26:r	T26:r	T27:r	T27:r
7	T28:r	T28:r	T29:r	T29:r	T30:r	T30:r	T31:r	T31:r

R1

Row\Col	0	1	2	3	4	5	6	7
0	T0:r	T0:r	T1:r	T1:r	T2:r	T2:r	T3:r	T3:r
1	T4:r	T4:r	T5:r	T5:r	T6:r	T6:r	T7:r	T7:r
2	T8:r	T8:r	T9:r	T9:r	T10:r	T10:r	T11:r	T11:r
3	T12:r	T12:r	T13:r	T13:r	T14:r	T14:r	T15:r	T15:r
4	T16:r	T16:r	T17:r	T17:r	T18:r	T18:r	T19:r	T19:r
5	T20:r	T20:r	T21:r	T21:r	T22:r	T22:r	T23:r	T23:r
6	T24:r	T24:r	T25:r	T25:r	T26:r	T26:r	T27:r	T27:r
7	T28:r	T28:r	T29:r	T29:r	T30:r	T30:r	T31:r	T31:r

R3

Row\Col	0	1	2	3	4	5	6	7
0	T0:r	T0:r	T1:r	T1:r	T2:r	T2:r	T3:r	T3:r
1	T4:r	T4:r	T5:r	T5:r	T6:r	T6:r	T7:r	T7:r
2	T8:r	T8:r	T9:r	T9:r	T10:r	T10:r	T11:r	T11:r
3	T12:r	T12:r	T13:r	T13:r	T14:r	T14:r	T15:r	T15:r
4	T16:r	T16:r	T17:r	T17:r	T18:r	T18:r	T19:r	T19:r
5	T20:r	T20:r	T21:r	T21:r	T22:r	T22:r	T23:r	T23:r
6	T24:r	T24:r	T25:r	T25:r	T26:r	T26:r	T27:r	T27:r
7	T28:r	T28:r	T29:r	T29:r	T30:r	T30:r	T31:r	T31:r

.x4 一个warp(32个Thread)搬运8*8 x4个half

```
uint32_t RA[4];
uint32_t load_smem_a_ptr = __cvta_generic_to_shared(&s_a[lane_id % 16][(lane_id / 16) * 8]);
LDMATRIX_X4(RA[0], RA[1], RA[2], RA[3], load_smem_a_ptr);

#define LDMATRIX_X4(R0, R1, R2, R3, addr)                                \
  asm volatile("ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n" \
          : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3)                       \
          : "r"(addr))
```

Float Toy

进制转换（正整数） - 锤子在线工具

Tensor Core详解之mma

hgemm_mma_m16n8k16_naive_kernel

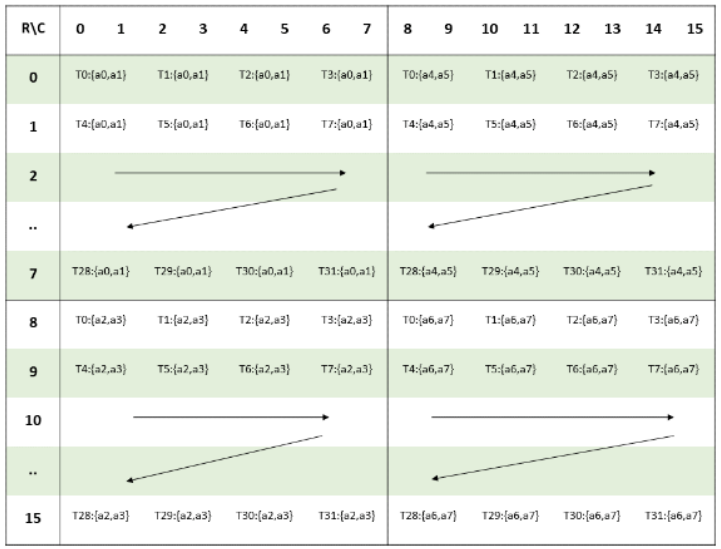
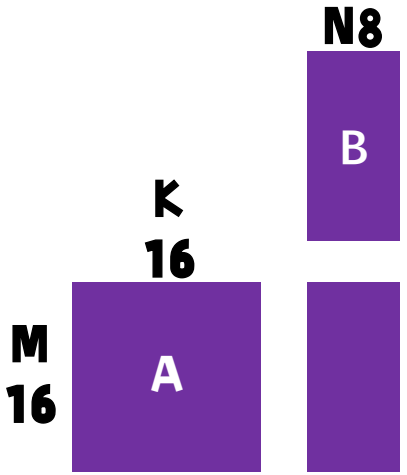


Figure 78: MMA .m16n8k16 fragment layout for matrix A with .f16 / .bf16 type.



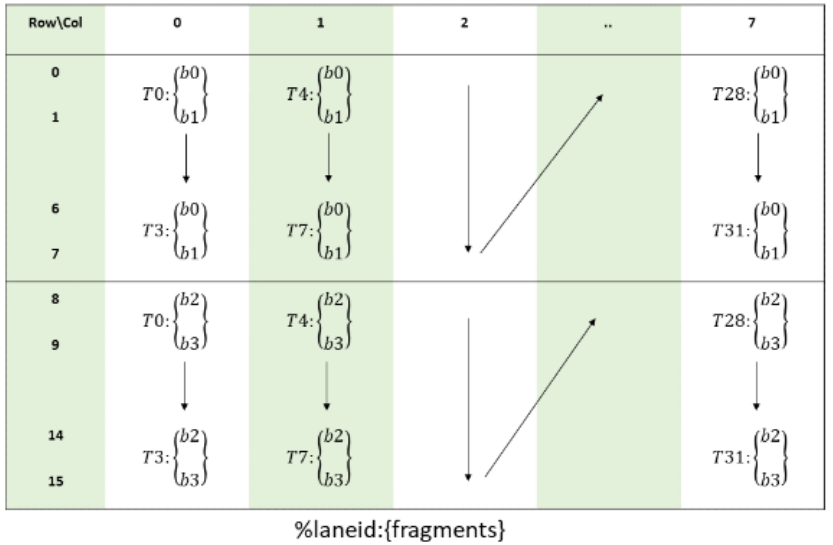


Figure 80: MMA .m16n8k16 fragment layout for matrix B with .f16 / .bf16 type.

- Multiplicand A:
  - .f16 and .bf16 :

包含四个.f16x2 寄存器的向量表达式，每个寄存器包含来自矩阵 A 的两个.f16 / .bf16 元素。

.atype	Fragment	Elements (low to high)
.f16 / .bf16	A vector expression containing four .f16x2 registers, with each register containing two .f16 / .bf16 elements from the matrix A.	a0, a1, a2, a3, a4, a5, a6, a7

The layout of the fragments held by different threads is shown in Figure 78.

- Multiplicand B:
  - .f16 and .bf16 :

包含两个 .f16x2 寄存器的向量表达式，每个寄存器包含来自矩阵 B 的两个 .f16 / .bf16 元素。

.btype	Fragment	Elements (low to high)
.f16 / .bf16	A vector expression containing two .f16x2 registers, with each register containing two .f16 / .bf16 elements from the matrix B.	b0, b1, b2, b3

The layout of the fragments held by different threads is shown in Figure 80.

```
#define HMMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1)
asm volatile("mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 {%0, %1}, {%2, %3, %4, %5},
{%6, %7}, {%8, %9};\n" : "=r"(RD0), "=r"(RD1) : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3),
"r"(RB0), "r"(RB1), "r"(RC0), "r"(RC1))
```

Tensor Core详解之mma

hgemm_mma_m16n8k16_naive_kernel

Row\Col	0 1	2 3	4 5	6 7
0	T0: {c0, c1}	T1: {c0, c1}	T2: {c0, c1}	T3: {c0, c1}
1	T4: {c0, c1}	T5: {c0, c1}	T6: {c0, c1}	T7: {c0, c1}
2				
..				
7	T28: {c0, c1}	T29: {c0, c1}	T30: {c0, c1}	T31: {c0, c1}
8	T0:{c2, c3}	T1:{c2, c3}	T2:{c2, c3}	T3:{c2, c3}
9	T4:{c2, c3}	T5:{c2, c3}	T6:{c2, c3}	T7:{c2, c3}
10				
..				
15	T28:{c2, c3}	T29:{c2, c3}	T30:{c2, c3}	T31:{c2, c3}

%laneid:{fragments}

Figure 82: MMA .m16n8k16 fragment layout for accumulator matrix matrix C/D.

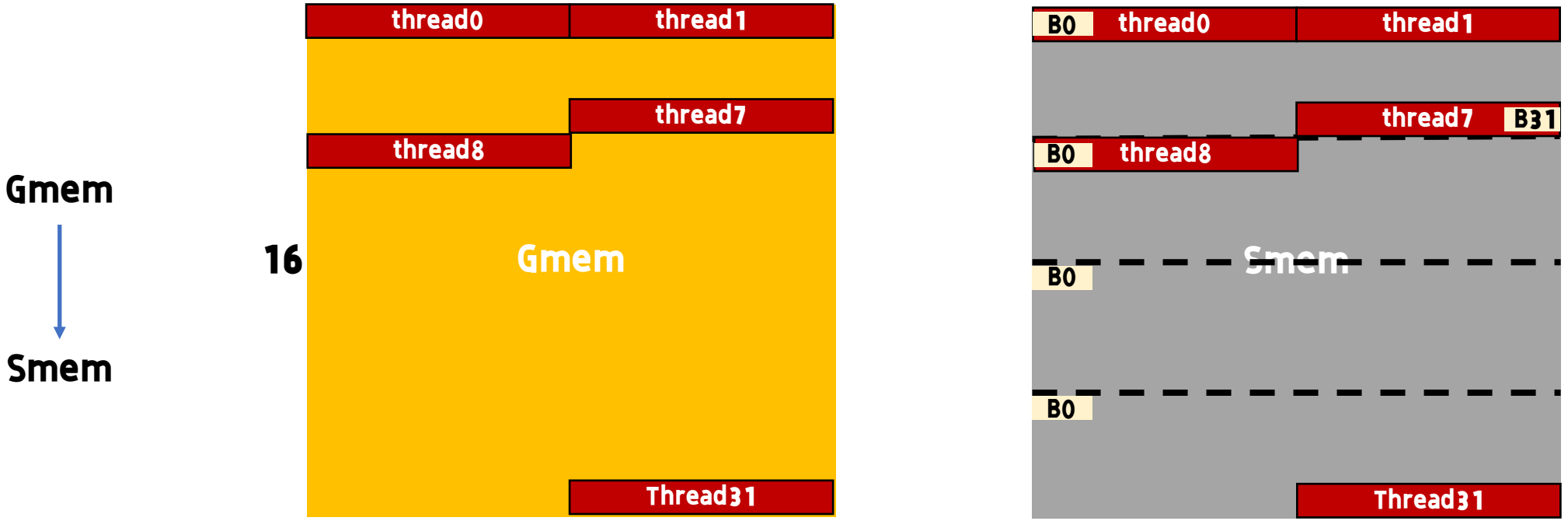
- Accumulators (C or D):

.ctype / .dtype	Fragment	Elements (low to high)
.f64	A vector expression containing four .f64 registers containing .f64 elements from the matrix C (or D).	c0, c1, c2, c3
.f32	A vector expression containing four .f32 registers containing four .f32 elements from the matrix C (or D).	
.f16	A vector expression containing two .f16x2 registers, with each register containing two .f16 elements from the matrix C (or D).	

包含两个.f16x2 寄存器的向量表达式，每个寄存器包含矩阵 C（或 D）中的两个.f16 元素

```
#define HMMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1)
asm volatile("mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 {%0, %1}, {%2, %3, %4,
%5}, {%6, %7}, {%8, %9};\n" : "=r"(RD0), "=r"(RD1) : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3),
"r"(RB0), "r"(RB1), "r"(RC0), "r"(RC1))
```

Tensor Core详解之Bank Conflict



因为每个thread连续读取4个4Byte，那么只要8个thread不访问到同一个bank就不会bank conflict

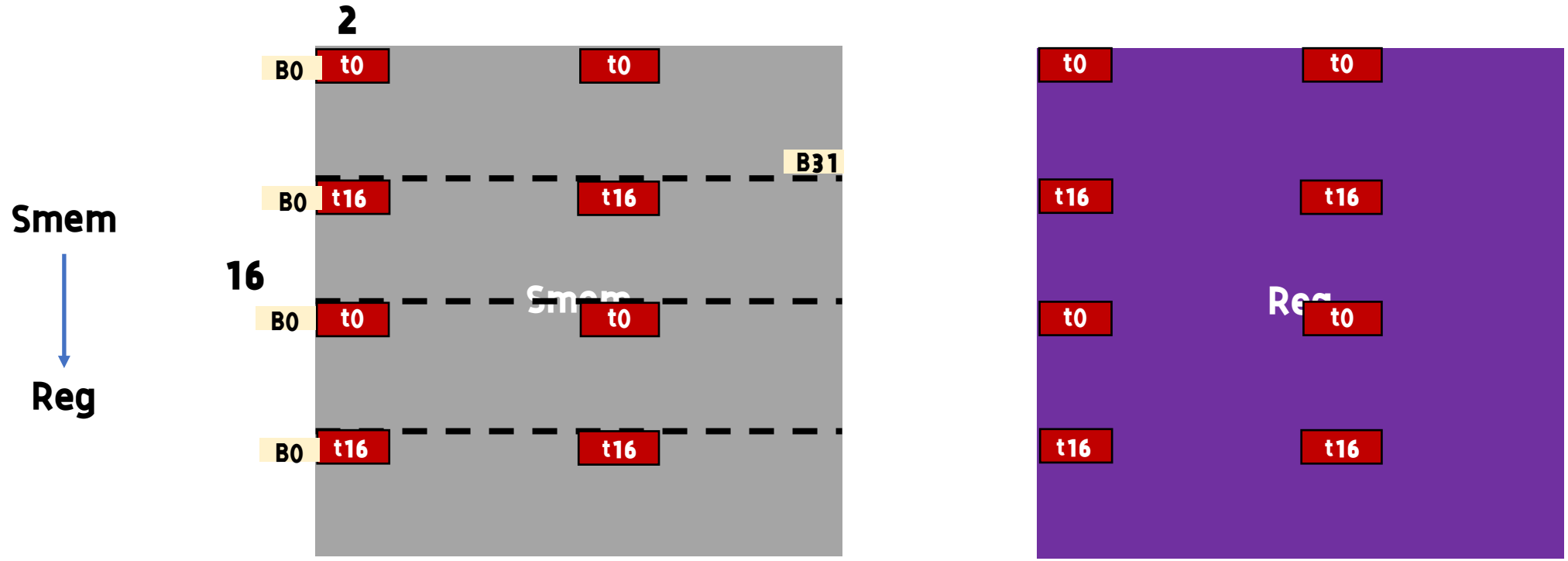
一个当前最常见的应用场景就是矩阵乘的时候从共享内存到寄存器的内存拷贝操作会涉及到 Bank Conflict 的问题。在 NVIDIA GPU 中一个 Cache line 的大小为 128 Bytes，即为 1024 bits。而每个 Cache line 中分为多个 Banks。**当一个线程访问超过 4 bytes(32 bits)，即每个 warp 超过 128 bytes(1024 bits)的时候，GPU 不会发出单个事务(transaction)**，每个事务的最大为 128 bytes。如果每个线程按照向量化访问最大 16 bytes（128 bits）进行访问时，那个 warp wide 将为 512 bytes，此时 GPU 会将其分解为 4 个事务，其中 T0-T7，T8-15，T16-T23，T24-T31 分别产生一个事务，每个事务的宽度为 128 bytes。Bank Conflict 将按照事务产生而并非按照请求或者 warp，指令产生。

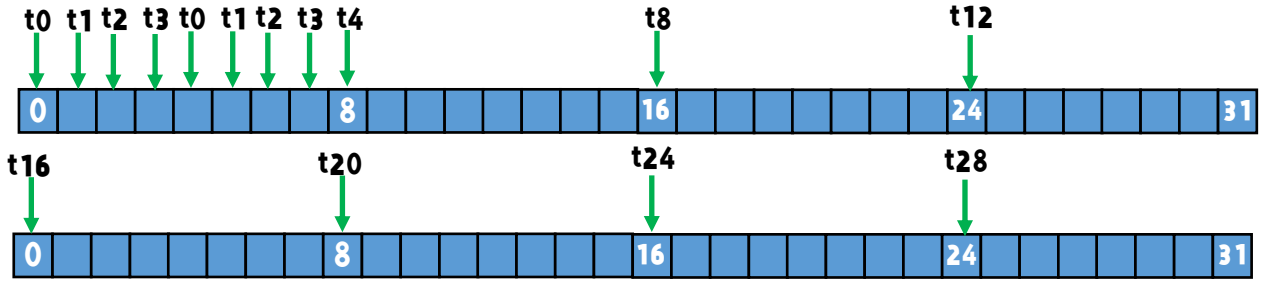
[关于 Bank Conflict 与 Swizzle - 知乎]

[实用 Swizzle 教程（一） - 知乎]

[Chtholly-Boss/swizzle: A practical way of learning Swizzle]

Tensor Core详解之Bank Conflict



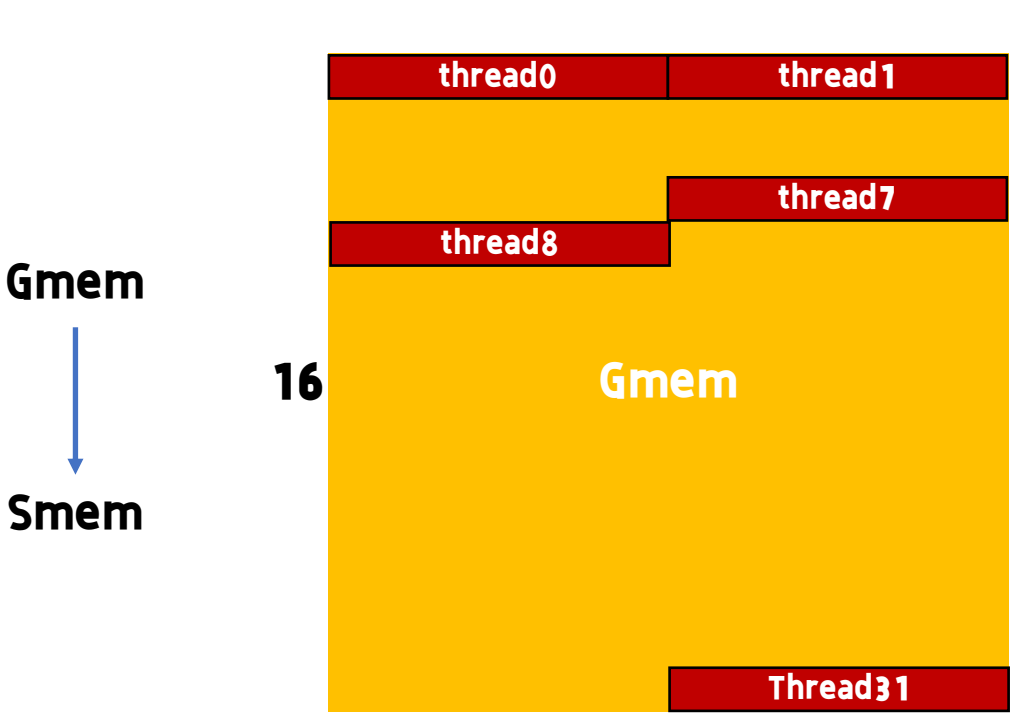


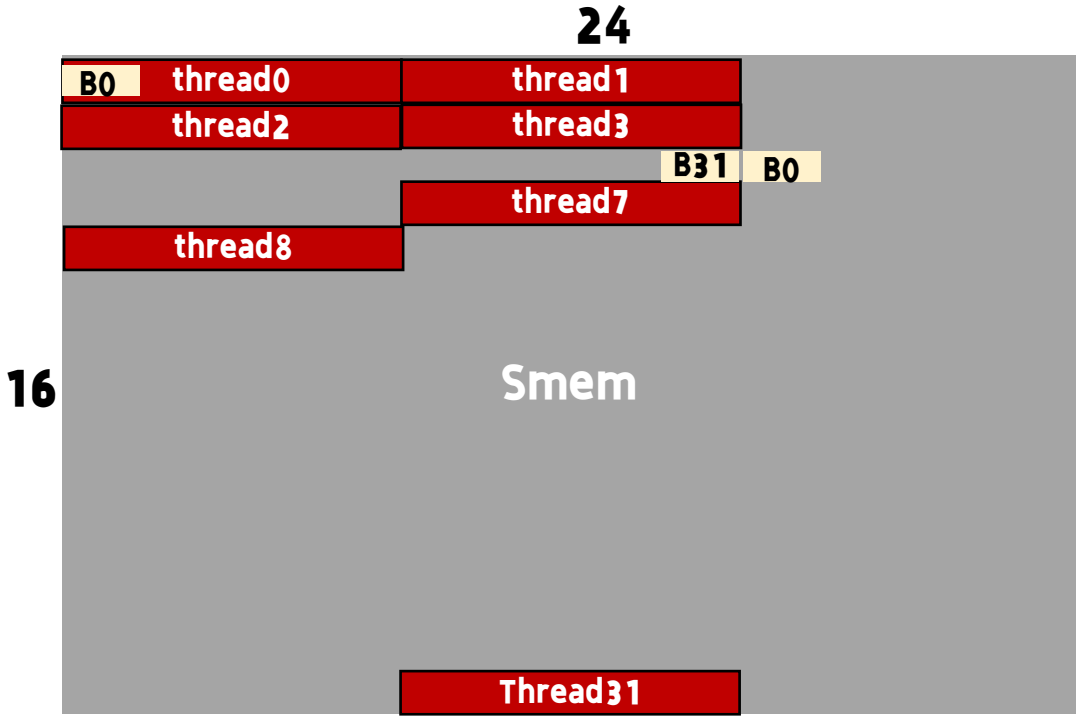
[实用 Swizzle 教程（一） - 知乎]

[Chtholly-Boss/swizzle: A practical way of learning Swizzle]

Tensor Core详解之Bank Conflict

通过padding去解决bank conflict

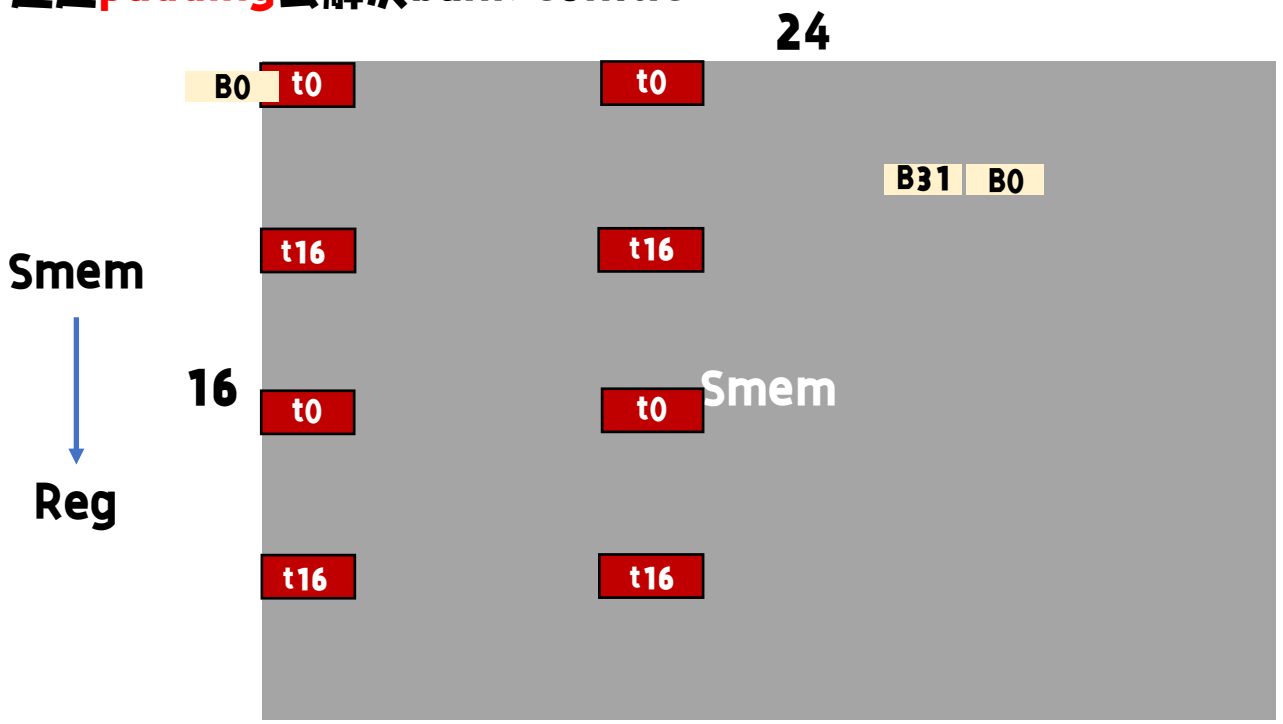


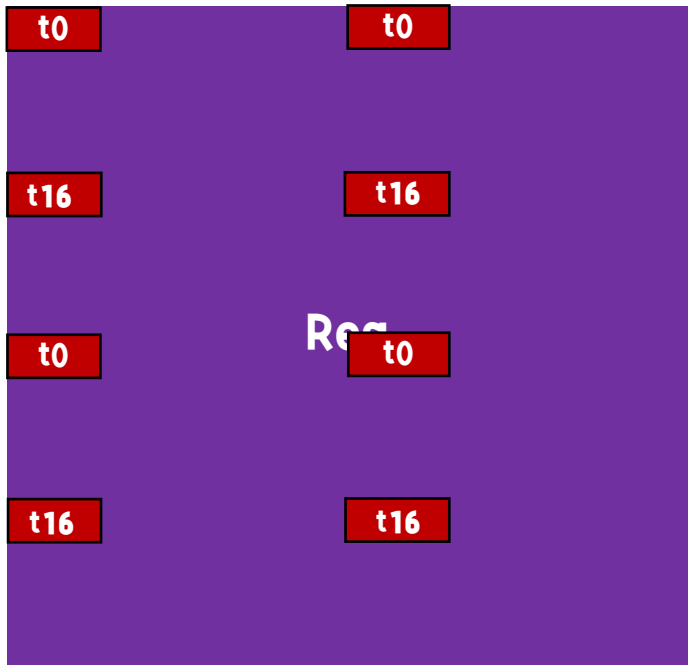


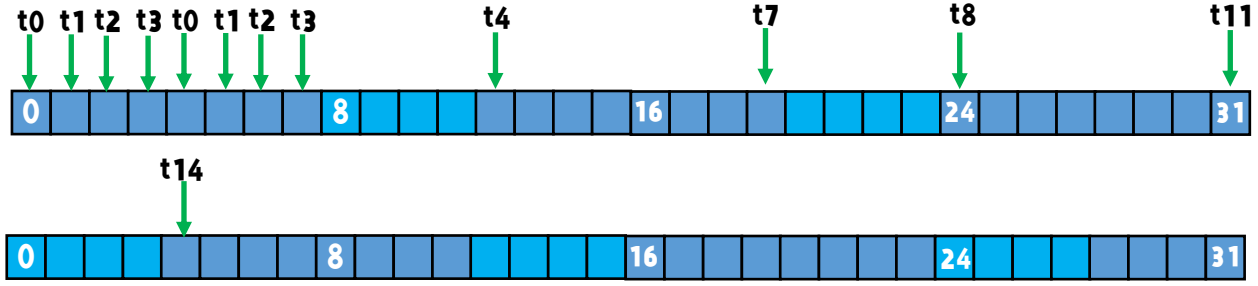
Nvidia Tensor Core-CUDA HGEMM优化进阶 - 知乎

Tensor Core详解之Bank Conflict

通过padding去解决bank conflict

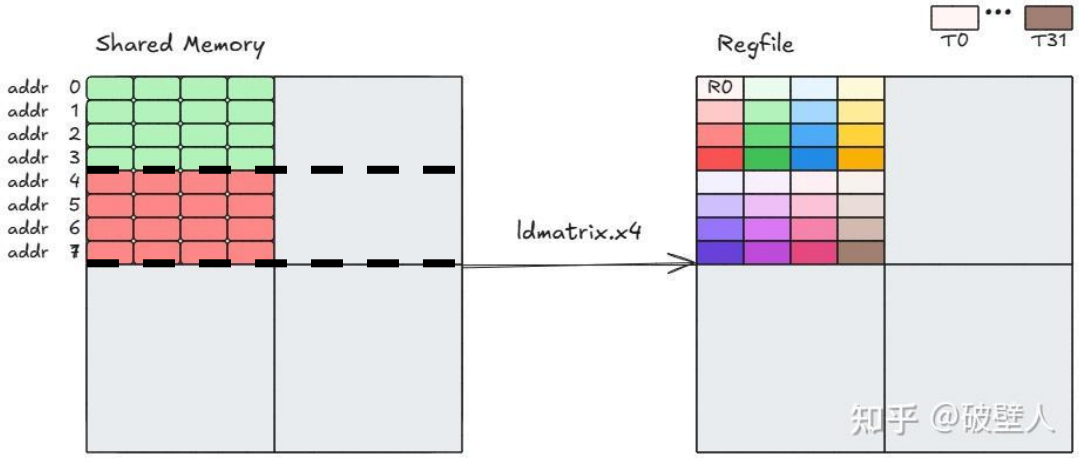


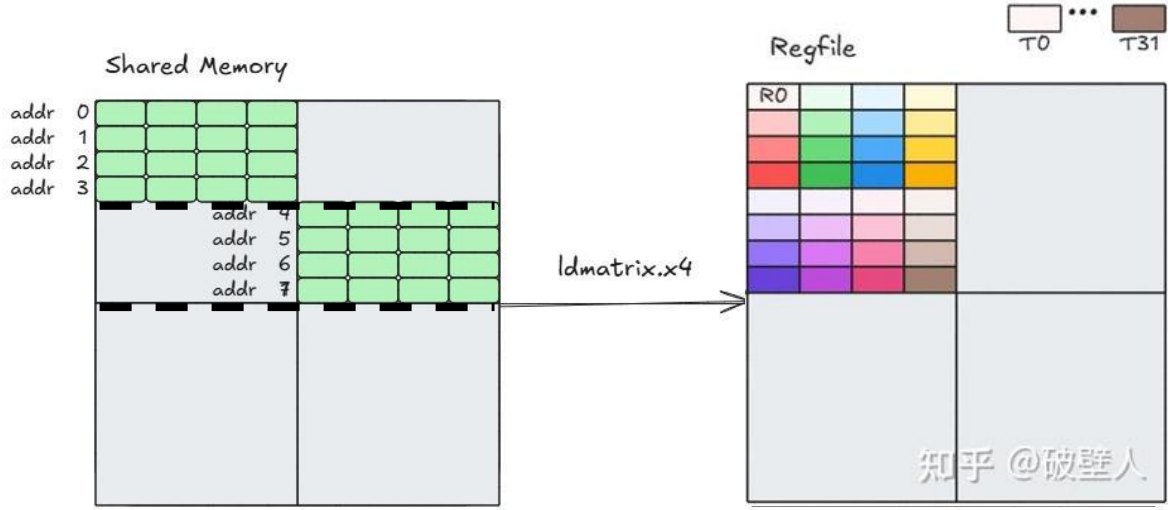




[Nvidia Tensor Core-CUDA HGEMM优化进阶 - 知乎]

Tensor Core详解之Bank Conflict





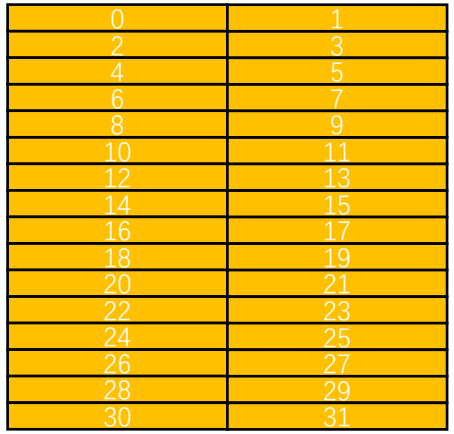
conflict

[实用 Swizzle 教程（一） - 知乎]

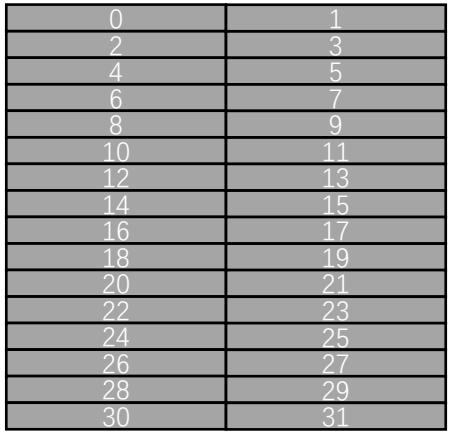
[Chtholly-Boss/swizzle: A practical way of learning Swizzle]

Tensor Core详解之Bank Conflict

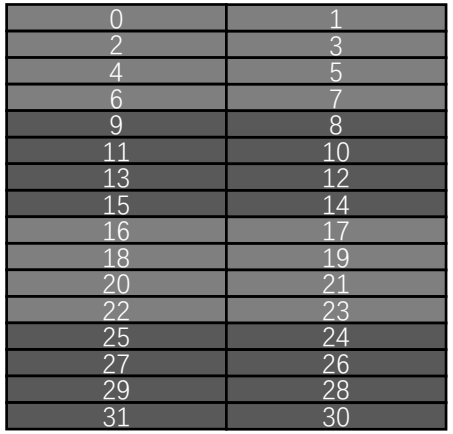
通过swizzle去解决bank conflict



Gmem

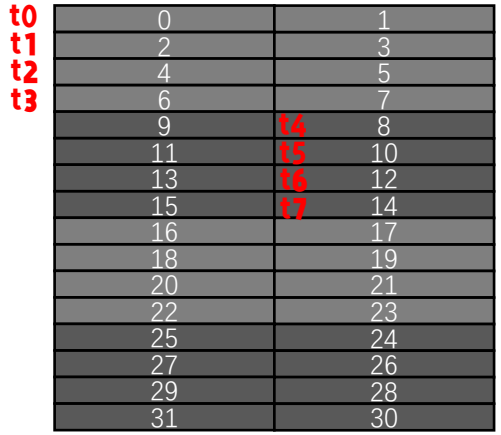


~~Smem~~



Smem

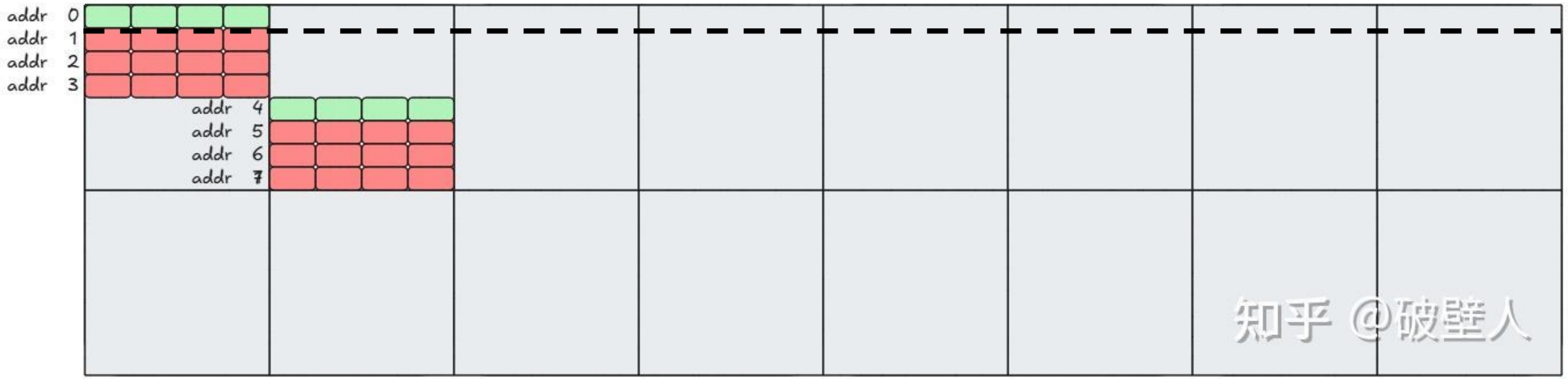
.num	Threads 0–7	Threads 8–15	Threads 16–23	Threads 24–31
.x1	addr0–addr7	–	–	–
.x2	addr0–addr7	addr8–addr15	–	–
.x4	addr0–addr7	addr8–addr15	addr16–addr23	addr24–addr31



[实用 Swizzle 教程（一） - 知乎]

[Chtholly-Boss/swizzle: A practical way of learning Swizzle]

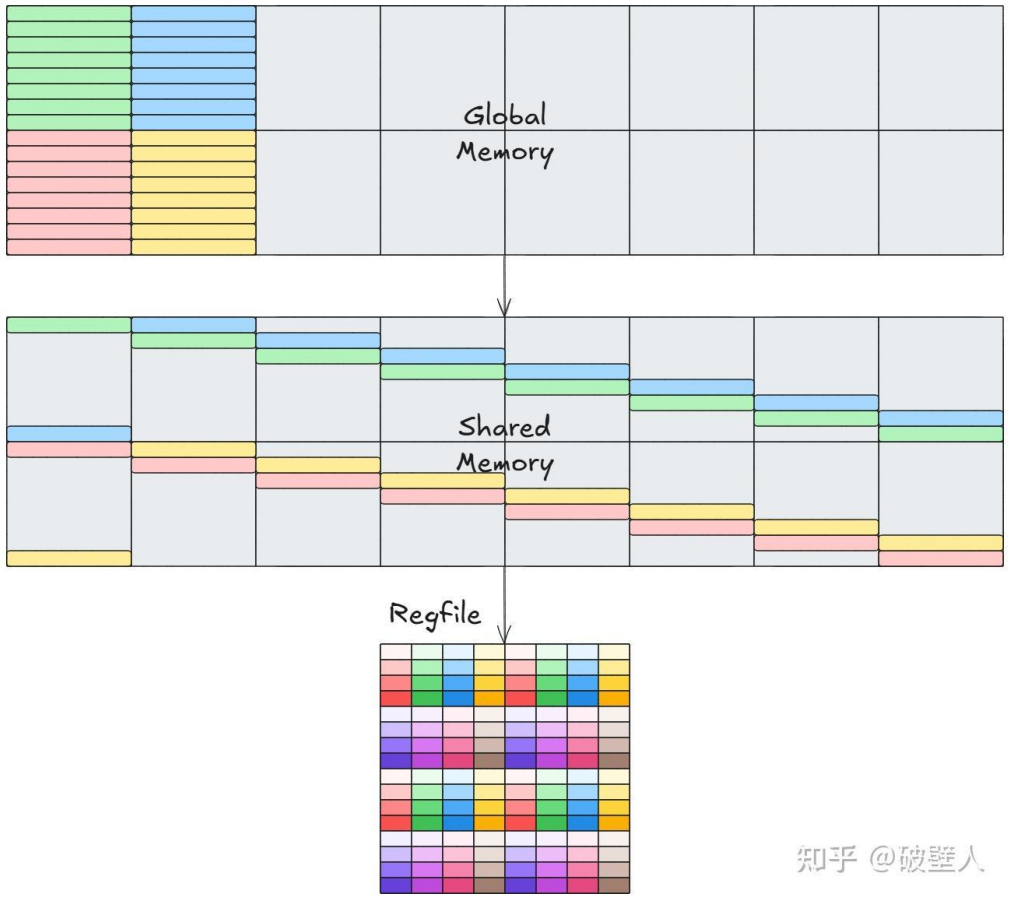
Tensor Core详解之Bank Conflict



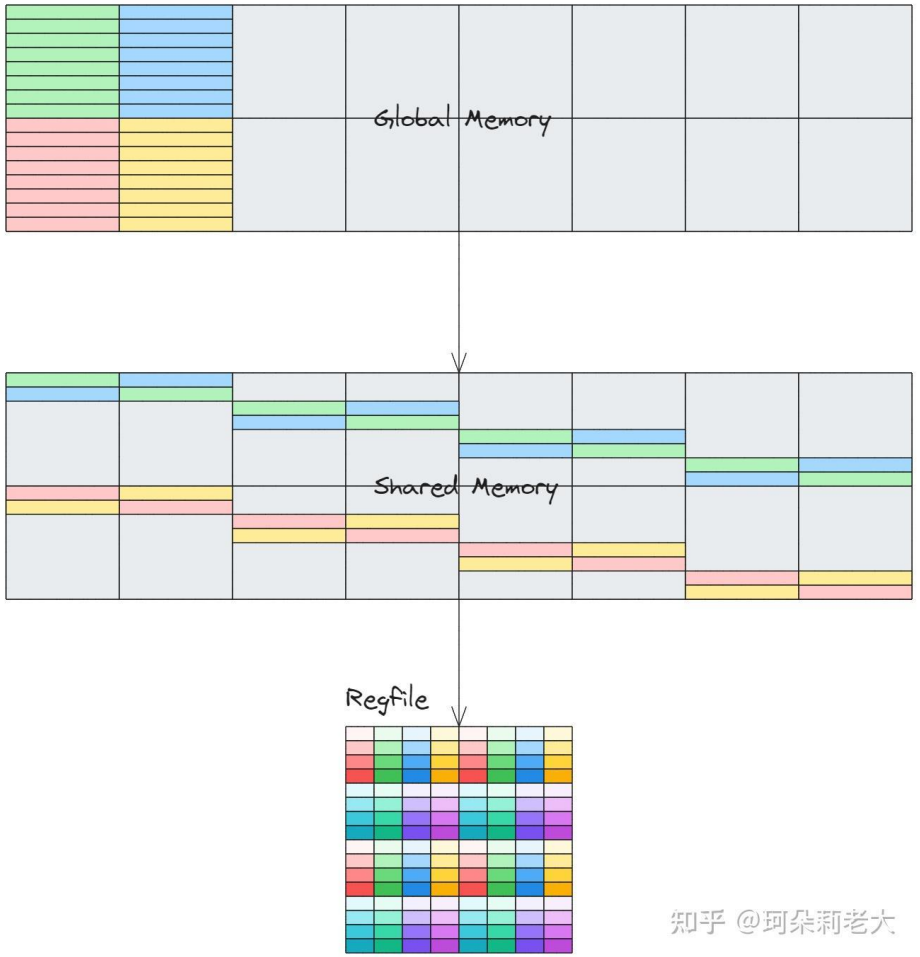
[实用 Swizzle 教程（一） - 知乎]

[Chtholly-Boss/swizzle: A practical way of learning Swizzle]

Tensor Core详解之Bank Conflict



知乎 @破壁人



知乎 @珂朵莉老大

[实用 Swizzle 教程（一） - 知乎]

[Chtholly-Boss/swizzle: A practical way of learning Swizzle]

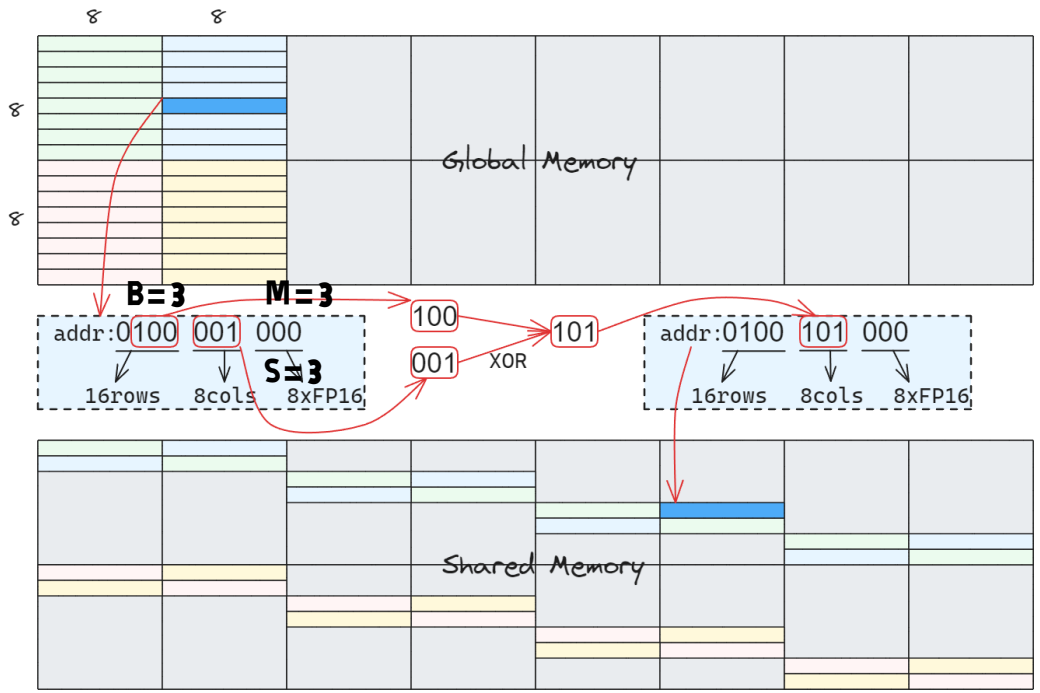
Tensor Core详解之Bank Conflict

通过swizzle去解决bank conflict

- `MBase`：表示一个块所需的位，上例中为 3，一个块有 8 个 `FP16`
- `BBits`：表示 bank 分布的位，上例中为 3，即中间 3 位所表示的 `8cols`
- `SShift`：包含 `gAddr` 信息的位移动到 `BBits` 的距离，用于映射 bank 分布 `BBits`

```
/**
 * \tparam S: SShift, right shift the addr for swizzling
 * \tparam B: BShift, bits to be swizzled
 * \tparam M: MBase, bits keep the same
 */
template <uint32_t S, uint32_t B, uint32_t M>
__device__ __forceinline__ uint32_t swizzle(uint32_t addr) {
    constexpr auto Bmask = ((1 << B) - 1) << M;
    return ((addr >> S) & Bmask) ^ addr;
}
```

知乎 @珂朵莉老大



1 << B → 0b 1000 - 1 → 0b 111 << M → 0b 111 000 (Bmask)
Addr 0b 0 100 001 000 >> S → 0b 0 100 001 & Bmask → 0b 100 000

[实用 Swizzle 教程（二） - 知乎]

[Chtholly-Boss/swizzle: A practical way of learning Swizzle]

cutlass

软件抽象分层与源码浅析



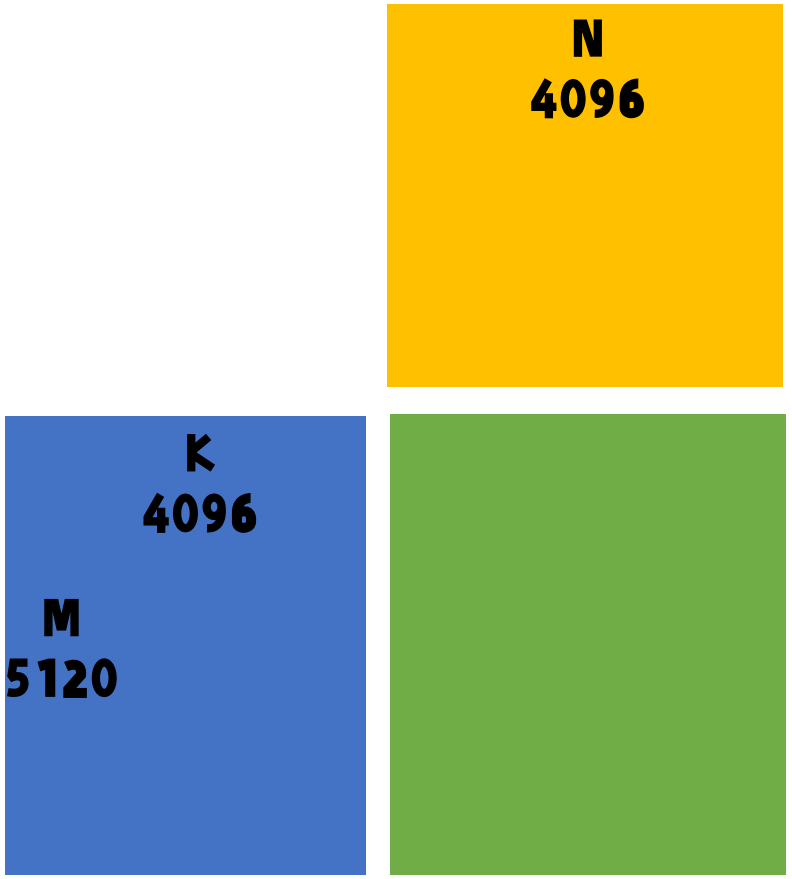
cutlass源码导读（1）——API与设计理念 - 知乎

cutlass源码导读（2）——Gemm的计算流程 - 知乎

cutlass源码导读（3）——核心软件抽象 - 知乎

cutlass源码导读（4）——软件分层与源码目录 - 知乎

Hierarchical GEMM Structure



we create template variables of tile sizes for thread-block, warp and mma-op to 128x256x64, 64x64x16, 8x8x16 (MxNxK) respectively.

我们分别将线程块、线程束和 MMA 操作的分块大小模板变量设置为 128x256x64、64x64x16、8x8x16（M x N x K）

```
// This code section describes the tile size a thread block will compute
using ShapeMMAThreadBlock = cutlass::gemm::GemmShape<128, 256, 64>; // <- threadblock tile M = 128, N = 256, K = 64
// This code section describes tile size a warp will compute
using ShapeMMAWarp = cutlass::gemm::GemmShape<64, 64, 16>; // <- warp tile M = 64, N = 64, K = 64
// This code section describes the size of MMA op
using ShapeMMAOp = cutlass::gemm::GemmShape<8, 8, 16>; // <- MMA Op tile M = 8, N = 8, K = 16
```

Hierarchical GEMM Structure

```
// This code section describes the tile size a thread block will compute
using ShapeMMAThreadBlock = cutlass::gemm::GemmShape<128, 256, 64>;
// <- threadblock tile M = 128, N = 256, K = 64
```

我们称之为tile size。含义是每个CTA负责结果矩阵中M * N（128 * 256）的部分，每次把A矩阵的M * K（128 * 64）和B矩阵的K * M放入Shared Memory中。

```
// This code section describes tile size a warp will compute
using ShapeMMAWarp = cutlass::gemm::GemmShape<64, 64, 16>;
// <- warp tile M = 64, N = 64, K = 16
```

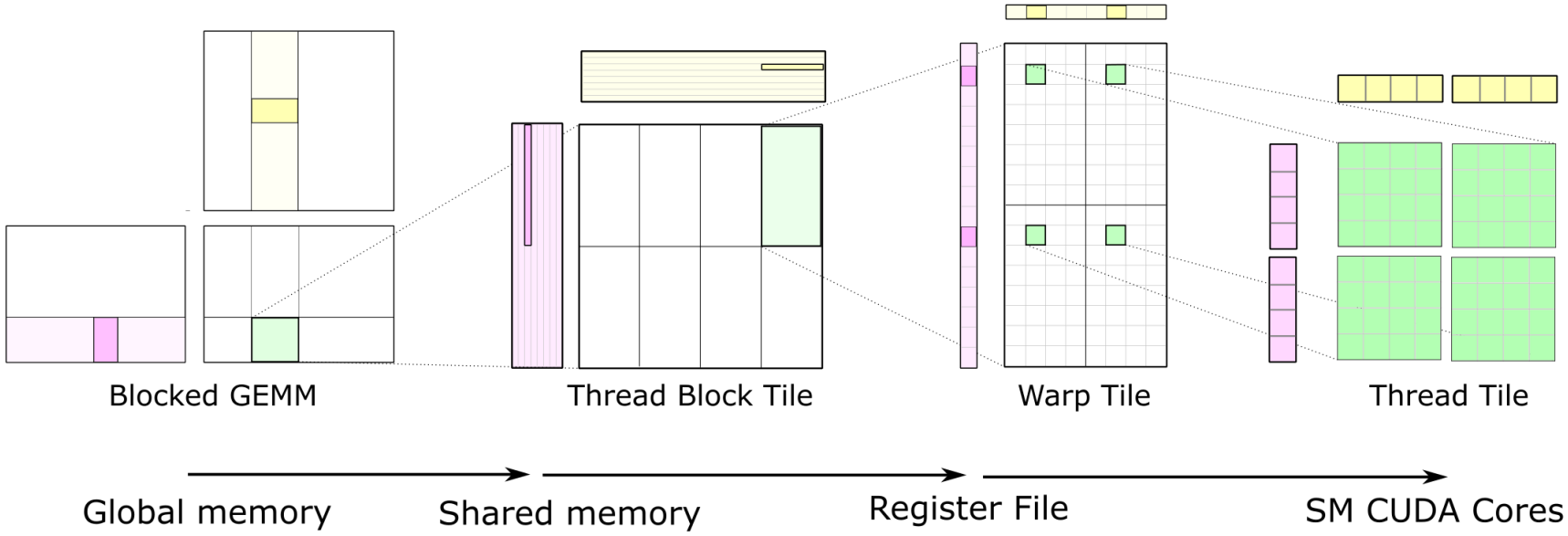
我们称之为warp size。用tile size来除以warp size就是每个CTA的warp数（每一维前者都必须是后者的整数倍）。M与N的含义是该warp负责的大小。

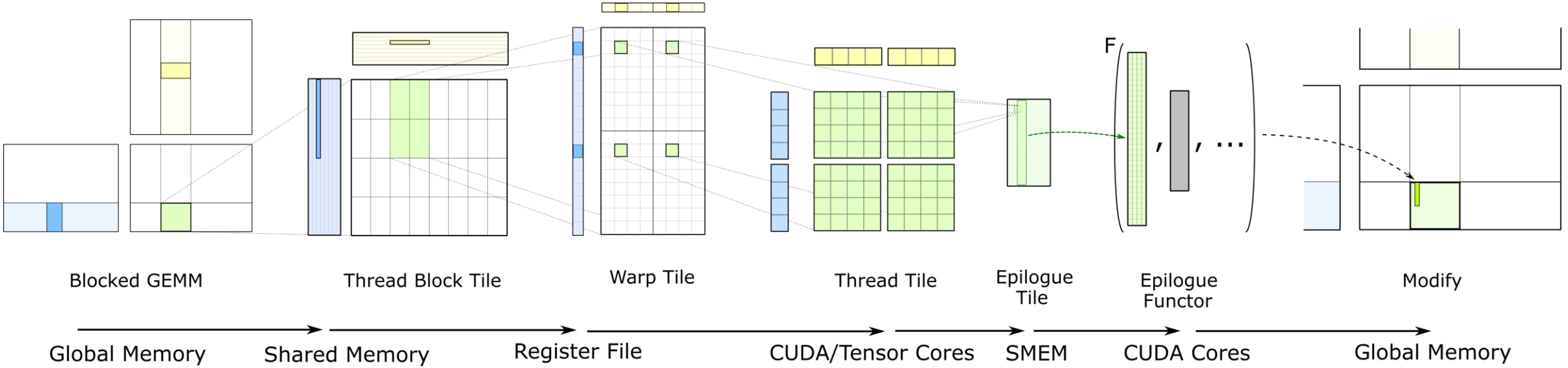
```
// This code section describes the size of MMA op
using ShapeMMAOp = cutlass::gemm::GemmShape<8, 8, 16>;
// <- MMA Op tile M = 8, N = 8, K = 16
```

单条Tensor Core指令负责的部分的大小，这个比较好理解。

[cutlass源码导读（2）——Gemm的计算流程 - 知乎]

Hierarchical GEMM Structure

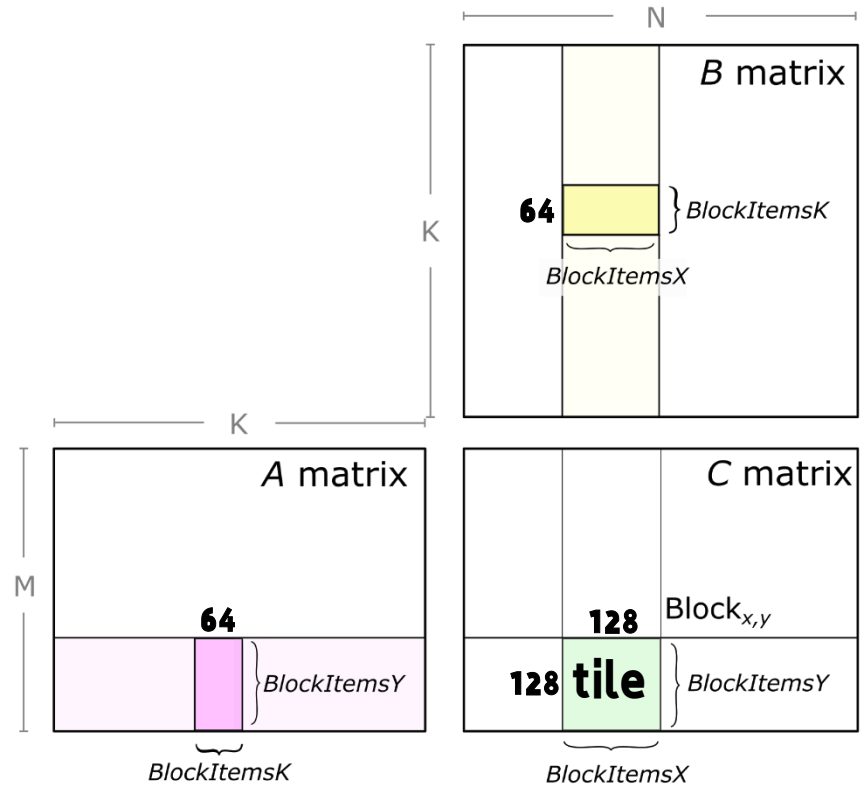


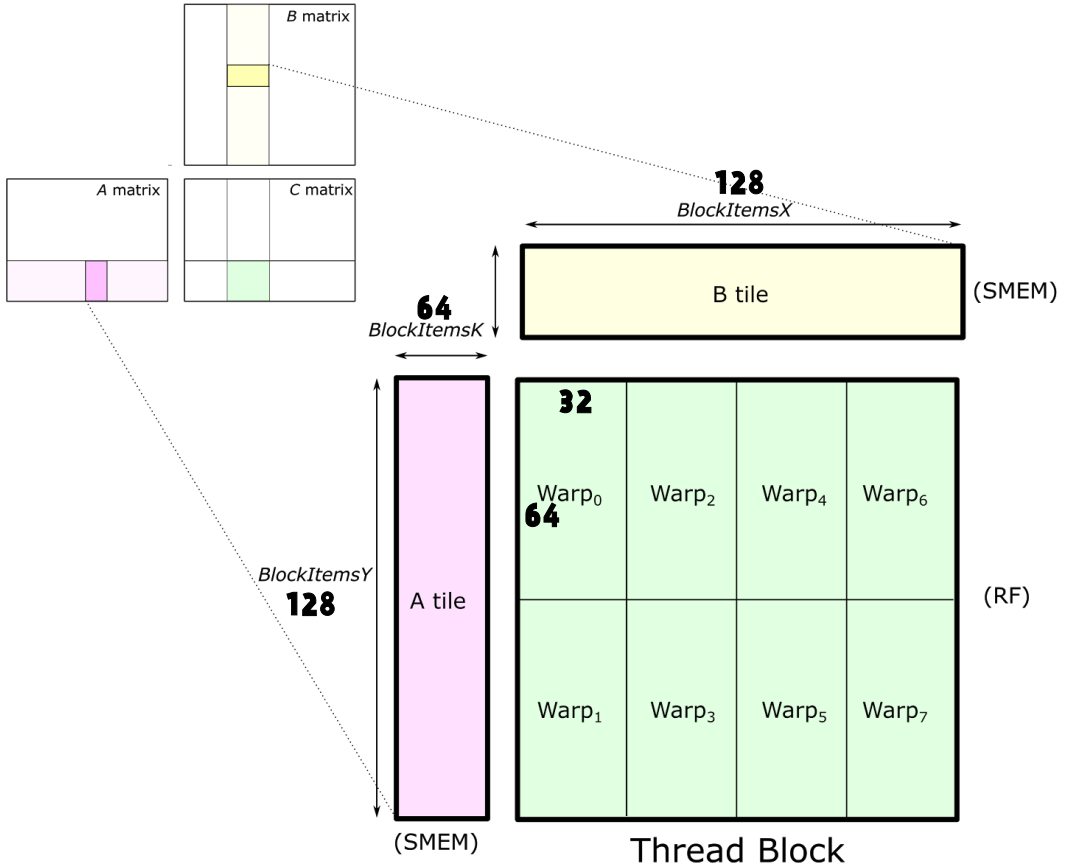


[CUTLASS: Fast Linear Algebra in CUDA C++ | NVIDIA Technical Blog]

Hierarchical GEMM Structure

```
// This code section describes tile size a warp will compute
using ShapeMMAWarp = cutlass::gemm::GemmShape<64, 32, 64>;
// <- warp tile M = 64, N = 64, K = 64
```

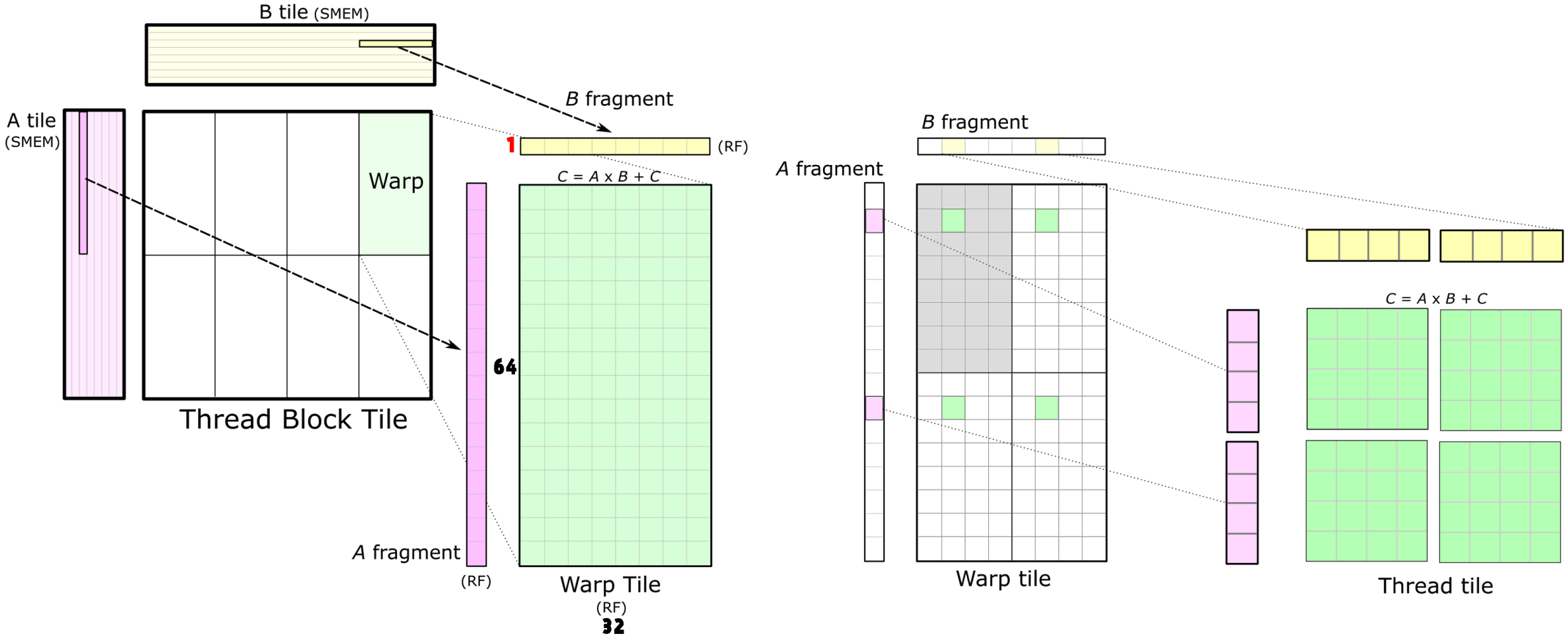





```
// This code section describes the tile size a thread block will compute
using ShapeMMAThreadBlock = cutlass::gemm::GemmShape<128, 128, 64>;
// <- threadblock tile M = 128, N = 128, K = 64
```

[CUTLASS: Fast Linear Algebra in CUDA C++ | NVIDIA Technical Blog]

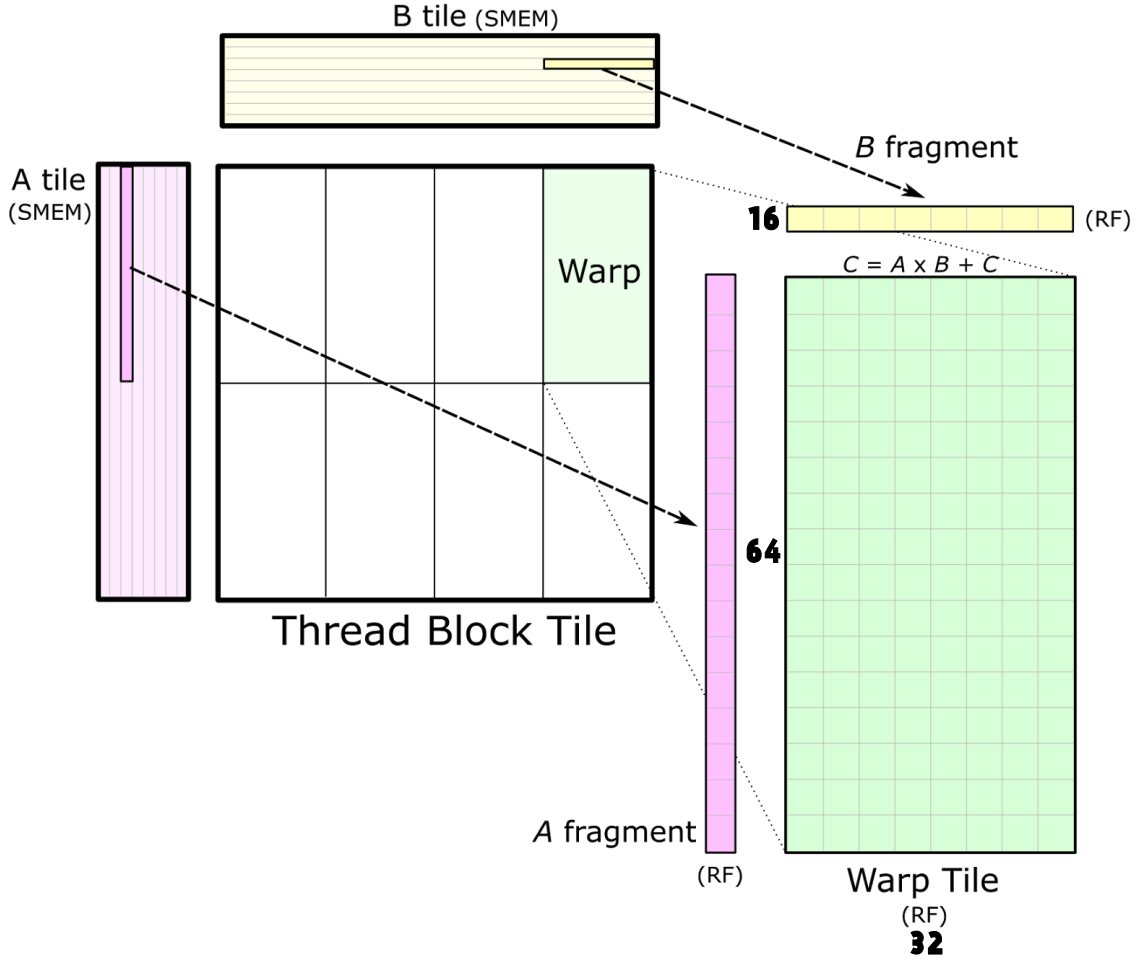
Hierarchical GEMM Structure

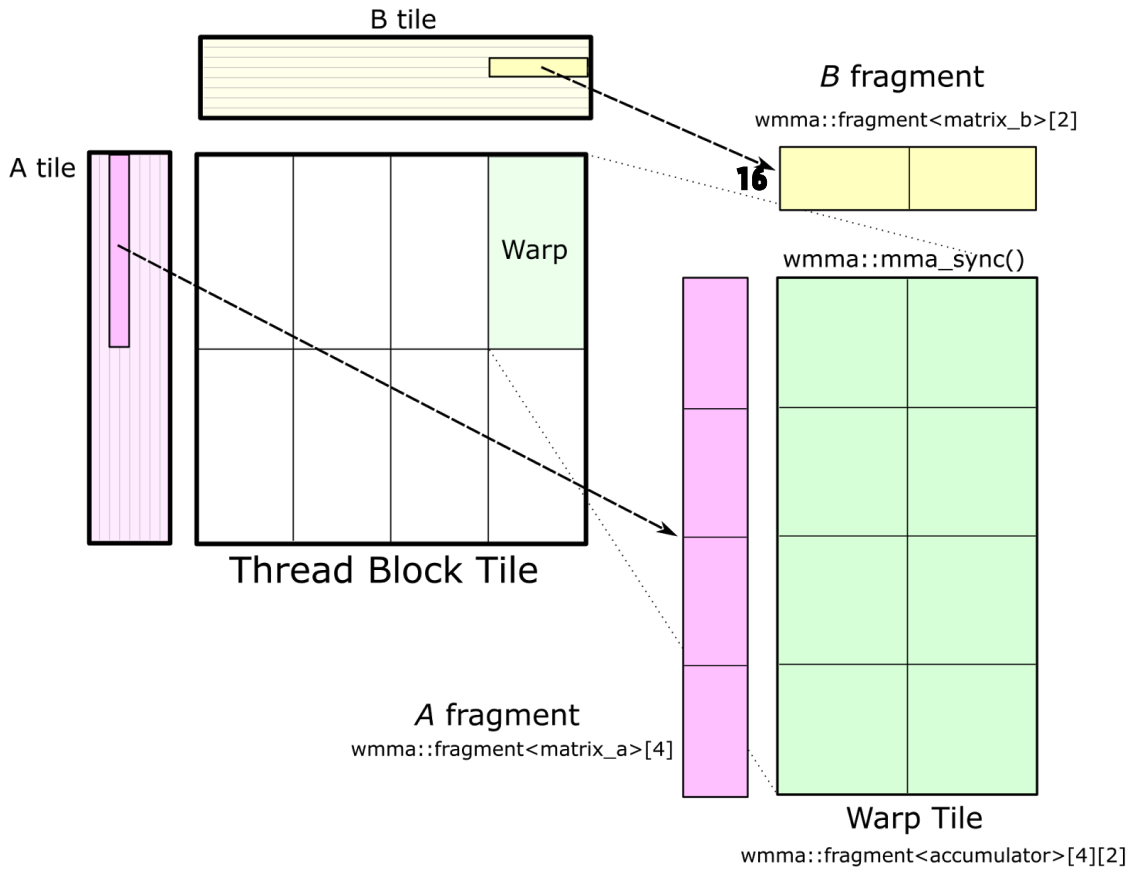


Warp tile的左上象限以灰色阴影显示。32个单元格对应于一个warp内的32个线程。这种排列导致同一行或同一列的多个线程分别获取A和B片段的相同元素。为了最大化计算强度，这种基本结构可以从完整的warp-level累加器tile中复制，从而得到一个由 8x1 和 1x8 片段的外积计算出的 8x8 整体线程warp。

[CUTLASS: Fast Linear Algebra in CUDA C++ | NVIDIA Technical Blog]

Hierarchical GEMM Structure

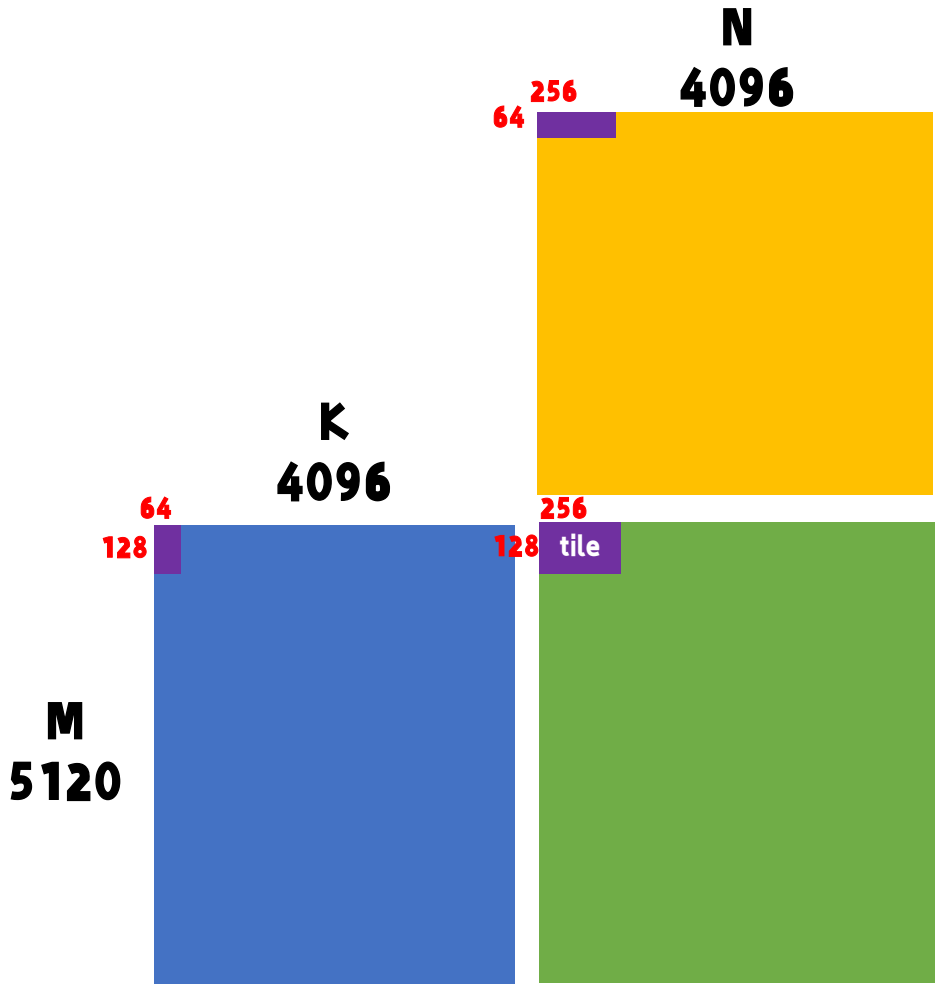





```
// This code section describes the size of MMA op
using ShapeMMAOp = cutlass::gemm::GemmShape<16, 16, 16>;
// <- MMA Op tile M = 16, N = 16, K = 16
```

[CUTLASS: Fast Linear Algebra in CUDA C++ | NVIDIA Technical Blog]

D = alpha * A * B + beta * C



tile_num
4096/256=16, 5120/128=40

```
// This code section describes the tile size a thread block will compute
using ShapeMMAThreadBlock = cutlass::gemm::GemmShape<128, 256, 64>;
// <- threadblock tile M = 128, N = 256, K = 64
```

软件分层之device

device->kernel->threadblock->warp->thread

它对外提供API的接口类，对内完成对Kernel的调用，可以理解是host端的代码

```
using Gemm = cutlass::gemm::device::Gemm<
                            ElementInputA,
                            LayoutInputA,
                            ElementInputB,
                            LayoutInputB,
                            ElementOutput,
                            LayoutOutput,
                            ElementAccumulator,
                            MMAOp,
                            SmArch,
                            ShapeMMAThreadBlock,
                            ShapeMMAWarp,
                            ShapeMMAOp,
                            EpilogueOp,
                            SwizzleThreadBlock,
                            NumStages>;
```

```
using namespace nvcuda;
__global__ void wmma_simple_kernel(half *A, half *B, half *C)
{
    wmma::fragment<wmma::matrix_a, 16, 16, 16, half, wmma::row_major> a_frag;
    wmma::fragment<wmma::matrix_b, 16, 16, 16, half, wmma::row_major> b_frag;
    wmma::fragment<wmma::accumulator, 16, 16, 16, half> c_frag;

    wmma::load_matrix_sync(a_frag, A, 16);
    wmma::load_matrix_sync(b_frag, B, 16);
    wmma::fill_fragment(c_frag, 0.0f);
    wmma::mma_sync(c_frag, a_frag, b_frag, c_frag);
    wmma::store_matrix_sync(C, c_frag, 16, wmma::mem_row_major);
}

void wmma_simple(half *A, half *B, half *C, int M, int N, int K)
{
    dim3 block(32);
    dim3 grid(1);

    wmma_simple_kernel<<<grid, block>>>(A, B, C);
}
```

[cutlass源码导读（3）——核心软件抽象 - 知乎]

[cutlass源码导读（4）——软件分层与源码目录 - 知乎]

软件分层之device

device->kernel->threadblock->warp->thread

它对外提供API的接口类，对内完成对Kernel的调用，可以理解是host端的代码

```cpp
/// Runs the kernel using initialized state.
Status run(cudaStream_t stream = nullptr) {

  ThreadblockSwizzle threadblock_swizzle;

  dim3 grid = threadblock_swizzle.get_grid_shape(params_.grid_tiled_shape);
  dim3 block(GemmKernel::kThreadCount, 1, 1);

  cudaError_t result;

  int smem_size = int(sizeof(typename GemmKernel::SharedStorage));

  if (smem_size >= (48 << 10)) {
    result = cudaFuncSetAttribute(Kernel<GemmKernel>,
                                  cudaFuncAttributeMaxDynamicSharedMemorySize,
                                  smem_size);

    if (result != cudaSuccess) {
      return Status::kErrorInternal;
    }
  }

  cutlass::arch::synclog_setup();
  cutlass::Kernel<GemmKernel><<<grid, block, smem_size, stream>>>(params_);

  result = cudaGetLastError();

  return result == cudaSuccess ? Status::kSuccess : Status::kErrorInternal;
}
```

```cpp
/// Warp count (concept: GemmShape)
using WarpCount = typename Mma::WarpCount;
static int const kThreadCount = 32 * WarpCount::kCount;
```

```cpp
/// Number of warps present
using WarpCount = GemmShape<
  Shape::kM / WarpShape::kM,
  Shape::kN / WarpShape::kN,
  Shape::kK / WarpShape::kK
>;
```

```cpp
struct GemmShape {
  static int const kM = M;
  static int const kN = N;
  static int const kK = K;

  static int const kMN = M * N;
  static int const kMK = M * K;
  static int const kKN = N * K;
  static int const kMNK = M * N * K;

  static int const kCount = kMNK;
```

```cpp
// This code section describes the tile size a thread block will compute
using ShapeMMAThreadBlock = cutlass::gemm::GemmShape<128, 256, 64>;
// <- threadblock tile M = 128, N = 256, K = 64
// This code section describes tile size a warp will compute
using ShapeMMAWarp = cutlass::gemm::GemmShape<64, 64, 64>;
// <- warp tile M = 64, N = 64, K = 64
```

软件分层之device

device->kernel->threadblock->warp->thread

它对外提供API的接口类，对内完成对Kernel的调用，可以理解是host端的代码

```cpp
/// Runs the kernel using initialized state.
Status run(cudaStream_t stream = nullptr) {

  ThreadblockSwizzle threadblock_swizzle;

  dim3 grid = threadblock_swizzle.get_grid_shape(params_.grid_tiled_shape);
  dim3 block(GemmKernel::kThreadCount, 1, 1);

  cudaError_t result;

  int smem_size = int(sizeof(typename GemmKernel::SharedStorage));

  if (smem_size >= (48 << 10)) {
    result = cudaFuncSetAttribute(Kernel<GemmKernel>,
                                  cudaFuncAttributeMaxDynamicSharedMemorySize,
                                  smem_size);

    if (result != cudaSuccess) {
      return Status::kErrorInternal;
    }
  }

  cutlass::arch::synclog_setup();
  cutlass::Kernel<GemmKernel><<<grid, block, smem_size, stream>>>(params_);

  result = cudaGetLastError();

  return result == cudaSuccess ? Status::kSuccess : Status::kErrorInternal;
}
```

```cpp
/// Shared memory storage structure
union SharedStorage {
  typename Mma::SharedStorage main_loop;
  typename Epilogue::SharedStorage epilogue;
};
```

```cpp
/// Buffer for A operand
AlignedBuffer<typename Operator::ElementA, ShapeA::kCount> operand_A;

/// Buffer for B operand
AlignedBuffer<typename Operator::ElementB, ShapeB::kCount> operand_B;
```

```cpp
  /// Number of storage elements
  static int const kBytes =
    (sizeof_bits<T>::value * N + 7) / 8;

private:

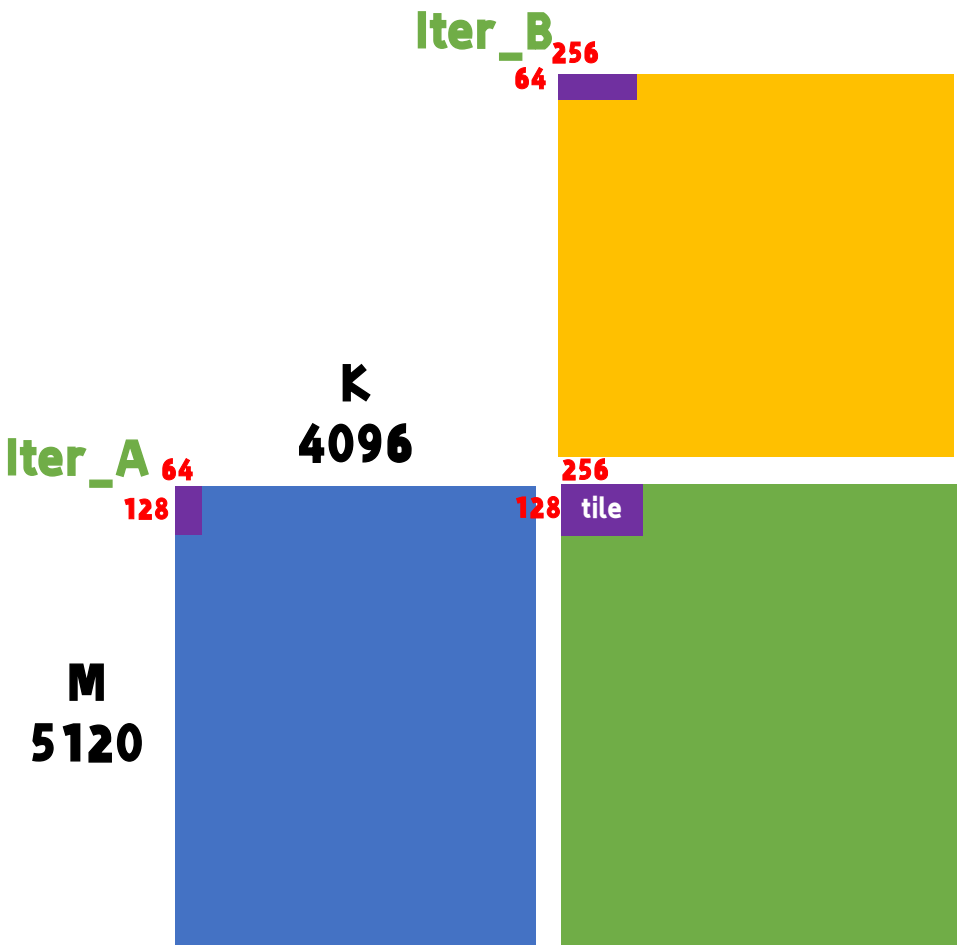
  /// Internal storage
  alignas(Align) Storage storage[kBytes];
```

软件分层之kernel

device->kernel->threadblock->warp->thread

```
namespace device {
class Gemm {
  using WarpShape = WarpShape_;
  using InstructionShape = InstructionShape_;
  using EpilogueOutputOp = EpilogueOutputOp_;
  using ThreadblockSwizzle = ThreadblockSwizzle_;
  using Operator = Operator_;
  static int const kStages = Stages;
  static int const kAlignmentA = AlignmentA;
  static int const kAlignmentB = AlignmentB;
  static int const kAlignmentC = EpilogueOutputOp::kCount;
  static bool const kSplitKSerial = SplitKSerial;
  static ComplexTransform const kTransformA = ComplexTransform::kNone;
  static ComplexTransform const kTransformB = ComplexTransform::kNone;

  /// Define the kernel
  using GemmKernel = typename kernel::DefaultGemm<
    ElementA,
    LayoutA,
    kAlignmentA,
    ElementB,
    LayoutB,
    kAlignmentB,
    ElementC,
```



软件分层之threadblock

device->kernel->threadblock->warp->thread

```
  /// Perform a threadblock-scoped matrix multiply-accumulate
  CUTLASS_DEVICE
  void operator()(
      int gemm_k_iterations,      ///< number of iterations of the mainloop
      FragmentC &accum,           ///< destination accumulator tile
      IteratorA iterator_A,       ///< iterator over A operand in global memory
      IteratorB iterator_B,       ///< iterator over B operand in global memory
      FragmentC const &src_accum) ///< source accumulator tile
  {
    // Prologue
    prologue(iterator_A, iterator_B, gemm_k_iterations);

    // Wait until we have at least one completed global fetch stage
    gmem_wait();

    // Perform accumulation in the 'd' output operand
    accum = src_accum;

    // Perform the MAC-iterations
    gemm_iters(gemm_k_iterations, accum, iterator_A, iterator_B);
  }
};
```

软件分层之threadblock

device->kernel->threadblock->warp->thread

2 数据移动

我们现在来细化上文说的“数据依次进入寄存器”这个过程，也就是引入GPU上的数据移动的过程

在GPU中，数据是多级存储的，主要是三级：
Global memory -- Shared Memory✦ -- Register

整个CUDA Kernel共享一个Global memory地址空间，每个CTA有自己的Shared Memory，该CTA的warps可以共用这份Shared Memory(Hopper引入了distrubute shared memory，暂时先不讨论). 每个thread拥有自己独立的多个Register，也称为Register file✦。

在整个GEMM的流程中，数据的移动轨迹是这样的：
Global memory -> Register -> Shared Memory -> Register -> Compute -> Shared Memory -> Register -> Gloable memory

如果引入LDGSTS指令，Global memory的数据可以直接进Shared Memory，不需要经过Register周转。

```
// Load A fragment from global A
FragmentA tb_frag_A;
tb_frag_A.clear();
iterator_A.load(tb_frag_A);
++iterator_A;

// Load B fragment from global B
FragmentB tb_frag_B;
tb_frag_B.clear();
iterator_B.load(tb_frag_B);
++iterator_B;

// Store A and B fragments to shared
this->smem_iterator_A_.store(transform_A_(tb_frag_A));
this->smem_iterator_B_.store(transform_B_(tb_frag_B));
```

cutlass源码导读（2）——Gemm的计算流程 - 知乎

软件分层之warp

device->kernel->threadblock->warp->arch?

```
if (AccumulatorsInRowMajor)
{ // matrix B is reordered
  mma(
      ptr_D[n + m_serpentine * MmaIterations::kColumn],
      ptr_A[m_serpentine],
      ptr_B[n],
      ptr_D[n + m_serpentine * MmaIterations::kColumn]);
}
else
{
  mma(
      ptr_D[m_serpentine + n * MmaIterations::kRow],
      ptr_A[m_serpentine],
      ptr_B[n],
      ptr_D[m_serpentine + n * MmaIterations::kRow]);
}
```

```
#if defined(CUTLASS_ARCH_MMA_SM75_ENABLED)

  unsigned const & A = reinterpret_cast<unsigned const &>(a);
  unsigned const & B = reinterpret_cast<unsigned const &>(b);

  int const *C = reinterpret_cast<int const *>(&c);
  int *D = reinterpret_cast<int *>(&d);

  asm volatile("mma.sync.aligned.m8n8k16.row.col.satfinite.s32.s8.s8.s32 {%0,%1}, {%2}, {%3}, {%4,%5};\n"
      : "=r"(D[0]), "=r"(D[1])
      : "r"(A), "r"(B), "r"(C[0]), "r"(C[1]));
```

软件分层之epilogue

epologue_op_t 是一个在矩阵乘法运算结束后的用来更新输出矩阵的functor，这个functor会被作为（ block_task_t 的）模板参数。这种设计让我们能轻松的将矩阵乘法和用户定义的元素操作相结合，详细内容见后文。CUTLASS提供 gemm::blas_scaled_epilogue 函子来计算GEMM操作 $\mathbf{C} = \alpha * \mathbf{AB} + \beta * \mathbf{C}$ （在(epilogue_function.h 被定义）。

D = alpha * A * B + beta * C

```
namespace cutlass {
namespace gemm {
namespace kernel {
struct Gemm {
  void operator()(Params const &params, SharedStorage &shared_storage) {

    // Construct thread-scoped matrix multiply
    Mma mma(shared_storage.main_loop, thread_idx, warp_idx, lane_idx);

    typename Mma::FragmentC accumulators;

    accumulators.clear();

    if (!kSplitKSerial || gemm_k_iterations > 0) {
      // Compute threadblock-scoped matrix multiply-add
      mma(gemm_k_iterations, accumulators, iterator_A, iterator_B, accumulators);
    }

    //
    // Epilogue
    //

    OutputOp output_op(params.output_op);
```

在kernel这个层级里面包含epilogue，并且最关键的点在于mma函数的返回值存储在寄存器中

CUTLASS: Fast Linear Algebra in CUDA C++ - 知乎